AuxiRuta

Código explicado, archivo por archivo

Guía de lectura técnica: Java, SQL, seguridad, integración, configuración, despliegue, pruebas y empaquetado.

Edición: 10 de noviembre de 2026. Fuente: C:/Users/Enzo/BACK-TPI-G1. Describe la implementación encontrada; no sustituye una auditoría ni supone centralización completa de reglas.

Cada archivo incluye ruta, propósito y SHA-256. Cada línea no vacía del código propio se reproduce numerada y comentada. Las líneas largas se envuelven sin cambiar números; las vacías se enumeran como separadores. Los tests se resumen por clase/casos. Maven Wrapper es herramienta de terceros y se explica por función. Los JAR se inspeccionan como ZIP de bytecode, no texto fuente.

Exclusiones: .git, .gitignore, .dockerignore, documentación/diagramas previos, .env real, temporales y clases/reportes/cachés de target. Excepción: JAR presentes y sus originales. No se incluyeron secretos de la instalación.

Ruta de lectura: construcción -> Commons/seguridad -> persistencia -> flota/solicitudes -> evaluación -> propuestas -> ejecución/automatización -> pruebas/JAR. El índice y los marcadores del PDF permiten saltar directamente a cada archivo.

Índice de temas y archivos

1. Construcción, despliegue y entorno 6

.env.example . . . 6
api-gateway/Dockerfile . . . 7
api-gateway/pom.xml . . . 8
assistance-service/Dockerfile . . . 12
assistance-service/pom.xml . . . 13
auxiruta-commons/pom.xml . . . 19
docker-compose.yml . . . 21
fleet-service/Dockerfile . . . 29
fleet-service/pom.xml . . . 30
infra/postgres/01-databases.sh . . . 35
scripts/Start-AuxiRuta.ps1 . . . 37
scripts/Verify-AuxiRuta.ps1 . . . 39

2. Biblioteca común 42

GeoMath (auxiruta-commons) . . . 42
Problems (auxiruta-commons) . . . 42
TokenSupport (auxiruta-commons) . . . 44
UtcClock (auxiruta-commons) . . . 47

3. Arranque, seguridad, Gateway y errores 48

GatewayApplication (api-gateway) . . . 48
GatewayRoutes (api-gateway) . . . 48
RequestLoggingFilter (api-gateway) . . . 50
SecurityConfiguration (api-gateway) . . . 51
SecurityErrors (api-gateway) . . . 58
application-docker (api-gateway) . . . 61
application (api-gateway) . . . 61
AssistanceApplication (assistance-service) . . . 62
ClockConfiguration (assistance-service) . . . 62
OpenApiConfiguration (assistance-service) . . . 63
RequestLoggingFilter (assistance-service) . . . 64
SecurityConfiguration (assistance-service) . . . 66
SecurityErrors (assistance-service) . . . 71
ApiExceptionHandler (assistance-service) . . . 73
application-docker (assistance-service) . . . 75
application (assistance-service) . . . 78
FleetApplication (fleet-service) . . . 79
ClockConfiguration (fleet-service) . . . 80
OpenApiConfiguration (fleet-service) . . . 81
RequestLoggingFilter (fleet-service) . . . 82
SecurityConfiguration (fleet-service) . . . 83

SecurityErrors (fleet-service) . . . 89
ApiExceptionHandler (fleet-service) . . . 90
application-docker (fleet-service) . . . 93
application (fleet-service) . . . 94
auxiruta-realm (infra) . . . 96

4. Persistencia: entidades, repositorios y SQL 111

AssistanceRequest (assistance-service) . . . 111
CandidateEvaluation (assistance-service) . . . 114
CandidateEvaluationRepository (assistance-service) . . . 117
Evaluation (assistance-service) . . . 118
EvaluationRepository (assistance-service) . . . 120
HistoryRepository (assistance-service) . . . 121
RequestRepository (assistance-service) . . . 122
RequestStatusHistory (assistance-service) . . . 123
Vehicle (assistance-service) . . . 125
VehicleRepository (assistance-service) . . . 127
001-assistance (assistance-service) . . . 128
002-evaluation (assistance-service) . . . 131
003-assignments (assistance-service) . . . 136
004-execution (assistance-service) . . . 137
005-automation (assistance-service) . . . 138
Operator (fleet-service) . . . 139
OperatorRepository (fleet-service) . . . 141
ReservationRepository (fleet-service) . . . 142
ResourceReservation (fleet-service) . . . 143
TowTruck (fleet-service) . . . 144
TowTruckRepository (fleet-service) . . . 147
TruckStatusHistory (fleet-service) . . . 148
TruckStatusHistoryRepository (fleet-service) . . . 149
001-fleet-catalog (fleet-service) . . . 150
002-reservations (fleet-service) . . . 153
003-automation (fleet-service) . . . 154

5. Flota: administración, candidatos y reservas 155

CandidateController (fleet-service) . . . 155
CandidateService (fleet-service) . . . 156
FleetController (fleet-service) . . . 158
FleetDtos (fleet-service) . . . 164
FleetService (fleet-service) . . . 167
FleetTypes (fleet-service) . . . 177
ReservationController (fleet-service) . . . 178
ReservationJobs (fleet-service) . . . 180
ReservationService (fleet-service) . . . 181

6. Vehículos, solicitudes y reglas 189

DomainTypes (assistance-service) . . . 189
RequestController (assistance-service) . . . 189
RequestDtos (assistance-service) . . . 194
RequestService (assistance-service) . . . 197
RulesConfiguration (assistance-service) . . . 206
mvp (assistance-service) . . . 209

7. Evaluación, rutas e integración externa 214

DispatchValidator (assistance-service) . . . 214
EvaluationReadService (assistance-service) . . . 217
EvaluationService (assistance-service) . . . 219
EvaluationStore (assistance-service) . . . 225
FleetClient (assistance-service) . . . 228
OpenRouteServiceClient (assistance-service) . . . 232
RouteProvider (assistance-service) . . . 238
RoutingBudget (assistance-service) . . . 238
RoutingFailure (assistance-service) . . . 240

8. Propuestas, aceptación y recuperación durable 242

AssignmentController (assistance-service) . . . 242
AssignmentCoordinator (assistance-service) . . . 244
AssignmentStore (assistance-service) . . . 247

9. Ejecución y automatización 257

AutomationJobs (assistance-service) . . . 257
AutomationStore (assistance-service) . . . 259
ExecutionController (assistance-service) . . . 265
ExecutionService (assistance-service) . . . 268
ExecutionStore (assistance-service) . . . 269

10. Pruebas y utilidades de demostración 276

BackendClient (api-gateway) . . . 276
BackendIntegrationTest (api-gateway) . . . 276
DeliveryArtifacts (api-gateway) . . . 276
DeliveryArtifactsTest (api-gateway) . . . 276
DeliveryIntegrationTest (api-gateway) . . . 276
DemoDataSetupTest (api-gateway) . . . 277
KeycloakScopeSetupTest (api-gateway) . . . 277
Phase7IntegrationTest (api-gateway) . . . 277
Phase8IntegrationTest (api-gateway) . . . 277
Phase9IntegrationTest (api-gateway) . . . 278
PrivacyDemoSetupTest (api-gateway) . . . 278
SecurityConfigurationTest (assistance-service) . . . 278
AssignmentCoordinatorTest (assistance-service) . . . 278
AutomationConfigurationTest (assistance-service) . . . 279
AutomationStoreTest (assistance-service) . . . 279
DispatchValidatorTest (assistance-service) . . . 279

EvaluationServiceTest (assistance-service) . . . 279
EvaluationStoreTest (assistance-service) . . . 280
ExecutionRulesTest (assistance-service) . . . 280
ExecutionServiceTest (assistance-service) . . . 280
RequestRulesTest (assistance-service) . . . 280
OpenRouteServiceClientTest (assistance-service) . . . 280
RoutingBudgetTest (assistance-service) . . . 281
TokenSupportTest (auxiruta-commons) . . . 281
CandidateServiceTest (fleet-service) . . . 281
FleetServiceTest (fleet-service) . . . 281
ReservationServiceTest (fleet-service) . . . 282
SecurityConfigurationTest (fleet-service) . . . 282

11. Maven Wrapper: herramientas distribuidas 283

api-gateway/.mvn/wrapper/maven-wrapper.properties . . . 283
api-gateway/mvnw . . . 283
api-gateway/mvnw.cmd . . . 283
assistance-service/.mvn/wrapper/maven-wrapper.properties . . . 283
assistance-service/mvnw . . . 284
assistance-service/mvnw.cmd . . . 284
auxiruta-commons/.mvn/wrapper/maven-wrapper.properties . . . 284
auxiruta-commons/mvnw . . . 284
auxiruta-commons/mvnw.cmd . . . 284
fleet-service/.mvn/wrapper/maven-wrapper.properties . . . 285
fleet-service/mvnw . . . 285
fleet-service/mvnw.cmd . . . 285

12. JAR: binarios y contenido empaquetado 286

api-gateway/target/app.jar . . . 286
api-gateway/target/app.jar.original . . . 287
assistance-service/target/app.jar . . . 287
assistance-service/target/app.jar.original . . . 290
auxiruta-commons/target/auxiruta-commons-1.0.0.jar . . . 292
fleet-service/target/app.jar . . . 293
fleet-service/target/app.jar.original . . . 295

13. Glosario de lectura 297

1. Construcción, despliegue y entorno

Commons se instala antes de compilar dependientes. Docker construye y arranca cinco contenedores; no hace falta frontend para ejecutar el proceso.

.env.example

Ruta: .env.example

SHA-256: 4e5287c7f20702d4b20b5dde5463dac02b0535920d887c0a7b91a41a4181a726

Plantilla pública de variables. El .env real no se inspecciona ni se reproduce en este documento.

L1

```
# Copiar a .env y reemplazar cada valor antes del primer arranque.
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
POSTGRES_ADMIN_PASSWORD=CAMBIAR_POSTGRES_ADMIN_PASSWORD
```

Variable POSTGRES_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L3

```
ASSISTANCE_DB_PASSWORD=CAMBIAR_ASSISTANCE_DB_PASSWORD
```

Variable ASSISTANCE_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L4

```
FLEET_DB_PASSWORD=CAMBIAR_FLEET_DB_PASSWORD
```

Variable FLEET_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L5

```
KEYCLOAK_DB_PASSWORD=CAMBIAR_KEYCLOAK_DB_PASSWORD
```

Variable KEYCLOAK_DB_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L6

```
KC_BOOTSTRAP_ADMIN_PASSWORD=CAMBIAR_KC_BOOTSTRAP_ADMIN_PASSWORD
```

Variable KC_BOOTSTRAP_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L7

```
KEYCLOAK_CLIENT_SECRET=CAMBIAR_KEYCLOAK_CLIENT_SECRET
```

Variable KEYCLOAK_CLIENT_SECRET. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L8

```
DEMO_DRIVER_PASSWORD=CAMBIAR_DEMO_DRIVER_PASSWORD
```

Variable DEMO_DRIVER_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L9

```
DEMO_OPERATOR_PASSWORD=CAMBIAR_DEMO_OPERATOR_PASSWORD
```

Variable DEMO_OPERATOR_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L10

```
DEMO_ADMIN_PASSWORD=CAMBIAR_DEMO_ADMIN_PASSWORD
```

Variable DEMO_ADMIN_PASSWORD. Contraseña/secreto elegido antes del primer arranque; aquí solo hay marcador.

L12

```
# OpenRouteService (local secret; never commit a real key)
```

Comentario o directiva del intérprete; no es regla de negocio.

L13

```
ORS_API_KEY=
```

Variable ORS_API_KEY. Clave externa obtenida de ORS.

L14

```
ORS_BASE_URL=https://api.heigit.org/openrouteservice/v2
```

Variable ORS_BASE_URL. Configuración pública del servicio; conservar el default salvo cambio de entorno.

L16

```
# Normal: docker. Demo: docker,demo. Temporal quotas remain unchanged.
```

Comentario o directiva del intérprete; no es regla de negocio.

L17

```
SPRING_PROFILES_ACTIVE=docker
```

Variable SPRING_PROFILES_ACTIVE. Configuración pública del servicio; conservar el default salvo cambio de entorno.

L18

```
AUXIRUTA_AUTOMATION_ENABLED=true
```

Variable AUXIRUTA_AUTOMATION_ENABLED. Configuración pública del servicio; conservar el default salvo cambio de entorno.

Líneas vacías 11, 15: separadores sin instrucciones.

api-gateway/Dockerfile

`Ruta: api-gateway/Dockerfile`

`SHA-256: 0ca230dac29fd9d6b49716f7b717bce8decd5d5b25ad20c8078a8c342f33a18c`

Construcción en etapas: JDK para compilar/verificar y JRE para ejecutar app.jar como usuario sin privilegios.

L1

```
FROM eclipse-temurin:21-jdk-jammy AS build
```

Selecciona imagen base eclipse-temurin:21-jdk-jammy AS build. AS build nombra la etapa cuando aparece.

L2

```
WORKDIR /workspace
```

Establece directorio de trabajo /workspace.

L3

```
COPY auxiruta-commons/ auxiruta-commons/
```

Copia auxiruta-commons/ auxiruta-commons/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L4

```
COPY api-gateway/ api-gateway/
```

Copia api-gateway/ api-gateway/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L5

```
COPY docs/api/ docs/api/
```

Copia docs/api/ docs/api/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L6

```
RUN chmod +x auxiruta-commons/mvnw api-gateway/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
auxiruta-commons/pom.xml install && ./api-gateway/mvnw -B -ntp -f api-gateway/pom.xml verify
```

Durante construcción ejecuta chmod +x auxiruta-commons/mvnw api-gateway/mvnw && ./auxiruta-commons/mvnw -B -ntp -f auxiruta-commons/pom.xml install && ./api-gateway/mvnw -B -ntp -f api-gateway/pom.xml verify. La compilación ocurre en la imagen JDK.

L7

```
FROM eclipse-temurin:21-jre-jammy
```

Selecciona imagen base eclipse-temurin:21-jre-jammy. AS build nombra la etapa cuando aparece.

L8

```
RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Durante construcción ejecuta groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta. La compilación ocurre en la imagen JDK.

L9

```
WORKDIR /app
```

Establece directorio de trabajo /app.

L10

```
COPY --from=build /workspace/api-gateway/target/app.jar app.jar
```

Copia --from=build /workspace/api-gateway/target/app.jar app.jar. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L11

```
USER auxiruta
```

Ejecuta como auxiruta, sin privilegios de root.

L12

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Comando de arranque: Java ejecuta el JAR indicado.

api-gateway/pom.xml

Ruta: api-gateway/pom.xml

SHA-256: 50cdc6dddef3e3915e49cabd84f0b301b113b243054eac809fba5cee1a6d44ba

Configuración Maven del módulo api-gateway: versión Java, dependencias, scopes y empaquetado. No es código ejecutado por una petición.

L1

```
<?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
    <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
    <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
        <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
        <artifactId>spring-boot-starter-parent</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-parent.

L7

```
        <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
        <relativePath/>
```

Vacío: resuelve el padre desde repositorios, no desde un POM vecino.

L9

```
    </parent>
```

Cierra bloque parent.

L10

```
    <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
    <artifactId>api-gateway</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: api-gateway.

L12

```
    <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
    <properties>
```

Variables de la construcción.

L14

```
        <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
    </properties>
```

Cierra bloque properties.

L17

```
    <dependencyManagement>
```

Administra versiones mediante BOM sin añadir por sí mismo bibliotecas al classpath.

L18

```
        <dependencies>
```

Lista de dependencias.

L19

```
            <dependency>
```

Declara una dependencia Maven.

L20

```
                <groupId>org.springframework.cloud</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.cloud.

L21

```
                <artifactId>spring-cloud-dependencies</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-cloud-dependencies.

L22

```
                <version>2025.0.3</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2025.0.3.

L23

```
                <type>pom</type>
```

Configura type en este elemento Maven. Valor: pom.

L24

```
                <scope>import</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: import.

L25

```
            </dependency>
```

Cierra bloque dependency.

L26

```
        </dependencies>
```

Cierra bloque dependencies.

L27

```
    </dependencyManagement>
```

Cierra bloque dependencyManagement.

L28

```
    <dependencies>
```

Lista de dependencias.

L29

```
        <dependency>
```

Declara una dependencia Maven.

L30

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L31

```
            <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-oauth2-resource-server.

L32

```
        </dependency>
```

Cierra bloque dependency.

L33

```
        <dependency>
```

Declara una dependencia Maven.

L34

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L35

```
            <artifactId>spring-boot-starter-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-test.

L36

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L37

```
        </dependency>
```

Cierra bloque dependency.

L38

```
        <dependency>
```

Declara una dependencia Maven.

L39

```
            <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L40

```
            <artifactId>spring-security-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-test.

L41

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L42

```
        </dependency>
```

Cierra bloque dependency.

L43

```
        <dependency>
```

Declara una dependencia Maven.

L44

```
            <groupId>org.springframework.cloud</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.cloud.

L45

```
            <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-cloud-starter-gateway-server-webflux.

L46

```
        </dependency>
```

Cierra bloque dependency.

L47

```
        <dependency>
```

Declara una dependencia Maven.

L48

```
            <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L49

```
            <artifactId>auxiruta-commons</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: auxiruta-commons.

L50

```
            <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L51

```
        </dependency>
```

Cierra bloque dependency.

L52

```
    </dependencies>
```

Cierra bloque dependencies.

L53

```
    <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L54

```
        <finalName>app</finalName>
```

Nombre base app del JAR generado. Valor: app.

L55

```
        <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L56

```
            <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L57

```
                <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L58

```
                <artifactId>spring-boot-maven-plugin</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-maven-plugin.

L59

```
            </plugin>
```

Cierra bloque plugin.

L60

```
        </plugins>
```

Cierra bloque plugins.

L61

```
    </build>
```

Cierra bloque build.

L62

```
</project>
```

Cierra bloque project.

assistance-service/Dockerfile

`Ruta: assistance-service/Dockerfile`

`SHA-256: d5f286c7a946ff30bfd6ebe80efc5acc6cb33999d492d35a8ff61bb8bde3a8a8`

Construcción en etapas: JDK para compilar/verificar y JRE para ejecutar app.jar como usuario sin privilegios.

L1

```
FROM eclipse-temurin:21-jdk-jammy AS build
```

Selecciona imagen base eclipse-temurin:21-jdk-jammy AS build. AS build nombra la etapa cuando aparece.

L2

```
WORKDIR /workspace
```

Establece directorio de trabajo /workspace.

L3

```
COPY auxiruta-commons/ auxiruta-commons/
```

Copia auxiruta-commons/ auxiruta-commons/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L4

```
COPY assistance-service/ assistance-service/
```

Copia assistance-service/ assistance-service/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L5

```
RUN chmod +x auxiruta-commons/mvnw assistance-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
auxiruta-commons/pom.xml install && ./assistance-service/mvnw -B -ntp -f assistance-service/pom.xml
verify
```

Durante construcción ejecuta chmod +x auxiruta-commons/mvnw assistance-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f auxiruta-commons/pom.xml install && ./assistance-service/mvnw -B -ntp -f assistance-service/pom.xml verify. La compilación ocurre en la imagen JDK.

L6

```
FROM eclipse-temurin:21-jre-jammy
```

Selecciona imagen base eclipse-temurin:21-jre-jammy. AS build nombra la etapa cuando aparece.

L7

```
RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Durante construcción ejecuta groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta. La compilación ocurre en la imagen JDK.

L8

```
WORKDIR /app
```

Establece directorio de trabajo /app.

L9

```
COPY --from=build /workspace/assistance-service/target/app.jar app.jar
```

Copia --from=build /workspace/assistance-service/target/app.jar app.jar. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L10

```
USER auxiruta
```

Ejecuta como auxiruta, sin privilegios de root.

L11

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Comando de arranque: Java ejecuta el JAR indicado.

assistance-service/pom.xml

`Ruta: assistance-service/pom.xml`

`SHA-256: ac89b5450f62709ea767fb629e9356f9c5531070a8537f97db160a3b338d9acd`

Configuración Maven del módulo assistance-service: versión Java, dependencias, scopes y empaquetado. No es código ejecutado por una petición.

L1

```
<?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
    <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
    <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
        <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
        <artifactId>spring-boot-starter-parent</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-parent.

L7

```
        <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
        <relativePath/>
```

Vacío: resuelve el padre desde repositorios, no desde un POM vecino.

L9

```
    </parent>
```

Cierra bloque parent.

L10

```
    <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
    <artifactId>assistance-service</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: assistance-service.

L12

```
    <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
    <properties>
```

Variables de la construcción.

L14

```
        <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
    </properties>
```

Cierra bloque properties.

L17

```
    <dependencies>
```

Lista de dependencias.

L18

```
        <dependency>
```

Declara una dependencia Maven.

L19

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L20

```
            <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-oauth2-resource-server.

L21

```
        </dependency>
```

Cierra bloque dependency.

L22

```
        <dependency>
```

Declara una dependencia Maven.

L23

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L24

```
            <artifactId>spring-boot-starter-web</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-web.

L25

```
        </dependency>
```

Cierra bloque dependency.

L26

```
        <dependency>
```

Declara una dependencia Maven.

L27

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L28

```
            <artifactId>spring-boot-starter-data-jpa</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-data-jpa.

L29

```
        </dependency>
```

Cierra bloque dependency.

L30

```
        <dependency>
```

Declara una dependencia Maven.

L31

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
            <artifactId>spring-boot-starter-validation</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-validation.

L33

```
        </dependency>
```

Cierra bloque dependency.

L34

```
        <dependency>
```

Declara una dependencia Maven.

L35

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L36

```
            <artifactId>spring-boot-starter-oauth2-client</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-oauth2-client.

L37

```
        </dependency>
```

Cierra bloque dependency.

L38

```
        <dependency>
```

Declara una dependencia Maven.

L39

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L40

```
            <artifactId>spring-boot-starter-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-test.

L41

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L42

```
        </dependency>
```

Cierra bloque dependency.

L43

```
        <dependency>
```

Declara una dependencia Maven.

L44

```
            <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L45

```
            <artifactId>spring-security-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-test.

L46

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L47

```
        </dependency>
```

Cierra bloque dependency.

L48

```
        <dependency>
```

Declara una dependencia Maven.

L49

```
            <groupId>org.postgresql</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.postgresql.

L50

```
            <artifactId>postgresql</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: postgresql.

L51

```
            <scope>runtime</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: runtime.

L52

```
        </dependency>
```

Cierra bloque dependency.

L53

```
        <dependency>
```

Declara una dependencia Maven.

L54

```
            <groupId>org.springdoc</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springdoc.

L55

```
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: springdoc-openapi-starter-webmvc-ui.

L56

```
            <version>2.10.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2.10.0.

L57

```
        </dependency>
```

Cierra bloque dependency.

L58

```
        <dependency>
```

Declara una dependencia Maven.

L59

```
            <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L60

```
            <artifactId>auxiruta-commons</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: auxiruta-commons.

L61

```
            <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L62

```
        </dependency>
```

Cierra bloque dependency.

L63

```
    </dependencies>
```

Cierra bloque dependencies.

L64

```
    <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L65

```
        <finalName>app</finalName>
```

Nombre base app del JAR generado. Valor: app.

L66

```
        <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L67

```
            <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L68

```
                <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L69

```
                <artifactId>spring-boot-maven-plugin</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-maven-plugin.

L70

```
            </plugin>
```

Cierra bloque plugin.

L71

```
        </plugins>
```

Cierra bloque plugins.

L72

```
    </build>
```

Cierra bloque build.

L73

```
</project>
```

Cierra bloque project.

auxiruta-commons/pom.xml

`Ruta: auxiruta-commons/pom.xml`

`SHA-256: 5cb1b1351a70fe19bc14f35ae959dba1c9c80ba7a54c84215b9bdee372c14478`

Configuración Maven del módulo auxiruta-commons: versión Java, dependencias, scopes y empaquetado. No es código ejecutado por una petición.

L1

```
<?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
    <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
    <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
        <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
        <artifactId>spring-boot-starter-parent</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-parent.

L7

```
        <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
        <relativePath/>
```

Vacío: resuelve el padre desde repositorios, no desde un POM vecino.

L9

```
    </parent>
```

Cierra bloque parent.

L10

```
    <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
    <artifactId>auxiruta-commons</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: auxiruta-commons.

L12

```
    <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
    <properties>
```

Variables de la construcción.

L14

```
        <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
    </properties>
```

Cierra bloque properties.

L17

```
    <dependencies>
```

Lista de dependencias.

L18

```
        <dependency>
```

Declara una dependencia Maven.

L19

```
            <groupId>org.springframework</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.

L20

```
            <artifactId>spring-web</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-web.

L21

```
        </dependency>
```

Cierra bloque dependency.

L22

```
        <dependency>
```

Declara una dependencia Maven.

L23

```
            <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L24

```
            <artifactId>spring-security-oauth2-jose</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-oauth2-jose.

L25

```
        </dependency>
```

Cierra bloque dependency.

L26

```
        <dependency>
```

Declara una dependencia Maven.

L27

```
            <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L28

```
            <artifactId>spring-security-core</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-core.

L29

```
        </dependency>
```

Cierra bloque dependency.

L30

```
        <dependency>
```

Declara una dependencia Maven.

L31

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
            <artifactId>spring-boot-starter-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-test.

L33

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L34

```
        </dependency>
```

Cierra bloque dependency.

L35

```
    </dependencies>
```

Cierra bloque dependencies.

L36

```
</project>
```

Cierra bloque project.

docker-compose.yml

`Ruta: docker-compose.yml`

`SHA-256: bcb293e45ae0e73438d85be5ed494906fd0900a1505ab866ce9dcd2e5be50a69`

Declara cinco servicios, red, volumen PostgreSQL, secretos por entorno, dependencias y puertos públicos. Tres bases lógicas comparten un servidor PostgreSQL.

L1

```
name: auxiruta
```

Establece name = auxiruta dentro del bloque. Se interpreta al cargar la configuración.

L2

```
x-logging: &logging
```

Establece x-logging = &logging dentro del bloque. Se interpreta al cargar la configuración.

L3

```
  driver: json-file
```

Establece driver = json-file dentro del bloque. Se interpreta al cargar la configuración.

L4

```
  options:
```

Abre bloque options; sus hijos configuran este componente.

L5

```
    max-size: "10m"
```

Establece max-size = "10m" dentro del bloque. Se interpreta al cargar la configuración.

L6

```
    max-file: "3"
```

Establece max-file = "3" dentro del bloque. Se interpreta al cargar la configuración.

L7

```
x-jwt: &jwt
```

Establece x-jwt = &jwt dentro del bloque. Se interpreta al cargar la configuración.

L8

```
  SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-docker}
```

Configura SPRING_PROFILES_ACTIVE desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L9

```
  AUXIRUTA_AUTOMATION_ENABLED: ${AUXIRUTA_AUTOMATION_ENABLED:-true}
```

Configura AUXIRUTA_AUTOMATION_ENABLED desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L10

```
  KEYCLOAK_ISSUER_URI: http://localhost:8080/realms/auxiruta
```

Establece KEYCLOAK_ISSUER_URI = http://localhost:8080/realms/auxiruta dentro del bloque. Se interpreta al cargar la configuración.

L11

```
  KEYCLOAK_JWK_SET_URI: http://keycloak:8080/realms/auxiruta/protocol/openid-connect/certs
```

Establece KEYCLOAK_JWK_SET_URI = http://keycloak:8080/realms/auxiruta/protocol/openid-connect/certs dentro del bloque. Se interpreta al cargar la configuración.

L12

```
services:
```

Abre bloque services; sus hijos configuran este componente.

L13

```
  postgres:
```

Abre bloque postgres; sus hijos configuran este componente.

L14

```
    image: postgres:17.11
```

Establece image = postgres:17.11 dentro del bloque. Se interpreta al cargar la configuración.

L15

```
    environment:
```

Abre bloque environment; sus hijos configuran este componente.

L16

```
      POSTGRES_USER: postgres
```

Establece POSTGRES_USER = postgres dentro del bloque. Se interpreta al cargar la configuración.

L17

```
      POSTGRES_DB: postgres
```

Establece POSTGRES_DB = postgres dentro del bloque. Se interpreta al cargar la configuración.

L18

```
      POSTGRES_PASSWORD: ${POSTGRES_ADMIN_PASSWORD:?Configurar .env}
```

Configura POSTGRES_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L19

```
      ASSISTANCE_DB_PASSWORD: ${ASSISTANCE_DB_PASSWORD:?Configurar .env}
```

Configura ASSISTANCE_DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L20

```
      FLEET_DB_PASSWORD: ${FLEET_DB_PASSWORD:?Configurar .env}
```

Configura FLEET_DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L21

```
      KEYCLOAK_DB_PASSWORD: ${KEYCLOAK_DB_PASSWORD:?Configurar .env}
```

Configura KEYCLOAK_DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L22

```
    volumes:
```

Abre bloque volumes; sus hijos configuran este componente.

L23

```
      - postgres-data:/var/lib/postgresql/data
```

Añade a la lista superior: postgres-data:/var/lib/postgresql/data.

L24

```
      - ./infra/postgres/01-databases.sh:/docker-entrypoint-initdb.d/01-databases.sh:ro
```

Añade a la lista superior: ./infra/postgres/01-databases.sh:/docker-entrypoint-initdb.d/01-databases.sh:ro.

L25

```
    healthcheck:
```

Abre bloque healthcheck; sus hijos configuran este componente.

L26

```
      test: [CMD-SHELL, "pg_isready -U postgres -d postgres"]
```

Establece test = [CMD-SHELL, "pg_isready -U postgres -d postgres"] dentro del bloque. Se interpreta al cargar la configuración.

L27

```
      interval: 5s
```

Establece interval = 5s dentro del bloque. Se interpreta al cargar la configuración.

L28

```
      timeout: 3s
```

Establece timeout = 3s dentro del bloque. Se interpreta al cargar la configuración.

L29

```
      retries: 30
```

Establece retries = 30 dentro del bloque. Se interpreta al cargar la configuración.

L30

```
    logging: *logging
```

Establece logging = *logging dentro del bloque. Se interpreta al cargar la configuración.

L31

```
  keycloak:
```

Abre bloque keycloak; sus hijos configuran este componente.

L32

```
    image: quay.io/keycloak/keycloak:26.7.5
```

Establece image = quay.io/keycloak/keycloak:26.7.5 dentro del bloque. Se interpreta al cargar la configuración.

L33

```
    command: [start-dev, --import-realm]
```

Establece command = [start-dev, --import-realm] dentro del bloque. Se interpreta al cargar la configuración.

L34

```
    environment:
```

Abre bloque environment; sus hijos configuran este componente.

L35

```
      KC_DB: postgres
```

Establece KC_DB = postgres dentro del bloque. Se interpreta al cargar la configuración.

L36

```
      KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak_db
```

Establece KC_DB_URL = jdbc:postgresql://postgres:5432/keycloak_db dentro del bloque. Se interpreta al cargar la configuración.

L37

```
      KC_DB_USERNAME: keycloak_user
```

Establece KC_DB_USERNAME = keycloak_user dentro del bloque. Se interpreta al cargar la configuración.

L38

```
      KC_DB_PASSWORD: ${KEYCLOAK_DB_PASSWORD:?Configurar .env}
```

Configura KC_DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L39

```
      KC_BOOTSTRAP_ADMIN_USERNAME: bootstrap-admin
```

Establece KC_BOOTSTRAP_ADMIN_USERNAME = bootstrap-admin dentro del bloque. Se interpreta al cargar la configuración.

L40

```
      KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_BOOTSTRAP_ADMIN_PASSWORD:?Configurar .env}
```

Configura KC_BOOTSTRAP_ADMIN_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L41

```
      KC_HOSTNAME: http://localhost:8080
```

Establece KC_HOSTNAME = http://localhost:8080 dentro del bloque. Se interpreta al cargar la configuración.

L42

```
      KC_HOSTNAME_BACKCHANNEL_DYNAMIC: "true"
```

Establece KC_HOSTNAME_BACKCHANNEL_DYNAMIC = "true" dentro del bloque. Se interpreta al cargar la configuración.

L43

```
      KC_HEALTH_ENABLED: "true"
```

Establece KC_HEALTH_ENABLED = "true" dentro del bloque. Se interpreta al cargar la configuración.

L44

```
      KEYCLOAK_CLIENT_SECRET: ${KEYCLOAK_CLIENT_SECRET:?Configurar .env}
```

Configura KEYCLOAK_CLIENT_SECRET desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L45

```
      DEMO_DRIVER_PASSWORD: ${DEMO_DRIVER_PASSWORD:?Configurar .env}
```

Configura DEMO_DRIVER_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L46

```
      DEMO_OPERATOR_PASSWORD: ${DEMO_OPERATOR_PASSWORD:?Configurar .env}
```

Configura DEMO_OPERATOR_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L47

```
      DEMO_ADMIN_PASSWORD: ${DEMO_ADMIN_PASSWORD:?Configurar .env}
```

Configura DEMO_ADMIN_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L48

```
    ports: ["127.0.0.1:8080:8080"]
```

Establece ports = ["127.0.0.1:8080:8080"] dentro del bloque. Se interpreta al cargar la configuración.

L49

```
    volumes:
```

Abre bloque volumes; sus hijos configuran este componente.

L50

```
      - ./infra/keycloak/auxiruta-realm.json:/opt/keycloak/data/import/auxiruta-realm.json:ro
```

Añade a la lista superior: ./infra/keycloak/auxiruta-realm.json:/opt/keycloak/data/import/auxiruta-realm.json:ro.

L51

```
    depends_on:
```

Abre bloque depends_on; sus hijos configuran este componente.

L52

```
      postgres:
```

Abre bloque postgres; sus hijos configuran este componente.

L53

```
        condition: service_healthy
```

Establece condition = service_healthy dentro del bloque. Se interpreta al cargar la configuración.

L54

```
    healthcheck:
```

Abre bloque healthcheck; sus hijos configuran este componente.

L55

```
      test: [CMD-SHELL, "bash -c 'exec 3<>/dev/tcp/127.0.0.1/9000; printf \"GET /health/ready
HTTP/1.0\\r\\n\\r\\n\" >&3; grep -q \"200 OK\" <&3'"]
```

Establece test = [CMD-SHELL, "bash -c 'exec 3<>/dev/tcp/127.0.0.1/9000; printf \"GET /health/ready HTTP/1.0\\r\\n\\r\\n\" >&3; grep -q \"200 OK\" <&3'"] dentro del bloque. Se interpreta al cargar la configuración.

L56

```
      interval: 10s
```

Establece interval = 10s dentro del bloque. Se interpreta al cargar la configuración.

L57

```
      timeout: 5s
```

Establece timeout = 5s dentro del bloque. Se interpreta al cargar la configuración.

L58

```
      start_period: 60s
```

Establece start_period = 60s dentro del bloque. Se interpreta al cargar la configuración.

L59

```
      retries: 30
```

Establece retries = 30 dentro del bloque. Se interpreta al cargar la configuración.

L60

```
    logging: *logging
```

Establece logging = *logging dentro del bloque. Se interpreta al cargar la configuración.

L61

```
  fleet-service:
```

Abre bloque fleet-service; sus hijos configuran este componente.

L62

```
    build:
```

Abre bloque build; sus hijos configuran este componente.

L63

```
      context: .
```

Establece context = . dentro del bloque. Se interpreta al cargar la configuración.

L64

```
      dockerfile: fleet-service/Dockerfile
```

Establece dockerfile = fleet-service/Dockerfile dentro del bloque. Se interpreta al cargar la configuración.

L65

```
    environment:
```

Abre bloque environment; sus hijos configuran este componente.

L66

```
      <<: *jwt
```

Continúa la configuración del bloque anterior con la referencia indicada.

L67

```
      DB_URL: jdbc:postgresql://postgres:5432/fleet_db
```

Establece DB_URL = jdbc:postgresql://postgres:5432/fleet_db dentro del bloque. Se interpreta al cargar la configuración.

L68

```
      DB_USER: fleet_user
```

Establece DB_USER = fleet_user dentro del bloque. Se interpreta al cargar la configuración.

L69

```
      DB_PASSWORD: ${FLEET_DB_PASSWORD:?Configurar .env}
```

Configura DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L70

```
    depends_on:
```

Abre bloque depends_on; sus hijos configuran este componente.

L71

```
      keycloak:
```

Abre bloque keycloak; sus hijos configuran este componente.

L72

```
        condition: service_healthy
```

Establece condition = service_healthy dentro del bloque. Se interpreta al cargar la configuración.

L73

```
    logging: *logging
```

Establece logging = *logging dentro del bloque. Se interpreta al cargar la configuración.

L74

```
  assistance-service:
```

Abre bloque assistance-service; sus hijos configuran este componente.

L75

```
    build:
```

Abre bloque build; sus hijos configuran este componente.

L76

```
      context: .
```

Establece context = . dentro del bloque. Se interpreta al cargar la configuración.

L77

```
      dockerfile: assistance-service/Dockerfile
```

Establece dockerfile = assistance-service/Dockerfile dentro del bloque. Se interpreta al cargar la configuración.

L78

```
    environment:
```

Abre bloque environment; sus hijos configuran este componente.

L79

```
      <<: *jwt
```

Continúa la configuración del bloque anterior con la referencia indicada.

L80

```
      DB_URL: jdbc:postgresql://postgres:5432/assistance_db
```

Establece DB_URL = jdbc:postgresql://postgres:5432/assistance_db dentro del bloque. Se interpreta al cargar la configuración.

L81

```
      DB_USER: assistance_user
```

Establece DB_USER = assistance_user dentro del bloque. Se interpreta al cargar la configuración.

L82

```
      DB_PASSWORD: ${ASSISTANCE_DB_PASSWORD:?Configurar .env}
```

Configura DB_PASSWORD desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L83

```
      KEYCLOAK_TOKEN_URI: http://keycloak:8080/realms/auxiruta/protocol/openid-connect/token
```

Establece KEYCLOAK_TOKEN_URI = http://keycloak:8080/realms/auxiruta/protocol/openid-connect/token dentro del bloque. Se interpreta al cargar la configuración.

L84

```
      KEYCLOAK_CLIENT_ID: assistance-service
```

Establece KEYCLOAK_CLIENT_ID = assistance-service dentro del bloque. Se interpreta al cargar la configuración.

L85

```
      KEYCLOAK_CLIENT_SECRET: ${KEYCLOAK_CLIENT_SECRET:?Configurar .env}
```

Configura KEYCLOAK_CLIENT_SECRET desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L86

```
      FLEET_URL: http://fleet-service:8081
```

Establece FLEET_URL = http://fleet-service:8081 dentro del bloque. Se interpreta al cargar la configuración.

L87

```
      ORS_API_KEY: ${ORS_API_KEY:-}
```

Configura ORS_API_KEY desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L88

```
      ORS_BASE_URL: ${ORS_BASE_URL:-https://api.heigit.org/openrouteservice/v2}
```

Configura ORS_BASE_URL desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L89

```
    depends_on:
```

Abre bloque depends_on; sus hijos configuran este componente.

L90

```
      keycloak:
```

Abre bloque keycloak; sus hijos configuran este componente.

L91

```
        condition: service_healthy
```

Establece condition = service_healthy dentro del bloque. Se interpreta al cargar la configuración.

L92

```
    logging: *logging
```

Establece logging = *logging dentro del bloque. Se interpreta al cargar la configuración.

L93

```
  api-gateway:
```

Abre bloque api-gateway; sus hijos configuran este componente.

L94

```
    build:
```

Abre bloque build; sus hijos configuran este componente.

L95

```
      context: .
```

Establece context = . dentro del bloque. Se interpreta al cargar la configuración.

L96

```
      dockerfile: api-gateway/Dockerfile
```

Establece dockerfile = api-gateway/Dockerfile dentro del bloque. Se interpreta al cargar la configuración.

L97

```
    environment:
```

Abre bloque environment; sus hijos configuran este componente.

L98

```
      <<: *jwt
```

Continúa la configuración del bloque anterior con la referencia indicada.

L99

```
    ports: ["127.0.0.1:8000:8000"]
```

Establece ports = ["127.0.0.1:8000:8000"] dentro del bloque. Se interpreta al cargar la configuración.

L100

```
    depends_on:
```

Abre bloque depends_on; sus hijos configuran este componente.

L101

```
      keycloak:
```

Abre bloque keycloak; sus hijos configuran este componente.

L102

```
        condition: service_healthy
```

Establece condition = service_healthy dentro del bloque. Se interpreta al cargar la configuración.

L103

```
    logging: *logging
```

Establece logging = *logging dentro del bloque. Se interpreta al cargar la configuración.

L104

```
  volumes:
```

Abre bloque volumes; sus hijos configuran este componente.

L105

```
   postgres-data:
```

Abre bloque postgres-data; sus hijos configuran este componente.

fleet-service/Dockerfile

`Ruta: fleet-service/Dockerfile`

`SHA-256: cd222b12a2b4cbd7e3dd9b6d441564bf95fd2a128d96c30a052f95c949ee3fee`

Construcción en etapas: JDK para compilar/verificar y JRE para ejecutar app.jar como usuario sin privilegios.

L1

```
  FROM eclipse-temurin:21-jdk-jammy AS build
```

Selecciona imagen base eclipse-temurin:21-jdk-jammy AS build. AS build nombra la etapa cuando aparece.

L2

```
WORKDIR /workspace
```

Establece directorio de trabajo /workspace.

L3

```
COPY auxiruta-commons/ auxiruta-commons/
```

Copia auxiruta-commons/ auxiruta-commons/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L4

```
COPY fleet-service/ fleet-service/
```

Copia fleet-service/ fleet-service/. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L5

```
RUN chmod +x auxiruta-commons/mvnw fleet-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f
auxiruta-commons/pom.xml install && ./fleet-service/mvnw -B -ntp -f fleet-service/pom.xml verify
```

Durante construcción ejecuta chmod +x auxiruta-commons/mvnw fleet-service/mvnw && ./auxiruta-commons/mvnw -B -ntp -f auxiruta-commons/pom.xml install && ./fleet-service/mvnw -B -ntp -f fleet-service/pom.xml verify. La compilación ocurre en la imagen JDK.

L6

```
FROM eclipse-temurin:21-jre-jammy
```

Selecciona imagen base eclipse-temurin:21-jre-jammy. AS build nombra la etapa cuando aparece.

L7

```
RUN groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta
```

Durante construcción ejecuta groupadd --system auxiruta && useradd --system --gid auxiruta --home-dir /app auxiruta. La compilación ocurre en la imagen JDK.

L8

```
WORKDIR /app
```

Establece directorio de trabajo /app.

L9

```
COPY --from=build /workspace/fleet-service/target/app.jar app.jar
```

Copia --from=build /workspace/fleet-service/target/app.jar app.jar. --from=build obtiene artefactos de la etapa anterior; no copia .env.

L10

```
USER auxiruta
```

Ejecuta como auxiruta, sin privilegios de root.

L11

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Comando de arranque: Java ejecuta el JAR indicado.

fleet-service/pom.xml

`Ruta: fleet-service/pom.xml`

`SHA-256: 989dbc53026cf4dec79b5b6a418cf314ff666659f263fa0fda7faf2e618d76ba`

Configuración Maven del módulo fleet-service: versión Java, dependencias, scopes y empaquetado. No es código ejecutado por una petición.

L1

```
<?xml version="1.0" ?>
```

Declara sintaxis XML.

L2

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Abre bloque del modelo Maven y agrupa sus hijos.

L3

```
    <modelVersion>4.0.0</modelVersion>
```

Versión del modelo POM, no de Maven. Valor: 4.0.0.

L4

```
    <parent>
```

Herencia del padre que administra plugins y versiones.

L5

```
        <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L6

```
        <artifactId>spring-boot-starter-parent</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-parent.

L7

```
        <version>3.5.16</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 3.5.16.

L8

```
        <relativePath/>
```

Vacío: resuelve el padre desde repositorios, no desde un POM vecino.

L9

```
    </parent>
```

Cierra bloque parent.

L10

```
    <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L11

```
    <artifactId>fleet-service</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: fleet-service.

L12

```
    <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L13

```
    <properties>
```

Variables de la construcción.

L14

```
        <java.version>21</java.version>
```

Versión de Java para compilar. Valor: 21.

L15

```
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

Codificación del fuente. Valor: UTF-8.

L16

```
    </properties>
```

Cierra bloque properties.

L17

```
    <dependencies>
```

Lista de dependencias.

L18

```
        <dependency>
```

Declara una dependencia Maven.

L19

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L20

```
            <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-oauth2-resource-server.

L21

```
        </dependency>
```

Cierra bloque dependency.

L22

```
        <dependency>
```

Declara una dependencia Maven.

L23

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L24

```
            <artifactId>spring-boot-starter-web</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-web.

L25

```
        </dependency>
```

Cierra bloque dependency.

L26

```
        <dependency>
```

Declara una dependencia Maven.

L27

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L28

```
            <artifactId>spring-boot-starter-data-jpa</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-data-jpa.

L29

```
        </dependency>
```

Cierra bloque dependency.

L30

```
        <dependency>
```

Declara una dependencia Maven.

L31

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L32

```
            <artifactId>spring-boot-starter-validation</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-validation.

L33

```
        </dependency>
```

Cierra bloque dependency.

L34

```
        <dependency>
```

Declara una dependencia Maven.

L35

```
            <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L36

```
            <artifactId>spring-boot-starter-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-starter-test.

L37

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L38

```
        </dependency>
```

Cierra bloque dependency.

L39

```
        <dependency>
```

Declara una dependencia Maven.

L40

```
            <groupId>org.springframework.security</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.security.

L41

```
            <artifactId>spring-security-test</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-security-test.

L42

```
            <scope>test</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: test.

L43

```
        </dependency>
```

Cierra bloque dependency.

L44

```
        <dependency>
```

Declara una dependencia Maven.

L45

```
            <groupId>org.postgresql</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.postgresql.

L46

```
            <artifactId>postgresql</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: postgresql.

L47

```
            <scope>runtime</scope>
```

test solo en pruebas; runtime en ejecución; import incorpora un BOM. Valor: runtime.

L48

```
        </dependency>
```

Cierra bloque dependency.

L49

```
        <dependency>
```

Declara una dependencia Maven.

L50

```
            <groupId>org.springdoc</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springdoc.

L51

```
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: springdoc-openapi-starter-webmvc-ui.

L52

```
            <version>2.10.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 2.10.0.

L53

```
        </dependency>
```

Cierra bloque dependency.

L54

```
        <dependency>
```

Declara una dependencia Maven.

L55

```
            <groupId>com.auxiruta</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: com.auxiruta.

L56

```
            <artifactId>auxiruta-commons</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: auxiruta-commons.

L57

```
            <version>1.0.0</version>
```

Versión del elemento Maven o configuración del bloque. Valor: 1.0.0.

L58

```
        </dependency>
```

Cierra bloque dependency.

L59

```
    </dependencies>
```

Cierra bloque dependencies.

L60

```
    <build>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L61

```
        <finalName>app</finalName>
```

Nombre base app del JAR generado. Valor: app.

L62

```
        <plugins>
```

Abre bloque del modelo Maven y agrupa sus hijos.

L63

```
            <plugin>
```

Plugin de construcción; el de Spring Boot reempaqueta el ejecutable.

L64

```
                <groupId>org.springframework.boot</groupId>
```

Grupo Maven del artefacto o proveedor. Valor: org.springframework.boot.

L65

```
                <artifactId>spring-boot-maven-plugin</artifactId>
```

Nombre de proyecto, dependencia o plugin del bloque actual. Valor: spring-boot-maven-plugin.

L66

```
            </plugin>
```

Cierra bloque plugin.

L67

```
        </plugins>
```

Cierra bloque plugins.

L68

```
    </build>
```

Cierra bloque build.

L69

```
</project>
```

Cierra bloque project.

infra/postgres/01-databases.sh

`Ruta: infra/postgres/01-databases.sh`

`SHA-256: 85d1dacac8858cdea77d6c35cbf181f0799562f558e7b207c7d758e2901441e6`

Inicialización de PostgreSQL: crea usuarios y bases separados para Fleet, Assistance y Keycloak con contraseñas suministradas por el entorno.

L1

```
#!/bin/sh
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
set -eu
```

Shell aborta por comando fallido o variable indefinida.

L3

```
psql --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" --set=ON_ERROR_STOP=1
--set=assistance_password="$ASSISTANCE_DB_PASSWORD" --set=fleet_password="$FLEET_DB_PASSWORD"
--set=keycloak_password="$KEYCLOAK_DB_PASSWORD" <<'SQL'
```

Invoca psql con errores fatales y contraseñas parametrizadas en heredoc.

L4

```
CREATE ROLE assistance_user LOGIN PASSWORD :'assistance_password';
```

Crea usuario PostgreSQL assistance_user con clave parametrizada.

L5

```
CREATE ROLE fleet_user LOGIN PASSWORD :'fleet_password';
```

Crea usuario PostgreSQL fleet_user con clave parametrizada.

L6

```
CREATE ROLE keycloak_user LOGIN PASSWORD :'keycloak_password';
```

Crea usuario PostgreSQL keycloak_user con clave parametrizada.

L7

```
CREATE DATABASE assistance_db OWNER assistance_user;
```

Crea base assistance_db propiedad de assistance_user.

L8

```
CREATE DATABASE fleet_db OWNER fleet_user;
```

Crea base fleet_db propiedad de fleet_user.

L9

```
CREATE DATABASE keycloak_db OWNER keycloak_user;
```

Crea base keycloak_db propiedad de keycloak_user.

L10

```
REVOKE ALL ON DATABASE assistance_db FROM PUBLIC;
```

Retira permisos de PUBLIC sobre assistance_db.

L11

```
REVOKE ALL ON DATABASE fleet_db FROM PUBLIC;
```

Retira permisos de PUBLIC sobre fleet_db.

L12

```
REVOKE ALL ON DATABASE keycloak_db FROM PUBLIC;
```

Retira permisos de PUBLIC sobre keycloak_db.

L13

```
GRANT CONNECT ON DATABASE assistance_db TO assistance_user;
```

Permite conectar a assistance_db como assistance_user.

L14

```
GRANT CONNECT ON DATABASE fleet_db TO fleet_user;
```

Permite conectar a fleet_db como fleet_user.

L15

```
GRANT CONNECT ON DATABASE keycloak_db TO keycloak_user;
```

Permite conectar a keycloak_db como keycloak_user.

L16

```
SQL
```

Cierra heredoc enviado a psql.

scripts/Start-AuxiRuta.ps1

Ruta: scripts/Start-AuxiRuta.ps1

SHA-256: 3aaecd59a93c9bf24b6e39f6f5c15841ef76d3249fcfb4ef1d151b5ebc95f8d5

Arranca el backend con perfil normal o demo, construye si corresponde y espera respuestas 401 como prueba de disponibilidad protegida.

L1

```
param(
```

Define parámetros del script.

L2

```
    [ValidateSet('docker', 'docker,demo')][string]$Profile = 'docker',
```

Limita perfiles a docker y docker,demo.

L3

```
    [switch]$NoBuild
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L4

```
)
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L5

```
$ErrorActionPreference = 'Stop'
```

Stop detiene ejecución ante errores PowerShell.

L6

```
$repo = Split-Path $PSScriptRoot -Parent
```

Calcula raíz desde ubicación del script.

L7

```
$previousProfile = $env:SPRING_PROFILES_ACTIVE
```

Conserva/restaura perfil del proceso; no cambia por sí solo contenedores ya arrancados. Selecciona perfil transmitido a aplicaciones.

L8

```
Push-Location $repo
```

Entra temporalmente en la raíz para resolver rutas/Compose.

L9

```
try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L10

```
    if (!(Test-Path -LiteralPath '.env')) { throw 'Copiar .env.example a .env y completar los secretos
locales antes de iniciar.' }
```

Comprueba existencia de .env sin imprimir valores.

L11

```
    & docker info --format '{{.ServerVersion}}'
```

Verifica motor Docker.

L12

```
    if ($LASTEXITCODE -ne 0) { throw 'Iniciar Docker Desktop y esperar a que el motor este disponible.'
}
```

Comprueba resultado de comando externo.

L13

```
    $env:SPRING_PROFILES_ACTIVE = $Profile
```

Selecciona perfil transmitido a aplicaciones.

L14

```
    & docker compose config --quiet
```

Valida Compose sin mostrar secretos.

L15

```
    if ($LASTEXITCODE -ne 0) { throw 'Configuracion Compose incompleta.' }
```

Comprueba resultado de comando externo.

L16

```
    if ($NoBuild) { & docker compose up -d } else { & docker compose up -d --build }
```

Construye imágenes e inicia/recrea contenedores. Arranca/recrea servicios sin borrar volúmenes.

L17

```
    if ($LASTEXITCODE -ne 0) { throw 'Fallo la construccion o el arranque.' }
```

Comprueba resultado de comando externo.

L18

```
    $deadline = (Get-Date).AddSeconds(120)
```

Plazo de 120 s para disponibilidad.

L19

```
    do {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L20

```
        $ready = $true
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L21

```
        foreach ($probe in @('http://fleet-service:8081/api/v1/operators/me',
'http://assistance-service:8082/api/v1/vehicles', 'http://localhost:8000/api/v1/vehicles')) {
```

Sondea Fleet, Assistance y Gateway.

L22

```
            $result = & docker compose exec -T api-gateway curl -s -o /dev/null --max-time 3 -w
'%{http_code}' $probe 2>$null
```

Ejecuta curl dentro de Gateway, con DNS interno Docker.

L23

```
            if ($LASTEXITCODE -ne 0 -or $result -ne '401') { $ready = $false }
```

Comprueba resultado de comando externo. 401 demuestra servidor disponible y protegido, no login completo.

L24

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L25

```
        if (!$ready) { Start-Sleep -Seconds 2 }
```

Espera dos segundos entre sondeos.

L26

```
    } while (!$ready -and (Get-Date) -lt $deadline)
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L27

```
    if (!$ready) { throw 'Servicios no disponibles en 120 s. Revisar docker compose logs --tail 80.' }
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L28

```
    Write-Host "AuxiRuta disponible en http://localhost:8000; Keycloak en http://localhost:8080. Perfil:
  $Profile"
```

Informa resultado sin credenciales.

L29

```
} finally {
```

Restaura entorno o ubicación aun con error.

L30

```
    $env:SPRING_PROFILES_ACTIVE = $previousProfile
```

Conserva/restaura perfil del proceso; no cambia por sí solo contenedores ya arrancados. Selecciona perfil transmitido a aplicaciones.

L31

```
    Pop-Location
```

Restaura directorio anterior incluso ante fallo.

L32

```
}
```

Delimita objeto, arreglo o bloque del archivo.

scripts/Verify-AuxiRuta.ps1

`Ruta: scripts/Verify-AuxiRuta.ps1`

`SHA-256: b09634d7e2f8333678798c677e09b35fb7593bf78f27c3c60cdc87637fa8695f`

Compila/verifica los cuatro módulos y opcionalmente ejecuta integración secuencial, con restauración del perfil normal al terminar demo.

L1

```
param([switch]$Integration, [switch]$IncludeDemo)
```

Define parámetros del script. Demo solo procede con integración habilitada.

L2

```
$ErrorActionPreference = 'Stop'
```

Stop detiene ejecución ante errores PowerShell.

L3

```
if ($IncludeDemo -and !$Integration) { throw 'IncludeDemo requiere Integration.' }
```

Demo solo procede con integración habilitada.

L4

```
$repo = Split-Path $PSScriptRoot -Parent
```

Calcula raíz desde ubicación del script.

L5

```
function Invoke-Maven([string]$Module, [string[]]$Arguments) {
```

Ejecuta Wrapper del módulo y comprueba salida.

L6

```
    & ".\$Module\mvnw.cmd" -f "$Module/pom.xml" @Arguments
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L7

```
    if ($LASTEXITCODE -ne 0) { throw "Fallo Maven en $Module. Revisar target/surefire-reports." }
```

Comprueba resultado de comando externo.

L8

```
}
```

Delimita objeto, arreglo o bloque del archivo.

L9

```
Push-Location $repo
```

Entra temporalmente en la raíz para resolver rutas/Compose.

L10

```
try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L11

```
    Invoke-Maven 'auxiruta-commons' @('install')
```

Ejecuta Wrapper del módulo y comprueba salida.

L12

```
    foreach ($module in @('fleet-service', 'assistance-service', 'api-gateway')) { Invoke-Maven $module
@('verify') }
```

Ejecuta Wrapper del módulo y comprueba salida.

L13

```
    if ($Integration) {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L14

```
        & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile docker
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L15

```
        Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.integration=true',
'-Dauxiruta.realRouting=true', '-Dtest=BackendIntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L16

```
        Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase7=true', '-Dtest=Phase7IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L17

```
        Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase8=true', '-Dtest=Phase8IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L18

```
        Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.delivery=true',
'-Dtest=DeliveryIntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L19

```
        if ($IncludeDemo) {
```

Demo solo procede con integración habilitada.

L20

```
            try {
```

Controla flujo según condición/bucle; una excepción se propaga al llamador.

L21

```
                & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile 'docker,demo' -NoBuild
```

Asigna los valores o ejecuta el comando indicado dentro del bloque actual.

L22

```
                Invoke-Maven 'api-gateway' @('test', '-Dauxiruta.phase9=true',
'-Dtest=Phase9IntegrationTest')
```

Ejecuta Wrapper del módulo y comprueba salida.

L23

```
            } finally { & "$PSScriptRoot\Start-AuxiRuta.ps1" -Profile docker -NoBuild }
```

Restaura entorno o ubicación aun con error.

L24

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L25

```
    }
```

Delimita objeto, arreglo o bloque del archivo.

L26

```
    Write-Host 'Verificacion completada. Resultados JUnit en target/surefire-reports de cada modulo.'
```

Informa resultado sin credenciales.

L27

```
} finally { Pop-Location }
```

Restaura directorio anterior incluso ante fallo. Restaura entorno o ubicación aun con error.

2. Biblioteca común

La biblioteca comparte infraestructura estable, no entidades ni DTO de negocio.

GeoMath (auxiruta-commons)

Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/GeoMath.java

SHA-256: ab682c974d8c97560a74649591e83d2bdfc7780d4def2e95721953dca5f547e1

Calcula distancia geográfica con Haversine para cobertura y preselección. No calcula rutas viales ni reemplaza ORS.

L1

```
package com.auxiruta.commons;
```

Ubica el archivo en com.auxiruta.commons. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
/** Pure geographic calculation; coordinates are latitude/longitude in degrees. */
```

Comentario de intención o documentación; no se ejecuta.

L3

```
public final class GeoMath {
```

Declara clase GeoMath.

L4

```
  private GeoMath() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L5

```
  public static double distance(double lat1,double lon1,double lat2,double lon2,double radius) {
```

Función pura de distancia: recibe dos coordenadas en grados y radio terrestre; devuelve metros si el radio está en metros.

L6

```
    double p1=Math.toRadians(lat1),p2=Math.toRadians(lat2),dp=p2-p1,dl=Math.toRadians(lon2-lon1);
```

Convierte grados a radianes.

L7

```
    double a=Math.pow(Math.sin(dp/2),2)+Math.cos(p1)*Math.cos(p2)*Math.pow(Math.sin(dl/2),2);
```

Calcula término de Haversine con latitudes y diferencia de longitudes.

L8

```
    return 2*radius*Math.asin(Math.sqrt(Math.max(0,Math.min(1,a))));
```

Obtiene distancia angular; clamp 0..1 evita errores numéricos.

L9

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L10

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Problems (auxiruta-commons)

Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/errors/Problems.java

SHA-256: c25b283288b5ffd30ca242f88e5b9b851e2e89d55a372ffef7e30a62e0c35107

Construye errores ProblemDetail homogéneos sin exponer consultas SQL, credenciales ni trazas internas.

L1

```
package com.auxiruta.commons.errors;
```

Ubica el archivo en com.auxiruta.commons.errors. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L4

```
public final class Problems {
```

Declara clase Problems.

L5

```
    private Problems() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L6

```
    public static ProblemDetail of(int status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L7

```
        HttpStatus http = HttpStatus.resolve(status);
```

Busca nombre HTTP conocido; puede ser null para código no enumerado.

L8

```
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatusCode.valueOf(status),
switch(status) {
```

Crea problema con estado y descripción elegida por switch.

L9

```
            case 400 -> "Entrada invalida";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L10

```
            case 401 -> "Autenticacion requerida o token invalido";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L11

```
            case 403 -> "Permiso insuficiente";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L12

```
            case 404 -> "Recurso no encontrado";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L13

```
            case 409 -> "Conflicto con el estado del recurso";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L14

```
            case 503 -> "Servicio temporalmente no disponible";
```

Asocia este código HTTP con el mensaje genérico escrito; evita revelar detalles internos.

L15

```
            default -> status >= 500 ? "Error interno" : "La solicitud no pudo procesarse";
```

Errores 5xx desconocidos se presentan como Error interno; otros como solicitud no procesable.

L16

```
        });
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L17

```
        p.setTitle(http == null ? "Error" : http.getReasonPhrase());
```

Usa razón HTTP estándar como título, con fallback Error.

L18

```
        p.setProperty("code", "HTTP_" + status);
```

Añade código estable HTTP_n para facilitar interpretación del cliente.

L19

```
        return p;
```

Entrega p al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L20

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L21

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2: separadores sin instrucciones.

TokenSupport (auxiruta-commons)

`Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/security/TokenSupport.java`

`SHA-256: ecddf0cb5660350a064a67a1797cca061f2ba6197920216806ce720c53545bf2`

Centraliza los roles admitidos, validaciones complementarias de JWT y propiedad. Distingue tokens humanos de tokens técnicos por audiencia y cliente autorizado; ADMIN no hereda las acciones de DRIVER u OPERATOR.

L1

```
package com.auxiruta.commons.security;
```

Ubica el archivo en com.auxiruta.commons.security. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import org.springframework.security.core.authority.SimpleGrantedAuthority;
```

Importa tipos: org.springframework.security.core.authority.SimpleGrantedAuthority. Validación JWT, Authentication, autorización y cadenas de seguridad.

L5

```
import org.springframework.security.oauth2.core.*;
```

Importa tipos: org.springframework.security.oauth2.core.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L6

```
import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L7

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L10

```
public final class TokenSupport {
```

Declara clase TokenSupport.

L11

```
    private TokenSupport() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L12

```
    public static Collection<SimpleGrantedAuthority> roles(Jwt jwt) {
```

Extrae solo roles de negocio reconocidos desde realm_access.roles.

L13

```
        Object realm = jwt.getClaims().get("realm_access");
```

Extrae realm_access como Object porque el claim puede faltar o no tener el tipo esperado.

L14

```
        if (!(realm instanceof Map<?, ?> map) || !(map.get("roles") instanceof Collection<?> roles))
return List.of();
```

La rama condicional selecciona la actualización o retorno correspondiente.

L15

```
        return roles.stream().filter(String.class::isInstance).map(String.class::cast)
```

Conserva solo elementos que cumplen el predicado. Transforma cada elemento al valor/tipo de salida.

L16

```
                .filter(Set.of("DRIVER", "OPERATOR", "ADMIN", "FLEET_SERVICE")::contains)
```

Conserva solo elementos que cumplen el predicado.

L17

```
                .distinct().map(role -> new SimpleGrantedAuthority("ROLE_" + role)).toList();
```

Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista.

L18

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L19

```
    public static OAuth2TokenValidator<Jwt> validator(String issuer, Set<String> audiences) {
```

Compone validadores de emisor/tiempo con presencia de sub, expiración y audiencia aceptada.

L20

```
        return new DelegatingOAuth2TokenValidator<>(JwtValidators.createDefaultWithIssuer(issuer), jwt
-> {
```

Combina los validadores estándar de emisor/tiempo con el predicado adicional de audiencia, subject y expiración.

L21

```
            boolean valid = jwt.getExpiresAt() != null && jwt.getSubject() != null &&
!jwt.getSubject().isBlank()
```

Identidad obtenida del JWT, no del JSON enviado por el usuario.

L22

```
                    && jwt.getAudience() != null &&
jwt.getAudience().stream().anyMatch(audiences::contains);
```

Verifica destinatario del JWT; estar firmado no basta para autorizar otro servicio.

L23

```
            return valid ? OAuth2TokenValidatorResult.success() : OAuth2TokenValidatorResult.failure(
```

Devuelve éxito del validador únicamente si todas las condiciones adicionales se cumplen.

L24

```
                    new OAuth2Error("invalid_token", "Token no valido para este servicio", null));
```

Construye error invalid_token sin revelar detalles de la identidad.

L25

```
        });
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L26

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L27

```
    public static boolean permits(Jwt jwt, boolean technical, String... requiredRoles) {
```

Autoriza por audiencia, azp y rol, separando acceso humano y técnico.

L28

```
        String audience = technical ? "auxiruta-fleet" : "auxiruta-api";
```

Selecciona auxiruta-fleet para tráfico técnico y auxiruta-api para humano.

L29

```
        String client = technical ? "assistance-service" : "auxiruta-api";
```

Exige assistance-service como cliente técnico y auxiruta-api para login humano.

L30

```
        return jwt.getAudience() != null && jwt.getAudience().contains(audience) &&
client.equals(jwt.getClaimAsString("azp"))
```

Verifica cliente que obtuvo el token. Verifica destinatario del JWT; estar firmado no basta para autorizar otro servicio.

L31

```
                && roles(jwt).stream().anyMatch(a -> Arrays.stream(requiredRoles).anyMatch(role ->
a.getAuthority().equals("ROLE_" + role)));
```

Exige al menos uno de los roles solicitados después de validar audiencia y cliente.

L32

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L33

```
    public static String subject(Jwt jwt) { return jwt.getSubject(); }
```

Identidad obtenida del JWT, no del JSON enviado por el usuario.

L34

```
    public static void requireOwner(Jwt jwt, String ownerSubject) {
```

Devuelve 404 si el sub del JWT no coincide con el propietario. Campos: ownerSubject: sub de Keycloak del propietario.

L35

```
        if (ownerSubject == null || !ownerSubject.equals(subject(jwt)))
```

La rama condicional selecciona la actualización o retorno correspondiente.

L36

```
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Recurso no encontrado");
```

Oculta recursos ajenos con el mismo 404 que los inexistentes.

L37

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L38

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 9: separadores sin instrucciones.

UtcClock (auxiruta-commons)

`Ruta: auxiruta-commons/src/main/java/com/auxiruta/commons/time/UtcClock.java`

`SHA-256: c90343ab6dc53f9aba20089b71fabe93a7f78bd467fc29d9c519c848721d151a`

Proporciona un reloj UTC compartido. Inyectar Clock permite controlar el tiempo en pruebas sin alterar la hora del sistema.

L1

```
package com.auxiruta.commons.time;
```

Ubica el archivo en com.auxiruta.commons.time. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
/** Provee el reloj UTC que las aplicaciones pueden sustituir en pruebas temporales. */
```

Comentario de intención o documentación; no se ejecuta.

L6

```
public final class UtcClock {
```

Declara clase UtcClock.

L7

```
    private UtcClock() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
    public static Clock system() {
```

Fábrica común del reloj; permite que cada servicio comparta la decisión de usar UTC.

L10

```
        return Clock.systemUTC();
```

Crea reloj de sistema con zona UTC; las pruebas pueden sustituirlo por Clock fijo.

L11

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L12

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 4, 8: separadores sin instrucciones.

3. Arranque, seguridad, Gateway y errores

Keycloak autentica. Gateway y servicios verifican JWT por separado; la propiedad se controla adicionalmente en servicios/repositorios.

GatewayApplication (api-gateway)

Ruta: api-gateway/src/main/java/com/auxiruta/gateway/GatewayApplication.java

SHA-256: 9dafd319b21dbd96e8173113b788c4fc0cd6d65c98c218af74dffb140db2b0fa

Punto de entrada del Gateway reactivo: recibe tráfico público y lo dirige a los microservicios.

L1

```
package com.auxiruta.gateway;
```

Ubica el archivo en com.auxiruta.gateway. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación.

L4

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación.

L6

```
@SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
public class GatewayApplication {
```

Declara clase GatewayApplication.

L8

```
    public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
        SpringApplication.run(GatewayApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L11

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 5: separadores sin instrucciones.

GatewayRoutes (api-gateway)

Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/GatewayRoutes.java

SHA-256: 0028904075f87954b8f749bb863662b63c5080b3711a3f0a0a31b6a6da86a769

Enruta los endpoints públicos hacia Fleet o Assistance. Las rutas /internal no se publican. Ajusta caché DNS para recuperar contenedores recreados.

L1

```
package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L4

```
import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring.

L5

```
import org.springframework.cloud.gateway.route.*;
```

Importa tipos: org.springframework.cloud.gateway.route.*. Rutas y builder del Gateway reactivo.

L6

```
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
```

Importa tipos: org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder. Rutas y builder del Gateway reactivo.

L7

```
@Configuration
```

Declara configuración Spring.

L8

```
public class GatewayRoutes {
```

Declara clase GatewayRoutes.

L9

```
    @Bean
```

Registra el objeto retornado en Spring.

L10

```
    org.springframework.cloud.gateway.config.HttpClientCustomizer containerDns() {
```

Expone personalización del cliente HTTP de Gateway para recuperación DNS.

L11

```
        return client -> client.resolver(spec -> spec.cacheMaxTimeToLive(java.time.Duration.ofSeconds(10
)).cacheNegativeTimeToLive(java.time.Duration.ZERO));
```

DNS positivo hasta 10 s y negativo 0 s para recuperarse de recreación de contenedores.

L12

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L13

```
    @Bean
```

Registra el objeto retornado en Spring.

L14

```
    RouteLocator routes(RouteLocatorBuilder builder,
```

Define bean de rutas con builder y URIs de servicios inyectadas.

L15

```
            @Value("${FLEET_URI:http://fleet-service:8081}") String fleet,
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición.

L16

```
            @Value("${ASSISTANCE_URI:http://assistance-service:8082}") String assistance) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición.

L17

```
        return builder.routes()
```

Inicia el builder de rutas; cada .route posterior incorpora un destino.

L18

```
            .route("fleet", r -> r.path("/api/v1/operators/**", "/api/v1/tow-trucks/**",
"/api/v1/configuration/fleet").uri(fleet))
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L19

```
            .route("assistance", r -> r.path("/api/v1/vehicles/**", "/api/v1/assistance-requests/**",
"/api/v1/assignments/**", "/api/v1/operator/assignments",
"/api/v1/configuration/assistance").uri(assistance))
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L20

```
            .route("fleet-docs", r -> r.path("/docs/fleet/v3/api-docs").filters(f ->
f.stripPrefix(2)).uri(fleet))
```

Retira /docs/fleet o /docs/assistance al reenviar /v3/api-docs. Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L21

```
            .route("assistance-docs", r -> r.path("/docs/assistance/v3/api-docs").filters(f ->
f.stripPrefix(2)).uri(assistance))
```

Retira /docs/fleet o /docs/assistance al reenviar /v3/api-docs. Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L22

```
            .build();
```

Construye el objeto final a partir del builder configurado en líneas previas.

L23

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L24

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2: separadores sin instrucciones.

RequestLoggingFilter (api-gateway)

`Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/RequestLoggingFilter.java`

`SHA-256: 1e24aa62974f3a29a3feb2715bc38e611f9ae7350e6bb80c644ec20bbff00b6a`

Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L1

```
package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L4

```
import org.springframework.core.annotation.Order;
```

Importa tipos: org.springframework.core.annotation.Order. Tipo auxiliar usado por las expresiones del archivo.

L5

```
import org.springframework.web.server.*;
```

Importa tipos: org.springframework.web.server.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import reactor.core.publisher.Mono;
```

Importa tipos: reactor.core.publisher.Mono. Mono y operadores de finalización del flujo reactivo.

L7

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L8

```
@Component
```

Registra componente inyectable.

L9

```
@Order(-200)
```

Ordena filtro/manejador.

L10

```
public class RequestLoggingFilter implements WebFilter {
```

Declara clase RequestLoggingFilter.

L11

```
    public Mono<Void> filter(ServerWebExchange x, WebFilterChain chain) {
```

Implementa filtro reactivo; Mono expresa finalización sin cuerpo adicional.

L12

```
        long start=System.nanoTime();
```

Captura tiempo monotónico inicial para medir duración, no fecha de negocio.

L13

```
        return chain.filter(x).doFinally(signal -> LoggerFactory.getLogger(getClass()).info("HTTP
method={} status={} durationMs={}",x.getRequest().getMethod(),x.getResponse().getStatusCode(),(System.na
noTime()-start)/1_000_000));
```

Conserva solo elementos que cumplen el predicado. Registra los campos indicados, sin token ni secreto.

L14

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L15

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2: separadores sin instrucciones.

SecurityConfiguration (api-gateway)

`Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/SecurityConfiguration.java`

`SHA-256: d88682fe1532f39d8a78f3d5a361f65a758a5677c609d7a788039ec9f6a54e17`

Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L1

```
package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía.

L5

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L6

```
import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring.

L7

```
import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP.

L8

```
import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.config.web.server.ServerHttpSecurity;
```

Importa tipos: org.springframework.security.config.web.server.ServerHttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad.

L12

```
import org.springframework.security.web.server.SecurityWebFilterChain;
```

Importa tipos: org.springframework.security.web.server.SecurityWebFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad.

L13

```
import org.springframework.security.web.server.context.NoOpServerSecurityContextRepository;
```

Importa tipos: org.springframework.security.web.server.context.NoOpServerSecurityContextRepository. Validación JWT, Authentication, autorización y cadenas de seguridad.

L14

```
import org.springframework.security.authorization.ReactiveAuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.ReactiveAuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad.

L15

```
import org.springframework.security.web.server.authorization.AuthorizationContext;
```

Importa tipos: org.springframework.security.web.server.authorization.AuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad.

L17

```
@Configuration
```

Declara configuración Spring.

L18

```
public class SecurityConfiguration {
```

Declara clase SecurityConfiguration.

L19

```
    @Bean
```

Registra el objeto retornado en Spring.

L20

```
    ReactiveJwtDecoder jwtDecoder(@Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}")
String issuer,
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración.

L21

```
            @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición.

L22

```
        var decoder = NimbusReactiveJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Construye decodificador que verifica firma con claves JWKS.

L23

```
        decoder.setJwtValidator(TokenSupport.validator(issuer, Set.of("auxiruta-api")));
```

Añade validación de emisor, tiempo, sub y audiencia.

L24

```
        return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring.

L25

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L26

```
    @Bean
```

Registra el objeto retornado en Spring.

L27

```
    SecurityWebFilterChain security(ServerHttpSecurity http, SecurityErrors errors)  {
```

Construye la cadena reactiva de seguridad del Gateway.

L28

```
        var converter = new JwtAuthenticationConverter();
```

Crea convertidor de JWT a Authentication para incorporar authorities.

L29

```
        converter.setJwtGrantedAuthoritiesConverter(jwt -> new
java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L30

```
http.csrf(ServerHttpSecurity.CsrfSpec::disable).httpBasic(ServerHttpSecurity.HttpBasicSpec::disable)
```

Desactiva CSRF y mecanismos de login propios de esta API Bearer; Keycloak autentica.

L31

```
            .formLogin(ServerHttpSecurity.FormLoginSpec::disable)
```

Desactiva formulario propio del Gateway; la pantalla de login pertenece a Keycloak.

L32

```
            .securityContextRepository(NoOpServerSecurityContextRepository.getInstance())
```

No persiste contexto de seguridad en sesión.

L33

```
            .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
```

Conecta errores 401/403 con el escritor ProblemDetail propio.

L34

```
            .oauth2ResourceServer(o -> o.authenticationEntryPoint(errors).accessDeniedHandler(errors)
```

Activa procesamiento de Bearer JWT y errores propios.

L35

```
                .jwt(j -> j.jwtAuthenticationConverter(new
ReactiveJwtAuthenticationConverterAdapter(converter))))
```

Configura conversión de claims y roles cuando se valida JWT; en Gateway usa adaptador reactivo.

L36

```
            .authorizeExchange(auth -> auth
```

Inicia reglas de autorización del Gateway; se evalúan en orden.

L37

```
                .pathMatchers(HttpMethod.GET, "/api/v1/operators/me").access(allowed(false, "OPERATOR"))
```

Autoriza GET /api/v1/operators/me a OPERATOR con audiencia/azp válidos.

L38

```
                .pathMatchers(HttpMethod.PATCH,
"/api/v1/operators/me/availability").access(allowed(false, "OPERATOR"))
```

Autoriza PATCH /api/v1/operators/me/availability a OPERATOR con audiencia/azp válidos.

L39

```
                .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks/assigned-to-me").access(allowed(false,
 "OPERATOR"))
```

Autoriza GET /api/v1/tow-trucks/assigned-to-me a OPERATOR con audiencia/azp válidos.

L40

```
                .pathMatchers(HttpMethod.PUT,
"/api/v1/tow-trucks/assigned-to-me/location").access(allowed(false, "OPERATOR"))
```

Autoriza PUT /api/v1/tow-trucks/assigned-to-me/location a OPERATOR con audiencia/azp válidos.

L41

```
                .pathMatchers("/api/v1/operators/me", "/api/v1/operators/me/**",
"/api/v1/tow-trucks/assigned-to-me", "/api/v1/tow-trucks/assigned-to-me/**").denyAll()
```

Deniega /api/v1/operators/me, /api/v1/operators/me/**, /api/v1/tow-trucks/assigned-to-me, /api/v1/tow-trucks/assigned-to-me/**; impide caer en reglas generales.

L42

```
                .pathMatchers(HttpMethod.GET, "/api/v1/operators").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/operators a ADMIN con audiencia/azp válidos.

L43

```
                .pathMatchers(HttpMethod.GET, "/api/v1/operators/{id}").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/operators/{id} a ADMIN con audiencia/azp válidos.

L44

```
                .pathMatchers(HttpMethod.POST, "/api/v1/operators").access(allowed(false, "ADMIN"))
```

Autoriza POST /api/v1/operators a ADMIN con audiencia/azp válidos.

L45

```
                .pathMatchers(HttpMethod.PATCH, "/api/v1/operators/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza PATCH /api/v1/operators/{id} a ADMIN con audiencia/azp válidos.

L46

```
                .pathMatchers(HttpMethod.PATCH,
"/api/v1/operators/{id}/registration-status").access(allowed(false, "ADMIN"))
```

Autoriza PATCH /api/v1/operators/{id}/registration-status a ADMIN con audiencia/azp válidos.

L47

```
                .pathMatchers(HttpMethod.PUT,
"/api/v1/operators/{id}/qualification").access(allowed(false, "ADMIN"))
```

Autoriza PUT /api/v1/operators/{id}/qualification a ADMIN con audiencia/azp válidos.

L48

```
                .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/tow-trucks a ADMIN con audiencia/azp válidos.

L49

```
                .pathMatchers(HttpMethod.GET, "/api/v1/tow-trucks/{id}").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/tow-trucks/{id} a ADMIN con audiencia/azp válidos.

L50

```
                .pathMatchers(HttpMethod.POST, "/api/v1/tow-trucks").access(allowed(false, "ADMIN"))
```

Autoriza POST /api/v1/tow-trucks a ADMIN con audiencia/azp válidos.

L51

```
                .pathMatchers(HttpMethod.PATCH, "/api/v1/tow-trucks/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza PATCH /api/v1/tow-trucks/{id} a ADMIN con audiencia/azp válidos.

L52

```
                .pathMatchers(HttpMethod.PUT, "/api/v1/tow-trucks/{id}/operator").access(allowed(false,
"ADMIN"))
```

Autoriza PUT /api/v1/tow-trucks/{id}/operator a ADMIN con audiencia/azp válidos.

L53

```
                .pathMatchers(HttpMethod.DELETE,
"/api/v1/tow-trucks/{id}/operator").access(allowed(false, "ADMIN"))
```

Autoriza DELETE /api/v1/tow-trucks/{id}/operator a ADMIN con audiencia/azp válidos.

L54

```
                .pathMatchers(HttpMethod.PATCH,
"/api/v1/tow-trucks/{id}/operational-status").access(allowed(false, "ADMIN"))
```

Autoriza PATCH /api/v1/tow-trucks/{id}/operational-status a ADMIN con audiencia/azp válidos.

L55

```
                .pathMatchers(HttpMethod.GET, "/api/v1/configuration/fleet").access(allowed(false,
"ADMIN"))
```

Autoriza GET /api/v1/configuration/fleet a ADMIN con audiencia/azp válidos.

L56

```
                .pathMatchers(HttpMethod.POST, "/api/v1/vehicles").access(allowed(false, "DRIVER"))
```

Autoriza POST /api/v1/vehicles a DRIVER con audiencia/azp válidos.

L57

```
                .pathMatchers(HttpMethod.GET, "/api/v1/vehicles").access(allowed(false, "DRIVER"))
```

Autoriza GET /api/v1/vehicles a DRIVER con audiencia/azp válidos.

L58

```
                .pathMatchers(HttpMethod.GET, "/api/v1/vehicles/{id}").access(allowed(false, "DRIVER"))
```

Autoriza GET /api/v1/vehicles/{id} a DRIVER con audiencia/azp válidos.

L59

```
                .pathMatchers(HttpMethod.PATCH, "/api/v1/vehicles/{id}").access(allowed(false,
"DRIVER"))
```

Autoriza PATCH /api/v1/vehicles/{id} a DRIVER con audiencia/azp válidos.

L60

```
                .pathMatchers(HttpMethod.PATCH,
"/api/v1/vehicles/{id}/registration-status").access(allowed(false, "DRIVER"))
```

Autoriza PATCH /api/v1/vehicles/{id}/registration-status a DRIVER con audiencia/azp válidos.

L61

```
                .pathMatchers(HttpMethod.POST, "/api/v1/assistance-requests").access(allowed(false,
"DRIVER"))
```

Autoriza POST /api/v1/assistance-requests a DRIVER con audiencia/azp válidos.

L62

```
                .pathMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/cancellations").access(allowed(false, "DRIVER"))
```

Autoriza POST /api/v1/assistance-requests/{id}/cancellations a DRIVER con audiencia/azp válidos.

L63

```
                .pathMatchers(HttpMethod.GET, "/api/v1/assistance-requests").access(allowed(false,
"DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L64

```
                .pathMatchers(HttpMethod.GET, "/api/v1/assistance-requests/{id}").access(allowed(false,
"DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests/{id} a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L65

```
                .pathMatchers(HttpMethod.GET,
"/api/v1/assistance-requests/{id}/history").access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests/{id}/history a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L66

```
                .pathMatchers(HttpMethod.GET, "/api/v1/operator/assignments").access(allowed(false,
"OPERATOR"))
```

Autoriza GET /api/v1/operator/assignments a OPERATOR con audiencia/azp válidos.

L67

```
                .pathMatchers(HttpMethod.GET, "/api/v1/assignments/{id}").access(allowed(false,
  "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assignments/{id} a OPERATOR, ADMIN con audiencia/azp válidos.

L68

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assignments/{id}/acceptance").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assignments/{id}/acceptance a OPERATOR con audiencia/azp válidos.

L69

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assignments/{id}/rejection").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assignments/{id}/rejection a OPERATOR con audiencia/azp válidos.

L70

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assistance-requests/{id}/dispatch").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/dispatch a OPERATOR con audiencia/azp válidos.

L71

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assistance-requests/{id}/arrival").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/arrival a OPERATOR con audiencia/azp válidos.

L72

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assistance-requests/{id}/loading").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/loading a OPERATOR con audiencia/azp válidos.

L73

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assistance-requests/{id}/transport-start").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/transport-start a OPERATOR con audiencia/azp válidos.

L74

```
                .pathMatchers(HttpMethod.POST,
  "/api/v1/assistance-requests/{id}/completion").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/completion a OPERATOR con audiencia/azp válidos.

L75

```
                .pathMatchers(HttpMethod.GET, "/api/v1/configuration/assistance").access(allowed(false,
  "ADMIN"))
```

Autoriza GET /api/v1/configuration/assistance a ADMIN con audiencia/azp válidos.

L76

```
                .pathMatchers(HttpMethod.GET, "/docs/fleet/v3/api-docs").access(allowed(false, "DRIVER",
   "OPERATOR", "ADMIN"))
```

Autoriza GET /docs/fleet/v3/api-docs a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L77

```
                .pathMatchers(HttpMethod.GET, "/docs/assistance/v3/api-docs").access(allowed(false,
  "DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /docs/assistance/v3/api-docs a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L78

```
                .anyExchange().denyAll());
```

Deniega todo acceso no autorizado antes.

L79

```
        return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L80

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L81

```
    private ReactiveAuthorizationManager<AuthorizationContext> allowed(boolean technical, String...
roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado.

L82

```
        return (authentication, context) -> authentication.map(auth -> new AuthorizationDecision(
```

Transforma cada elemento al valor/tipo de salida.

L83

```
            auth instanceof JwtAuthenticationToken token && TokenSupport.permits(token.getToken(),
technical, roles)))
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport.

L84

```
            .defaultIfEmpty(new AuthorizationDecision(false));
```

Si no existe Authentication, la decisión reactiva por defecto es denegar.

L85

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L86

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 16: separadores sin instrucciones.

SecurityErrors (api-gateway)

`Ruta: api-gateway/src/main/java/com/auxiruta/gateway/config/SecurityErrors.java`

`SHA-256: e9ecdfb98810b91864f9bb496852f8f4bb5a21abf356555bdf7b7b54bc03527a`

Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L1

```
package com.auxiruta.gateway.config;
```

Ubica el archivo en com.auxiruta.gateway.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L5

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.core.annotation.Order;
```

Importa tipos: org.springframework.core.annotation.Order. Tipo auxiliar usado por las expresiones del archivo.

L7

```
import org.springframework.web.server.*;
```

Importa tipos: org.springframework.web.server.*. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.springframework.security.web.server.*;
```

Importa tipos: org.springframework.security.web.server.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.web.server.authorization.ServerAccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.server.authorization.ServerAccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L12

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L13

```
import reactor.core.publisher.Mono;
```

Importa tipos: reactor.core.publisher.Mono. Mono y operadores de finalización del flujo reactivo.

L15

```
@Component
```

Registra componente inyectable.

L16

```
@Order(-2)
```

Ordena filtro/manejador.

L17

```
public class SecurityErrors implements ServerAuthenticationEntryPoint, ServerAccessDeniedHandler,
WebExceptionHandler {
```

Declara clase SecurityErrors.

L18

```
    private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
    public SecurityErrors(ObjectMapper mapper) { this.mapper = mapper; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L20

```
    public Mono<Void> commence(ServerWebExchange x, AuthenticationException e) { return write(x,401); }
```

Una falla de autenticación produce HTTP 401.

L21

```
    public Mono<Void> handle(ServerWebExchange x, AccessDeniedException e) { return write(x,403); }
```

Una falla de autorización produce HTTP 403.

L22

```
    public Mono<Void> handle(ServerWebExchange x, Throwable e) {
```

Manejador reactivo de errores generales; debe respetar si la respuesta ya fue enviada.

L23

```
        if (x.getResponse().isCommitted()) return Mono.error(e);
```

La rama condicional selecciona la actualización o retorno correspondiente.

L24

```
        return write(x, e instanceof ResponseStatusException status ? status.getStatusCode().value() :
503);
```

Conserva el código de ResponseStatusException; un error de enrutamiento no clasificado se presenta como 503.

L25

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L26

```
    private Mono<Void> write(ServerWebExchange x,int status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L27

```
        x.getResponse().setStatusCode(HttpStatusCode.valueOf(status));
```

Establece el código HTTP de la respuesta reactiva.

L28

```
        x.getResponse().getHeaders().setContentType(MediaType.APPLICATION_PROBLEM_JSON);
```

Declara application/problem+json para errores uniformes.

L29

```
        if(status==401) x.getResponse().getHeaders().set("WWW-Authenticate","Bearer");
```

Añade desafío Bearer en 401. La rama condicional selecciona la actualización o retorno correspondiente.

L30

```
        try { return x.getResponse().writeWith(Mono.just(x.getResponse().bufferFactory().wrap(mapper.wri
teValueAsBytes(Problems.of(status))))); }
```

Serializa ProblemDetail a bytes y los escribe en el buffer reactivo.

L31

```
        catch (Exception e) { return Mono.error(e); }
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L32

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L33

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 14: separadores sin instrucciones.

application-docker (api-gateway)

`Ruta: api-gateway/src/main/resources/application-docker.yml`

`SHA-256: 89c1f5a186b9e2a184c0da5dfa10be8df94e879273c34d139c4f7b2155871caf`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
  security:
```

Abre bloque security; sus hijos configuran este componente.

L3

```
    oauth2:
```

Abre bloque oauth2; sus hijos configuran este componente.

L4

```
      resourceserver:
```

Abre bloque resourceserver; sus hijos configuran este componente.

L5

```
        jwt:
```

Abre bloque jwt; sus hijos configuran este componente.

L6

```
          issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. Valor: ${KEYCLOAK_ISSUER_URI}.

L7

```
          jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. Valor: ${KEYCLOAK_JWK_SET_URI}.

application (api-gateway)

`Ruta: api-gateway/src/main/resources/application.yml`

`SHA-256: 3942f1d02a0f9ca65c6ea7ac62d14818280e85d15f29b7b5c4da5ff55aef1aa1`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
  application:
```

Abre bloque application; sus hijos configuran este componente.

L3

```
    name: api-gateway
```

Establece name = api-gateway dentro del bloque. Se interpreta al cargar la configuración.

L4

```
server:
```

Abre bloque server; sus hijos configuran este componente.

L5

```
  port: ${SERVER_PORT:8000}
```

Puerto de escucha; Compose controla su exposición al host. Valor: ${SERVER_PORT:8000}.

AssistanceApplication (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/AssistanceApplication.java`

`SHA-256: 456ecaa6914c1201f176d5e79f5b04d78d4747fd45d1c4feb28fba7c23ad533e`

Punto de entrada del microservicio propietario de vehículos, solicitudes, evaluaciones y ejecución.

L1

```
package com.auxiruta.assistance;
```

Ubica el archivo en com.auxiruta.assistance. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación.

L4

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación.

L6

```
@SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
public class AssistanceApplication {
```

Declara clase AssistanceApplication.

L8

```
    public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
        SpringApplication.run(AssistanceApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L11

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 5: separadores sin instrucciones.

ClockConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/ClockConfiguration.java`

SHA-256: 43d1c82f7599d17ebe747064ae61154ae177ba7f02099e265b419c0ba10e8ce1

Registra el Clock UTC como bean para servicios y reglas temporales.

L1

```
package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.time.UtcClock;
```

Importa tipos: com.auxiruta.commons.time.UtcClock. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import org.springframework.context.annotation.Bean;
```

Importa tipos: org.springframework.context.annotation.Bean. Beans, configuración y eventos del contexto Spring.

L6

```
import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring.

L8

```
@Configuration
```

Declara configuración Spring.

L9

```
public class ClockConfiguration {
```

Declara clase ClockConfiguration.

L10

```
    @Bean
```

Registra el objeto retornado en Spring.

L11

```
    Clock clock() {
```

Método fábrica del bean Clock que Spring inyecta en reglas temporales.

L12

```
        return UtcClock.system();
```

Obtiene el reloj común UTC y lo entrega al contenedor de beans.

L13

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L14

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 7: separadores sin instrucciones.

OpenApiConfiguration (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/OpenApiConfiguration.java

SHA-256: c2555b390b0dd1a9d7ef60fa65c90022376c39a984b83ff99bbe989606a82ed7

Declara metadatos OpenAPI y el esquema Bearer JWT. Una anotación de documentación no concede permisos: esa decisión está en SecurityConfiguration.

L1

```
package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring.

L3

```
import io.swagger.v3.oas.annotations.OpenAPIDefinition;
```

Importa tipos: io.swagger.v3.oas.annotations.OpenAPIDefinition. Metadatos OpenAPI de rutas, respuestas y autenticación.

L4

```
import io.swagger.v3.oas.annotations.info.Info;
```

Importa tipos: io.swagger.v3.oas.annotations.info.Info. Metadatos OpenAPI de rutas, respuestas y autenticación.

L5

```
import io.swagger.v3.oas.annotations.servers.Server;
```

Importa tipos: io.swagger.v3.oas.annotations.servers.Server. Metadatos OpenAPI de rutas, respuestas y autenticación.

L6

```
import io.swagger.v3.oas.annotations.security.SecurityScheme;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityScheme. Metadatos OpenAPI de rutas, respuestas y autenticación.

L7

```
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
```

Importa tipos: io.swagger.v3.oas.annotations.enums.SecuritySchemeType. Metadatos OpenAPI de rutas, respuestas y autenticación.

L8

```
@Configuration
```

Declara configuración Spring.

L9

```
@OpenAPIDefinition(info=@Info(title="AuxiRuta Assistance API",version="1.0.0",description="Vehículos y
solicitudes. Bearer JWT. Coordenadas WGS84 hasta seis decimales. Fechas UTC. page>=0, size 1..100.
Creación, cancelación, respuestas a propuestas y etapas exigen Idempotency-Key de hasta 100 caracteres.
Evaluación real con OpenRouteService; recuperación automática de operaciones remotas. Una respuesta
exitosa puede conservar resourceReleasePending hasta confirmar liberación. Los errores usan
application/problem+json."),servers=@Server(url="/"))
```

Metadatos documentales de API.

L10

```
@SecurityScheme(name="bearerAuth",type=SecuritySchemeType.HTTP,scheme="bearer",bearerFormat="JWT")
```

Describe Bearer JWT.

L11

```
public class OpenApiConfiguration {}
```

Declara clase OpenApiConfiguration.

RequestLoggingFilter (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/RequestLoggingFilter.java`

`SHA-256: 05711d2e47e4fc3c3aae1032d423e9c98f7f9ebe9c390e03ac5b8c07a202ba0e`

Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L1

```
package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import jakarta.servlet.*;
```

Importa tipos: jakarta.servlet.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L4

```
import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L5

```
import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo.

L6

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L7

```
import org.springframework.web.filter.OncePerRequestFilter;
```

Importa tipos: org.springframework.web.filter.OncePerRequestFilter. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L9

```
@Component
```

Registra componente inyectable.

L10

```
public class RequestLoggingFilter extends OncePerRequestFilter {
```

Declara clase RequestLoggingFilter.

L11

```
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
throws ServletException, IOException {
```

Filtro Servlet por petición: envuelve el resto de la cadena y mide su resultado incluso si falla.

L12

```
        long start = System.nanoTime();
```

Captura tiempo monotónico inicial para medir duración, no fecha de negocio.

L13

```
        try { chain.doFilter(req,res); }
```

Delega a los filtros/controladores siguientes; finally registrará duración y estado aunque lancen excepción.

L14

```
        finally { LoggerFactory.getLogger(getClass()).debug("HTTP method={} status={}
durationMs={}",req.getMethod(),res.getStatus(),(System.nanoTime()-start)/1_000_000); }
```

Registra los campos indicados, sin token ni secreto.

L15

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L16

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2: separadores sin instrucciones.

SecurityConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/SecurityConfiguration.java`

`SHA-256: cfe51f4a7745ef5522015f810e05e143f86c69bc3bdc5677af9d818c1e363027`

Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L1

```
package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía.

L5

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L6

```
import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring.

L7

```
import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP.

L8

```
import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

Importa tipos: org.springframework.security.config.annotation.web.builders.HttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad.

L12

```
import org.springframework.security.web.SecurityFilterChain;
```

Importa tipos: org.springframework.security.web.SecurityFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad.

L13

```
import org.springframework.security.config.http.SessionCreationPolicy;
```

Importa tipos: org.springframework.security.config.http.SessionCreationPolicy. Validación JWT, Authentication, autorización y cadenas de seguridad.

L14

```
import org.springframework.security.authorization.AuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.AuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad.

L15

```
import org.springframework.security.web.access.intercept.RequestAuthorizationContext;
```

Importa tipos: org.springframework.security.web.access.intercept.RequestAuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad.

L17

```
@Configuration
```

Declara configuración Spring.

L18

```
public class SecurityConfiguration {
```

Declara clase SecurityConfiguration.

L19

```
    @Bean
```

Registra el objeto retornado en Spring.

L20

```
    JwtDecoder jwtDecoder(@Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String
issuer,
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración.

L21

```
            @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición.

L22

```
        var decoder = NimbusJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Construye decodificador que verifica firma con claves JWKS.

L23

```
        decoder.setJwtValidator(TokenSupport.validator(issuer, Set.of("auxiruta-api")));
```

Añade validación de emisor, tiempo, sub y audiencia.

L24

```
        return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring.

L25

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L26

```
    @Bean
```

Registra el objeto retornado en Spring.

L27

```
    SecurityFilterChain security(HttpSecurity http, SecurityErrors errors) throws Exception {
```

Construye la cadena Servlet de seguridad que intercepta peticiones antes del controlador.

L28

```
        var converter = new JwtAuthenticationConverter();
```

Crea convertidor de JWT a Authentication para incorporar authorities.

L29

```
        converter.setJwtGrantedAuthoritiesConverter(jwt -> new
java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L30

```
        http.csrf(c -> c.disable()).httpBasic(c -> c.disable()).formLogin(c -> c.disable())
```

Desactiva formulario propio del Gateway; la pantalla de login pertenece a Keycloak. Desactiva CSRF y mecanismos de login propios de esta API Bearer; Keycloak autentica.

L31

```
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

No guarda sesión HTTP; cada petición aporta token.

L32

```
            .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
```

Conecta errores 401/403 con el escritor ProblemDetail propio.

L33

```
            .oauth2ResourceServer(o -> o.authenticationEntryPoint(errors).accessDeniedHandler(errors)
```

Activa procesamiento de Bearer JWT y errores propios.

L34

```
                .jwt(j -> j.jwtAuthenticationConverter(converter)))
```

Configura conversión de claims y roles cuando se valida JWT; en Gateway usa adaptador reactivo.

L35

```
            .authorizeHttpRequests(auth -> auth
```

Inicia reglas de autorización Servlet; la primera coincidencia aplica.

L36

```
                .requestMatchers(HttpMethod.POST, "/api/v1/vehicles").access(allowed(false, "DRIVER"))
```

Autoriza POST /api/v1/vehicles a DRIVER con audiencia/azp válidos.

L37

```
                .requestMatchers(HttpMethod.GET, "/api/v1/vehicles").access(allowed(false, "DRIVER"))
```

Autoriza GET /api/v1/vehicles a DRIVER con audiencia/azp válidos.

L38

```
                .requestMatchers(HttpMethod.GET, "/api/v1/vehicles/{id}").access(allowed(false,
"DRIVER"))
```

Autoriza GET /api/v1/vehicles/{id} a DRIVER con audiencia/azp válidos.

L39

```
                .requestMatchers(HttpMethod.PATCH, "/api/v1/vehicles/{id}").access(allowed(false,
"DRIVER"))
```

Autoriza PATCH /api/v1/vehicles/{id} a DRIVER con audiencia/azp válidos.

L40

```
                .requestMatchers(HttpMethod.PATCH,
"/api/v1/vehicles/{id}/registration-status").access(allowed(false, "DRIVER"))
```

Autoriza PATCH /api/v1/vehicles/{id}/registration-status a DRIVER con audiencia/azp válidos.

L41

```
                .requestMatchers(HttpMethod.POST, "/api/v1/assistance-requests").access(allowed(false,
"DRIVER"))
```

Autoriza POST /api/v1/assistance-requests a DRIVER con audiencia/azp válidos.

L42

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/cancellations").access(allowed(false, "DRIVER"))
```

Autoriza POST /api/v1/assistance-requests/{id}/cancellations a DRIVER con audiencia/azp válidos.

L43

```
                .requestMatchers(HttpMethod.GET, "/api/v1/assistance-requests").access(allowed(false,
"DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L44

```
                .requestMatchers(HttpMethod.GET,
"/api/v1/assistance-requests/{id}").access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests/{id} a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L45

```
                .requestMatchers(HttpMethod.GET,
"/api/v1/assistance-requests/{id}/history").access(allowed(false, "DRIVER", "OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assistance-requests/{id}/history a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L46

```
                .requestMatchers(HttpMethod.GET, "/api/v1/operator/assignments").access(allowed(false,
"OPERATOR"))
```

Autoriza GET /api/v1/operator/assignments a OPERATOR con audiencia/azp válidos.

L47

```
                .requestMatchers(HttpMethod.GET, "/api/v1/assignments/{id}").access(allowed(false,
"OPERATOR", "ADMIN"))
```

Autoriza GET /api/v1/assignments/{id} a OPERATOR, ADMIN con audiencia/azp válidos.

L48

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assignments/{id}/acceptance").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assignments/{id}/acceptance a OPERATOR con audiencia/azp válidos.

L49

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assignments/{id}/rejection").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assignments/{id}/rejection a OPERATOR con audiencia/azp válidos.

L50

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/dispatch").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/dispatch a OPERATOR con audiencia/azp válidos.

L51

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/arrival").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/arrival a OPERATOR con audiencia/azp válidos.

L52

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/loading").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/loading a OPERATOR con audiencia/azp válidos.

L53

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/transport-start").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/transport-start a OPERATOR con audiencia/azp válidos.

L54

```
                .requestMatchers(HttpMethod.POST,
"/api/v1/assistance-requests/{id}/completion").access(allowed(false, "OPERATOR"))
```

Autoriza POST /api/v1/assistance-requests/{id}/completion a OPERATOR con audiencia/azp válidos.

L55

```
                .requestMatchers(HttpMethod.GET,
"/api/v1/configuration/assistance").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/configuration/assistance a ADMIN con audiencia/azp válidos.

L56

```
                .requestMatchers(HttpMethod.GET, "/v3/api-docs/**").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /v3/api-docs/** a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L57

```
                .requestMatchers(HttpMethod.GET, "/swagger-ui/**").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /swagger-ui/** a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L58

```
                .requestMatchers(HttpMethod.GET, "/swagger-ui.html").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /swagger-ui.html a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L59

```
                .anyRequest().denyAll());
```

Deniega todo acceso no autorizado antes.

L60

```
        return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L61

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L62

```
    private AuthorizationManager<RequestAuthorizationContext> allowed(boolean technical, String...
roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado.

L63

```
        return (authentication, context) -> new AuthorizationDecision(
```

Devuelve una función de autorización evaluada por petición, no durante el arranque.

L64

```
            authentication.get() instanceof JwtAuthenticationToken token &&
TokenSupport.permits(token.getToken(), technical, roles));
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport.

L65

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L66

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 16: separadores sin instrucciones.

SecurityErrors (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/config/SecurityErrors.java`

`SHA-256: 2c92982115f9f22dcce836153a9a271b7a5ee97242643ecddb7f78985de9751b`

Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L1

```
package com.auxiruta.assistance.config;
```

Ubica el archivo en com.auxiruta.assistance.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L5

```
import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L6

```
import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo.

L7

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L8

```
import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.web.AuthenticationEntryPoint;
```

Importa tipos: org.springframework.security.web.AuthenticationEntryPoint. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.web.access.AccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.access.AccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad.

L13

```
@Component
```

Registra componente inyectable.

L14

```
public class SecurityErrors implements AuthenticationEntryPoint, AccessDeniedHandler {
```

Declara clase SecurityErrors.

L15

```
    private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
    public SecurityErrors(ObjectMapper mapper) { this.mapper = mapper; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L17

```
    public void commence(HttpServletRequest req, HttpServletResponse res, AuthenticationException e)
throws IOException { write(res, 401); }
```

Una falla de autenticación produce HTTP 401 en Servlet.

L18

```
    public void handle(HttpServletRequest req, HttpServletResponse res, AccessDeniedException e) throws
IOException { write(res, 403); }
```

Una falla de autorización produce HTTP 403 en Servlet.

L19

```
    private void write(HttpServletResponse response, int status) throws IOException {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L20

```
        response.setStatus(status);
```

Establece el código HTTP Servlet.

L21

```
        response.setContentType("application/problem+json");
```

Declara application/problem+json para errores uniformes.

L22

```
        if (status == 401) response.setHeader("WWW-Authenticate", "Bearer");
```

Añade desafío Bearer en 401. La rama condicional selecciona la actualización o retorno correspondiente.

L23

```
        mapper.writeValue(response.getOutputStream(), Problems.of(status));
```

Serializa ProblemDetail directamente en el cuerpo Servlet.

L24

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L25

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 12: separadores sin instrucciones.

ApiExceptionHandler (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/exceptions/ApiExceptionHandler.java`

`SHA-256: a87478d2044884ba021587287452b72789ef4878c4e17f598715e7923f50245a`

Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L1

```
package com.auxiruta.assistance.exceptions;
```

Ubica el archivo en com.auxiruta.assistance.exceptions. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L5

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
```

Importa tipos: org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler. Controladores, errores HTTP y clientes REST según la clase importada.

L7

```
import org.springframework.web.context.request.WebRequest;
```

Importa tipos: org.springframework.web.context.request.WebRequest. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L9

```
import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L12

```
@RestControllerAdvice
```

Centraliza transformación de excepciones.

L13

```
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
```

Declara clase ApiExceptionHandler.

L14

```
    @Override
```

Redefine contrato heredado.

L15

```
    protected ResponseEntity<Object> handleExceptionInternal(Exception ex, Object body, HttpHeaders
headers, HttpStatusCode status, WebRequest request) {
```

Interviene en errores MVC de validación/parsing y preserva encabezados/estado.

L16

```
        return new ResponseEntity<>(Problems.of(status.value()), headers, status);
```

Reemplaza el cuerpo de error por el formato común conservando estado y headers.

L17

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L18

```
    @ExceptionHandler(ResponseStatusException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L19

```
    ResponseEntity<ProblemDetail> status(ResponseStatusException e) { return
ResponseEntity.status(e.getStatusCode()).body(statusProblem(e)); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail. Prepara ProblemDetail y puede conservar el código específico de confirmación pendiente.

L20

```
    private ProblemDetail statusProblem(ResponseStatusException e) {
```

Prepara ProblemDetail y puede conservar el código específico de confirmación pendiente.

L21

```
        var problem=Problems.of(e.getStatusCode().value());
```

Obtiene el error común correspondiente al estado de la excepción.

L22

```
        if("ACCEPTANCE_CONFIRMATION_PENDING".equals(e.getReason()))problem.setProperty("code","ACCEPTANC
E_CONFIRMATION_PENDING");
```

Protege la carrera aceptación/cancelación: una confirmación remota incierta puede haber ocurrido. Impide cancelar hasta reconciliar, con código distinguible en el error. La rama condicional selecciona la actualización o retorno correspondiente.

L23

```
        return problem;
```

Entrega problem al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L24

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L25

```
    @ExceptionHandler(AccessDeniedException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L26

```
    ResponseEntity<ProblemDetail> forbidden(AccessDeniedException e) { return
ResponseEntity.status(403).body(Problems.of(403)); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L27

```
    @ExceptionHandler({org.springframework.dao.DataIntegrityViolationException.class,org.springframework
.dao.ConcurrencyFailureException.class})
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L28

```
    ResponseEntity<ProblemDetail> conflict(Exception e) { return
ResponseEntity.status(409).body(Problems.of(409)); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L29

```
    @ExceptionHandler(Exception.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L30

```
    ResponseEntity<ProblemDetail> unexpected(Exception e) {
```

Captura errores no clasificados para devolver 500 genérico y registrar solo tipo de excepción.

L31

```
        LoggerFactory.getLogger(getClass()).error("Error inesperado tipo={}",
e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto.

L32

```
        return ResponseEntity.status(500).body(Problems.of(500));
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L33

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L34

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 11: separadores sin instrucciones.

application-docker (assistance-service)

`Ruta: assistance-service/src/main/resources/application-docker.yml`

`SHA-256: f2f8bac12925f3503c1e7df989b9fdc5dbf31175c8b6da4e4e59de7dc328550d`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
  security:
```

Abre bloque security; sus hijos configuran este componente.

L3

```
    oauth2:
```

Abre bloque oauth2; sus hijos configuran este componente.

L4

```
      client:
```

Abre bloque client; sus hijos configuran este componente.

L5

```
        registration:
```

Abre bloque registration; sus hijos configuran este componente.

L6

```
          fleet-client:
```

Abre bloque fleet-client; sus hijos configuran este componente.

L7

```
            provider: keycloak
```

Establece provider = keycloak dentro del bloque. Se interpreta al cargar la configuración.

L8

```
            client-id: ${KEYCLOAK_CLIENT_ID}
```

Identificador del cliente OAuth. Valor: ${KEYCLOAK_CLIENT_ID}.

L9

```
            client-secret: ${KEYCLOAK_CLIENT_SECRET}
```

Secreto técnico resuelto desde entorno, no token de usuario. Valor: ${KEYCLOAK_CLIENT_SECRET}.

L10

```
            authorization-grant-type: client_credentials
```

client_credentials autentica al servicio, no al conductor. Valor: client_credentials.

L11

```
        provider:
```

Abre bloque provider; sus hijos configuran este componente.

L12

```
          keycloak:
```

Abre bloque keycloak; sus hijos configuran este componente.

L13

```
            token-uri: ${KEYCLOAK_TOKEN_URI}
```

Endpoint de Keycloak para emitir tokens. Valor: ${KEYCLOAK_TOKEN_URI}.

L14

```
      resourceserver:
```

Abre bloque resourceserver; sus hijos configuran este componente.

L15

```
        jwt:
```

Abre bloque jwt; sus hijos configuran este componente.

L16

```
          issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. Valor: ${KEYCLOAK_ISSUER_URI}.

L17

```
          jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. Valor: ${KEYCLOAK_JWK_SET_URI}.

L18

```
  datasource:
```

Abre bloque datasource; sus hijos configuran este componente.

L19

```
    url: ${DB_URL}
```

Dirección de conexión o servidor del bloque. Valor: ${DB_URL}.

L20

```
    username: ${DB_USER}
```

Usuario de conexión o identidad. Valor: ${DB_USER}.

L21

```
    password: ${DB_PASSWORD}
```

Referencia de contraseña; no reproduce el secreto local. Valor: ${DB_PASSWORD}.

L22

```
  sql:
```

Abre bloque sql; sus hijos configuran este componente.

L23

```
    init:
```

Abre bloque init; sus hijos configuran este componente.

L24

```
      mode: always
```

always aplica inicialización SQL también a PostgreSQL. Valor: always.

L25

```
      schema-locations: classpath:db/001-assistance.sql,classpath:db/002-evaluation.sql,classpath:db/003
-assignments.sql,classpath:db/004-execution.sql,classpath:db/005-automation.sql
```

Scripts SQL ordenados para inicializar/extender esquema. Valor: classpath:db/001-assistance.sql,classpath:db/002-evaluation.sql,classpath:db/003-assignments.sql,classpath:db/004-execution.sql,classpath:db/005-automation.sql.

L26

```
  jpa:
```

Abre bloque jpa; sus hijos configuran este componente.

L27

```
    open-in-view: false
```

false evita mantener EntityManager durante serialización HTTP. Valor: false.

L28

```
    hibernate:
```

Abre bloque hibernate; sus hijos configuran este componente.

L29

```
      ddl-auto: validate
```

validate comprueba el esquema; Hibernate no crea tablas. Valor: validate.

L31

```
ors:
```

Abre bloque ors; sus hijos configuran este componente.

L32

```
    base-url: ${ORS_BASE_URL:https://api.heigit.org/openrouteservice/v2}
```

URL del proveedor ORS. Valor: ${ORS_BASE_URL:https://api.heigit.org/openrouteservice/v2}.

L33

```
    api-key: ${ORS_API_KEY:}
```

Referencia a la clave ORS; nunca versionar su valor local. Valor: ${ORS_API_KEY:}.

Líneas vacías 30: separadores sin instrucciones.

application (assistance-service)

`Ruta: assistance-service/src/main/resources/application.yml`

`SHA-256: e2cc3040013dfdd284d1c0e19aaed8f27fb62be791708afe276de7ae60985c73`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
  spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
    jackson:
```

Abre bloque jackson; sus hijos configuran este componente.

L3

```
      deserialization:
```

Abre bloque deserialization; sus hijos configuran este componente.

L4

```
        fail-on-unknown-properties: true
```

true rechaza campos JSON desconocidos. Valor: true.

L5

```
    application:
```

Abre bloque application; sus hijos configuran este componente.

L6

```
      name: assistance-service
```

Establece name = assistance-service dentro del bloque. Se interpreta al cargar la configuración.

L7

```
  server:
```

Abre bloque server; sus hijos configuran este componente.

L8

```
    port: ${SERVER_PORT:8082}
```

Puerto de escucha; Compose controla su exposición al host. Valor: ${SERVER_PORT:8082}.

L10

```
  springdoc:
```

Abre bloque springdoc; sus hijos configuran este componente.

L11

```
    paths-to-match: /api/v1/**
```

Patrones incluidos en OpenAPI público. Valor: /api/v1/**.

L13

```
  logging:
```

Abre bloque logging; sus hijos configuran este componente.

L14

```
    level:
```

Abre bloque level; sus hijos configuran este componente.

L15

```
      org.hibernate.engine.jdbc.spi.SqlExceptionHelper: "OFF"
```

Establece org.hibernate.engine.jdbc.spi.SqlExceptionHelper = "OFF" dentro del bloque. Se interpreta al cargar la configuración.

L17

```
  auxiruta:
```

Abre bloque auxiruta; sus hijos configuran este componente.

L18

```
    automation:
```

Abre bloque automation; sus hijos configuran este componente.

L19

```
      enabled: ${AUXIRUTA_AUTOMATION_ENABLED:true}
```

Activa el componente o identidad del bloque. Valor: ${AUXIRUTA_AUTOMATION_ENABLED:true}.

Líneas vacías 9, 12, 16: separadores sin instrucciones.

FleetApplication (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/FleetApplication.java`

`SHA-256: 71a520f5a207f076a877f912d38710031c69869dd30307069acd579c6363bff2`

Punto de entrada de Spring Boot para el microservicio propietario de operadores, grúas y reservas.

L1

```
  package com.auxiruta.fleet;
```

Ubica el archivo en com.auxiruta.fleet. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
  import org.springframework.boot.SpringApplication;
```

Importa tipos: org.springframework.boot.SpringApplication. Arranque/autoconfiguración y condiciones de activación.

L4

```
  import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Importa tipos: org.springframework.boot.autoconfigure.SpringBootApplication. Arranque/autoconfiguración y condiciones de activación.

L6

```
  @SpringBootApplication
```

Activa autoconfiguración y escaneo de componentes.

L7

```
  public class FleetApplication {
```

Declara clase FleetApplication.

L8

```
      public static void main(String[] args) {
```

Entrada JVM: recibe argumentos y el cuerpo arranca Spring Boot.

L9

```
        SpringApplication.run(FleetApplication.class, args);
```

Arranca contexto Spring y servidor HTTP del módulo.

L10

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L11

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 5: separadores sin instrucciones.

ClockConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/ClockConfiguration.java`

`SHA-256: 2ff94a34728fd1b690a76f22da0536db6b74c03a8b27f972fbb0140fd2be7ef3`

Registra el Clock UTC como bean para servicios y reglas temporales.

L1

```
package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.time.UtcClock;
```

Importa tipos: com.auxiruta.commons.time.UtcClock. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import org.springframework.context.annotation.Bean;
```

Importa tipos: org.springframework.context.annotation.Bean. Beans, configuración y eventos del contexto Spring.

L6

```
import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring.

L8

```
@Configuration
```

Declara configuración Spring.

L9

```
public class ClockConfiguration {
```

Declara clase ClockConfiguration.

L10

```
    @Bean
```

Registra el objeto retornado en Spring.

L11

```
    Clock clock() {
```

Método fábrica del bean Clock que Spring inyecta en reglas temporales.

L12

```
        return UtcClock.system();
```

Obtiene el reloj común UTC y lo entrega al contenedor de beans.

L13

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L14

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 7: separadores sin instrucciones.

OpenApiConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/OpenApiConfiguration.java`

`SHA-256: 0afaba709d0c2a3a71eac3e5130e3395ea0f7861ed064cf94974c01bcf190803`

Declara metadatos OpenAPI y el esquema Bearer JWT. Una anotación de documentación no concede permisos: esa decisión está en SecurityConfiguration.

L1

```
package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import org.springframework.context.annotation.Configuration;
```

Importa tipos: org.springframework.context.annotation.Configuration. Beans, configuración y eventos del contexto Spring.

L3

```
import io.swagger.v3.oas.annotations.OpenAPIDefinition;
```

Importa tipos: io.swagger.v3.oas.annotations.OpenAPIDefinition. Metadatos OpenAPI de rutas, respuestas y autenticación.

L4

```
import io.swagger.v3.oas.annotations.info.Info;
```

Importa tipos: io.swagger.v3.oas.annotations.info.Info. Metadatos OpenAPI de rutas, respuestas y autenticación.

L5

```
import io.swagger.v3.oas.annotations.servers.Server;
```

Importa tipos: io.swagger.v3.oas.annotations.servers.Server. Metadatos OpenAPI de rutas, respuestas y autenticación.

L6

```
import io.swagger.v3.oas.annotations.security.SecurityScheme;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityScheme. Metadatos OpenAPI de rutas, respuestas y autenticación.

L7

```
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
```

Importa tipos: io.swagger.v3.oas.annotations.enums.SecuritySchemeType. Metadatos OpenAPI de rutas, respuestas y autenticación.

L8

```
@Configuration
```

Declara configuración Spring.

L9

```
@OpenAPIDefinition(info=@Info(title="AuxiRuta Fleet API",version="1.0.0",description="Administración de
flota. Fechas UTC ISO-8601. Listados page>=0 y size 1..100. PATCH modifica campos presentes; null no
borra campos. Borrar teléfono usando cadena vacía. Identidades y roles se administran previamente en
Keycloak."),servers=@Server(url="/"))
```

Metadatos documentales de API.

L10

```
@SecurityScheme(name="bearerAuth",type=SecuritySchemeType.HTTP,scheme="bearer",bearerFormat="JWT")
```

Describe Bearer JWT.

L11

```
public class OpenApiConfiguration {}
```

Declara clase OpenApiConfiguration.

RequestLoggingFilter (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/RequestLoggingFilter.java

SHA-256: 0e80c615a99e53aa872c8284c655e6600a4aced28fa8b985ad31f073453a48f5

Mide duración y registra método y estado HTTP, sin guardar tokens ni cuerpos. Gateway usa una cadena reactiva; los servicios usan el filtro Servlet.

L1

```
package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import jakarta.servlet.*;
```

Importa tipos: jakarta.servlet.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L4

```
import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L5

```
import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo.

L6

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L7

```
import org.springframework.web.filter.OncePerRequestFilter;
```

Importa tipos: org.springframework.web.filter.OncePerRequestFilter. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L9

```
@Component
```

Registra componente inyectable.

L10

```
public class RequestLoggingFilter extends OncePerRequestFilter {
```

Declara clase RequestLoggingFilter.

L11

```
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
throws ServletException, IOException {
```

Filtro Servlet por petición: envuelve el resto de la cadena y mide su resultado incluso si falla.

L12

```
        long start = System.nanoTime();
```

Captura tiempo monotónico inicial para medir duración, no fecha de negocio.

L13

```
        try { chain.doFilter(req,res); }
```

Delega a los filtros/controladores siguientes; finally registrará duración y estado aunque lancen excepción.

L14

```
        finally { LoggerFactory.getLogger(getClass()).debug("HTTP method={} status={}
durationMs={}",req.getMethod(),res.getStatus(),(System.nanoTime()-start)/1_000_000); }
```

Registra los campos indicados, sin token ni secreto.

L15

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L16

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2: separadores sin instrucciones.

SecurityConfiguration (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/SecurityConfiguration.java`

`SHA-256: ef4ca19af0007f707509a6429fdc95b196a836acc0f994d31d3523107a959cd6`

Configura validación JWT y permisos por ruta y método. El primer matcher aplicable decide; las rutas específicas preceden a las generales y lo no enumerado se deniega.

L1

```
package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.Set;
```

Importa tipos: java.util.Set. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía.

L5

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L6

```
import org.springframework.context.annotation.*;
```

Importa tipos: org.springframework.context.annotation.*. Beans, configuración y eventos del contexto Spring.

L7

```
import org.springframework.http.HttpMethod;
```

Importa tipos: org.springframework.http.HttpMethod. Estados, headers, tipos de contenido y respuestas HTTP.

L8

```
import org.springframework.security.authorization.AuthorizationDecision;
```

Importa tipos: org.springframework.security.authorization.AuthorizationDecision. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.oauth2.jwt.*;
```

Importa tipos: org.springframework.security.oauth2.jwt.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.oauth2.server.resource.authentication.*;
```

Importa tipos: org.springframework.security.oauth2.server.resource.authentication.*. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

Importa tipos: org.springframework.security.config.annotation.web.builders.HttpSecurity. Validación JWT, Authentication, autorización y cadenas de seguridad.

L12

```
import org.springframework.security.web.SecurityFilterChain;
```

Importa tipos: org.springframework.security.web.SecurityFilterChain. Validación JWT, Authentication, autorización y cadenas de seguridad.

L13

```
import org.springframework.security.config.http.SessionCreationPolicy;
```

Importa tipos: org.springframework.security.config.http.SessionCreationPolicy. Validación JWT, Authentication, autorización y cadenas de seguridad.

L14

```
import org.springframework.security.authorization.AuthorizationManager;
```

Importa tipos: org.springframework.security.authorization.AuthorizationManager. Validación JWT, Authentication, autorización y cadenas de seguridad.

L15

```
import org.springframework.security.web.access.intercept.RequestAuthorizationContext;
```

Importa tipos: org.springframework.security.web.access.intercept.RequestAuthorizationContext. Validación JWT, Authentication, autorización y cadenas de seguridad.

L17

```
@Configuration
```

Declara configuración Spring.

L18

```
public class SecurityConfiguration {
```

Declara clase SecurityConfiguration.

L19

```
    @Bean
```

Registra el objeto retornado en Spring.

L20

```
    JwtDecoder jwtDecoder(@Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String
issuer,
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Define bean que decodifica y verifica el JWT; las URLs de emisor/claves provienen de configuración.

L21

```
            @Value("${spring.security.oauth2.resourceserver.jwt.jwk-set-uri}") String keys) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición.

L22

```
        var decoder = NimbusJwtDecoder.withJwkSetUri(keys).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Construye decodificador que verifica firma con claves JWKS.

L23

```
        decoder.setJwtValidator(TokenSupport.validator(issuer, Set.of("auxiruta-api",
 "auxiruta-fleet")));
```

Añade validación de emisor, tiempo, sub y audiencia.

L24

```
        return decoder;
```

Entrega el decodificador ya configurado al contenedor Spring.

L25

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L26

```
    @Bean
```

Registra el objeto retornado en Spring.

L27

```
    SecurityFilterChain security(HttpSecurity http, SecurityErrors errors) throws Exception {
```

Construye la cadena Servlet de seguridad que intercepta peticiones antes del controlador.

L28

```
        var converter = new JwtAuthenticationConverter();
```

Crea convertidor de JWT a Authentication para incorporar authorities.

L29

```
        converter.setJwtGrantedAuthoritiesConverter(jwt -> new
 java.util.ArrayList<>(TokenSupport.roles(jwt)));
```

Traduce roles de Keycloak a ROLE_ de Spring.

L30

```
        http.csrf(c -> c.disable()).httpBasic(c -> c.disable()).formLogin(c -> c.disable())
```

Desactiva formulario propio del Gateway; la pantalla de login pertenece a Keycloak. Desactiva CSRF y mecanismos de login propios de esta API Bearer; Keycloak autentica.

L31

```
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

No guarda sesión HTTP; cada petición aporta token.

L32

```
            .exceptionHandling(e -> e.authenticationEntryPoint(errors).accessDeniedHandler(errors))
```

Conecta errores 401/403 con el escritor ProblemDetail propio.

L33

```
            .oauth2ResourceServer(o -> o.authenticationEntryPoint(errors).accessDeniedHandler(errors)
```

Activa procesamiento de Bearer JWT y errores propios.

L34

```
                .jwt(j -> j.jwtAuthenticationConverter(converter)))
```

Configura conversión de claims y roles cuando se valida JWT; en Gateway usa adaptador reactivo.

L35

```
            .authorizeHttpRequests(auth -> auth
```

Inicia reglas de autorización Servlet; la primera coincidencia aplica.

L36

```
                .requestMatchers(HttpMethod.GET, "/api/v1/operators/me").access(allowed(false,
"OPERATOR"))
```

Autoriza GET /api/v1/operators/me a OPERATOR con audiencia/azp válidos.

L37

```
                .requestMatchers(HttpMethod.PATCH,
"/api/v1/operators/me/availability").access(allowed(false, "OPERATOR"))
```

Autoriza PATCH /api/v1/operators/me/availability a OPERATOR con audiencia/azp válidos.

L38

```
                .requestMatchers(HttpMethod.GET,
"/api/v1/tow-trucks/assigned-to-me").access(allowed(false, "OPERATOR"))
```

Autoriza GET /api/v1/tow-trucks/assigned-to-me a OPERATOR con audiencia/azp válidos.

L39

```
                .requestMatchers(HttpMethod.PUT,
"/api/v1/tow-trucks/assigned-to-me/location").access(allowed(false, "OPERATOR"))
```

Autoriza PUT /api/v1/tow-trucks/assigned-to-me/location a OPERATOR con audiencia/azp válidos.

L40

```
                .requestMatchers("/api/v1/operators/me", "/api/v1/operators/me/**",
"/api/v1/tow-trucks/assigned-to-me", "/api/v1/tow-trucks/assigned-to-me/**").denyAll()
```

Deniega /api/v1/operators/me, /api/v1/operators/me/**, /api/v1/tow-trucks/assigned-to-me, /api/v1/tow-trucks/assigned-to-me/**; impide caer en reglas generales.

L41

```
                .requestMatchers(HttpMethod.GET, "/api/v1/operators").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/operators a ADMIN con audiencia/azp válidos.

L42

```
                .requestMatchers(HttpMethod.GET, "/api/v1/operators/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza GET /api/v1/operators/{id} a ADMIN con audiencia/azp válidos.

L43

```
                .requestMatchers(HttpMethod.POST, "/api/v1/operators").access(allowed(false, "ADMIN"))
```

Autoriza POST /api/v1/operators a ADMIN con audiencia/azp válidos.

L44

```
                .requestMatchers(HttpMethod.PATCH, "/api/v1/operators/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza PATCH /api/v1/operators/{id} a ADMIN con audiencia/azp válidos.

L45

```
                .requestMatchers(HttpMethod.PATCH,
"/api/v1/operators/{id}/registration-status").access(allowed(false, "ADMIN"))
```

Autoriza PATCH /api/v1/operators/{id}/registration-status a ADMIN con audiencia/azp válidos.

L46

```
                .requestMatchers(HttpMethod.PUT,
"/api/v1/operators/{id}/qualification").access(allowed(false, "ADMIN"))
```

Autoriza PUT /api/v1/operators/{id}/qualification a ADMIN con audiencia/azp válidos.

L47

```
                .requestMatchers(HttpMethod.GET, "/api/v1/tow-trucks").access(allowed(false, "ADMIN"))
```

Autoriza GET /api/v1/tow-trucks a ADMIN con audiencia/azp válidos.

L48

```
                .requestMatchers(HttpMethod.GET, "/api/v1/tow-trucks/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza GET /api/v1/tow-trucks/{id} a ADMIN con audiencia/azp válidos.

L49

```
                .requestMatchers(HttpMethod.POST, "/api/v1/tow-trucks").access(allowed(false, "ADMIN"))
```

Autoriza POST /api/v1/tow-trucks a ADMIN con audiencia/azp válidos.

L50

```
                .requestMatchers(HttpMethod.PATCH, "/api/v1/tow-trucks/{id}").access(allowed(false,
"ADMIN"))
```

Autoriza PATCH /api/v1/tow-trucks/{id} a ADMIN con audiencia/azp válidos.

L51

```
                .requestMatchers(HttpMethod.PUT,
"/api/v1/tow-trucks/{id}/operator").access(allowed(false, "ADMIN"))
```

Autoriza PUT /api/v1/tow-trucks/{id}/operator a ADMIN con audiencia/azp válidos.

L52

```
                .requestMatchers(HttpMethod.DELETE,
"/api/v1/tow-trucks/{id}/operator").access(allowed(false, "ADMIN"))
```

Autoriza DELETE /api/v1/tow-trucks/{id}/operator a ADMIN con audiencia/azp válidos.

L53

```
                .requestMatchers(HttpMethod.PATCH,
"/api/v1/tow-trucks/{id}/operational-status").access(allowed(false, "ADMIN"))
```

Autoriza PATCH /api/v1/tow-trucks/{id}/operational-status a ADMIN con audiencia/azp válidos.

L54

```
                .requestMatchers(HttpMethod.GET, "/api/v1/configuration/fleet").access(allowed(false,
"ADMIN"))
```

Autoriza GET /api/v1/configuration/fleet a ADMIN con audiencia/azp válidos.

L55

```
                .requestMatchers(HttpMethod.GET, "/internal/fleet/candidates").access(allowed(true,
"FLEET_SERVICE"))
```

Autoriza GET /internal/fleet/candidates a FLEET_SERVICE con audiencia/azp válidos.

L56

```
                .requestMatchers(HttpMethod.PUT, "/internal/reservations/{id}").access(allowed(true,
"FLEET_SERVICE"))
```

Autoriza PUT /internal/reservations/{id} a FLEET_SERVICE con audiencia/azp válidos.

L57

```
                .requestMatchers(HttpMethod.GET, "/internal/reservations/{id}").access(allowed(true,
"FLEET_SERVICE"))
```

Autoriza GET /internal/reservations/{id} a FLEET_SERVICE con audiencia/azp válidos.

L58

```
                .requestMatchers(HttpMethod.POST,
"/internal/reservations/{id}/confirmation").access(allowed(true, "FLEET_SERVICE"))
```

Autoriza POST /internal/reservations/{id}/confirmation a FLEET_SERVICE con audiencia/azp válidos.

L59

```
                .requestMatchers(HttpMethod.POST,
"/internal/reservations/{id}/release").access(allowed(true, "FLEET_SERVICE"))
```

Autoriza POST /internal/reservations/{id}/release a FLEET_SERVICE con audiencia/azp válidos.

L60

```
                .requestMatchers(HttpMethod.GET,
"/internal/reservations/{id}/eligibility").access(allowed(true, "FLEET_SERVICE"))
```

Autoriza GET /internal/reservations/{id}/eligibility a FLEET_SERVICE con audiencia/azp válidos.

L61

```
                .requestMatchers(HttpMethod.POST,
"/internal/reservations/status-queries").access(allowed(true, "FLEET_SERVICE"))
```

Autoriza POST /internal/reservations/status-queries a FLEET_SERVICE con audiencia/azp válidos.

L62

```
                .requestMatchers(HttpMethod.GET, "/v3/api-docs/**").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /v3/api-docs/** a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L63

```
                .requestMatchers(HttpMethod.GET, "/swagger-ui/**").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /swagger-ui/** a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L64

```
                .requestMatchers(HttpMethod.GET, "/swagger-ui.html").access(allowed(false, "DRIVER",
"OPERATOR", "ADMIN"))
```

Autoriza GET /swagger-ui.html a DRIVER, OPERATOR, ADMIN con audiencia/azp válidos.

L65

```
                .anyRequest().denyAll());
```

Deniega todo acceso no autorizado antes.

L66

```
        return http.build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Termina la cadena de seguridad con todos los matchers y handlers configurados.

L67

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L68

```
    private AuthorizationManager<RequestAuthorizationContext> allowed(boolean technical, String...
roles) {
```

Combina autenticación JWT, audiencia, cliente autorizado y roles; no basta con un texto ROLE_ aislado.

L69

```
        return (authentication, context) -> new AuthorizationDecision(
```

Devuelve una función de autorización evaluada por petición, no durante el arranque.

L70

```
            authentication.get() instanceof JwtAuthenticationToken token &&
TokenSupport.permits(token.getToken(), technical, roles));
```

Solo acepta Authentication basada en JWT y delega la combinación de cliente, audiencia y rol en TokenSupport.

L71

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L72

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 16: separadores sin instrucciones.

SecurityErrors (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/config/SecurityErrors.java

SHA-256: ad40b14bbb11927cdda88c16561ce26be59fb3f41cb2f43308b798f532890831

Convierte fallos de autenticación/autorización a respuestas JSON 401/403. Gateway además transforma errores de enrutamiento. No añade un mecanismo de inicio de sesión propio.

L1

```
package com.auxiruta.fleet.config;
```

Ubica el archivo en com.auxiruta.fleet.config. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L5

```
import jakarta.servlet.http.*;
```

Importa tipos: jakarta.servlet.http.*. Tipos de petición, respuesta y cadena de filtros Servlet.

L6

```
import java.io.IOException;
```

Importa tipos: java.io.IOException. Tipo auxiliar usado por las expresiones del archivo.

L7

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L8

```
import org.springframework.security.core.AuthenticationException;
```

Importa tipos: org.springframework.security.core.AuthenticationException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.springframework.security.web.AuthenticationEntryPoint;
```

Importa tipos: org.springframework.security.web.AuthenticationEntryPoint. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import org.springframework.security.web.access.AccessDeniedHandler;
```

Importa tipos: org.springframework.security.web.access.AccessDeniedHandler. Validación JWT, Authentication, autorización y cadenas de seguridad.

L13

```
@Component
```

Registra componente inyectable.

L14

```
public class SecurityErrors implements AuthenticationEntryPoint, AccessDeniedHandler {
```

Declara clase SecurityErrors.

L15

```
    private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
    public SecurityErrors(ObjectMapper mapper) { this.mapper = mapper; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L17

```
    public void commence(HttpServletRequest req, HttpServletResponse res, AuthenticationException e)
throws IOException { write(res, 401); }
```

Una falla de autenticación produce HTTP 401 en Servlet.

L18

```
    public void handle(HttpServletRequest req, HttpServletResponse res, AccessDeniedException e) throws
IOException { write(res, 403); }
```

Una falla de autorización produce HTTP 403 en Servlet.

L19

```
    private void write(HttpServletResponse response, int status) throws IOException {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L20

```
        response.setStatus(status);
```

Establece el código HTTP Servlet.

L21

```
        response.setContentType("application/problem+json");
```

Declara application/problem+json para errores uniformes.

L22

```
        if (status == 401) response.setHeader("WWW-Authenticate", "Bearer");
```

Añade desafío Bearer en 401. La rama condicional selecciona la actualización o retorno correspondiente.

L23

```
        mapper.writeValue(response.getOutputStream(), Problems.of(status));
```

Serializa ProblemDetail directamente en el cuerpo Servlet.

L24

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L25

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 12: separadores sin instrucciones.

ApiExceptionHandler (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/exceptions/ApiExceptionHandler.java`

`SHA-256: af6976efc91d9d1d55f11a20e0ceb23791c600a1b61e412599b11bf0674c03a4`

Normaliza errores de controladores, validación y persistencia. Las restricciones de integridad/concurrencia se exponen como 409, no como detalle SQL.

L1

```
package com.auxiruta.fleet.exceptions;
```

Ubica el archivo en com.auxiruta.fleet.exceptions. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import com.auxiruta.commons.errors.Problems;
```

Importa tipos: com.auxiruta.commons.errors.Problems. Utilidades compartidas de seguridad, errores, reloj o geografía.

L4

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L5

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
```

Importa tipos: org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler. Controladores, errores HTTP y clientes REST según la clase importada.

L7

```
import org.springframework.web.context.request.WebRequest;
```

Importa tipos: org.springframework.web.context.request.WebRequest. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L9

```
import org.springframework.security.access.AccessDeniedException;
```

Importa tipos: org.springframework.security.access.AccessDeniedException. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L12

```
@RestControllerAdvice
```

Centraliza transformación de excepciones.

L13

```
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
```

Declara clase ApiExceptionHandler.

L14

```
    @Override
```

Redefine contrato heredado.

L15

```
    protected ResponseEntity<Object> handleExceptionInternal(Exception ex, Object body, HttpHeaders
 headers, HttpStatusCode status, WebRequest request) {
```

Interviene en errores MVC de validación/parsing y preserva encabezados/estado.

L16

```
        return new ResponseEntity<>(Problems.of(status.value()), headers, status);
```

Reemplaza el cuerpo de error por el formato común conservando estado y headers.

L17

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L18

```
    @ExceptionHandler(ResponseStatusException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L19

```
    ResponseEntity<ProblemDetail> status(ResponseStatusException e) { return
 ResponseEntity.status(e.getStatusCode()).body(Problems.of(e.getStatusCode().value())); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L20

```
    @ExceptionHandler(AccessDeniedException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L21

```
    ResponseEntity<ProblemDetail> forbidden(AccessDeniedException e) { return
 ResponseEntity.status(403).body(Problems.of(403)); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L22

```
    @ExceptionHandler({org.springframework.dao.DataIntegrityViolationException.class,
 org.springframework.dao.ConcurrencyFailureException.class})
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L23

```
    ResponseEntity<ProblemDetail> conflict(Exception e) { return
 ResponseEntity.status(409).body(Problems.of(409)); }
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L24

```
    @ExceptionHandler(jakarta.validation.ConstraintViolationException.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L25

```
    ResponseEntity<ProblemDetail> invalidConstraint(Exception e) { return
 ResponseEntity.badRequest().body(Problems.of(400)); }
```

Devuelve 400 ante validación de restricción inválida.

L26

```
    @ExceptionHandler(Exception.class)
```

Asocia el método siguiente al tipo de excepción indicado; no captura errores de otros procesos.

L27

```
    ResponseEntity<ProblemDetail> unexpected(Exception e) {
```

Captura errores no clasificados para devolver 500 genérico y registrar solo tipo de excepción.

L28

```
        LoggerFactory.getLogger(getClass()).error("Error inesperado tipo={}",
e.getClass().getSimpleName());
```

Registra los campos indicados, sin token ni secreto.

L29

```
        return ResponseEntity.status(500).body(Problems.of(500));
```

Construye respuesta con estado explícito y cuerpo ProblemDetail.

L30

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L31

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 11: separadores sin instrucciones.

application-docker (fleet-service)

`Ruta: fleet-service/src/main/resources/application-docker.yml`

`SHA-256: d7169f4b65e321f359da990392d6c87d1c00ed2d22a0ed6fc4025517a912cc01`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
  security:
```

Abre bloque security; sus hijos configuran este componente.

L3

```
    oauth2:
```

Abre bloque oauth2; sus hijos configuran este componente.

L4

```
      resourceserver:
```

Abre bloque resourceserver; sus hijos configuran este componente.

L5

```
        jwt:
```

Abre bloque jwt; sus hijos configuran este componente.

L6

```
          issuer-uri: ${KEYCLOAK_ISSUER_URI}
```

Emisor esperado del JWT. Valor: ${KEYCLOAK_ISSUER_URI}.

L7

```
          jwk-set-uri: ${KEYCLOAK_JWK_SET_URI}
```

URL de claves públicas para verificar firma. Valor: ${KEYCLOAK_JWK_SET_URI}.

L8

```
  datasource:
```

Abre bloque datasource; sus hijos configuran este componente.

L9

```
    url: ${DB_URL}
```

Dirección de conexión o servidor del bloque. Valor: ${DB_URL}.

L10

```
    username: ${DB_USER}
```

Usuario de conexión o identidad. Valor: ${DB_USER}.

L11

```
    password: ${DB_PASSWORD}
```

Referencia de contraseña; no reproduce el secreto local. Valor: ${DB_PASSWORD}.

L12

```
  sql:
```

Abre bloque sql; sus hijos configuran este componente.

L13

```
    init:
```

Abre bloque init; sus hijos configuran este componente.

L14

```
      mode: always
```

always aplica inicialización SQL también a PostgreSQL. Valor: always.

L15

```
      schema-locations:
classpath:db/001-fleet-catalog.sql,classpath:db/002-reservations.sql,classpath:db/003-automation.sql
```

Scripts SQL ordenados para inicializar/extender esquema. Valor: classpath:db/001-fleet-catalog.sql,classpath:db/002-reservations.sql,classpath:db/003-automation.sql.

L16

```
  jpa:
```

Abre bloque jpa; sus hijos configuran este componente.

L17

```
    open-in-view: false
```

false evita mantener EntityManager durante serialización HTTP. Valor: false.

L18

```
    hibernate:
```

Abre bloque hibernate; sus hijos configuran este componente.

L19

```
      ddl-auto: validate
```

validate comprueba el esquema; Hibernate no crea tablas. Valor: validate.

application (fleet-service)

`Ruta: fleet-service/src/main/resources/application.yml`

`SHA-256: b17184f8451ba691381128b4a644c577c38bf59949214a9d1883905147fc49b0`

Configuración de arranque y dependencias del módulo. Las variables se resuelven desde el entorno al iniciar; no contiene secretos reales.

L1

```
spring:
```

Abre bloque spring; sus hijos configuran este componente.

L2

```
  jackson:
```

Abre bloque jackson; sus hijos configuran este componente.

L3

```
    deserialization:
```

Abre bloque deserialization; sus hijos configuran este componente.

L4

```
      fail-on-unknown-properties: true
```

true rechaza campos JSON desconocidos. Valor: true.

L5

```
  application:
```

Abre bloque application; sus hijos configuran este componente.

L6

```
    name: fleet-service
```

Establece name = fleet-service dentro del bloque. Se interpreta al cargar la configuración.

L7

```
server:
```

Abre bloque server; sus hijos configuran este componente.

L8

```
  port: ${SERVER_PORT:8081}
```

Puerto de escucha; Compose controla su exposición al host. Valor: ${SERVER_PORT:8081}.

L10

```
springdoc:
```

Abre bloque springdoc; sus hijos configuran este componente.

L11

```
  paths-to-match: /api/v1/**
```

Patrones incluidos en OpenAPI público. Valor: /api/v1/**.

L13

```
# SQL constraint messages may contain personal data; API errors are handled centrally.
```

Comentario o directiva del intérprete; no es regla de negocio.

L14

```
logging:
```

Abre bloque logging; sus hijos configuran este componente.

L15

```
  level:
```

Abre bloque level; sus hijos configuran este componente.

L16

```
    org.hibernate.engine.jdbc.spi.SqlExceptionHelper: "OFF"
```

Establece org.hibernate.engine.jdbc.spi.SqlExceptionHelper = "OFF" dentro del bloque. Se interpreta al cargar la configuración.

L18

```
auxiruta:
```

Abre bloque auxiruta; sus hijos configuran este componente.

L19

```
  automation:
```

Abre bloque automation; sus hijos configuran este componente.

L20

```
    enabled: ${AUXIRUTA_AUTOMATION_ENABLED:true}
```

Activa el componente o identidad del bloque. Valor: ${AUXIRUTA_AUTOMATION_ENABLED:true}.

Líneas vacías 9, 12, 17: separadores sin instrucciones.

auxiruta-realm (infra)

`Ruta: infra/keycloak/auxiruta-realm.json`

`SHA-256: 2b4f78b03c0c56da61157f391b25191d0c9d75db3752f6896fc46dcce0e26cd3`

Importación inicial de Keycloak: realm, roles, clientes, scopes, audiencias y cuentas demo. Los valores secretos son referencias de entorno. Un realm persistido no se reimporta al reiniciar.

L1

```
{
```

Delimita objeto, arreglo o bloque del archivo.

L2

```
  "realm": "auxiruta",
```

Establece realm = "auxiruta" dentro del bloque. Se interpreta al cargar la configuración.

L3

```
  "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L4

```
  "sslRequired": "external",
```

Establece sslRequired = "external" dentro del bloque. Se interpreta al cargar la configuración.

L5

```
  "registrationAllowed": false,
```

Establece registrationAllowed = false dentro del bloque. Se interpreta al cargar la configuración.

L6

```
  "resetPasswordAllowed": false,
```

Establece resetPasswordAllowed = false dentro del bloque. Se interpreta al cargar la configuración.

L7

```
  "roles": {
```

Abre bloque roles; sus hijos configuran este componente.

L8

```
    "realm": [
```

Abre bloque realm; sus hijos configuran este componente.

L9

```
      {
```

Delimita objeto, arreglo o bloque del archivo.

L10

```
        "name": "DRIVER"
```

Establece name = "DRIVER" dentro del bloque. Se interpreta al cargar la configuración.

L11

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L12

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L13

```
      "name": "OPERATOR"
```

Establece name = "OPERATOR" dentro del bloque. Se interpreta al cargar la configuración.

L14

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L15

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L16

```
      "name": "ADMIN"
```

Establece name = "ADMIN" dentro del bloque. Se interpreta al cargar la configuración.

L17

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L18

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L19

```
      "name": "FLEET_SERVICE"
```

Establece name = "FLEET_SERVICE" dentro del bloque. Se interpreta al cargar la configuración.

L20

```
    }
```

Delimita objeto, arreglo o bloque del archivo.

L21

```
  ]
```

Delimita objeto, arreglo o bloque del archivo.

L22

```
},
```

Delimita objeto, arreglo o bloque del archivo.

L23

```
"clients": [
```

Abre bloque clients; sus hijos configuran este componente.

L24

```
  {
```

Delimita objeto, arreglo o bloque del archivo.

L25

```
    "clientId": "auxiruta-api",
```

Establece clientId = "auxiruta-api" dentro del bloque. Se interpreta al cargar la configuración.

L26

```
    "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L27

```
    "publicClient": true,
```

true significa cliente sin secreto para login humano. Valor: true.

L28

```
    "standardFlowEnabled": true,
```

Habilita Authorization Code. Valor: true.

L29

```
    "directAccessGrantsEnabled": false,
```

Controla password grant; el humano utiliza PKCE. Valor: false.

L30

```
    "fullScopeAllowed": false,
```

false restringe roles a los mappings configurados. Valor: false.

L31

```
    "redirectUris": [
```

Abre bloque redirectUris; sus hijos configuran este componente.

L32

```
      "https://oauth.pstmn.io/v1/callback",
```

Establece https = //oauth.pstmn.io/v1/callback" dentro del bloque. Se interpreta al cargar la configuración.

L33

```
      "http://127.0.0.1:8765/callback"
```

Establece http = //127.0.0.1:8765/callback" dentro del bloque. Se interpreta al cargar la configuración.

L34

```
    ],
```

Delimita objeto, arreglo o bloque del archivo.

L35

```
    "webOrigins": [],
```

Orígenes web admitidos. Valor: [].

L36

```
    "attributes": {
```

Abre bloque attributes; sus hijos configuran este componente.

L37

```
      "pkce.code.challenge.method": "S256"
```

Establece pkce.code.challenge.method = "S256" dentro del bloque. Se interpreta al cargar la configuración.

L38

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L39

```
      "defaultClientScopes": [
```

Abre bloque defaultClientScopes; sus hijos configuran este componente.

L40

```
        "profile",
```

Valor "profile" de la lista de roles/callbacks/scopes del bloque superior.

L41

```
        "email",
```

Valor "email" de la lista de roles/callbacks/scopes del bloque superior.

L42

```
        "roles",
```

Valor "roles" de la lista de roles/callbacks/scopes del bloque superior.

L43

```
        "basic"
```

Valor "basic" de la lista de roles/callbacks/scopes del bloque superior.

L44

```
      ],
```

Delimita objeto, arreglo o bloque del archivo.

L45

```
      "protocolMappers": [
```

Abre bloque protocolMappers; sus hijos configuran este componente.

L46

```
        {
```

Delimita objeto, arreglo o bloque del archivo.

L47

```
          "name": "audience-auxiruta-api",
```

Establece name = "audience-auxiruta-api" dentro del bloque. Se interpreta al cargar la configuración.

L48

```
          "protocol": "openid-connect",
```

Establece protocol = "openid-connect" dentro del bloque. Se interpreta al cargar la configuración.

L49

```
          "protocolMapper": "oidc-audience-mapper",
```

Establece protocolMapper = "oidc-audience-mapper" dentro del bloque. Se interpreta al cargar la configuración.

L50

```
          "config": {
```

Abre bloque config; sus hijos configuran este componente.

L51

```
            "included.custom.audience": "auxiruta-api",
```

Audiencia añadida. Valor: "auxiruta-api".

L52

```
            "access.token.claim": "true",
```

Incluye claim en access token. Valor: "true".

L53

```
            "id.token.claim": "false"
```

Controla inclusión en ID token; no es access token. Valor: "false".

L54

```
          }
```

Delimita objeto, arreglo o bloque del archivo.

L55

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L56

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L57

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L58

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L59

```
      "clientId": "assistance-service",
```

Establece clientId = "assistance-service" dentro del bloque. Se interpreta al cargar la configuración.

L60

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L61

```
      "publicClient": false,
```

true significa cliente sin secreto para login humano. Valor: false.

L62

```
      "secret": "${KEYCLOAK_CLIENT_SECRET}",
```

Configura secret desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L63

```
      "serviceAccountsEnabled": true,
```

Permite client_credentials. Valor: true.

L64

```
      "standardFlowEnabled": false,
```

Habilita Authorization Code. Valor: false.

L65

```
      "directAccessGrantsEnabled": false,
```

Controla password grant; el humano utiliza PKCE. Valor: false.

L66

```
      "fullScopeAllowed": false,
```

false restringe roles a los mappings configurados. Valor: false.

L67

```
    "defaultClientScopes": [
```

Abre bloque defaultClientScopes; sus hijos configuran este componente.

L68

```
      "roles",
```

Valor "roles" de la lista de roles/callbacks/scopes del bloque superior.

L69

```
      "basic"
```

Valor "basic" de la lista de roles/callbacks/scopes del bloque superior.

L70

```
    ],
```

Delimita objeto, arreglo o bloque del archivo.

L71

```
    "protocolMappers": [
```

Abre bloque protocolMappers; sus hijos configuran este componente.

L72

```
      {
```

Delimita objeto, arreglo o bloque del archivo.

L73

```
        "name": "audience-auxiruta-fleet",
```

Establece name = "audience-auxiruta-fleet" dentro del bloque. Se interpreta al cargar la configuración.

L74

```
        "protocol": "openid-connect",
```

Establece protocol = "openid-connect" dentro del bloque. Se interpreta al cargar la configuración.

L75

```
        "protocolMapper": "oidc-audience-mapper",
```

Establece protocolMapper = "oidc-audience-mapper" dentro del bloque. Se interpreta al cargar la configuración.

L76

```
        "config": {
```

Abre bloque config; sus hijos configuran este componente.

L77

```
          "included.custom.audience": "auxiruta-fleet",
```

Audiencia añadida. Valor: "auxiruta-fleet".

L78

```
          "access.token.claim": "true",
```

Incluye claim en access token. Valor: "true".

L79

```
          "id.token.claim": "false"
```

Controla inclusión en ID token; no es access token. Valor: "false".

L80

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L81

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L82

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L83

```
    }
```

Delimita objeto, arreglo o bloque del archivo.

L84

```
  ],
```

Delimita objeto, arreglo o bloque del archivo.

L85

```
  "scopeMappings": [
```

Abre bloque scopeMappings; sus hijos configuran este componente.

L86

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L87

```
      "client": "auxiruta-api",
```

Establece client = "auxiruta-api" dentro del bloque. Se interpreta al cargar la configuración.

L88

```
      "roles": [
```

Abre bloque roles; sus hijos configuran este componente.

L89

```
        "DRIVER",
```

Valor "DRIVER" de la lista de roles/callbacks/scopes del bloque superior.

L90

```
        "OPERATOR",
```

Valor "OPERATOR" de la lista de roles/callbacks/scopes del bloque superior.

L91

```
        "ADMIN"
```

Valor "ADMIN" de la lista de roles/callbacks/scopes del bloque superior.

L92

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L93

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L94

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L95

```
      "client": "assistance-service",
```

Establece client = "assistance-service" dentro del bloque. Se interpreta al cargar la configuración.

L96

```
      "roles": [
```

Abre bloque roles; sus hijos configuran este componente.

L97

```
        "FLEET_SERVICE"
```

Valor "FLEET_SERVICE" de la lista de roles/callbacks/scopes del bloque superior.

L98

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L99

```
    }
```

Delimita objeto, arreglo o bloque del archivo.

L100

```
  ],
```

Delimita objeto, arreglo o bloque del archivo.

L101

```
  "users": [
```

Abre bloque users; sus hijos configuran este componente.

L102

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L103

```
      "username": "service-account-assistance-service",
```

Usuario de conexión o identidad. Valor: "service-account-assistance-service".

L104

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L105

```
      "serviceAccountClientId": "assistance-service",
```

Cliente dueño de la cuenta técnica. Valor: "assistance-service".

L106

```
      "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L107

```
        "FLEET_SERVICE"
```

Valor "FLEET_SERVICE" de la lista de roles/callbacks/scopes del bloque superior.

L108

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L109

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L110

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L111

```
      "username": "driver.demo",
```

Usuario de conexión o identidad. Valor: "driver.demo".

L112

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L113

```
      "emailVerified": true,
```

Correo demo considerado verificado; no envía correo. Valor: true.

L114

```
      "firstName": "Driver",
```

Establece firstName = "Driver" dentro del bloque. Se interpreta al cargar la configuración.

L115

```
      "lastName": "Demo",
```

Establece lastName = "Demo" dentro del bloque. Se interpreta al cargar la configuración.

L116

```
      "email": "driver@auxiruta.example",
```

Establece email = "driver@auxiruta.example" dentro del bloque. Se interpreta al cargar la configuración.

L117

```
      "credentials": [
```

Abre bloque credentials; sus hijos configuran este componente.

L118

```
        {
```

Delimita objeto, arreglo o bloque del archivo.

L119

```
          "type": "password",
```

Establece type = "password" dentro del bloque. Se interpreta al cargar la configuración.

L120

```
          "value": "${DEMO_DRIVER_PASSWORD}",
```

Configura value desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L121

```
          "temporary": false
```

false evita cambio forzado de clave en demo. Valor: false.

L122

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L123

```
    ],
```

Delimita objeto, arreglo o bloque del archivo.

L124

```
    "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L125

```
      "DRIVER"
```

Valor "DRIVER" de la lista de roles/callbacks/scopes del bloque superior.

L126

```
    ]
```

Delimita objeto, arreglo o bloque del archivo.

L127

```
  },
```

Delimita objeto, arreglo o bloque del archivo.

L128

```
  {
```

Delimita objeto, arreglo o bloque del archivo.

L129

```
    "username": "operator.demo",
```

Usuario de conexión o identidad. Valor: "operator.demo".

L130

```
    "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L131

```
    "emailVerified": true,
```

Correo demo considerado verificado; no envía correo. Valor: true.

L132

```
    "firstName": "Operator",
```

Establece firstName = "Operator" dentro del bloque. Se interpreta al cargar la configuración.

L133

```
    "lastName": "Demo",
```

Establece lastName = "Demo" dentro del bloque. Se interpreta al cargar la configuración.

L134

```
    "email": "operator@auxiruta.example",
```

Establece email = "operator@auxiruta.example" dentro del bloque. Se interpreta al cargar la configuración.

L135

```
    "credentials": [
```

Abre bloque credentials; sus hijos configuran este componente.

L136

```
      {
```

Delimita objeto, arreglo o bloque del archivo.

L137

```
          "type": "password",
```

Establece type = "password" dentro del bloque. Se interpreta al cargar la configuración.

L138

```
          "value": "${DEMO_OPERATOR_PASSWORD}",
```

Configura value desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L139

```
          "temporary": false
```

false evita cambio forzado de clave en demo. Valor: false.

L140

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L141

```
      ],
```

Delimita objeto, arreglo o bloque del archivo.

L142

```
      "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L143

```
        "OPERATOR"
```

Valor "OPERATOR" de la lista de roles/callbacks/scopes del bloque superior.

L144

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L145

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L146

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L147

```
      "username": "admin.demo",
```

Usuario de conexión o identidad. Valor: "admin.demo".

L148

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L149

```
      "emailVerified": true,
```

Correo demo considerado verificado; no envía correo. Valor: true.

L150

```
      "firstName": "Admin",
```

Establece firstName = "Admin" dentro del bloque. Se interpreta al cargar la configuración.

L151

```
      "lastName": "Demo",
```

Establece lastName = "Demo" dentro del bloque. Se interpreta al cargar la configuración.

L152

```
      "email": "admin@auxiruta.example",
```

Establece email = "admin@auxiruta.example" dentro del bloque. Se interpreta al cargar la configuración.

L153

```
      "credentials": [
```

Abre bloque credentials; sus hijos configuran este componente.

L154

```
        {
```

Delimita objeto, arreglo o bloque del archivo.

L155

```
          "type": "password",
```

Establece type = "password" dentro del bloque. Se interpreta al cargar la configuración.

L156

```
          "value": "${DEMO_ADMIN_PASSWORD}",
```

Configura value desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L157

```
          "temporary": false
```

false evita cambio forzado de clave en demo. Valor: false.

L158

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L159

```
      ],
```

Delimita objeto, arreglo o bloque del archivo.

L160

```
      "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L161

```
        "ADMIN"
```

Valor "ADMIN" de la lista de roles/callbacks/scopes del bloque superior.

L162

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L163

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L164

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L165

```
      "username": "driver2.demo",
```

Usuario de conexión o identidad. Valor: "driver2.demo".

L166

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L167

```
      "emailVerified": true,
```

Correo demo considerado verificado; no envía correo. Valor: true.

L168

```
      "firstName": "Driver",
```

Establece firstName = "Driver" dentro del bloque. Se interpreta al cargar la configuración.

L169

```
      "lastName": "Demo 2",
```

Establece lastName = "Demo 2" dentro del bloque. Se interpreta al cargar la configuración.

L170

```
      "email": "driver2@auxiruta.example",
```

Establece email = "driver2@auxiruta.example" dentro del bloque. Se interpreta al cargar la configuración.

L171

```
      "credentials": [
```

Abre bloque credentials; sus hijos configuran este componente.

L172

```
        {
```

Delimita objeto, arreglo o bloque del archivo.

L173

```
          "type": "password",
```

Establece type = "password" dentro del bloque. Se interpreta al cargar la configuración.

L174

```
          "value": "${DEMO_DRIVER_PASSWORD}",
```

Configura value desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L175

```
          "temporary": false
```

false evita cambio forzado de clave en demo. Valor: false.

L176

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L177

```
      ],
```

Delimita objeto, arreglo o bloque del archivo.

L178

```
      "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L179

```
        "DRIVER"
```

Valor "DRIVER" de la lista de roles/callbacks/scopes del bloque superior.

L180

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L181

```
    },
```

Delimita objeto, arreglo o bloque del archivo.

L182

```
    {
```

Delimita objeto, arreglo o bloque del archivo.

L183

```
      "username": "operator2.demo",
```

Usuario de conexión o identidad. Valor: "operator2.demo".

L184

```
      "enabled": true,
```

Activa el componente o identidad del bloque. Valor: true.

L185

```
      "emailVerified": true,
```

Correo demo considerado verificado; no envía correo. Valor: true.

L186

```
      "firstName": "Operator",
```

Establece firstName = "Operator" dentro del bloque. Se interpreta al cargar la configuración.

L187

```
      "lastName": "Demo 2",
```

Establece lastName = "Demo 2" dentro del bloque. Se interpreta al cargar la configuración.

L188

```
      "email": "operator2@auxiruta.example",
```

Establece email = "operator2@auxiruta.example" dentro del bloque. Se interpreta al cargar la configuración.

L189

```
      "credentials": [
```

Abre bloque credentials; sus hijos configuran este componente.

L190

```
        {
```

Delimita objeto, arreglo o bloque del archivo.

L191

```
          "type": "password",
```

Establece type = "password" dentro del bloque. Se interpreta al cargar la configuración.

L192

```
          "value": "${DEMO_OPERATOR_PASSWORD}",
```

Configura value desde entorno. :? exige valor; :- o : indican default según Compose/Spring. No muestra el secreto resuelto.

L193

```
          "temporary": false
```

false evita cambio forzado de clave en demo. Valor: false.

L194

```
        }
```

Delimita objeto, arreglo o bloque del archivo.

L195

```
      ],
```

Delimita objeto, arreglo o bloque del archivo.

L196

```
      "realmRoles": [
```

Abre bloque realmRoles; sus hijos configuran este componente.

L197

```
        "OPERATOR"
```

Valor "OPERATOR" de la lista de roles/callbacks/scopes del bloque superior.

L198

```
      ]
```

Delimita objeto, arreglo o bloque del archivo.

L199

```
    }
```

Delimita objeto, arreglo o bloque del archivo.

L200

```
  ]
```

Delimita objeto, arreglo o bloque del archivo.

L201

```
}
```

Delimita objeto, arreglo o bloque del archivo.

4. Persistencia: entidades, repositorios y SQL

Cada servicio administra su base. UUID remotos no son FK entre bases. Keycloak administra sus tablas internas, no definidas por Java del proyecto.

AssistanceRequest (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssistanceRequest.java

SHA-256: b89954f4bdb7ed1180eb5319a4e0e373611a3c597453ffc3d17ca7c51f49d143

Entidad central con estado, ubicación, datos declarados del vehículo, primera aceptación y snapshot de reglas. Los campos de automatización permiten reanudar después de reinicios.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L7

```
@Entity @Table(name="assistance_requests",uniqueConstraints=@UniqueConstraint(columnNames={"driver_subje
ct","idempotency_key"}))
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L8

```
public class AssistanceRequest {
```

Declara clase AssistanceRequest.

L9

```
  protected AssistanceRequest() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L11

```
  @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia.

L12

```
@Column(nullable=false) Instant createdAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdAt: fecha de creación.

L13

```
@Column(nullable=false) Instant updatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: updatedAt: fecha de modificación.

L14

```
public UUID getId() { return id; }
```

Expone el UUID de la entidad; permite acceder mediante proxy sin manipular el campo directamente.

L15

```
@ManyToOne(fetch=FetchType.LAZY,optional=false) @JoinColumn(name="vehicle_id") Vehicle vehicle;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L16

```
@Column(nullable=false,length=100) String driverSubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: driverSubject: sub del conductor que creó la solicitud.

L17

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=30) Status status=Status.REQUESTED;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L18

```
Instant firstAssignedAt;
```

Campos: firstAssignedAt: primera aceptación que impide cancelar incluso durante reemplazos.

L19

```
@Column(length=100) String closureReasonCode;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: closureReasonCode: causa codificada del cierre.

L20

```
@Column(nullable=false,precision=9,scale=6) BigDecimal originLatitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: originLatitude: latitud de recogida.

L21

```
@Column(nullable=false,precision=9,scale=6) BigDecimal originLongitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: originLongitude: longitud de recogida.

L22

```
@Column(nullable=false,precision=9,scale=6) BigDecimal destinationLatitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: destinationLatitude: latitud de entrega.

L23

```
@Column(nullable=false,precision=9,scale=6) BigDecimal destinationLongitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: destinationLongitude: longitud de entrega.

L24

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) VehicleType declaredVehicleType;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: declaredVehicleType: copia del tipo de vehículo al solicitar.

L25

```
@Column(nullable=false,precision=9,scale=2) BigDecimal declaredWeightKg;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: declaredWeightKg: peso declarado en kg.

L26

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Mobility mobilityCondition;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: mobilityCondition: condición ROLLABLE/NON_ROLLABLE.

L27

```
@Column(nullable=false,length=1000) String description;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: description: descripción de asistencia.

L28

```
@Column(nullable=false) int searchAttemptNumber=1;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: searchAttemptNumber: número de búsqueda de 1 a 3.

L29

```
@Column(nullable=false) Instant searchDeadline;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: searchDeadline: vencimiento del intento.

L30

```
@Column(nullable=false,length=30) String configurationRevision;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: configurationRevision: versión de reglas.

L31

```
@Column(nullable=false,columnDefinition="text") String configurationSnapshot;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: configurationSnapshot: JSON de parámetros vigente al crear.

L32

```
@Column(nullable=false,length=100) String idempotencyKey;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: idempotencyKey: clave para repetir sin duplicar.

L33

```
@Column(nullable=false,length=64) String requestFingerprint;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: requestFingerprint: SHA-256 del contenido asociado a la clave.

L34

```
@Column(nullable=false) boolean resourceReleasePending;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: resourceReleasePending: liberación remota pendiente que impide reutilización.

L35

```
  @org.hibernate.annotations.OptimisticLock(excluded=true) @Column(nullable=false) Instant
nextEvaluationAt=Instant.EPOCH;
```

excluded=true evita incrementar versión por este campo de planificación. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: nextEvaluationAt: próximo trabajo de evaluación.

L36

```
  @org.hibernate.annotations.OptimisticLock(excluded=true) @Column(nullable=false) Instant
nextReconciliationAt=Instant.EPOCH;
```

excluded=true evita incrementar versión por este campo de planificación. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: nextReconciliationAt: próxima revisión del compromiso.

L37

```
  @org.hibernate.annotations.OptimisticLock(excluded=true) @Column(nullable=false) Instant
evaluationLeaseUntil=Instant.EPOCH;
```

excluded=true evita incrementar versión por este campo de planificación. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Usa 1970-01-01 UTC como planificación inicialmente vencida. Campos: evaluationLeaseUntil: límite de trabajo de evaluación en curso.

L38

```
  @Column(nullable=false) boolean replacementPending;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: replacementPending: necesidad de iniciar reemplazo después de liberar.

L39

```
  Instant closedAt;
```

Campos: closedAt: fecha de cierre; null indica abierto.

L40

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

CandidateEvaluation (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/CandidateEvaluation.java`

`SHA-256: 3b16748c128ef7389ef94e46af5adab37f5d5f651a7c65d669dc1b738a6012cb`

Snapshot por candidato de compatibilidad, elegibilidad, recorridos y posición en el ranking. Referencia UUID remotos sin FK hacia Fleet.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
@Entity @Table(name="candidate_evaluations")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L7

```
public class CandidateEvaluation {
```

Declara clase CandidateEvaluation.

L8

```
  protected CandidateEvaluation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L10

```
  @ManyToOne(fetch=FetchType.LAZY,optional=false) @JoinColumn(name="evaluation_id") Evaluation
evaluation;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L11

```
  @Column(nullable=false) UUID towTruckId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: towTruckId: UUID de grúa.

L12

```
  @Column(nullable=false) UUID operatorId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operatorId: UUID del perfil operador.

L13

```
  @Column(nullable=false,length=100) String operatorSubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operatorSubject: sub del operador destinatario.

L14

```
  @Column(nullable=false,precision=9,scale=2) BigDecimal truckCapacityKg;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: truckCapacityKg: snapshot de capacidad.

L15

```
  @Column(nullable=false,length=30) String qualificationCategory;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: qualificationCategory: habilitación LIGHT_TOWING.

L16

```
  @Column(nullable=false) Instant qualificationValidUntil;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L17

```
  @Column(nullable=false,precision=10,scale=7) BigDecimal truckLatitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: truckLatitude: snapshot de latitud de grúa.

L18

```
@Column(nullable=false,precision=10,scale=7) BigDecimal truckLongitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: truckLongitude: snapshot de longitud de grúa.

L19

```
@Column(nullable=false) Instant truckLocationUpdatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: truckLocationUpdatedAt: fecha de posición usada al evaluar.

L20

```
@Column(nullable=false) boolean compatible;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: compatible: compatibilidad de capacidad, tipo y movilidad.

L21

```
@Column(nullable=false) boolean eligible;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: eligible: aptitud completa del candidato/compromiso.

L22

```
@Column(nullable=true,length=100) String reasonCode;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: reasonCode: causa codificada.

L23

```
@Column(name="approach_distance_m",columnDefinition="numeric") BigDecimal approachDistanceM;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: approachDistanceM: distancia vial grúa-recogida.

L24

```
@Column(nullable=true,columnDefinition="numeric") BigDecimal approachDurationSeconds;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: approachDurationSeconds: duración original de aproximación.

L25

```
@Column(nullable=true) Integer adjustedApproachSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: adjustedApproachSeconds: aproximación ajustada.

L26

```
@Column(nullable=true) Instant approachRouteObtainedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: approachRouteObtainedAt: fecha de ruta de aproximación.

L27

```
@Column(name="transport_distance_m",columnDefinition="numeric") BigDecimal transportDistanceM;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: transportDistanceM: distancia vial origen-destino en metros.

L28

```
@Column(nullable=true,columnDefinition="numeric") BigDecimal transportDurationSeconds;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: transportDurationSeconds: duración vial original del traslado.

L29

```
  @Column(nullable=true) Integer adjustedTransportSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: adjustedTransportSeconds: duración ajustada y redondeada hacia arriba.

L30

```
  @Column(nullable=true) Instant transportRouteObtainedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: transportRouteObtainedAt: fecha de ruta de traslado.

L31

```
  @Column(nullable=true) Instant estimatedServiceEndAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: estimatedServiceEndAt: fin planificado para controlar habilitación.

L32

```
  @Column(nullable=true) Integer rankingPosition;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: rankingPosition: posición entre elegibles, comenzando en 1.

L33

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

CandidateEvaluationRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/CandidateEvaluationRepository.java`

`SHA-256: 8aa2411c609b41f153b6b5dc36b2a687db9b9a0e12811cebb66b259ea6a915ad`

Interfaz Spring Data JPA para CandidateEvaluation. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
public interface CandidateEvaluationRepository extends JpaRepository<CandidateEvaluation,UUID> {
```

Declara interfaz CandidateEvaluationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L5

```
  List<CandidateEvaluation> findByEvaluationId(UUID id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L6

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Evaluation (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/Evaluation.java`

`SHA-256: a75c97179dd3765991e2083989b964bb2fb07b1ac3ee0f58fc24d2d23b3881dd`

Entidad del resultado de una evaluación, con distancias, tiempos, motivos y datos usados. No equivale a una asignación aceptada.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
@Entity @Table(name="evaluations")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L7

```
public class Evaluation {
```

Declara clase Evaluation.

L8

```
  protected Evaluation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L10

```
  @ManyToOne(fetch=FetchType.LAZY,optional=false) @JoinColumn(name="request_id") AssistanceRequest
request;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L11

```
  @Column(nullable=false) int searchAttemptNumber;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: searchAttemptNumber: número de búsqueda de 1 a 3.

L12

```
@Column(nullable=false,precision=9,scale=6) BigDecimal originLatitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: originLatitude: latitud de recogida.

L13

```
@Column(nullable=false,precision=9,scale=6) BigDecimal originLongitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: originLongitude: longitud de recogida.

L14

```
@Column(nullable=false,precision=9,scale=6) BigDecimal destinationLatitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: destinationLatitude: latitud de entrega.

L15

```
@Column(nullable=false,precision=9,scale=6) BigDecimal destinationLongitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: destinationLongitude: longitud de entrega.

L16

```
@Column(name="transport_distance_m",columnDefinition="numeric") BigDecimal transportDistanceM;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: transportDistanceM: distancia vial origen-destino en metros.

L17

```
@Column(nullable=true,columnDefinition="numeric") BigDecimal transportDurationSeconds;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: transportDurationSeconds: duración vial original del traslado.

L18

```
@Column(nullable=true) Integer adjustedTransportSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: adjustedTransportSeconds: duración ajustada y redondeada hacia arriba.

L19

```
@Column(nullable=true) Instant routeObtainedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: routeObtainedAt: fecha del recorrido.

L20

```
@Column(nullable=false,length=20) String vehicleType;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: vehicleType: tipo CAR/SUV/PICKUP.

L21

```
@Column(nullable=false,precision=9,scale=2) BigDecimal declaredWeightKg;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: declaredWeightKg: peso declarado en kg.

L22

```
@Column(nullable=false,length=20) String mobilityCondition;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: mobilityCondition: condición ROLLABLE/NON_ROLLABLE.

L23

```
@Column(nullable=false) int maximumArrivalSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: maximumArrivalSeconds: presupuesto máximo hasta llegada.

L24

```
@Column(nullable=false) int loadingSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: loadingSeconds: carga prevista.

L25

```
@Column(nullable=false) int unloadingSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: unloadingSeconds: descarga prevista.

L26

```
@Column(nullable=false) int operationalMarginSeconds;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operationalMarginSeconds: contingencia de planificación.

L27

```
@Column(nullable=false,length=30) String configurationRevision;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: configurationRevision: versión de reglas.

L28

```
@Column(nullable=false,columnDefinition="text") String coverageAndAdmissionSummary;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: coverageAndAdmissionSummary: resumen de cobertura y admisión.

L29

```
@Column(nullable=false,length=30) String result;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: result: resultado de evaluación, distinto del estado de solicitud.

L30

```
@Column(nullable=true,length=100) String reasonCode;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: reasonCode: causa codificada.

L31

```
@Column(nullable=true,columnDefinition="text") String reasonDetail;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: reasonDetail: detalle diagnóstico acotado.

L32

```
@Column(nullable=false) Instant evaluatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: evaluatedAt: instante de evaluación.

L33

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

EvaluationRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationRepository.java`

`SHA-256: 90fe65f1c192b5e066d34cbc2a7770361dd412ce2a736de34e35c4ca42aa1fd9`

Interfaz Spring Data JPA para Evaluation. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
public interface EvaluationRepository extends JpaRepository<Evaluation,UUID> {
```

Declara interfaz EvaluationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L5

```
 Optional<Evaluation> findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(UUID id);
```

Recupera evaluación más reciente por fecha e ID. Campos: id: identificador UUID del registro.

L6

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

HistoryRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/HistoryRepository.java`

`SHA-256: a15300ba98f697c83a44b8ab5e4c978161c46310e819c7daae9ce5e930e5eb62`

Interfaz Spring Data JPA para History. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L5

```
public interface HistoryRepository extends JpaRepository<RequestStatusHistory,UUID> {
```

Declara interfaz HistoryRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L6

```
 Page<RequestStatusHistory> findByRequestId(UUID id,Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L7

```
  Optional<RequestStatusHistory> findByRequestIdAndOperationKey(UUID id,String key);
```

Busca resultado anterior para repetir sin duplicar. Campos: id: identificador UUID del registro.

L8

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RequestRepository (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestRepository.java

SHA-256: 18a7e513fe5c65585e1552888e7bc96a40b1cae73eac5209ba4dda80720e6c9c

Interfaz Spring Data JPA para Request. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L5

```
import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L6

```
public interface RequestRepository extends JpaRepository<AssistanceRequest,UUID> {
```

Declara interfaz RequestRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
  @Query("select r.vehicle.id from AssistanceRequest r where r.id=:id") Optional<UUID> vehicleId(UUID
id);
```

Consulta AssistanceRequest con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Cambios: r.id recibe :id") Optional<UUID> vehicleId(UUID id).

L8

```
  Optional<AssistanceRequest> findByDriverSubjectAndIdempotencyKey(String subject,String key);
```

Busca resultado anterior para repetir sin duplicar.

L9

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select r from AssistanceRequest r where r.id=:id")
Optional<AssistanceRequest> lockById(UUID id);
```

Adquiere bloqueo pesimista. Consulta AssistanceRequest con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Cambios: r.id recibe :id") Optional<AssistanceRequest> lockById(UUID id).

L10

```
  @Query("select count(r)>0 from AssistanceRequest r where r.vehicle.id=:id and (r.closedAt is null or
 r.resourceReleasePending=true)") boolean hasActive(UUID id);
```

Consulta AssistanceRequest contando coincidencias. Campos: id: identificador UUID del registro. Cambios: r.resourceReleasePending recibe true)") boolean hasActive(UUID id).

L11

```
  @Query("select r from AssistanceRequest r where (:subject is null or r.driverSubject=:subject) and
 (:status is null or r.status=:status)") Page<AssistanceRequest> visible(String
 subject,DomainTypes.Status status,Pageable page);
```

Consulta AssistanceRequest con el filtro y proyección escritos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L12

```
  @Query(value="select r.* from assistance_requests r where exists(select 1 from assignments a where
 a.request_id=r.id and a.operator_subject=:subject) and (:status is null or
 r.status=:status)",countQuery="select count(*) from assistance_requests r where exists(select 1 from
 assignments a where a.request_id=r.id and a.operator_subject=:subject) and (:status is null or
 r.status=:status)",nativeQuery=true)
```

Consulta assistance_requests, assignments con el filtro y proyección escritos. Consulta assistance_requests, assignments contando coincidencias. SQL nativo sobre tablas; @Query sin esa bandera es JPQL sobre entidades.

L13

```
  Page<AssistanceRequest> forOperator(String subject,String status,Pageable page);
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L14

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RequestStatusHistory (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestStatusHistory.java`

`SHA-256: 37f9182ed8fce3a9184d2cb398fca51ffb95036a4b76e2f4fa5016d64012246f`

Auditoría de cambios de solicitud con actor, motivo, fecha, idempotencia y evidencia. Es persistencia de negocio, distinta de los logs de consola.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L7

```
@Entity @Table(name="request_status_history",uniqueConstraints=@UniqueConstraint(columnNames={"request_i
d","operation_key"}))
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L8

```
public class RequestStatusHistory {
```

Declara clase RequestStatusHistory.

L9

```
  protected RequestStatusHistory() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L11

```
  @ManyToOne(fetch=FetchType.LAZY,optional=false) @JoinColumn(name="request_id") AssistanceRequest
request;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L12

```
  @Enumerated(EnumType.STRING) @Column(length=30) Status previousStatus;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: previousStatus: estado previo.

L13

```
  @Enumerated(EnumType.STRING) @Column(nullable=false,length=30) Status newStatus;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: newStatus: estado resultante.

L14

```
  @Column(length=100) String actorSubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: actorSubject: sub del usuario que produjo el evento.

L15

```
  @Column(nullable=false,length=20) String actorType;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: actorType: tipo de actor SYSTEM/DRIVER/OPERATOR.

L16

```
  @Column(columnDefinition="text") String reason;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: reason: motivo legible.

L17

```
  @Column(columnDefinition="text") String details;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: details: evidencia adicional.

L18

```
  @Column(length=100) String operationKey;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operationKey: clave durable de acción.

L19

```
  @Column(length=64) String requestFingerprint;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: requestFingerprint: SHA-256 del contenido asociado a la clave.

L20

```
  @Column(length=100) String correlationId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: correlationId: identificador opcional de correlación; su campo no implica generación automática.

L21

```
  @Column(nullable=false) Instant occurredAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: occurredAt: instante del evento.

L22

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Vehicle (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/Vehicle.java`

`SHA-256: 6345f53442c477a7d36ebdff23f49e26d675778badf1a8c195fd6746c578ce5c`

Entidad de vehículo del conductor. ownerSubject enlaza con sub de Keycloak sin FK entre bases; el repositorio filtra por propietario.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L7

```
@Entity @Table(name="vehicles")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L8

```
public class Vehicle {
```

Declara clase Vehicle.

L9

```
  protected Vehicle() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L11

```
  @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia.

L12

```
  @Column(nullable=false) Instant createdAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdAt: fecha de creación.

L13

```
  @Column(nullable=false) Instant updatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: updatedAt: fecha de modificación.

L14

```
  public UUID getId() { return id; }
```

Expone el UUID de la entidad; permite acceder mediante proxy sin manipular el campo directamente.

L15

```
  @Column(nullable=false,length=100) String ownerSubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: ownerSubject: sub de Keycloak del propietario.

L16

```
  @Column(nullable=false,unique=true,length=10) String licensePlate;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: licensePlate: patente normalizada y única.

L17

```
  @Column(length=100) String brand;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: brand: marca declarada.

L18

```
  @Column(length=100) String model;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: model: modelo declarado.

L19

```
  @Enumerated(EnumType.STRING) @Column(nullable=false,length=20) VehicleType vehicleType;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: vehicleType: tipo CAR/SUV/PICKUP.

L20

```
  @Column(nullable=false,precision=9,scale=2) BigDecimal declaredWeightKg;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: declaredWeightKg: peso declarado en kg.

L21

```
  @Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Registration
registrationStatus=Registration.ACTIVE;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L22

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

VehicleRepository (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/VehicleRepository.java`

`SHA-256: 93838d0cb029bb802e4e27341478ebf72ca6a756e0d305c87e71d736bf595224`

Interfaz Spring Data JPA para Vehicle. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L5

```
import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L6

```
public interface VehicleRepository extends JpaRepository<Vehicle,UUID> {
```

Declara interfaz VehicleRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
  Optional<Vehicle> findByIdAndOwnerSubject(UUID id,String subject);
```

Filtra por propietario en la consulta misma. Campos: id: identificador UUID del registro.

L8

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select v from Vehicle v where v.id=:id and
v.ownerSubject=:subject") Optional<Vehicle> lockOwned(UUID id,String subject);
```

Adquiere bloqueo pesimista. Consulta Vehicle con el filtro y proyección escritos. Campos: id: identificador UUID del registro.

L9

```
  Page<Vehicle> findByOwnerSubject(String subject,Pageable page);
```

Filtra por propietario en la consulta misma.

L10

```
  Page<Vehicle> findByOwnerSubjectAndRegistrationStatus(String subject,DomainTypes.Registration
status,Pageable page);
```

Filtra por propietario en la consulta misma. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L11

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

001-assistance (assistance-service)

`Ruta: assistance-service/src/main/resources/db/001-assistance.sql`

`SHA-256: f535d38c9952f4c1d5ea0f336972f467dabcbec681dd575e1c351859d7d403f1`

Esquema/actualización SQL aditiva de assistance-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
CREATE TABLE IF NOT EXISTS vehicles (
```

Crea vehicles solo si la tabla no existe.

L2

```
  id UUID PRIMARY KEY,owner_subject VARCHAR(100) NOT NULL,license_plate VARCHAR(10) NOT NULL UNIQUE
CHECK(license_plate ~ '^[A-Z0-9]{6,10}$'),
```

PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, owner_subject, license_plate. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
  brand VARCHAR(100),model VARCHAR(100),vehicle_type VARCHAR(20) NOT NULL CHECK(vehicle_type IN
('CAR','SUV','PICKUP')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: brand, model, vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
  declared_weight_kg NUMERIC(9,2) NOT NULL CHECK(declared_weight_kg BETWEEN 500 AND 3000),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
  registration_status VARCHAR(20) NOT NULL CHECK(registration_status IN ('ACTIVE','INACTIVE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: registration_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
  created_at TIMESTAMPTZ NOT NULL,updated_at TIMESTAMPTZ NOT NULL,version BIGINT NOT NULL
```

NOT NULL exige valor. Columnas: created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L7

```
);
```

Cierra la definición/sentencia SQL.

L8

```
CREATE TABLE IF NOT EXISTS assistance_requests (
```

Crea assistance_requests solo si la tabla no existe.

L9

```
  id UUID PRIMARY KEY,vehicle_id UUID NOT NULL REFERENCES vehicles(id),driver_subject VARCHAR(100) NOT
NULL,
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, vehicle_id, driver_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L10

```
  status VARCHAR(30) NOT NULL CHECK(status IN ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','AS
SIGNED','SEARCHING_ALTERNATIVE','DISPATCHED','ARRIVED','VEHICLE_LOADED','IN_TRANSIT','COMPLETED','REJECT
ED','UNFULFILLED','CANCELLED')),first_assigned_at TIMESTAMPTZ,closure_reason_code VARCHAR(100),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status, first_assigned_at, closure_reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L11

```
  origin_latitude NUMERIC(9,6) NOT NULL CHECK(origin_latitude BETWEEN -90 AND 90),origin_longitude
NUMERIC(9,6) NOT NULL CHECK(origin_longitude BETWEEN -180 AND 180),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: origin_latitude, origin_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L12

```
  destination_latitude NUMERIC(9,6) NOT NULL CHECK(destination_latitude BETWEEN -90 AND
90),destination_longitude NUMERIC(9,6) NOT NULL CHECK(destination_longitude BETWEEN -180 AND 180),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: destination_latitude, destination_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L13

```
  declared_vehicle_type VARCHAR(20) NOT NULL CHECK(declared_vehicle_type IN ('CAR','SUV','PICKUP')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L14

```
  declared_weight_kg NUMERIC(9,2) NOT NULL CHECK(declared_weight_kg BETWEEN 500 AND 3000),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L15

```
  mobility_condition VARCHAR(20) NOT NULL CHECK(mobility_condition IN ('ROLLABLE','NON_ROLLABLE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: mobility_condition. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L16

```
  description VARCHAR(1000) NOT NULL CHECK(length(trim(description))>0),search_attempt_number INTEGER NOT
  NULL CHECK(search_attempt_number BETWEEN 1 AND 3),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: description, search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L17

```
  search_deadline TIMESTAMPTZ NOT NULL,configuration_revision VARCHAR(30) NOT NULL,configuration_snapshot
  TEXT NOT NULL,
```

NOT NULL exige valor. Columnas: search_deadline, configuration_revision, configuration_snapshot. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L18

```
  idempotency_key VARCHAR(100) NOT NULL CHECK(length(trim(idempotency_key))>0),request_fingerprint
VARCHAR(64) NOT NULL,
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: idempotency_key, request_fingerprint. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L19

```
  created_at TIMESTAMPTZ NOT NULL,updated_at TIMESTAMPTZ NOT NULL,closed_at TIMESTAMPTZ,version BIGINT
NOT NULL,
```

NOT NULL exige valor. Columnas: created_at, updated_at, closed_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L20

```
  UNIQUE(driver_subject,idempotency_key),CHECK(origin_latitude<>destination_latitude OR
origin_longitude<>destination_longitude),
```

La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L21

```
  CHECK(search_deadline>created_at),CHECK((closed_at IS NOT NULL)=(status IN
('COMPLETED','REJECTED','UNFULFILLED','CANCELLED'))),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L22

```
  CHECK(status<>'CANCELLED' OR first_assigned_at IS NULL)
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L23

```
);
```

Cierra la definición/sentencia SQL.

L24

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_requests_active_vehicle ON assistance_requests(vehicle_id) WHERE
closed_at IS NULL;
```

Índice único ux_requests_active_vehicle sobre assistance_requests. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado.

L25

```
CREATE INDEX IF NOT EXISTS ix_requests_driver_status ON
assistance_requests(driver_subject,status,created_at,id);
```

Índice de búsqueda ix_requests_driver_status sobre assistance_requests.

L26

```
CREATE INDEX IF NOT EXISTS ix_vehicle_owner ON vehicles(owner_subject,created_at,id);
```

Índice de búsqueda ix_vehicle_owner sobre vehicles.

L27

```
CREATE TABLE IF NOT EXISTS request_status_history (
```

Crea request_status_history solo si la tabla no existe.

L28

```
  id UUID PRIMARY KEY,request_id UUID NOT NULL REFERENCES assistance_requests(id),previous_status
VARCHAR(30) CHECK(previous_status IN ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','ASSIGNED',
'SEARCHING_ALTERNATIVE','DISPATCHED','ARRIVED','VEHICLE_LOADED','IN_TRANSIT','COMPLETED','REJECTED','UNF
ULFILLED','CANCELLED')),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, request_id, previous_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L29

```
  new_status VARCHAR(30) NOT NULL CHECK(new_status IN ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTA
NCE','ASSIGNED','SEARCHING_ALTERNATIVE','DISPATCHED','ARRIVED','VEHICLE_LOADED','IN_TRANSIT','COMPLETED'
,'REJECTED','UNFULFILLED','CANCELLED')),actor_subject VARCHAR(100),actor_type VARCHAR(20) NOT NULL,
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: new_status, actor_subject, actor_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L30

```
  reason TEXT,details TEXT,operation_key VARCHAR(100),request_fingerprint VARCHAR(64),correlation_id
VARCHAR(100),occurred_at TIMESTAMPTZ NOT NULL,
```

NOT NULL exige valor. Columnas: reason, details, operation_key, request_fingerprint, correlation_id, occurred_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L31

```
  UNIQUE(request_id,operation_key)
```

La unicidad impide repetir la combinación indicada.

L32

```
);
```

Cierra la definición/sentencia SQL.

L33

```
CREATE INDEX IF NOT EXISTS ix_request_history ON request_status_history(request_id,occurred_at,id);
```

Índice de búsqueda ix_request_history sobre request_status_history.

002-evaluation (assistance-service)

`Ruta: assistance-service/src/main/resources/db/002-evaluation.sql`

`SHA-256: 602729ab9a9aba0c7ffd0d1be8d5e8aa06f467695685ae546ca0949766f6a5ec`

Esquema/actualización SQL aditiva de assistance-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
CREATE TABLE IF NOT EXISTS evaluations (
```

Crea evaluations solo si la tabla no existe.

L2

```
  id UUID PRIMARY KEY, request_id UUID NOT NULL REFERENCES assistance_requests(id),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
  search_attempt_number integer NOT NULL,
```

NOT NULL exige valor. Columnas: search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
  origin_latitude numeric(9,6) NOT NULL,
```

NOT NULL exige valor. Columnas: origin_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
  origin_longitude numeric(9,6) NOT NULL,
```

NOT NULL exige valor. Columnas: origin_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
destination_latitude numeric(9,6) NOT NULL,
```

NOT NULL exige valor. Columnas: destination_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L7

```
destination_longitude numeric(9,6) NOT NULL,
```

NOT NULL exige valor. Columnas: destination_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L8

```
transport_distance_m numeric,
```

Columnas: transport_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L9

```
transport_duration_seconds numeric,
```

Columnas: transport_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L10

```
adjusted_transport_seconds integer,
```

Columnas: adjusted_transport_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L11

```
route_obtained_at timestamptz,
```

Columnas: route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L12

```
vehicle_type varchar(20) NOT NULL,
```

NOT NULL exige valor. Columnas: vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L13

```
declared_weight_kg numeric(9,2) NOT NULL,
```

NOT NULL exige valor. Columnas: declared_weight_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L14

```
mobility_condition varchar(20) NOT NULL,
```

NOT NULL exige valor. Columnas: mobility_condition. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L15

```
maximum_arrival_seconds integer NOT NULL,
```

NOT NULL exige valor. Columnas: maximum_arrival_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L16

```
loading_seconds integer NOT NULL,
```

NOT NULL exige valor. Columnas: loading_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L17

```
unloading_seconds integer NOT NULL,
```

NOT NULL exige valor. Columnas: unloading_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L18

```
  operational_margin_seconds integer NOT NULL,
```

NOT NULL exige valor. Columnas: operational_margin_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L19

```
  configuration_revision varchar(30) NOT NULL,
```

NOT NULL exige valor. Columnas: configuration_revision. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L20

```
  coverage_and_admission_summary text NOT NULL,
```

NOT NULL exige valor. Columnas: coverage_and_admission_summary. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L21

```
  result varchar(30) NOT NULL,
```

NOT NULL exige valor. Columnas: result. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L22

```
  reason_code varchar(100),
```

Columnas: reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L23

```
  reason_detail text,
```

Columnas: reason_detail. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L24

```
  evaluated_at timestamptz NOT NULL
```

NOT NULL exige valor. Columnas: evaluated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L25

```
);
```

Cierra la definición/sentencia SQL.

L26

```
CREATE TABLE IF NOT EXISTS candidate_evaluations (
```

Crea candidate_evaluations solo si la tabla no existe.

L27

```
  id UUID PRIMARY KEY, evaluation_id UUID NOT NULL REFERENCES evaluations(id),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, evaluation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L28

```
  tow_truck_id uuid NOT NULL,
```

NOT NULL exige valor. Columnas: tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L29

```
  operator_id uuid NOT NULL,
```

NOT NULL exige valor. Columnas: operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L30

```
  operator_subject varchar(100) NOT NULL,
```

NOT NULL exige valor. Columnas: operator_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L31

```
  truck_capacity_kg numeric(9,2) NOT NULL,
```

NOT NULL exige valor. Columnas: truck_capacity_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L32

```
  qualification_category varchar(30) NOT NULL,
```

NOT NULL exige valor. Columnas: qualification_category. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L33

```
  qualification_valid_until timestamptz NOT NULL,
```

NOT NULL exige valor. Columnas: qualification_valid_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L34

```
  truck_latitude numeric(10,7) NOT NULL,
```

NOT NULL exige valor. Columnas: truck_latitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L35

```
  truck_longitude numeric(10,7) NOT NULL,
```

NOT NULL exige valor. Columnas: truck_longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L36

```
  truck_location_updated_at timestamptz NOT NULL,
```

NOT NULL exige valor. Columnas: truck_location_updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L37

```
  compatible boolean NOT NULL,
```

NOT NULL exige valor. Columnas: compatible. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L38

```
  eligible boolean NOT NULL,
```

NOT NULL exige valor. Columnas: eligible. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L39

```
  reason_code varchar(100),
```

Columnas: reason_code. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L40

```
  approach_distance_m numeric,
```

Columnas: approach_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L41

```
  approach_duration_seconds numeric,
```

Columnas: approach_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L42

```
  adjusted_approach_seconds integer,
```

Columnas: adjusted_approach_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L43

```
  approach_route_obtained_at timestamptz,
```

Columnas: approach_route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L44

```
  transport_distance_m numeric,
```

Columnas: transport_distance_m. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L45

```
  transport_duration_seconds numeric,
```

Columnas: transport_duration_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L46

```
  adjusted_transport_seconds integer,
```

Columnas: adjusted_transport_seconds. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L47

```
  transport_route_obtained_at timestamptz,
```

Columnas: transport_route_obtained_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L48

```
  estimated_service_end_at timestamptz,
```

Columnas: estimated_service_end_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L49

```
  ranking_position integer
```

Columnas: ranking_position. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L50

```
);
```

Cierra la definición/sentencia SQL.

L51

```
CREATE INDEX IF NOT EXISTS ix_evaluations_request ON evaluations(request_id,evaluated_at,id);
```

Índice de búsqueda ix_evaluations_request sobre evaluations.

L52

```
CREATE INDEX IF NOT EXISTS ix_candidate_evaluation ON
candidate_evaluations(evaluation_id,ranking_position);
```

Índice de búsqueda ix_candidate_evaluation sobre candidate_evaluations.

L53

```
CREATE TABLE IF NOT EXISTS routing_budget(id INTEGER PRIMARY KEY CHECK(id=1),blocked_until TIMESTAMPTZ
NOT NULL);
```

Crea routing_budget solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, blocked_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L54

```
INSERT INTO routing_budget(id,blocked_until) VALUES(1,'1970-01-01T00:00:00Z') ON CONFLICT(id) DO
NOTHING;
```

Inserta en routing_budget. DO NOTHING evita duplicar la intención/fila.

L55

```
CREATE TABLE IF NOT EXISTS routing_calls(id UUID PRIMARY KEY,called_at TIMESTAMPTZ NOT NULL);
```

Crea routing_calls solo si la tabla no existe. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, called_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L56

```
CREATE INDEX IF NOT EXISTS ix_routing_calls_time ON routing_calls(called_at);
```

Índice de búsqueda ix_routing_calls_time sobre routing_calls.

003-assignments (assistance-service)

`Ruta: assistance-service/src/main/resources/db/003-assignments.sql`

`SHA-256: b51fd33e89a81538489ce0f35eac961b512cb589166b698aaec4e39205995dca`

Esquema/actualización SQL aditiva de assistance-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS resource_release_pending boolean NOT NULL
DEFAULT false;
```

Añade resource_release_pending a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: resource_release_pending. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L2

```
CREATE TABLE IF NOT EXISTS assignments (
```

Crea assignments solo si la tabla no existe.

L3

```
  id uuid PRIMARY KEY,request_id uuid NOT NULL REFERENCES assistance_requests(id),candidate_evaluation_id
  uuid NOT NULL REFERENCES candidate_evaluations(id),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, request_id, candidate_evaluation_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
  tow_truck_id uuid NOT NULL,operator_id uuid NOT NULL,operator_subject varchar(100) NOT NULL,
```

NOT NULL exige valor. Columnas: tow_truck_id, operator_id, operator_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
  status varchar(20) NOT NULL CHECK(status IN
('RESERVING','PROPOSED','ACCEPTED','REJECTED','EXPIRED','CANCELLED','CLOSED')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
  created_at timestamptz NOT NULL,expires_at timestamptz,accepted_at timestamptz,closed_at timestamptz,
```

NOT NULL exige valor. Columnas: created_at, expires_at, accepted_at, closed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L7

```
  response_key varchar(100),response_fingerprint varchar(64),response_kind
varchar(20),search_attempt_number integer NOT NULL,
```

NOT NULL exige valor. Columnas: response_key, response_fingerprint, response_kind, search_attempt_number. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L8

```
  reserve_payload text NOT NULL,reason text
```

NOT NULL exige valor. Columnas: reserve_payload, reason. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L9

```
);
```

Cierra la definición/sentencia SQL.

L10

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_assignment_active ON assignments(request_id) WHERE status IN
('RESERVING','PROPOSED','ACCEPTED');
```

Índice único ux_assignment_active sobre assignments. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado.

L11

```
CREATE INDEX IF NOT EXISTS ix_assignment_operator ON assignments(operator_subject,created_at,id);
```

Índice de búsqueda ix_assignment_operator sobre assignments.

L12

```
CREATE TABLE IF NOT EXISTS pending_actions (
```

Crea pending_actions solo si la tabla no existe.

L13

```
 id uuid PRIMARY KEY,assignment_id uuid NOT NULL REFERENCES assignments(id),kind varchar(20) NOT NULL
CHECK(kind IN ('RESERVE','CONFIRM','RELEASE')),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, assignment_id, kind. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L14

```
 status varchar(20) NOT NULL CHECK(status IN ('PENDING','COMPLETED','RETRYING')),created_at timestamptz
NOT NULL,updated_at timestamptz NOT NULL,
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status, created_at, updated_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L15

```
  UNIQUE(assignment_id,kind)
```

La unicidad impide repetir la combinación indicada.

L16

```
);
```

Cierra la definición/sentencia SQL.

004-execution (assistance-service)

`Ruta: assistance-service/src/main/resources/db/004-execution.sql`

`SHA-256: 75cb69ffe3c38d0abd8c83dd4e209dda40ef2f851655b00d0db70efb92f1d576`

Esquema/actualización SQL aditiva de assistance-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS dispatched_at timestamptz;
```

Añade dispatched_at a assignments sin borrar filas; omite si ya existe. Columnas: dispatched_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L2

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS arrived_at timestamptz;
```

Añade arrived_at a assignments sin borrar filas; omite si ya existe. Columnas: arrived_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS loaded_at timestamptz;
```

Añade loaded_at a assignments sin borrar filas; omite si ya existe. Columnas: loaded_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS transport_started_at timestamptz;
```

Añade transport_started_at a assignments sin borrar filas; omite si ya existe. Columnas: transport_started_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS completed_at timestamptz;
```

Añade completed_at a assignments sin borrar filas; omite si ya existe. Columnas: completed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
ALTER TABLE assignments ADD COLUMN IF NOT EXISTS completion_note varchar(500);
```

Añade completion_note a assignments sin borrar filas; omite si ya existe. Columnas: completion_note. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

005-automation (assistance-service)

Ruta: assistance-service/src/main/resources/db/005-automation.sql

SHA-256: b64a0fab2b794c87a57c03a0caaddb2dc5eea337d5df1c9539951dfef5c8b3a1

Esquema/actualización SQL aditiva de assistance-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
ALTER TABLE pending_actions ADD COLUMN IF NOT EXISTS failure_count integer NOT NULL DEFAULT 0;
```

Añade failure_count a pending_actions sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: failure_count. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L2

```
ALTER TABLE pending_actions ADD COLUMN IF NOT EXISTS next_attempt_at timestamptz NOT NULL DEFAULT now();
```

Añade next_attempt_at a pending_actions sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_attempt_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
CREATE INDEX IF NOT EXISTS ix_pending_actions_due ON pending_actions(next_attempt_at) WHERE
status<>'COMPLETED';
```

Índice de búsqueda ix_pending_actions_due sobre pending_actions.

L4

```
ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS next_evaluation_at timestamptz NOT NULL DEFAULT
 now();
```

Añade next_evaluation_at a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_evaluation_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS next_reconciliation_at timestamptz NOT NULL
DEFAULT now();
```

Añade next_reconciliation_at a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: next_reconciliation_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
CREATE INDEX IF NOT EXISTS ix_requests_automation ON
assistance_requests(status,next_evaluation_at,created_at);
```

Índice de búsqueda ix_requests_automation sobre assistance_requests.

L7

```
ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS evaluation_lease_until timestamptz NOT NULL
DEFAULT '1970-01-01T00:00:00Z';
```

Añade evaluation_lease_until a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: evaluation_lease_until. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L8

```
ALTER TABLE assistance_requests ADD COLUMN IF NOT EXISTS replacement_pending boolean NOT NULL DEFAULT
false;
```

Añade replacement_pending a assistance_requests sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: replacement_pending. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

Operator (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/Operator.java`

`SHA-256: a992cd77973cb255f2d4cf76a018495388737d8b50e86c01abdcc87d185f7b18`

Entidad JPA del perfil de negocio del operador. keycloakSubject identifica la cuenta externa; no guarda contraseña ni reemplaza Keycloak.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L6

```
@Entity @Table(name="operators")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L7

```
public class Operator {
```

Declara clase Operator.

L8

```
protected Operator() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
@Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L10

```
@Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia.

L11

```
@Column(nullable=false) Instant createdAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdAt: fecha de creación.

L12

```
@Column(nullable=false) Instant updatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: updatedAt: fecha de modificación.

L13

```
@Column(nullable=false,unique=true,length=100) String keycloakSubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: keycloakSubject: identidad externa del operador.

L14

```
@Column(nullable=false,length=150) String fullName;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: fullName: nombre del perfil de negocio.

L15

```
@Column(length=40) String contactPhone;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: contactPhone: teléfono opcional.

L16

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Registration registrationStatus =
Registration.ACTIVE;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L17

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Availability availabilityStatus =
Availability.UNAVAILABLE;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: availabilityStatus: disposición a recibir propuestas.

L18

```
@Enumerated(EnumType.STRING) @Column(length=30) Qualification qualificationCategory;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: qualificationCategory: habilitación LIGHT_TOWING.

L19

```
Instant qualificationValidFrom;
```

Campos: qualificationValidFrom: inicio de habilitación.

L20

```
  Instant qualificationValidUntil;
```

Campos: qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L21

```
  Instant availableSince;
```

Campos: availableSince: inicio de disponibilidad usado para desempate.

L22

```
  public UUID getId() { return id; }
```

Expone el UUID de la entidad; permite acceder mediante proxy sin manipular el campo directamente.

L23

```
  public String getKeycloakSubject() { return keycloakSubject; }
```

Expone identidad del operador para comprobar propiedad del equipo.

L24

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

OperatorRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/OperatorRepository.java`

`SHA-256: 7fa6033a6dffc11140e790cff8112fe48bdaef571ef567317f75f3a38f3716ba`

Interfaz Spring Data JPA para Operator. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L5

```
import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L6

```
public interface OperatorRepository extends JpaRepository<Operator,UUID> {
```

Declara interfaz OperatorRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
  Optional<Operator> findByKeycloakSubject(String subject);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L8

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select o from Operator o where o.keycloakSubject =
 :subject") Optional<Operator> lockBySubject(String subject);
```

Adquiere bloqueo pesimista. Consulta Operator con el filtro y proyección escritos. Cambios: o.keycloakSubject recibe :subject") Optional<Operator> lockBySubject(String subject).

L9

```
  Page<Operator> findByRegistrationStatus(FleetTypes.Registration status,Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L10

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select o from Operator o where o.id = :id")
 Optional<Operator> lockById(UUID id);
```

Adquiere bloqueo pesimista. Consulta Operator con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Cambios: o.id recibe :id") Optional<Operator> lockById(UUID id).

L11

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ReservationRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationRepository.java`

`SHA-256: c90a55be603708f631f688af2d9c73f6fb36fb4fb410d980b15925efbe4217fe`

Interfaz Spring Data JPA para Reservation. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
 package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
 import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
 import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
 import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L5

```
 public interface ReservationRepository extends JpaRepository<ResourceReservation,UUID> {
```

Declara interfaz ReservationRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L6

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select r from ResourceReservation r where r.id=:id")
 Optional<ResourceReservation> lockById(UUID id);
```

Adquiere bloqueo pesimista. Consulta ResourceReservation con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Cambios: r.id recibe :id") Optional<ResourceReservation> lockById(UUID id).

L7

```
  @Query("select count(r)>0 from ResourceReservation r where (r.towTruckId=:truck or
 r.operatorId=:operator) and r.status in ('HELD','CONFIRMED')") boolean busy(UUID truck,UUID operator);
```

Consulta ResourceReservation contando coincidencias. Cambios: r.towTruckId recibe :truck or r.operatorId=:operator) and r.status in ('HELD','CONFIRMED')") boolean busy(UUID truck,UUID operator).

L8

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ResourceReservation (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ResourceReservation.java

SHA-256: f9d4bbc11cd6a099407cb54ab8ac6a9ee8c16e3f0bcf27112ebf174e34ad8ae3

Entidad Fleet del compromiso exclusivo de una grúa y un operador: HELD, CONFIRMED, RELEASED o EXPIRED. El ID coincide con el intento de asignación remoto.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
@Entity @Table(name="resource_reservations")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L6

```
public class ResourceReservation {
```

Declara clase ResourceReservation.

L7

```
  @Id UUID id;
```

Identifica clave primaria. Campos: id: identificador UUID del registro.

L8

```
  @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia.

L9

```
  @Column(nullable=false) boolean createdHold;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdHold: indica si el intento llegó a retener recursos.

L10

```
  @Column(nullable=false) UUID requestId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: requestId: UUID de solicitud.

L11

```
  @Column(nullable=false) UUID towTruckId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: towTruckId: UUID de grúa.

L12

```
  @Column(nullable=false) UUID operatorId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operatorId: UUID del perfil operador.

L13

```
  @Column(nullable=false,length=20) String status;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L14

```
  @Column(nullable=false,columnDefinition="text") String payload;
```

Declara el tipo SQL esperado para el campo JPA; el esquema real lo crean los scripts y Hibernate lo valida. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: payload: contenido normalizado para reintentos.

L15

```
  @Column(nullable=false) Instant createdAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdAt: fecha de creación.

L16

```
  @Column(nullable=false) Instant expiresAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: expiresAt: fin exclusivo de retención/propuesta.

L17

```
  @Column(nullable=false) Instant searchDeadline;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: searchDeadline: vencimiento del intento.

L18

```
  @Column(nullable=false) Instant estimatedServiceEndAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: estimatedServiceEndAt: fin planificado para controlar habilitación.

L19

```
  Instant confirmedAt;
```

Campos: confirmedAt: instante de confirmación en Fleet.

L20

```
  Instant releasedAt;
```

Campos: releasedAt: instante de liberación.

L21

```
  protected ResourceReservation() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L22

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

TowTruck (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TowTruck.java`

`SHA-256: 19ad180f79b95f05be31d5b598447502e256ea985e57acf4b6abdd402d83209c`

Entidad JPA de la grúa: capacidad, tipos admitidos, vínculo, estado y última ubicación declarada. La disponibilidad humana está en Operator y el compromiso en ResourceReservation.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L7

```
@Entity @Table(name="tow_trucks")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L8

```
public class TowTruck {
```

Declara clase TowTruck.

L9

```
  protected TowTruck() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
  @Id UUID id = UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L11

```
  @Version long version;
```

Activa concurrencia optimista mediante comparación/incremento de versión. Campos: version: versión para control optimista de concurrencia.

L12

```
  @Column(nullable=false) Instant createdAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: createdAt: fecha de creación.

L13

```
  @Column(nullable=false) Instant updatedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: updatedAt: fecha de modificación.

L14

```
  @Column(nullable=false,unique=true,length=10) String licensePlate;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: licensePlate: patente normalizada y única.

L15

```
  @ManyToOne(fetch=FetchType.LAZY) @JoinColumn(name="operator_id",unique=true) Operator operator;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L16

```
  @Column(nullable=false,precision=9,scale=2) BigDecimal maximumLoadKg;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: maximumLoadKg: capacidad de carga de la grúa en kg.

L17

```
  @Column(nullable=false) boolean supportsNonRollable;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L18

```
  @Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Operational operationalStatus =
 Operational.INACTIVE;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operationalStatus: estado administrativo de la grúa.

L19

```
  @Column(precision=10,scale=7) BigDecimal latitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: latitude: latitud declarada.

L20

```
  @Column(precision=10,scale=7) BigDecimal longitude;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: longitude: longitud declarada.

L21

```
  Instant locationUpdatedAt;
```

Campos: locationUpdatedAt: instante servidor de última ubicación declarada.

L22

```
  @ElementCollection
 @CollectionTable(name="tow_truck_supported_types",joinColumns=@JoinColumn(name="tow_truck_id"))
```

Persiste valores en tabla auxiliar. Nombra tabla auxiliar y referencia al dueño. Nombra FK local.

L23

```
  @Enumerated(EnumType.STRING) @Column(name="vehicle_type",length=20) Set<VehicleType> supportedTypes =
 new HashSet<>();
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real.

L24

```
  public UUID getId() { return id; }
```

Expone el UUID de la entidad; permite acceder mediante proxy sin manipular el campo directamente.

L25

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

TowTruckRepository (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TowTruckRepository.java

SHA-256: a471d53429a473ef17c66b21f9513cc3fcd07729974c01db8506a7fdccf00b09

Interfaz Spring Data JPA para TowTruck. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.*;
```

Importa tipos: org.springframework.data.jpa.repository.*. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L5

```
import jakarta.persistence.LockModeType;
```

Importa tipos: jakarta.persistence.LockModeType. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L6

```
public interface TowTruckRepository extends JpaRepository<TowTruck,UUID> {
```

Declara interfaz TowTruckRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L7

```
  @EntityGraph(attributePaths={"operator","supportedTypes"})
```

Carga asociaciones indicadas con la consulta.

L8

```
  @Query("select t from TowTruck t where t.operationalStatus=:operational and
t.operator.registrationStatus=:registration and t.operator.availabilityStatus=:availability and not
exists (select r.id from ResourceReservation r where (r.towTruckId=t.id or r.operatorId=t.operator.id)
and r.status in ('HELD','CONFIRMED'))")
```

Consulta TowTruck, ResourceReservation con el filtro y proyección escritos.

L9

```
  List<TowTruck> findCandidates(FleetTypes.Operational operational,FleetTypes.Registration
registration,FleetTypes.Availability availability);
```

La firma aplica el JPQL y EntityGraph anteriores para obtener recursos libres por estado.

L10

```
  Optional<TowTruck> findByOperatorId(UUID id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L11

```
  @Query("select t.id from TowTruck t where t.operator.keycloakSubject = :subject") Optional<UUID>
assignedId(String subject);
```

Consulta TowTruck con el filtro y proyección escritos.

L12

```
  Page<TowTruck> findByOperationalStatus(FleetTypes.Operational status,Pageable page);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L13

```
  @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("select t from TowTruck t where t.id = :id")
 Optional<TowTruck> lockById(UUID id);
```

Adquiere bloqueo pesimista. Consulta TowTruck con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Cambios: t.id recibe :id") Optional<TowTruck> lockById(UUID id).

L14

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

TruckStatusHistory (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TruckStatusHistory.java`

`SHA-256: 73ab5ea8359d5e0813c4445df984ec8a23e25c96d544fc8474d4ec5f27d1f494`

Entidad de historial administrativo de la grúa. Conserva antes/después, motivo, actor y clave de idempotencia.

L1

```
 package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
 import jakarta.persistence.*;
```

Importa tipos: jakarta.persistence.*. Anotaciones y contratos JPA para entidades, relaciones, versión y bloqueos.

L3

```
 import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
 import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
 import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L6

```
 @Entity @Table(name="truck_status_history")
```

Marca entidad JPA. Define tabla y restricciones de mapeo.

L7

```
 public class TruckStatusHistory {
```

Declara clase TruckStatusHistory.

L8

```
  protected TruckStatusHistory() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  @Id UUID id=UUID.randomUUID();
```

Identifica clave primaria. Genera ID nuevo en la aplicación. Campos: id: identificador UUID del registro.

L10

```
@ManyToOne(fetch=FetchType.LAZY,optional=false) @JoinColumn(name="tow_truck_id") TowTruck truck;
```

Relaciona muchas filas con una entidad; LAZY difiere la carga. Nombra FK local.

L11

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Operational previousStatus;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: previousStatus: estado previo.

L12

```
@Enumerated(EnumType.STRING) @Column(nullable=false,length=20) Operational newStatus;
```

Guarda nombre del enum, no posición ordinal. Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: newStatus: estado resultante.

L13

```
@Column(nullable=false,length=500) String reason;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: reason: motivo legible.

L14

```
@Column(nullable=false,length=100) String changedBySubject;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: changedBySubject: sub del administrador responsable.

L15

```
@Column(unique=true,length=100) String operationKey;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: operationKey: clave durable de acción.

L16

```
@Column(length=100) String correlationId;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: correlationId: identificador opcional de correlación; su campo no implica generación automática.

L17

```
@Column(nullable=false) Instant changedAt;
```

Especifica nulabilidad, longitud o precisión esperada; SQL define el esquema real. Campos: changedAt: instante del cambio.

L18

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

TruckStatusHistoryRepository (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.java`

`SHA-256: 557505435aaeb9527d5dc985c5d60fe2295d258b731ec0904c6fd283fb034f37`

Interfaz Spring Data JPA para TruckStatusHistory. El proxy generado implementa CRUD y consultas; @Query define JPQL o SQL y @Lock controla concurrencia.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.data.jpa.repository.JpaRepository;
```

Importa tipos: org.springframework.data.jpa.repository.JpaRepository. Repositorios Spring Data y estructuras de paginación/orden.

L4

```
public interface TruckStatusHistoryRepository extends JpaRepository<TruckStatusHistory,UUID> {
```

Declara interfaz TruckStatusHistoryRepository. Spring implementa CRUD de la entidad/ID indicados mediante proxy.

L5

```
 Optional<TruckStatusHistory> findByOperationKey(String key);
```

Busca resultado anterior para repetir sin duplicar.

L6

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

001-fleet-catalog (fleet-service)

`Ruta: fleet-service/src/main/resources/db/001-fleet-catalog.sql`

`SHA-256: f9bd6eea6ac0dcd54f11c7c08dc20e29a523dd15d765c5e62034ab67895fefa4`

Esquema/actualización SQL aditiva de fleet-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
-- Additive bootstrap for new and existing phase-3 databases. Hibernate validates the result.
```

Continúa la condición o lista de columnas de la sentencia anterior; se ejecuta como una sola instrucción.

L2

```
CREATE TABLE IF NOT EXISTS operators (
```

Crea operators solo si la tabla no existe.

L3

```
  id UUID PRIMARY KEY, keycloak_subject VARCHAR(100) NOT NULL UNIQUE,
```

PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. Columnas: id, keycloak_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
  full_name VARCHAR(150) NOT NULL CHECK (length(trim(full_name)) > 0), contact_phone VARCHAR(40),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: full_name, contact_phone. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
  registration_status VARCHAR(20) NOT NULL CHECK (registration_status IN ('ACTIVE','INACTIVE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: registration_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
  availability_status VARCHAR(20) NOT NULL CHECK (availability_status IN ('AVAILABLE','UNAVAILABLE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: availability_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L7

```
  qualification_category VARCHAR(30) CHECK (qualification_category = 'LIGHT_TOWING'),
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: qualification_category. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L8

```
  qualification_valid_from TIMESTAMPTZ, qualification_valid_until TIMESTAMPTZ, available_since
TIMESTAMPTZ,
```

Columnas: qualification_valid_from, qualification_valid_until, available_since. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L9

```
  created_at TIMESTAMPTZ NOT NULL, updated_at TIMESTAMPTZ NOT NULL, version BIGINT NOT NULL,
```

NOT NULL exige valor. Columnas: created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L10

```
  CHECK (length(trim(keycloak_subject)) > 0),
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L11

```
  CHECK ((qualification_category IS NULL AND qualification_valid_from IS NULL AND
qualification_valid_until IS NULL)
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L12

```
  OR (qualification_category IS NOT NULL AND qualification_valid_from IS NOT NULL AND
qualification_valid_until IS NOT NULL AND qualification_valid_from < qualification_valid_until)),
```

NOT NULL exige valor.

L13

```
  CHECK ((availability_status='AVAILABLE' AND available_since IS NOT NULL AND
registration_status='ACTIVE') OR (availability_status='UNAVAILABLE' AND available_since IS NULL))
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L14

```
);
```

Cierra la definición/sentencia SQL.

L15

```
CREATE TABLE IF NOT EXISTS tow_trucks (
```

Crea tow_trucks solo si la tabla no existe.

L16

```
  id UUID PRIMARY KEY, license_plate VARCHAR(10) NOT NULL UNIQUE CHECK (license_plate ~
'^[A-Z0-9]{6,10}$'),
```

PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: id, license_plate. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L17

```
  operator_id UUID UNIQUE REFERENCES operators(id), maximum_load_kg NUMERIC(9,2) NOT NULL CHECK
(maximum_load_kg BETWEEN 500 AND 5000),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. La unicidad impide repetir la combinación indicada. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: operator_id, maximum_load_kg. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L18

```
  supports_non_rollable BOOLEAN NOT NULL,
```

NOT NULL exige valor. Columnas: supports_non_rollable. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L19

```
  operational_status VARCHAR(20) NOT NULL CHECK (operational_status IN
('ACTIVE','INACTIVE','MAINTENANCE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: operational_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L20

```
  latitude NUMERIC(10,7) CHECK (latitude BETWEEN -90 AND 90), longitude NUMERIC(10,7) CHECK (longitude
BETWEEN -180 AND 180),
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: latitude, longitude. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L21

```
  location_updated_at TIMESTAMPTZ, created_at TIMESTAMPTZ NOT NULL, updated_at TIMESTAMPTZ NOT NULL,
version BIGINT NOT NULL,
```

NOT NULL exige valor. Columnas: location_updated_at, created_at, updated_at, version. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L22

```
  CHECK ((latitude IS NULL AND longitude IS NULL AND location_updated_at IS NULL) OR (latitude IS NOT
NULL AND longitude IS NOT NULL AND location_updated_at IS NOT NULL))
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L23

```
);
```

Cierra la definición/sentencia SQL.

L24

```
CREATE TABLE IF NOT EXISTS tow_truck_supported_types (
```

Crea tow_truck_supported_types solo si la tabla no existe.

L25

```
  tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id), vehicle_type VARCHAR(20) NOT NULL CHECK
(vehicle_type IN ('CAR','SUV','PICKUP')),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: tow_truck_id, vehicle_type. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L26

```
  PRIMARY KEY(tow_truck_id,vehicle_type)
```

PRIMARY KEY impide ID nulo o repetido.

L27

```
);
```

Cierra la definición/sentencia SQL.

L28

```
CREATE TABLE IF NOT EXISTS truck_status_history (
```

Crea truck_status_history solo si la tabla no existe.

L29

```
  id UUID PRIMARY KEY, tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, tow_truck_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L30

```
  previous_status VARCHAR(20) NOT NULL CHECK (previous_status IN ('ACTIVE','INACTIVE','MAINTENANCE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: previous_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L31

```
  new_status VARCHAR(20) NOT NULL CHECK (new_status IN ('ACTIVE','INACTIVE','MAINTENANCE')),
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: new_status. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L32

```
  reason VARCHAR(500) NOT NULL CHECK (length(trim(reason))>0),changed_by_subject VARCHAR(100) NOT NULL,
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: reason, changed_by_subject. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L33

```
  operation_key VARCHAR(100) UNIQUE,correlation_id VARCHAR(100),changed_at TIMESTAMPTZ NOT NULL,
```

NOT NULL exige valor. La unicidad impide repetir la combinación indicada. Columnas: operation_key, correlation_id, changed_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L34

```
  CHECK(previous_status<>new_status)
```

CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L35

```
);
```

Cierra la definición/sentencia SQL.

L36

```
CREATE INDEX IF NOT EXISTS ix_operators_status ON operators(registration_status,created_at,id);
```

Índice de búsqueda ix_operators_status sobre operators.

L37

```
CREATE INDEX IF NOT EXISTS ix_trucks_status ON tow_trucks(operational_status,created_at,id);
```

Índice de búsqueda ix_trucks_status sobre tow_trucks.

L38

```
CREATE INDEX IF NOT EXISTS ix_truck_history ON truck_status_history(tow_truck_id,changed_at);
```

Índice de búsqueda ix_truck_history sobre truck_status_history.

002-reservations (fleet-service)

`Ruta: fleet-service/src/main/resources/db/002-reservations.sql`

`SHA-256: b1bf412e01965e155b5996b2175d89904ba26cb216ef3b72d2a13649eb665da7`

Esquema/actualización SQL aditiva de fleet-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
CREATE TABLE IF NOT EXISTS resource_reservations (
```

Crea resource_reservations solo si la tabla no existe.

L2

```
  id UUID PRIMARY KEY,version bigint NOT NULL,request_id UUID NOT NULL,
```

PRIMARY KEY impide ID nulo o repetido. NOT NULL exige valor. Columnas: id, version, request_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L3

```
  tow_truck_id UUID NOT NULL REFERENCES tow_trucks(id),operator_id UUID NOT NULL REFERENCES
operators(id),
```

La FK exige un registro relacionado en esta misma base; no cruza microservicios. NOT NULL exige valor. Columnas: tow_truck_id, operator_id. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L4

```
  status varchar(20) NOT NULL CHECK(status IN ('HELD','CONFIRMED','RELEASED','EXPIRED')),payload text NOT
  NULL,
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita. Columnas: status, payload. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L5

```
  created_at timestamptz NOT NULL,expires_at timestamptz NOT NULL,search_deadline timestamptz NOT
NULL,estimated_service_end_at timestamptz NOT NULL,confirmed_at timestamptz,released_at timestamptz,
```

NOT NULL exige valor. Columnas: created_at, expires_at, search_deadline, estimated_service_end_at, confirmed_at, released_at. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L6

```
  CHECK(expires_at<=search_deadline),CHECK(status<>'CONFIRMED' OR confirmed_at IS NOT NULL)
```

NOT NULL exige valor. CHECK impone los rangos, estados o coherencia entre campos de la expresión escrita.

L7

```
);
```

Cierra la definición/sentencia SQL.

L8

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_reservation_truck ON resource_reservations(tow_truck_id) WHERE
status IN ('HELD','CONFIRMED');
```

Índice único ux_reservation_truck sobre resource_reservations. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado.

L9

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_reservation_operator ON resource_reservations(operator_id) WHERE
status IN ('HELD','CONFIRMED');
```

Índice único ux_reservation_operator sobre resource_reservations. La unicidad impide repetir la combinación indicada. El índice es parcial: solo restringe registros de los estados indicados; permite conservar historial cerrado.

003-automation (fleet-service)

`Ruta: fleet-service/src/main/resources/db/003-automation.sql`

`SHA-256: b211cf284da77b8a4499563792cfd9d0bcc8bde8f2bd352c0a626f9cf0583ce7`

Esquema/actualización SQL aditiva de fleet-service. IF NOT EXISTS evita recrear objetos; no sustituye una migración versionada ni actualiza automáticamente todas las restricciones antiguas.

L1

```
ALTER TABLE resource_reservations ADD COLUMN IF NOT EXISTS created_hold boolean NOT NULL DEFAULT false;
```

Añade created_hold a resource_reservations sin borrar filas; omite si ya existe. NOT NULL exige valor. DEFAULT aporta el valor indicado cuando no se suministra otro. Columnas: created_hold. UUID identifica; TIMESTAMPTZ conserva instantes; NUMERIC conserva precisión decimal.

L2

```
UPDATE resource_reservations SET created_hold=true WHERE status IN ('HELD','CONFIRMED','RELEASED') AND
created_hold=false;
```

Actualiza resource_reservations para las filas del filtro.

5. Flota: administración, candidatos y reservas

ACTIVE, AVAILABLE y HELD representan registro, disponibilidad y compromiso diferentes; no deben confundirse con contenedor running.

CandidateController (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/CandidateController.java

SHA-256: 269ecb4d31b1907c23c162f352289e84511e5b711cffdcaaaefd3a18263e47fd

API interna de preselección, consumida por Assistance con credenciales técnicas. Valida parámetros y delega el filtrado.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.List;
```

Importa tipos: java.util.List. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L4

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L5

```
import org.springframework.validation.annotation.Validated;
```

Importa tipos: org.springframework.validation.annotation.Validated. Tipo auxiliar usado por las expresiones del archivo.

L6

```
import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L7

```
import io.swagger.v3.oas.annotations.Hidden;
```

Importa tipos: io.swagger.v3.oas.annotations.Hidden. Metadatos OpenAPI de rutas, respuestas y autenticación.

L8

```
@RestController @Validated @Hidden
```

Expone HTTP y serializa JSON. Oculta operación interna en OpenAPI.

L9

```
public class CandidateController {
```

Declara clase CandidateController.

L10

```
  private final CandidateService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L11

```
  public CandidateController(CandidateService service) { this.service=service; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L12

```
  @GetMapping("/internal/fleet/candidates")
```

Ruta /internal/fleet/candidates combinada con el prefijo de clase cuando corresponda.

L13

```
  public List<CandidateService.Candidate> candidates(
```

Endpoint técnico de candidatos: los parámetros de líneas siguientes se validan antes de delegar.

L14

```
    @RequestParam @DecimalMin("-90") @DecimalMax("90") @Digits(integer=3,fraction=6) BigDecimal latitude,
```

Lee parámetro/filtro URL y su default. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: latitude: latitud declarada.

L15

```
    @RequestParam @DecimalMin("-180") @DecimalMax("180") @Digits(integer=3,fraction=6) BigDecimal
  longitude,
```

Lee parámetro/filtro URL y su default. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: longitude: longitud declarada.

L16

```
    @RequestParam FleetTypes.VehicleType vehicleType,@RequestParam @DecimalMin("500") @DecimalMax("3000")
  @Digits(integer=4,fraction=2) BigDecimal weightKg,
```

Lee parámetro/filtro URL y su default. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: vehicleType: tipo CAR/SUV/PICKUP.

L17

```
    @RequestParam boolean nonRollable) { return
  service.find(latitude,longitude,vehicleType,weightKg,nonRollable); }
```

Lee parámetro/filtro URL y su default.

L18

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

CandidateService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/CandidateService.java`

`SHA-256: 088fe5913fba49bf13bc681658f8edd30f2d289fe2b3d32c2048cc679eec2cbb`

Preselecciona como máximo cinco grúas libres y compatibles, dentro de cobertura y con ubicación vigente. Ordena por distancia recta; Assistance realiza después el ranking vial.

L1

```
  package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
  import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
  import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
  import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L7

```
import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía.

L8

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L9

```
@Service
```

Registra lógica de aplicación como bean.

L10

```
public class CandidateService {
```

Declara clase CandidateService.

L11

```
  private final TowTruckRepository trucks;private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
  public CandidateService(TowTruckRepository trucks,Clock clock) { this.trucks=trucks;this.clock=clock; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
  public record Candidate(UUID towTruckId,UUID operatorId,String operatorSubject,BigDecimal
capacityKg,Set<VehicleType> supportedTypes,boolean supportsNonRollable,
```

Declara record inmutable Candidate. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; supportsNonRollable: capacidad para atender vehículos que no ruedan.

L14

```
   String qualificationCategory,Instant qualificationValidFrom,Instant qualificationValidUntil,Instant
availableSince,
```

Campos: qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado; availableSince: inicio de disponibilidad usado para desempate.

L15

```
   BigDecimal latitude,BigDecimal longitude,Instant locationUpdatedAt,double straightDistanceM) {}
```

Campos: latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada.

L16

```
  @Transactional(readOnly=true)
```

Delimita transacción local; readOnly=true declara consulta.

L17

```
  public List<Candidate> find(BigDecimal latitude,BigDecimal longitude,VehicleType type,BigDecimal
 weight,boolean nonRollable) {
```

Campos: latitude: latitud declarada; longitude: longitud declarada.

L18

```
    Instant now=clock.instant();
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación.

L19

```
    return trucks.findCandidates(Operational.ACTIVE,Registration.ACTIVE,Availability.AVAILABLE).stream()
```

Obtiene grúas ACTIVE, operadores ACTIVE/AVAILABLE y sin reservas; inicia filtros adicionales en memoria.

L20

```
     .filter(t->t.maximumLoadKg.compareTo(weight)>=0 && t.supportedTypes.contains(type) && (!nonRollable
 || t.supportsNonRollable))
```

Conserva solo elementos que cumplen el predicado.

L21

```
     .filter(t->t.latitude!=null && t.longitude!=null && t.locationUpdatedAt!=null &&
 !t.locationUpdatedAt.isAfter(now) && !t.locationUpdatedAt.isBefore(now.minusSeconds(300)))
```

Conserva solo elementos que cumplen el predicado.

L22

```
     .filter(t->GeoMath.distance(-31.4167,-64.1833,t.latitude.doubleValue(),t.longitude.doubleValue(),6371
 000)<=15000)
```

Conserva solo elementos que cumplen el predicado.

L23

```
     .map(t->{Operator o=org.hibernate.Hibernate.unproxy(t.operator,Operator.class);return new Candidate(t
 .id,o.id,o.keycloakSubject,t.maximumLoadKg,Set.copyOf(t.supportedTypes),t.supportsNonRollable,o.qualific
 ationCategory==null?null:o.qualificationCategory.name(),o.qualificationValidFrom,o.qualificationValidUnt
 il,o.availableSince,t.latitude,t.longitude,t.locationUpdatedAt,GeoMath.distance(latitude.doubleValue(),l
 ongitude.doubleValue(),t.latitude.doubleValue(),t.longitude.doubleValue(),6371000));})
```

Transforma cada elemento al valor/tipo de salida.

L24

```
     .filter(c->"LIGHT_TOWING".equals(c.qualificationCategory()) && c.qualificationValidFrom()!=null &&
 c.qualificationValidUntil()!=null && !c.qualificationValidFrom().isAfter(now) &&
 c.qualificationValidUntil().isAfter(now) && c.availableSince()!=null)
```

Conserva solo elementos que cumplen el predicado.

L25

```
     .sorted(Comparator.comparingDouble(Candidate::straightDistanceM).thenComparing(c->c.towTruckId().toSt
 ring())).limit(5).toList();
```

Ordena por el comparador explícito para desempate reproducible. Acota cantidad de resultados. Materializa la secuencia en lista.

L26

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L27

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

FleetController (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetController.java`

`SHA-256: d5101fd4d7280195eac7e93ac9f3400e8ccf96e8b2f3b86750d9f84a8f3977fc`

API pública de flota: ADMIN administra recursos y OPERATOR opera su propio perfil/equipo. Delega las reglas y transacciones en FleetService.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.net.URI;
```

Importa tipos: java.net.URI. Tipo auxiliar usado por las expresiones del archivo.

L4

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas.

L6

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L7

```
import org.springframework.http.ResponseEntity;
```

Importa tipos: org.springframework.http.ResponseEntity. Estados, headers, tipos de contenido y respuestas HTTP.

L8

```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L10

```
import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación.

L11

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación.

L12

```
import io.swagger.v3.oas.annotations.tags.Tag;
```

Importa tipos: io.swagger.v3.oas.annotations.tags.Tag. Metadatos OpenAPI de rutas, respuestas y autenticación.

L13

```
import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación.

L14

```
import static com.auxiruta.fleet.catalog.FleetDtos.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetDtos.*. Tipos y DTO locales de Fleet.

L15

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L17

```
@RestController @RequestMapping("/api/v1")
```

Expone HTTP y serializa JSON. Ruta /api/v1 combinada con el prefijo de clase cuando corresponda.

L18

```
@Tag(name="Fleet",description="Administración: ADMIN. Rutas me y assigned-to-me: OPERATOR, identidad
tomada del JWT.")
```

Agrupa operaciones Fleet en OpenAPI y describe alcance de roles.

L19

```
@SecurityRequirement(name="bearerAuth")
```

Documenta autenticación; no concede permisos.

L20

```
@ApiResponses({@ApiResponse(responseCode="400",description="Entrada
inválida"),@ApiResponse(responseCode="401",description="JWT ausente o
inválido"),@ApiResponse(responseCode="403",description="Rol
insuficiente"),@ApiResponse(responseCode="404",description="Recurso inexistente o
ajeno"),@ApiResponse(responseCode="409",description="Duplicado o conflicto de estado")})
```

Documenta código HTTP.

L21

```
public class FleetController {
```

Declara clase FleetController.

L22

```
  private final FleetService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L23

```
  public FleetController(FleetService service) { this.service=service; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L24

```
  @ApiResponse(responseCode="201",description="Perfil creado; Location contiene su URL")
```

Documenta código HTTP.

L25

```
  @PostMapping("/operators") @Operation(summary="ADMIN: registrar perfil de una identidad OPERATOR creada
  previamente en Keycloak")
```

Ruta /operators combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: registrar perfil de una identidad OPERATOR creada previamente en Keycloak.

L26

```
  public ResponseEntity<OperatorView> createOperator(@Valid @RequestBody CreateOperator d) { var
o=service.create(d);return ResponseEntity.created(URI.create("/api/v1/operators/"+o.id())).body(o); }
```

Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Responde 201 con Location.

L27

```
  @GetMapping("/operators") @Operation(summary="ADMIN: consultar operadores paginados y filtrar por
registrationStatus")
```

Ruta /operators combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar operadores paginados y filtrar por registrationStatus.

L28

```
  public PageView<OperatorView> operators(@RequestParam(defaultValue="0") int
page,@RequestParam(defaultValue="20") int size,@RequestParam(required=false) Registration
registrationStatus) { return service.operators(page,size,registrationStatus); }
```

Lee parámetro/filtro URL y su default. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L29

```
  @GetMapping("/operators/{id}") @Operation(summary="ADMIN: consultar un operador")
```

Ruta /operators/{id} combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar un operador.

L30

```
  public OperatorView operator(@PathVariable UUID id) { return service.getOperator(id); }
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro.

L31

```
  @PatchMapping("/operators/{id}") @Operation(summary="ADMIN: actualizar nombre o teléfono; subject
inmutable")
```

Ruta /operators/{id} combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: actualizar nombre o teléfono; subject inmutable.

L32

```
  public OperatorView editOperator(@PathVariable UUID id,@Valid @RequestBody EditOperator d) { return
service.edit(id,d); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro.

L33

```
  @PatchMapping("/operators/{id}/registration-status") @Operation(summary="ADMIN: activar o inactivar el
perfil; inactivar retira su disponibilidad")
```

Ruta /operators/{id}/registration-status combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: activar o inactivar el perfil; inactivar retira su disponibilidad.

L34

```
  public OperatorView registration(@PathVariable UUID id,@Valid @RequestBody RegistrationChange d) {
return service.registration(id,d); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro.

L35

```
  @PutMapping("/operators/{id}/qualification") @Operation(summary="ADMIN: registrar o renovar la
habilitación LIGHT_TOWING")
```

Ruta /operators/{id}/qualification combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: registrar o renovar la habilitación LIGHT_TOWING.

L36

```
  public OperatorView qualification(@PathVariable UUID id,@Valid @RequestBody QualificationChange d) {
return service.qualification(id,d); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro.

L37

```
  @GetMapping("/operators/me") @Operation(summary="OPERATOR: consultar el perfil asociado al sub del
JWT")
```

Ruta /operators/me combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR: consultar el perfil asociado al sub del JWT.

L38

```
  public OperatorView me(@AuthenticationPrincipal Jwt jwt) { return service.me(jwt.getSubject()); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L39

```
  @PatchMapping("/operators/me/availability") @Operation(summary="OPERATOR: declarar disponibilidad
 propia")
```

Ruta /operators/me/availability combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR: declarar disponibilidad propia.

L40

```
   public OperatorView availability(@AuthenticationPrincipal Jwt jwt,@Valid @RequestBody
 AvailabilityChange d) { return service.availability(jwt.getSubject(),d); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L41

```
  @ApiResponse(responseCode="201",description="Grúa creada; Location contiene su URL")
```

Documenta código HTTP.

L42

```
  @PostMapping("/tow-trucks") @Operation(summary="ADMIN: registrar una grúa INACTIVE, sin operador ni
 ubicación")
```

Ruta /tow-trucks combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: registrar una grúa INACTIVE, sin operador ni ubicación.

L43

```
   public ResponseEntity<TruckView> createTruck(@Valid @RequestBody CreateTruck d) { var
 t=service.create(d);return ResponseEntity.created(URI.create("/api/v1/tow-trucks/"+t.id())).body(t); }
```

Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Responde 201 con Location.

L44

```
  @GetMapping("/tow-trucks") @Operation(summary="ADMIN: consultar grúas paginadas y filtrar por
 operationalStatus")
```

Ruta /tow-trucks combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar grúas paginadas y filtrar por operationalStatus.

L45

```
   public PageView<TruckView> trucks(@RequestParam(defaultValue="0") int
 page,@RequestParam(defaultValue="20") int size,@RequestParam(required=false) Operational
 operationalStatus) { return service.trucks(page,size,operationalStatus); }
```

Lee parámetro/filtro URL y su default. Campos: operationalStatus: estado administrativo de la grúa.

L46

```
  @GetMapping("/tow-trucks/{id}") @Operation(summary="ADMIN: consultar capacidades, vínculo y ubicación
 de una grúa")
```

Ruta /tow-trucks/{id} combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar capacidades, vínculo y ubicación de una grúa.

L47

```
   public TruckView truck(@PathVariable UUID id) { return service.getTruck(id); }
```

Extrae ID de la ruta. Campos: id: identificador UUID del registro.

L48

```
  @PatchMapping("/tow-trucks/{id}") @Operation(summary="ADMIN: actualizar datos y capacidades de una
 grúa")
```

Ruta /tow-trucks/{id} combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: actualizar datos y capacidades de una grúa.

L49

```
   public TruckView editTruck(@PathVariable UUID id,@Valid @RequestBody EditTruck d) { return
 service.edit(id,d); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro.

L50

```
  @PutMapping("/tow-trucks/{id}/operator") @Operation(summary="ADMIN: vincular un operador; rechaza
 sustitución implícita y vínculos duplicados")
```

Ruta /tow-trucks/{id}/operator combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: vincular un operador; rechaza sustitución implícita y vínculos duplicados.

L51

```
  public TruckView link(@PathVariable UUID id,@Valid @RequestBody LinkOperator d) { return
 service.link(id,d); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Campos: id: identificador UUID del registro.

L52

```
  @ApiResponse(responseCode="204",description="Vínculo retirado")
```

Documenta código HTTP.

L53

```
  @DeleteMapping("/tow-trucks/{id}/operator") @Operation(summary="ADMIN: desvincular el operador;
 repetición sin vínculo devuelve 204")
```

Ruta /tow-trucks/{id}/operator combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: desvincular el operador; repetición sin vínculo devuelve 204.

L54

```
  public ResponseEntity<Void> unlink(@PathVariable UUID id) { service.unlink(id);return
 ResponseEntity.noContent().build(); }
```

Construye el objeto final a partir del builder configurado en líneas previas. Extrae ID de la ruta. Responde 204 sin cuerpo. Campos: id: identificador UUID del registro.

L55

```
  @PatchMapping("/tow-trucks/{id}/operational-status") @Operation(summary="ADMIN: cambiar estado con
 motivo e Idempotency-Key; devuelve el evento durable")
```

Ruta /tow-trucks/{id}/operational-status combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: cambiar estado con motivo e Idempotency-Key; devuelve el evento durable.

L56

```
  public StatusView status(@PathVariable UUID id,@Valid @RequestBody StatusChange
 d,@RequestHeader("Idempotency-Key") String key,@AuthenticationPrincipal Jwt jwt) { return
 service.status(id,d,jwt.getSubject(),key); }
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Lee header indicado, normalmente Idempotency-Key. Obtiene JWT validado, no identidad suministrada por el cliente. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L57

```
  @GetMapping("/tow-trucks/assigned-to-me") @Operation(summary="OPERATOR: consultar exclusivamente su
 grúa vinculada")
```

Ruta /tow-trucks/assigned-to-me combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR: consultar exclusivamente su grúa vinculada.

L58

```
  public TruckView assigned(@AuthenticationPrincipal Jwt jwt) { return
 service.assigned(jwt.getSubject()); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L59

```
  @PutMapping("/tow-trucks/assigned-to-me/location") @Operation(summary="OPERATOR: registrar coordenadas
 de su grúa; el servidor fija locationUpdatedAt")
```

Ruta /tow-trucks/assigned-to-me/location combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR: registrar coordenadas de su grúa; el servidor fija locationUpdatedAt.

L60

```
  public TruckView location(@AuthenticationPrincipal Jwt jwt,@Valid @RequestBody Location d) { return
service.location(jwt.getSubject(),d); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L61

```
  @GetMapping("/configuration/fleet") @Operation(summary="ADMIN: consultar límites efectivos de
administración de Fleet")
```

Ruta /configuration/fleet combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar límites efectivos de administración de Fleet.

L62

```
  public AdministrationRules configuration() { return new
AdministrationRules("AR-MVP-5",500,5000,100,Qualification.LIGHT_TOWING); }
```

Devuelve límites administrativos 500..5000 kg, paginación máxima 100 y categoría LIGHT_TOWING.

L63

```
  public record AdministrationRules(String rulesVersion,int minTruckCapacityKg,int maxTruckCapacityKg,int
  maxPageSize,Qualification requiredQualification) {}
```

Declara record inmutable AdministrationRules. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L64

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 16: separadores sin instrucciones.

FleetDtos (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetDtos.java`

`SHA-256: 5d5cd1b8b7668596b30ff4a11cf8fccfd807ecac89c367fd0ffb21b59f263a17`

Contratos de entrada/salida de Fleet mediante records y Bean Validation. Los DTO no son tablas y evitan exponer directamente entidades JPA.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L3

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L6

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L7

```
public final class FleetDtos {
```

Declara clase FleetDtos.

L8

```
  private FleetDtos() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  public record CreateOperator(@NotBlank @Size(max=100) String keycloakSubject,@NotBlank @Size(max=150)
String fullName,@Size(max=40) String contactPhone) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable CreateOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: keycloakSubject: identidad externa del operador; fullName: nombre del perfil de negocio; contactPhone: teléfono opcional.

L10

```
  public record EditOperator(@Size(min=1,max=150) String fullName,@Size(max=40) String contactPhone) {}
```

Limita caracteres/elementos según min/max. Declara record inmutable EditOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: fullName: nombre del perfil de negocio; contactPhone: teléfono opcional.

L11

```
  public record RegistrationChange(@NotNull Registration status) {}
```

Exige no null. Declara record inmutable RegistrationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L12

```
  public record AvailabilityChange(@NotNull Availability status) {}
```

Exige no null. Declara record inmutable AvailabilityChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L13

```
  public record QualificationChange(@NotNull Qualification category,@NotNull Instant validFrom,@NotNull
Instant validUntil) {}
```

Exige no null. Declara record inmutable QualificationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L14

```
  public record CreateTruck(@NotBlank @Size(max=30) String licensePlate,
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable CreateTruck. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única.

L15

```
   @NotNull @DecimalMin("500") @DecimalMax("5000") @Digits(integer=4,fraction=2) BigDecimal
maximumLoadKg,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: maximumLoadKg: capacidad de carga de la grúa en kg.

L16

```
   @NotNull Boolean supportsNonRollable,@NotEmpty Set<@NotNull VehicleType> supportedTypes) {}
```

Exige no null. Exige colección/texto no vacío. Campos: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L17

```
  public record EditTruck(@Size(max=30) String licensePlate,
```

Limita caracteres/elementos según min/max. Declara record inmutable EditTruck. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única.

L18

```
    @DecimalMin("500") @DecimalMax("5000") @Digits(integer=4,fraction=2) BigDecimal maximumLoadKg,
```

Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: maximumLoadKg: capacidad de carga de la grúa en kg.

L19

```
    Boolean supportsNonRollable,@Size(min=1) Set<@NotNull VehicleType> supportedTypes) {}
```

Limita caracteres/elementos según min/max. Exige no null. Campos: supportsNonRollable: capacidad para atender vehículos que no ruedan.

L20

```
  public record LinkOperator(@NotNull UUID operatorId) {}
```

Exige no null. Declara record inmutable LinkOperator. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: operatorId: UUID del perfil operador.

L21

```
  public record StatusChange(@NotNull Operational status,@NotBlank @Size(max=500) String reason) {}
```

Exige no null. Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable StatusChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos; reason: motivo legible.

L22

```
  public record Location(@NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer=3,fraction=6)
BigDecimal latitude,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Declara record inmutable Location. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: latitude: latitud declarada.

L23

```
    @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer=3,fraction=6) BigDecimal longitude) {}
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: longitude: longitud declarada.

L24

```
  public record StatusView(UUID id,UUID towTruckId,Operational previousStatus,Operational
newStatus,String reason,String changedBySubject,String operationKey,Instant changedAt) {}
```

Declara record inmutable StatusView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; towTruckId: UUID de grúa; previousStatus: estado previo; newStatus: estado resultante; reason: motivo legible; changedBySubject: sub del administrador responsable; operationKey: clave durable de acción; changedAt: instante del cambio.

L25

```
  public record OperatorView(UUID id,String keycloakSubject,String fullName,String
contactPhone,Registration registrationStatus,
```

Declara record inmutable OperatorView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; keycloakSubject: identidad externa del operador; fullName: nombre del perfil de negocio; contactPhone: teléfono opcional; registrationStatus: alta administrativa ACTIVE/INACTIVE.

L26

```
    Availability availabilityStatus,Qualification qualificationCategory,Instant
qualificationValidFrom,Instant qualificationValidUntil,
```

Campos: availabilityStatus: disposición a recibir propuestas; qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L27

```
    Instant availableSince,Instant createdAt,Instant updatedAt,long version) {}
```

Campos: availableSince: inicio de disponibilidad usado para desempate; createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia.

L28

```
  public record TruckView(UUID id,String licensePlate,UUID operatorId,BigDecimal maximumLoadKg,boolean
supportsNonRollable,
```

Declara record inmutable TruckView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; licensePlate: patente normalizada y única; operatorId: UUID del perfil operador; maximumLoadKg: capacidad de carga de la grúa en kg; supportsNonRollable: capacidad para atender vehículos que no ruedan.

L29

```
   Set<VehicleType> supportedTypes,Operational operationalStatus,BigDecimal latitude,BigDecimal
longitude,Instant locationUpdatedAt,
```

Campos: operationalStatus: estado administrativo de la grúa; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada.

L30

```
   Instant createdAt,Instant updatedAt,long version) {}
```

Campos: createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia.

L31

```
  public record PageView<T>(List<T> content,int page,int size,long totalElements,int totalPages) {}
```

Declara record inmutable PageView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L32

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

FleetService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetService.java`

`SHA-256: dcecb928f5ffa109aa941e5149970a20c957dd6d039ce7b7e564b262b9dae13a`

Implementa administración de flota, propiedad, vínculos exclusivos, disponibilidad y ubicación. Los cambios de recursos ocupados fallan con 409; bloquear la grúa evita moverla mientras existe HELD/CONFIRMED.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L7

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L8

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L9

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L10

```
import org.slf4j.LoggerFactory;
```

Importa tipos: org.slf4j.LoggerFactory. Logging de eventos y fallos sin acoplarse a una implementación de consola.

L11

```
import static com.auxiruta.fleet.catalog.FleetDtos.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetDtos.*. Tipos y DTO locales de Fleet.

L12

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L14

```
@Service
```

Registra lógica de aplicación como bean.

L15

```
@Transactional(readOnly=true)
```

Delimita transacción local; readOnly=true declara consulta.

L16

```
public class FleetService {
```

Declara clase FleetService.

L17

```
  private final OperatorRepository operators;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L18

```
  private final TowTruckRepository trucks;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
  private final TruckStatusHistoryRepository history;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L20

```
  private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L21

```
  @org.springframework.beans.factory.annotation.Autowired private ReservationRepository reservations;
```

Inyecta dependencia Spring.

L22

```
  private void freeTruck(UUID id){require(!reservations.busy(id,null),HttpStatus.CONFLICT);}
```

Impide mutar una grúa retenida o confirmada. Campos: id: identificador UUID del registro. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L23

```
  private void freeOperator(UUID id){require(!reservations.busy(null,id),HttpStatus.CONFLICT);}
```

Impide mutar un operador retenido o confirmado. Campos: id: identificador UUID del registro. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L24

```
  public FleetService(OperatorRepository operators,TowTruckRepository trucks,TruckStatusHistoryRepository
  history,Clock clock) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L25

```
   this.operators=operators;this.trucks=trucks;this.history=history;this.clock=clock;
```

Asigna o consulta campos de la instancia.

L26

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L27

```
  private void committed(String event,UUID id) {
```

Campos: id: identificador UUID del registro.

L28

```
   org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronization(
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit.

L29

```
    new org.springframework.transaction.support.TransactionSynchronization() {
```

Implementa callback afterCommit para anunciar solo cambios confirmados.

L30

```
     @Override public void afterCommit() { LoggerFactory.getLogger(FleetService.class).info("Fleet
 event={} resourceId={}",event,id); }
```

Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos.

L31

```
    });
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L32

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L33

```
  private Instant now() { return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS); }
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L34

```
  private static ResponseStatusException missing() { return new
 ResponseStatusException(HttpStatus.NOT_FOUND); }
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L35

```
  private static void require(boolean condition,HttpStatus status) { if(!condition) throw new
 ResponseStatusException(status); }
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L36

```
  private Operator operator(UUID id) { return operators.findById(id).orElseThrow(FleetService::missing);
 }
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L37

```
  private Operator own(String subject) { return
 operators.findByKeycloakSubject(subject).orElseThrow(FleetService::missing); }
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L38

```
  private Operator lockedOperator(UUID id) { return org.hibernate.Hibernate.unproxy(operators.lockById(id
 ).orElseThrow(FleetService::missing),Operator.class); }
```

Si no encuentra resultado lanza la excepción indicada. Campos: id: identificador UUID del registro.

L39

```
  private TowTruck truck(UUID id) { return trucks.findById(id).orElseThrow(FleetService::missing); }
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L40

```
  private TowTruck lockedTruck(UUID id) { return trucks.lockById(id).orElseThrow(FleetService::missing);
 }
```

Si no encuentra resultado lanza la excepción indicada. Campos: id: identificador UUID del registro.

L41

```
  private TowTruck ownTruck(String subject) { return
 trucks.findByOperatorId(own(subject).id).orElseThrow(FleetService::missing); }
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L42

```
  private static String plate(String value) {
```

Quita espacios/guiones, convierte a mayúsculas y exige entre 6 y 10 caracteres alfanuméricos.

L43

```
    String normalized=value.replace(" ","").replace("-","").toUpperCase(Locale.ROOT);
```

Normaliza patente quitando espacios/guiones y convirtiendo a mayúsculas con Locale.ROOT.

L44

```
    require(normalized.matches("[A-Z0-9]{6,10}"),HttpStatus.BAD_REQUEST); return normalized;
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L45

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L46

```
  private static String name(String value) { require(!value.isBlank(),HttpStatus.BAD_REQUEST); return
 value.strip(); }
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L47

```
  private static Pageable page(int page,int size) {
```

Valida número/tamaño y aplica orden estable para paginar.

L48

```
   require(page>=0 && size>=1 && size<=100,HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L49

```
   return PageRequest.of(page,size,Sort.by("createdAt").ascending().and(Sort.by("id")));
```

Construye página ordenada por fecha y UUID, estable incluso cuando coinciden fechas.

L50

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L51

```
  private static <T> PageView<T> result(Page<T> p) { return new
PageView<>(p.getContent(),p.getNumber(),p.getSize(),p.getTotalElements(),p.getTotalPages()); }
```

Proyecta contenido, página, tamaño, total de elementos y páginas al contrato REST.

L52

```
  private OperatorView view(Operator o) { return new OperatorView(o.id,o.keycloakSubject,o.fullName,o.con
tactPhone,o.registrationStatus,o.availabilityStatus,o.qualificationCategory,o.qualificationValidFrom,o.q
ualificationValidUntil,o.availableSince,o.createdAt,o.updatedAt,o.version); }
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L53

```
  private TruckView view(TowTruck t) { return new TruckView(t.id,t.licensePlate,t.operator==null?null:t.o
perator.getId(),t.maximumLoadKg,t.supportsNonRollable,Set.copyOf(t.supportedTypes),t.operationalStatus,t
.latitude,t.longitude,t.locationUpdatedAt,t.createdAt,t.updatedAt,t.version); }
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L54

```
  public OperatorView getOperator(UUID id) { return view(operator(id)); }
```

Campos: id: identificador UUID del registro.

L55

```
  public OperatorView me(String subject) { return view(own(subject)); }
```

Resuelve el perfil por identidad autenticada y devuelve solo ese operador.

L56

```
  public TruckView getTruck(UUID id) { return view(truck(id)); }
```

Campos: id: identificador UUID del registro.

L57

```
  public TruckView assigned(String subject) { return view(ownTruck(subject)); }
```

Devuelve únicamente la grúa vinculada al operador autenticado.

L58

```
  public PageView<OperatorView> operators(int p,int s,Registration status) { Pageable
pageable=page(p,s);return result((status==null?operators.findAll(pageable):operators.findByRegistrationS
tatus(status,pageable)).map(this::view)); }
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L59

```
  public PageView<TruckView> trucks(int p,int s,Operational status) { Pageable pageable=page(p,s);return
result((status==null?trucks.findAll(pageable):trucks.findByOperationalStatus(status,pageable)).map(this:
:view)); }
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L60

```
  @Transactional public OperatorView create(CreateOperator d) {
```

Delimita transacción local; readOnly=true declara consulta.

L61

```
    Operator o=new Operator();o.keycloakSubject=d.keycloakSubject().strip();o.fullName=name(d.fullName());
 o.contactPhone=d.contactPhone();o.createdAt=o.updatedAt=now();
```

Cambios: o.keycloakSubject recibe d.keycloakSubject().strip(); o.fullName recibe name(d.fullName()); o.contactPhone recibe d.contactPhone(); o.createdAt recibe o.updatedAt=now().

L62

```
    operators.saveAndFlush(o);committed("OPERATOR_CREATED",o.id);return view(o);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción.

L63

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L64

```
  @Transactional public OperatorView edit(UUID id,EditOperator d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L65

```
    require(d.fullName()!=null || d.contactPhone()!=null,HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L66

```
    Operator o=lockedOperator(id);if(d.fullName()!=null)o.fullName=name(d.fullName());if(d.contactPhone()!
 =null)o.contactPhone=d.contactPhone();
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: o.fullName recibe name(d.fullName()); o.contactPhone recibe d.contactPhone().

L67

```
    o.updatedAt=now();operators.flush();return view(o);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: o.updatedAt recibe now().

L68

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L69

```
  @Transactional public OperatorView registration(UUID id,RegistrationChange d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L70

```
    Operator o=lockedOperator(id);freeOperator(id);o.registrationStatus=d.status();
```

Cambios: o.registrationStatus recibe d.status().

L71

```
    if(d.status()==Registration.INACTIVE) {
 o.availabilityStatus=Availability.UNAVAILABLE;o.availableSince=null; }
```

Inactivar también retira disponibilidad y limpia availableSince. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: o.availabilityStatus recibe Availability.UNAVAILABLE; o.availableSince recibe null.

L72

```
    o.updatedAt=now();operators.flush();return view(o);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: o.updatedAt recibe now().

L73

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L74

```
  @Transactional public OperatorView qualification(UUID id,QualificationChange d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L75

```
    require(d.validFrom().isBefore(d.validUntil()),HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L76

```
    Operator o=lockedOperator(id);freeOperator(id);o.qualificationCategory=d.category();o.qualificationVal
  idFrom=d.validFrom();o.qualificationValidUntil=d.validUntil();o.updatedAt=now();operators.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: o.qualificationCategory recibe d.category(); o.qualificationValidFrom recibe d.validFrom(); o.qualificationValidUntil recibe d.validUntil(); o.updatedAt recibe now().

L77

```
    committed("QUALIFICATION_UPDATED",o.id);return view(o);
```

Programa log de renovación después del commit y retorna perfil actualizado.

L78

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L79

```
  @Transactional public OperatorView availability(String subject,AvailabilityChange d) {
```

Delimita transacción local; readOnly=true declara consulta.

L80

```
    Operator o=operators.lockBySubject(subject).orElseThrow(FleetService::missing);
```

Si no encuentra resultado lanza la excepción indicada.

L81

```
    freeOperator(o.id);
```

Impide cambiar disponibilidad si el operador está retenido/confirmado.

L82

```
    require(d.status()!=Availability.AVAILABLE ||
  o.registrationStatus==Registration.ACTIVE,HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L83

```
    if(o.availabilityStatus!=d.status()) { o.availabilityStatus=d.status();o.availableSince=d.status()==Av
  ailability.AVAILABLE?now():null;o.updatedAt=now(); }
```

availableSince solo cambia si cambia disponibilidad: repetir AVAILABLE no altera el desempate. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: o.availabilityStatus recibe d.status(); o.availableSince recibe d.status()==Availability.AVAILABLE?now():null; o.updatedAt recibe now().

L84

```
    operators.flush();return view(o);
```

Sincroniza cambios JPA con SQL; no es commit independiente.

L85

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L86

```
  @Transactional public TruckView create(CreateTruck d) {
```

Delimita transacción local; readOnly=true declara consulta.

L87

```
    TowTruck t=new TowTruck();t.licensePlate=plate(d.licensePlate());t.maximumLoadKg=d.maximumLoadKg();t.s
upportsNonRollable=d.supportsNonRollable();t.supportedTypes.addAll(d.supportedTypes());t.createdAt=t.upd
atedAt=now();trucks.saveAndFlush(t);committed("TRUCK_CREATED",t.id);return view(t);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Cambios: t.licensePlate recibe plate(d.licensePlate()); t.maximumLoadKg recibe d.maximumLoadKg(); t.supportsNonRollable recibe d.supportsNonRollable(); t.createdAt recibe t.updatedAt=now().

L88

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L89

```
  @Transactional public TruckView edit(UUID id,EditTruck d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L90

```
    require(d.licensePlate()!=null || d.maximumLoadKg()!=null || d.supportsNonRollable()!=null ||
d.supportedTypes()!=null,HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L91

```
    TowTruck t=lockedTruck(id);freeTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L92

```
    if(d.licensePlate()!=null)t.licensePlate=plate(d.licensePlate());
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: t.licensePlate recibe plate(d.licensePlate()).

L93

```
    if(d.maximumLoadKg()!=null)t.maximumLoadKg=d.maximumLoadKg();
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: t.maximumLoadKg recibe d.maximumLoadKg().

L94

```
    if(d.supportsNonRollable()!=null)t.supportsNonRollable=d.supportsNonRollable();
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: t.supportsNonRollable recibe d.supportsNonRollable().

L95

```
    if(d.supportedTypes()!=null) { t.supportedTypes.clear();t.supportedTypes.addAll(d.supportedTypes()); }
```

La rama condicional selecciona la actualización o retorno correspondiente.

L96

```
    t.updatedAt=now();trucks.flush();return view(t);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: t.updatedAt recibe now().

L97

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L98

```
  @Transactional public TruckView link(UUID id,LinkOperator d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L99

```
    // All operations needing both locks take the truck first, then the operator.
```

Comentario de intención o documentación; no se ejecuta.

L100

```
    TowTruck t=lockedTruck(id);Operator o=lockedOperator(d.operatorId());freeTruck(id);freeOperator(o.id);
```

Impide cambiar disponibilidad si el operador está retenido/confirmado. Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L101

```
    require(t.operator==null || t.operator.getId().equals(o.id),HttpStatus.CONFLICT);
```

No sustituye otro operador silenciosamente; admite vacío o el mismo vínculo. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L102

```
    var assigned=trucks.findByOperatorId(o.id);
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L103

```
    require(assigned.isEmpty() || assigned.get().id.equals(t.id),HttpStatus.CONFLICT);
```

El operador no puede estar vinculado a otra grúa. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L104

```
    t.operator=o;t.updatedAt=now();trucks.flush();return view(t);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: t.operator recibe o; t.updatedAt recibe now().

L105

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L106

```
  @Transactional public void unlink(UUID id) { TowTruck
 t=lockedTruck(id);freeTruck(id);t.operator=null;t.updatedAt=now();trucks.flush(); }
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea. Rechaza mutación de grúa con reserva HELD/CONFIRMED. Delimita transacción local; readOnly=true declara consulta. Sincroniza cambios JPA con SQL; no es commit independiente. Campos: id: identificador UUID del registro. Cambios: t.operator recibe null; t.updatedAt recibe now().

L107

```
  @Transactional public StatusView status(UUID id,StatusChange d,String subject,String key) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L108

```
    require(key!=null && !key.isBlank() && key.length()<=100,HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L109

```
    TowTruck t=lockedTruck(id);
```

Bloquea la grúa antes de decidir la modificación; freeTruck impide cambiar un compromiso activo cuando aparece en esta línea.

L110

```
    var previous=history.findByOperationKey(key);
```

Busca resultado anterior para repetir sin duplicar.

L111

```
    if(previous.isPresent()) {
```

La rama condicional selecciona la actualización o retorno correspondiente.

L112

```
     TruckStatusHistory h=previous.get();
```

Recupera el evento previo asociado a la clave; sirve para devolver la misma respuesta.

L113

```
     require(h.truck.getId().equals(id) && h.newStatus==d.status() && h.reason.equals(d.reason().strip())
  && h.changedBySubject.equals(subject),HttpStatus.CONFLICT);
```

Una clave de estado de grúa exige mismo actor, grúa, estado y motivo. Repetir devuelve el evento sin reaplicarlo. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L114

```
     return statusView(h);
```

Proyecta el evento de estado sin aplicar de nuevo el cambio.

L115

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L116

```
    freeTruck(id);
```

Rechaza mutación de grúa con reserva HELD/CONFIRMED.

L117

```
    require(t.operationalStatus!=d.status(),HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L118

```
    TruckStatusHistory h=new TruckStatusHistory();h.truck=t;h.previousStatus=t.operationalStatus;h.newStat
  us=d.status();h.reason=d.reason().strip();h.changedBySubject=subject;h.operationKey=key;h.changedAt=now(
  );
```

Cambios: h.truck recibe t; h.previousStatus recibe t.operationalStatus; h.newStatus recibe d.status(); h.reason recibe d.reason().strip(); h.changedBySubject recibe subject; h.operationKey recibe key; h.changedAt recibe now().

L119

```
    history.save(h);t.operationalStatus=d.status();t.updatedAt=h.changedAt;trucks.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: t.operationalStatus recibe d.status(); t.updatedAt recibe h.changedAt.

L120

```
    committed("TRUCK_"+h.previousStatus+"_TO_"+h.newStatus,t.id);return statusView(h);
```

Proyecta el evento de estado sin aplicar de nuevo el cambio.

L121

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L122

```
   private StatusView statusView(TruckStatusHistory h) { return new StatusView(h.id,h.truck.getId(),h.prev
  iousStatus,h.newStatus,h.reason,h.changedBySubject,h.operationKey,h.changedAt); }
```

Proyecta el evento de cambio de estado administrativo, no una nueva transición.

L123

```
  @Transactional public TruckView location(String subject,Location d) {
```

Delimita transacción local; readOnly=true declara consulta.

L124

```
    TowTruck t=lockedTruck(trucks.assignedId(subject).orElseThrow(FleetService::missing));
```

Si no encuentra resultado lanza la excepción indicada.

L125

```
    require(t.operator!=null && t.operator.getKeycloakSubject().equals(subject),HttpStatus.NOT_FOUND);
```

Expone identidad del operador para comprobar propiedad del equipo. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L126

```
    freeTruck(t.id);
```

Impide actualizar incluso las mismas coordenadas cuando existe HELD/CONFIRMED. Así no se renueva artificialmente la vigencia ni cambia la ruta durante una propuesta.

L127

```
    t.latitude=d.latitude();t.longitude=d.longitude();t.locationUpdatedAt=t.updatedAt=now();trucks.flush()
  ;return view(t);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: t.latitude recibe d.latitude(); t.longitude recibe d.longitude(); t.locationUpdatedAt recibe t.updatedAt=now().

L128

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L129

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 13: separadores sin instrucciones.

FleetTypes (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/FleetTypes.java`

`SHA-256: 17e491886a634ec9d3cffef409a8f4a1c4e246817675bb990c562f324429926f`

Enumeraciones propias de Fleet. ACTIVE/INACTIVE/MAINTENANCE es el estado administrativo de la grúa; no representa los estados del traslado.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
public final class FleetTypes {
```

Declara clase FleetTypes.

L3

```
  private FleetTypes() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L4

```
  public enum Registration { ACTIVE, INACTIVE }
```

Declara enumeración cerrada Registration.

L5

```
  public enum Availability { AVAILABLE, UNAVAILABLE }
```

Declara enumeración cerrada Availability.

L6

```
  public enum Operational { ACTIVE, MAINTENANCE, INACTIVE }
```

Declara enumeración cerrada Operational.

L7

```
  public enum VehicleType { CAR, SUV, PICKUP }
```

Declara enumeración cerrada VehicleType.

L8

```
  public enum Qualification { LIGHT_TOWING }
```

Declara enumeración cerrada Qualification.

L9

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ReservationController (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationController.java`

`SHA-256: 3a84464cd110696ec14fb29a5903c3a23e754b1d9334456fb4d3eb3393ed74c3`

API interna de reservar, confirmar, liberar y consultar elegibilidad/estado. Se usa el mismo UUID para reintentos idempotentes.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas.

L4

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L5

```
import io.swagger.v3.oas.annotations.Hidden;
```

Importa tipos: io.swagger.v3.oas.annotations.Hidden. Metadatos OpenAPI de rutas, respuestas y autenticación.

L6

```
@Hidden @RestController @RequestMapping("/internal/reservations")
```

Oculta operación interna en OpenAPI. Expone HTTP y serializa JSON. Ruta /internal/reservations combinada con el prefijo de clase cuando corresponda.

L7

```
public class ReservationController {
```

Declara clase ReservationController.

L8

```
  private final ReservationService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L9

```
  public ReservationController(ReservationService service){this.service=service;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
  @PutMapping("/{id}") public ReservationService.View reserve(@PathVariable UUID id,@Valid @RequestBody
ReservationService.Input input){return service.reserve(id,input);}
```

Extrae ID de la ruta. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Ruta /{id} combinada con el prefijo de clase cuando corresponda. Campos: id: identificador UUID del registro.

L11

```
  public record StatusQueries(@jakarta.validation.constraints.NotNull
@jakarta.validation.constraints.Size(min=1,max=100)
java.util.List<@jakarta.validation.constraints.NotNull UUID> assignmentAttemptIds) {}
```

Exige no null. Limita caracteres/elementos según min/max. Declara record inmutable StatusQueries. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L12

```
  @PostMapping("/status-queries") public java.util.List<ReservationService.StatusQuery> statuses(@Valid
@RequestBody StatusQueries input){return
input.assignmentAttemptIds().stream().distinct().map(service::statusQuery).toList();}
```

Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Ruta /status-queries combinada con el prefijo de clase cuando corresponda. Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista.

L13

```
  @GetMapping("/{id}") public ReservationService.View get(@PathVariable UUID id){return service.get(id);}
```

Extrae ID de la ruta. Ruta /{id} combinada con el prefijo de clase cuando corresponda. Campos: id: identificador UUID del registro.

L14

```
  @PostMapping("/{id}/confirmation") public ReservationService.View confirm(@PathVariable UUID id){return
  service.confirm(id);}
```

Extrae ID de la ruta. Ruta /{id}/confirmation combinada con el prefijo de clase cuando corresponda. Campos: id: identificador UUID del registro.

L15

```
  @GetMapping("/{id}/eligibility") public ReservationService.Eligibility eligibility(@PathVariable UUID
id){return service.eligibility(id);}
```

Extrae ID de la ruta. Ruta /{id}/eligibility combinada con el prefijo de clase cuando corresponda. Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro.

L16

```
  @PostMapping("/{id}/release") public ReservationService.View release(@PathVariable UUID id){return
service.release(id);}
```

Extrae ID de la ruta. Ruta /{id}/release combinada con el prefijo de clase cuando corresponda. Campos: id: identificador UUID del registro.

L17

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ReservationJobs (fleet-service)

Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationJobs.java

SHA-256: 04f85ae53b01b45b58e17fc5c08f1ec55ae36c29055f46bc698b5d54f72d15a6

Materializa la expiración de retenciones mediante una tarea periódica; no cancela servicios ya confirmados.

L1

```
package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.Clock;
```

Importa tipos: java.time.Clock. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java.

L5

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables.

L7

```
import org.springframework.scheduling.annotation.*;
```

Importa tipos: org.springframework.scheduling.annotation.*. Tareas periódicas y scheduler.

L8

```
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
```

Importa tipos: org.springframework.boot.autoconfigure.condition.ConditionalOnProperty. Arranque/autoconfiguración y condiciones de activación.

L9

```
@Component @EnableScheduling
@ConditionalOnProperty(name="auxiruta.automation.enabled",havingValue="true",matchIfMissing=true)
```

Registra componente inyectable. Habilita tareas programadas. Activa bean solo si cumple propiedad configurada.

L10

```
public class ReservationJobs {
```

Declara clase ReservationJobs.

L11

```
 private final JdbcTemplate jdbc;private final ReservationService reservations;private final Clock
clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
 public ReservationJobs(JdbcTemplate jdbc,ReservationService reservations,Clock
clock){this.jdbc=jdbc;this.reservations=reservations;this.clock=clock;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
  @Scheduled(fixedDelayString="${auxiruta.automation.fleet-delay-ms:15000}",initialDelay=15000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa.

L14

```
   public void expire(){for(UUID id:jdbc.queryForList("select id from resource_reservations where
  status='HELD' and expires_at<=? order by expires_at,id limit
  100",UUID.class,Timestamp.from(clock.instant())))try{reservations.expireDue(id);}catch(Exception
  e){org.slf4j.LoggerFactory.getLogger(getClass()).warn("Reservation expiration deferred id={}
  type={}",id,e.getClass().getSimpleName());}}
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED. Consulta resource_reservations con el filtro y proyección escritos. Campos: id: identificador UUID del registro. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo. Registra los campos indicados, sin token ni secreto.

L15

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ReservationService (fleet-service)

`Ruta: fleet-service/src/main/java/com/auxiruta/fleet/catalog/ReservationService.java`

`SHA-256: 0bb0a7c0e0ca183e35a1d6a90a8c89ac728fda1fee4f9faf765b1de519cb752b`

Protege la exclusividad mediante bloqueos y restricciones. HELD caduca; CONFIRMED no caduca por el plazo antiguo de la oferta. Exige ubicación de hasta 300 s al reservar/confirmar, pero no vuelve a caducarla después de aceptar.

L1

```
  package com.auxiruta.fleet.catalog;
```

Ubica el archivo en com.auxiruta.fleet.catalog. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
  import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L3

```
  import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
  import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
  import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L6

```
  import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L7

```
  import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L8

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L9

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L10

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L11

```
import static com.auxiruta.fleet.catalog.FleetTypes.*;
```

Importa miembros estáticos: com.auxiruta.fleet.catalog.FleetTypes.*. Tipos y DTO locales de Fleet.

L12

```
@Service
```

Registra lógica de aplicación como bean.

L13

```
@Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L14

```
public class ReservationService {
```

Declara clase ReservationService.

L15

```
 public record Input(@NotNull UUID requestId,@NotNull UUID towTruckId,@NotNull UUID operatorId,@NotNull
VehicleType vehicleType,@NotNull @DecimalMin("500") @DecimalMax("3000") BigDecimal weightKg,boolean
nonRollable,@NotNull Instant searchDeadline,@NotNull Instant estimatedServiceEndAt,Integer
holdSeconds,Integer minimumOfferSeconds) {
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; vehicleType: tipo CAR/SUV/PICKUP; searchDeadline: vencimiento del intento; estimatedServiceEndAt: fin planificado para controlar habilitación.

L16

```
  public Input
{if(holdSeconds==null)holdSeconds=60;if(minimumOfferSeconds==null)minimumOfferSeconds=15;}
```

La rama condicional selecciona la actualización o retorno correspondiente.

L17

```
  public Input(UUID requestId,UUID towTruckId,UUID operatorId,VehicleType vehicleType,BigDecimal
weightKg,boolean nonRollable,Instant searchDeadline,Instant estimatedServiceEndAt){this(requestId,towTru
ckId,operatorId,vehicleType,weightKg,nonRollable,searchDeadline,estimatedServiceEndAt,60,15);}
```

Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; vehicleType: tipo CAR/SUV/PICKUP; searchDeadline: vencimiento del intento; estimatedServiceEndAt: fin planificado para controlar habilitación.

L18

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L19

```
 public record View(UUID assignmentAttemptId,UUID requestId,UUID towTruckId,UUID operatorId,String
status,Instant expiresAt,Instant confirmedAt,Instant releasedAt,boolean createdHold) {}
```

Declara record inmutable View. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; status: estado del objeto; su enum o CHECK define valores permitidos; expiresAt: fin exclusivo de retención/propuesta; confirmedAt: instante de confirmación en Fleet; releasedAt: instante de liberación; createdHold: indica si el intento llegó a retener recursos.

L20

```
  private final ReservationRepository reservations;private final TowTruckRepository trucks;private final
 OperatorRepository operators;private final Clock clock;private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L21

```
  public ReservationService(ReservationRepository reservations,TowTruckRepository
 trucks,OperatorRepository operators,Clock clock,ObjectMapper mapper){this.reservations=reservations;this
 .trucks=trucks;this.operators=operators;this.clock=clock;this.mapper=mapper;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L22

```
  @jakarta.persistence.PersistenceContext private jakarta.persistence.EntityManager entityManager;
```

Inyecta EntityManager transaccional.

L23

```
  private Instant now(){return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);}
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L24

```
  private static void require(boolean ok){if(!ok)throw new ResponseStatusException(HttpStatus.CONFLICT);}
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L25

```
  private static ResponseStatusException missing(){return new
 ResponseStatusException(HttpStatus.NOT_FOUND);}
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L26

```
  private View view(ResourceReservation r){return new View(r.id,r.requestId,r.towTruckId,r.operatorId,r.s
 tatus,r.expiresAt,r.confirmedAt,r.releasedAt,r.createdHold);}
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L27

```
  private String payload(Input d){try{return mapper.writeValueAsString(new Input(d.requestId(),d.towTruck
 Id(),d.operatorId(),d.vehicleType(),d.weightKg().stripTrailingZeros(),d.nonRollable(),d.searchDeadline()
 ,d.estimatedServiceEndAt(),d.holdSeconds(),d.minimumOfferSeconds()));}catch(Exception e){throw new
 IllegalArgumentException("Invalid reservation");}}
```

Serializa el contenido normalizado que se conserva para reintentar la misma reserva. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L28

```
  private String normalizeStored(String stored){try{return
 payload(mapper.readValue(stored,Input.class));}catch(Exception e){throw new
 IllegalStateException("Invalid reservation payload");}}
```

Reconstruye un payload antiguo con valores por defecto y lo normaliza para idempotencia. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L29

```
  private void eligible(TowTruck t,Operator o,Input d,Instant now){eligible(t,o,d,now,true);}
```

La sobrecarga por defecto exige ubicación fresca; se usa al reservar y confirmar. Agrupa validaciones de vínculo, registro, disponibilidad, capacidad, cobertura, ubicación y habilitación.

L30

```
  private void eligible(TowTruck t,Operator o,Input d,Instant now,boolean freshLocation){
```

Agrupa validaciones de vínculo, registro, disponibilidad, capacidad, cobertura, ubicación y habilitación.

L31

```
    require(t.operator!=null && t.operator.getId().equals(o.id));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L32

```
    require(t.operationalStatus==Operational.ACTIVE && o.registrationStatus==Registration.ACTIVE &&
  o.availabilityStatus==Availability.AVAILABLE);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L33

```
    require(t.maximumLoadKg.compareTo(d.weightKg())>=0 && t.supportedTypes.contains(d.vehicleType()) &&
  (!d.nonRollable() || t.supportsNonRollable));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L34

```
    require(t.locationUpdatedAt!=null && !t.locationUpdatedAt.isAfter(now) && (!freshLocation ||
  !t.locationUpdatedAt.isBefore(now.minusSeconds(300))));
```

Al reservar/confirmar exige ubicación no futura de hasta 300 s. Para elegibilidad de una reserva ya confirmada freshLocation=false elimina solo el límite de antigüedad. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L35

```
    require(t.latitude!=null && t.longitude!=null && com.auxiruta.commons.GeoMath.distance(-31.4167,-64.18
  33,t.latitude.doubleValue(),t.longitude.doubleValue(),6371000)<=15000);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L36

```
    require(o.qualificationCategory==Qualification.LIGHT_TOWING && o.qualificationValidFrom!=null &&
  !o.qualificationValidFrom.isAfter(now) && o.qualificationValidUntil!=null &&
  o.qualificationValidUntil.isAfter(d.estimatedServiceEndAt()));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L37

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L38

```
  public View reserve(UUID id,Input d){
```

Campos: id: identificador UUID del registro.

L39

```
    require((d.holdSeconds()==60 && d.minimumOfferSeconds()==15) || (d.holdSeconds()==10 &&
  d.minimumOfferSeconds()==3));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L40

```
    // Every operation involving both resources locks truck, then operator, then reservation.
```

Comentario de intención o documentación; no se ejecuta.

L41

```
    TowTruck t=trucks.lockById(d.towTruckId()).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada.

L42

```
    Operator o=org.hibernate.Hibernate.unproxy(operators.lockById(d.operatorId()).orElseThrow(ReservationS
  ervice::missing),Operator.class);
```

Si no encuentra resultado lanza la excepción indicada.

L43

```
    var previous=reservations.lockById(id);String payload=payload(d);
```

Campos: payload: contenido normalizado para reintentos.

L44

```
    if(previous.isPresent()){require(normalizeStored(previous.get().payload).equals(payload));expire(previ
  ous.get());return view(previous.get());}
```

Reutiliza intento con mismo payload normalizado; no revive una reserva cerrada. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L45

```
    Instant now=now();boolean eligible=true;
```

Campos: eligible: aptitud completa del candidato/compromiso.

L46

```
    try{require(Duration.between(now,d.searchDeadline()).getSeconds()>=d.minimumOfferSeconds() && d.estima
  tedServiceEndAt().isAfter(now));eligible(t,o,d,now);require(!reservations.busy(t.id,o.id));}catch(Respon
  seStatusException e){eligible=false;}
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L47

```
    ResourceReservation r=new ResourceReservation();r.id=id;r.requestId=d.requestId();r.towTruckId=t.id;r.
  operatorId=o.id;r.payload=payload;r.status="HELD";r.createdAt=now;r.searchDeadline=d.searchDeadline();r.
  estimatedServiceEndAt=d.estimatedServiceEndAt();r.expiresAt=now.plusSeconds(d.holdSeconds()).isBefore(d.
  searchDeadline())?now.plusSeconds(d.holdSeconds()):d.searchDeadline();r.createdHold=eligible;if(!eligibl
  e){r.status="EXPIRED";r.releasedAt=now;}return view(reservations.saveAndFlush(r));
```

Guarda un intento durable: HELD si pudo retener, EXPIRED si no. createdHold permite distinguir intento sin retención y oferta que sí ocupó recursos. Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.id recibe id; r.requestId recibe d.requestId(); r.towTruckId recibe t.id; r.operatorId recibe o.id; r.payload recibe payload; r.status recibe "HELD"; r.createdAt recibe now; r.searchDeadline recibe d.searchDeadline(); r.estimatedServiceEndAt recibe d.estimatedServiceEndAt(); r.expiresAt recibe now.plusSeconds(d.holdSeconds()).isBefore(d.searchDeadline())?now.plusSeconds(d.holdSeconds()):d.searchDeadline(); r.createdHold recibe eligible; r.status recibe "EXPIRED"; r.releasedAt recibe now.

L48

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L49

```
  private ResourceReservation lock(UUID id){
```

Adquiere bloqueos en orden para serializar operaciones competidoras. Campos: id: identificador UUID del registro.

L50

```
    // Read only IDs before taking resource locks; refresh state under reservation lock.
```

Comentario de intención o documentación; no se ejecuta.

L51

```
    ResourceReservation first=reservations.findById(id).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L52

```
    trucks.lockById(first.towTruckId).orElseThrow(ReservationService::missing);operators.lockById(first.op
  eratorId).orElseThrow(ReservationService::missing);
```

Si no encuentra resultado lanza la excepción indicada.

L53

```
    entityManager.refresh(first,jakarta.persistence.LockModeType.PESSIMISTIC_WRITE);return first;
```

Recarga bajo bloqueo pesimista después de bloquear grúa y operador: evita usar el estado leído antes de tomar los locks.

L54

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L55

```
  private void expire(ResourceReservation r){if(r.status.equals("HELD") &&
!r.expiresAt.isAfter(now())){r.status="EXPIRED";r.releasedAt=now();}}
```

Solo cambia HELD a EXPIRED al alcanzar expiresAt; no vence CONFIRMED. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.status recibe "EXPIRED"; r.releasedAt recibe now().

L56

```
  public void expireDue(UUID id){expire(lock(id));}
```

Campos: id: identificador UUID del registro.

L57

```
  public record StatusQuery(UUID assignmentAttemptId,String status,Eligibility resource) {}
```

Declara record inmutable StatusQuery. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L58

```
  public StatusQuery statusQuery(UUID id){var found=reservations.findById(id);if(found.isEmpty())return
new StatusQuery(id,"MISSING",null);var r=found.get();String state=r.status.equals("HELD") &&
!r.expiresAt.isAfter(now())?"EXPIRED":r.status;return new
StatusQuery(id,state,state.equals("CONFIRMED")?eligibility(id):null);}
```

Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro. La rama condicional selecciona la actualización o retorno correspondiente.

L59

```
  public View get(UUID id){ResourceReservation r=lock(id);expire(r);return view(r);}
```

Campos: id: identificador UUID del registro.

L60

```
  public View confirm(UUID id){ResourceReservation r=lock(id);if(r.status.equals("CONFIRMED"))return
view(r);if(!r.status.equals("HELD") || !r.expiresAt.isAfter(now()) ||
!r.searchDeadline.isAfter(now())){expire(r);return view(r);}
```

Una confirmación repetida devuelve el compromiso ya existente antes de evaluar el vencimiento de la antigua oferta. Campos: id: identificador UUID del registro. La rama condicional selecciona la actualización o retorno correspondiente.

L61

```
   Input d;try{d=mapper.readValue(r.payload,Input.class);}catch(Exception e){throw new
IllegalStateException("Invalid reservation payload");}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L62

```
   TowTruck t=trucks.findById(r.towTruckId).orElseThrow();Operator
o=org.hibernate.Hibernate.unproxy(operators.findById(r.operatorId).orElseThrow(),Operator.class);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L63

```
   try{eligible(t,o,d,now());}catch(ResponseStatusException
e){r.status="RELEASED";r.releasedAt=now();return view(r);}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Cambios: r.status recibe "RELEASED"; r.releasedAt recibe now().

L64

```
    if(!r.expiresAt.isAfter(now())){expire(r);return view(r);}
```

La rama condicional selecciona la actualización o retorno correspondiente.

L65

```
    r.status="CONFIRMED";r.confirmedAt=now();reservations.flush();return view(r);
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: r.status recibe "CONFIRMED"; r.confirmedAt recibe now().

L66

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L67

```
   public record Eligibility(UUID assignmentAttemptId,UUID requestId,UUID towTruckId,UUID
  operatorId,String operatorSubject,boolean eligible,String reason,BigDecimal latitude,BigDecimal
  longitude,Instant locationUpdatedAt,Instant qualificationValidUntil,Instant checkedAt) {}
```

Declara record inmutable Eligibility. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; eligible: aptitud completa del candidato/compromiso; reason: motivo legible; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L68

```
  public Eligibility eligibility(UUID id){
```

Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro.

L69

```
    ResourceReservation r=lock(id);TowTruck t=trucks.findById(r.towTruckId).orElseThrow();Operator
  o=org.hibernate.Hibernate.unproxy(operators.findById(r.operatorId).orElseThrow(),Operator.class);boolean
   valid=r.status.equals("CONFIRMED");String reason=valid?"ELIGIBLE":"RESERVATION_NOT_CONFIRMED";
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: reason: motivo legible.

L70

```
    if(valid)try{Input input=mapper.readValue(r.payload,Input.class);eligible(t,o,input,now(),false);}catc
  h(ResponseStatusException e){valid=false;reason="RESOURCE_NOT_ELIGIBLE";}catch(Exception e){throw new
  IllegalStateException("Invalid reservation payload");}
```

Revalida recurso confirmado sin caducidad por edad de ubicación. Conserva compatibilidad, disponibilidad, vínculo, cobertura y habilitación; transforma incumplimiento en eligible=false. La rama condicional aborta ante la condición inválida. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L71

```
    return new Eligibility(r.id,r.requestId,r.towTruckId,r.operatorId,o.keycloakSubject,valid,reason,t.lat
  itude,t.longitude,t.locationUpdatedAt,o.qualificationValidUntil,now());
```

Devuelve snapshot de aptitud y coordenadas para revalidar despacho; no crea reserva nueva.

L72

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L73

```
   public View release(UUID id){ResourceReservation r=lock(id);if(!Set.of("RELEASED","EXPIRED").contains(r
  .status)){if(r.status.equals("CONFIRMED")){Operator o=org.hibernate.Hibernate.unproxy(operators.findById
  (r.operatorId).orElseThrow(),Operator.class);if(o.availabilityStatus==Availability.AVAILABLE){o.availabl
  eSince=now();o.updatedAt=now();}}r.status="RELEASED";r.releasedAt=now();}return view(r);}
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro. La rama condicional selecciona la actualización o retorno

correspondiente. Cambios: o.availableSince recibe now(); o.updatedAt recibe now(); r.status recibe "RELEASED"; r.releasedAt recibe now().

L74

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

6. Vehículos, solicitudes y reglas

La exclusividad es por vehículo, no por conductor. firstAssignedAt hace irreversible la prohibición de cancelar.

DomainTypes (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/DomainTypes.java

SHA-256: 15ea37a4add9712df187fa559331c4dc5a11df04b8c72289884fcf8179a16033

Enumeraciones de Assistance: tipo de vehículo, movilidad, registro y ciclo de vida de la solicitud. Los estados de asignación y reserva se almacenan aparte.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
public final class DomainTypes {
```

Declara clase DomainTypes.

L3

```
  private DomainTypes() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L4

```
  public enum VehicleType { CAR, SUV, PICKUP }
```

Declara enumeración cerrada VehicleType.

L5

```
  public enum Registration { ACTIVE, INACTIVE }
```

Declara enumeración cerrada Registration.

L6

```
  public enum Mobility { ROLLABLE, NON_ROLLABLE }
```

Declara enumeración cerrada Mobility.

L7

```
  public enum Status { REQUESTED, PENDING_EVALUATION, AWAITING_ACCEPTANCE, ASSIGNED,
SEARCHING_ALTERNATIVE, DISPATCHED, ARRIVED, VEHICLE_LOADED, IN_TRANSIT, COMPLETED, REJECTED,
UNFULFILLED, CANCELLED }
```

Declara enumeración cerrada Status.

L8

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RequestController (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestController.java

SHA-256: f83d39971758751311f2627b06e5e7f02b58363b1c41678d17fad97db0846c37

API de vehículos propios y solicitudes. Crear una solicitud persiste, evalúa y propone; devolver 201 indica creación, no necesariamente asignación exitosa.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.net.URI;
```

Importa tipos: java.net.URI. Tipo auxiliar usado por las expresiones del archivo.

L4

```
import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas.

L5

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import org.springframework.http.ResponseEntity;
```

Importa tipos: org.springframework.http.ResponseEntity. Estados, headers, tipos de contenido y respuestas HTTP.

L7

```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad.

L8

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L9

```
import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación.

L10

```
import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación.

L11

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación.

L12

```
import static com.auxiruta.assistance.requests.RequestDtos.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.RequestDtos.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L13

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L14

```
@RestController @RequestMapping("/api/v1") @SecurityRequirement(name="bearerAuth")
```

Expone HTTP y serializa JSON. Documenta autenticación; no concede permisos. Ruta /api/v1 combinada con el prefijo de clase cuando corresponda.

L15

```
@ApiResponses({@ApiResponse(responseCode="400",description="Entrada
inválida"),@ApiResponse(responseCode="401",description="Autenticación
requerida"),@ApiResponse(responseCode="403",description="Rol
insuficiente"),@ApiResponse(responseCode="404",description="Recurso inexistente o
ajeno"),@ApiResponse(responseCode="409",description="Conflicto de estado, exclusividad o
idempotencia")})
```

Documenta código HTTP.

L16

```
public class RequestController {
```

Declara clase RequestController.

L17

```
 private final RequestService service;private final RulesConfiguration rules;private final
EvaluationService evaluation;private final AssignmentCoordinator assignments;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L18

```
 public RequestController(RequestService service,RulesConfiguration rules,EvaluationService
evaluation,AssignmentCoordinator assignments) {
this.service=service;this.rules=rules;this.evaluation=evaluation;this.assignments=assignments; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L19

```
@ApiResponse(responseCode="201",description="Vehículo creado con Location")
```

Documenta código HTTP.

L20

```
@PostMapping("/vehicles") @Operation(summary="DRIVER: registrar vehículo propio ACTIVE")
```

Ruta /vehicles combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: registrar vehículo propio ACTIVE.

L21

```
 public ResponseEntity<VehicleView> createVehicle(@AuthenticationPrincipal Jwt jwt,@Valid @RequestBody
CreateVehicle d) { var v=service.createVehicle(jwt.getSubject(),d);return
ResponseEntity.created(URI.create("/api/v1/vehicles/"+v.id())).body(v); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Responde 201 con Location.

L22

```
@GetMapping("/vehicles") @Operation(summary="DRIVER: listar exclusivamente vehículos propios")
```

Ruta /vehicles combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: listar exclusivamente vehículos propios.

L23

```
 public PageView<VehicleView> vehicles(@AuthenticationPrincipal Jwt jwt,@RequestParam(defaultValue="0")
int page,@RequestParam(defaultValue="20") int size,@RequestParam(required=false) Registration
registrationStatus) { return service.vehicles(jwt.getSubject(),page,size,registrationStatus); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: registrationStatus: alta administrativa ACTIVE/INACTIVE.

L24

```
 @GetMapping("/vehicles/{id}") @Operation(summary="DRIVER: consultar vehículo propio; ajeno e
inexistente devuelven 404")
```

Ruta /vehicles/{id} combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: consultar vehículo propio; ajeno e inexistente devuelven 404.

L25

```
  public VehicleView vehicle(@PathVariable UUID id,@AuthenticationPrincipal Jwt jwt) { return
service.vehicle(id,jwt.getSubject()); }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L26

```
  @PatchMapping("/vehicles/{id}") @Operation(summary="DRIVER: editar vehículo propio sin solicitud
activa")
```

Ruta /vehicles/{id} combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: editar vehículo propio sin solicitud activa.

L27

```
  public VehicleView edit(@PathVariable UUID id,@AuthenticationPrincipal Jwt jwt,@Valid @RequestBody
EditVehicle d) { return service.editVehicle(id,jwt.getSubject(),d); }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L28

```
  @PatchMapping("/vehicles/{id}/registration-status") @Operation(summary="DRIVER: activar/inactivar
vehículo sin solicitud activa")
```

Ruta /vehicles/{id}/registration-status combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: activar/inactivar vehículo sin solicitud activa.

L29

```
  public VehicleView registration(@PathVariable UUID id,@AuthenticationPrincipal Jwt jwt,@Valid
@RequestBody RegistrationChange d) { return service.registration(id,jwt.getSubject(),d); }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L30

```
  @ApiResponses({@ApiResponse(responseCode="201",description="Solicitud creada con
Location"),@ApiResponse(responseCode="200",description="Solicitud recuperada por Idempotency-Key")})
```

Documenta código HTTP.

L31

```
  @PostMapping("/assistance-requests") @Operation(summary="DRIVER: crear y evaluar solicitud con
snapshot; repetir Idempotency-Key y cuerpo recupera la solicitud")
```

Ruta /assistance-requests combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: crear y evaluar solicitud con snapshot; repetir Idempotency-Key y cuerpo recupera la solicitud.

L32

```
  public ResponseEntity<RequestView> create(@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key,@Valid @RequestBody CreateRequest d) {
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON.

L33

```
   var result=service.createRequest(jwt.getSubject(),key,d);if(result.created()){evaluation.evaluate(resu
lt.request().id());assignments.propose(result.request().id());}return ResponseEntity.status(result.creat
ed()?201:200).location(URI.create("/api/v1/assistance-requests/"+result.request().id())).body(service.re
quest(result.request().id(),jwt));
```

Construye respuesta con estado explícito y cuerpo ProblemDetail. Identidad obtenida del JWT, no del JSON enviado por el usuario. La rama condicional selecciona la actualización o retorno correspondiente.

L34

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L35

```
  @GetMapping("/assistance-requests") @Operation(summary="DRIVER: solicitudes propias; ADMIN: globales;
 OPERATOR: solicitudes con propuestas o asignaciones propias")
```

Ruta /assistance-requests combinada con el prefijo de clase cuando corresponda. Operación: DRIVER: solicitudes propias; ADMIN: globales; OPERATOR: solicitudes con propuestas o asignaciones propias.

L36

```
  public PageView<RequestView> requests(@AuthenticationPrincipal Jwt jwt,@RequestParam(defaultValue="0")
 int page,@RequestParam(defaultValue="20") int size,@RequestParam(required=false) Status status) { return
  service.requests(jwt,page,size,status); }
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L37

```
  @GetMapping("/assistance-requests/{id}") @Operation(summary="Consultar solicitud autorizada; fuera del
 ámbito devuelve 404")
```

Ruta /assistance-requests/{id} combinada con el prefijo de clase cuando corresponda. Operación: Consultar solicitud autorizada; fuera del ámbito devuelve 404.

L38

```
  public RequestView request(@PathVariable UUID id,@AuthenticationPrincipal Jwt jwt) { return
 service.request(id,jwt); }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Campos: id: identificador UUID del registro.

L39

```
  @GetMapping("/assistance-requests/{id}/history") @Operation(summary="Consultar historial paginado
 dentro del mismo ámbito de la solicitud")
```

Ruta /assistance-requests/{id}/history combinada con el prefijo de clase cuando corresponda. Operación: Consultar historial paginado dentro del mismo ámbito de la solicitud.

L40

```
  public PageView<HistoryView> history(@PathVariable UUID id,@AuthenticationPrincipal Jwt
 jwt,@RequestParam(defaultValue="0") int page,@RequestParam(defaultValue="20") int size) { return
 service.history(id,jwt,page,size); }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Campos: id: identificador UUID del registro.

L41

```
  @PostMapping("/assistance-requests/{id}/cancellations") @Operation(summary="DRIVER propietario:
 cancelar antes de primera aceptación; Idempotency-Key identifica el evento durable")
```

Ruta /assistance-requests/{id}/cancellations combinada con el prefijo de clase cuando corresponda. Operación: DRIVER propietario: cancelar antes de primera aceptación; Idempotency-Key identifica el evento durable.

L42

```
  public HistoryView cancel(@PathVariable UUID id,@AuthenticationPrincipal Jwt
 jwt,@RequestHeader("Idempotency-Key") String key,@Valid @RequestBody Cancellation d) { var
 result=service.cancel(id,jwt.getSubject(),key,d);assignments.recover(id);return result; }
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L43

```
  @GetMapping("/configuration/assistance") @Operation(summary="ADMIN: consultar reglas efectivas sin
 secretos")
```

Ruta /configuration/assistance combinada con el prefijo de clase cuando corresponda. Operación: ADMIN: consultar reglas efectivas sin secretos.

L44

```
  public java.util.Map<String,Object> configuration() { return rules.effective(); }
```

Expone reglas efectivas al ADMIN; no devuelve secretos de infraestructura.

L45

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RequestDtos (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestDtos.java

SHA-256: 27a881b1526e3e4f97d0c5e6fb526a8b965fb461ba31c499f7aba1a573929d6c

DTO de vehículos, solicitudes, historial y paginación. Los cuerpos se validan antes de ingresar en las reglas del servicio.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L6

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L7

```
public final class RequestDtos {
```

Declara clase RequestDtos.

L8

```
  private RequestDtos() {}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  public record CreateVehicle(@NotBlank @Size(max=30) String licensePlate,@Size(max=100) String
brand,@Size(max=100) String model,
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable CreateVehicle. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado.

L10

```
    @NotNull VehicleType vehicleType,@NotNull @DecimalMin("500") @DecimalMax("3000")
@Digits(integer=4,fraction=2) BigDecimal declaredWeightKg) {}
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg.

L11

```
  public record EditVehicle(@Size(max=30) String licensePlate,@Size(max=100) String brand,@Size(max=100)
 String model,
```

Limita caracteres/elementos según min/max. Declara record inmutable EditVehicle. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado.

L12

```
   VehicleType vehicleType,@DecimalMin("500") @DecimalMax("3000") @Digits(integer=4,fraction=2)
 BigDecimal declaredWeightKg) {}
```

Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg.

L13

```
  public record RegistrationChange(@NotNull Registration status) {}
```

Exige no null. Declara record inmutable RegistrationChange. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L14

```
  public record CreateRequest(@NotNull UUID vehicleId,
```

Exige no null. Declara record inmutable CreateRequest. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L15

```
   @NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer=3,fraction=6) BigDecimal originLatitude,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: originLatitude: latitud de recogida.

L16

```
   @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer=3,fraction=6) BigDecimal
 originLongitude,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: originLongitude: longitud de recogida.

L17

```
   @NotNull @DecimalMin("-90") @DecimalMax("90") @Digits(integer=3,fraction=6) BigDecimal
 destinationLatitude,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: destinationLatitude: latitud de entrega.

L18

```
   @NotNull @DecimalMin("-180") @DecimalMax("180") @Digits(integer=3,fraction=6) BigDecimal
 destinationLongitude,
```

Exige no null. Mínimo decimal inclusive por defecto. Máximo decimal inclusive por defecto. Limita cifras enteras y decimales. Campos: destinationLongitude: longitud de entrega.

L19

```
   @NotNull Mobility mobilityCondition,@NotBlank @Size(max=1000) String description) {}
```

Exige no null. Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Campos: mobilityCondition: condición ROLLABLE/NON_ROLLABLE; description: descripción de asistencia.

L20

```
  public record Cancellation(@NotBlank @Size(max=500) String reason) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Cancellation. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: reason: motivo legible.

L21

```
 public record VehicleView(UUID id,String licensePlate,String brand,String model,VehicleType
vehicleType,BigDecimal declaredWeightKg,Registration registrationStatus,Instant createdAt,Instant
updatedAt,long version) {}
```

Declara record inmutable VehicleView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; licensePlate: patente normalizada y única; brand: marca declarada; model: modelo declarado; vehicleType: tipo CAR/SUV/PICKUP; declaredWeightKg: peso declarado en kg; registrationStatus: alta administrativa ACTIVE/INACTIVE; createdAt: fecha de creación; updatedAt: fecha de modificación; version: versión para control optimista de concurrencia.

L22

```
 public record RequestView(UUID id,UUID vehicleId,Status status,Instant firstAssignedAt,String
closureReasonCode,
```

Declara record inmutable RequestView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; status: estado del objeto; su enum o CHECK define valores permitidos; firstAssignedAt: primera aceptación que impide cancelar incluso durante reemplazos; closureReasonCode: causa codificada del cierre.

L23

```
  BigDecimal originLatitude,BigDecimal originLongitude,BigDecimal destinationLatitude,BigDecimal
destinationLongitude,
```

Campos: originLatitude: latitud de recogida; originLongitude: longitud de recogida; destinationLatitude: latitud de entrega; destinationLongitude: longitud de entrega.

L24

```
  VehicleType declaredVehicleType,BigDecimal declaredWeightKg,Mobility mobilityCondition,String
description,
```

Campos: declaredVehicleType: copia del tipo de vehículo al solicitar; declaredWeightKg: peso declarado en kg; mobilityCondition: condición ROLLABLE/NON_ROLLABLE; description: descripción de asistencia.

L25

```
  int searchAttemptNumber,Instant searchDeadline,String configurationRevision,String
configurationSnapshot,
```

Campos: searchAttemptNumber: número de búsqueda de 1 a 3; searchDeadline: vencimiento del intento; configurationRevision: versión de reglas; configurationSnapshot: JSON de parámetros vigente al crear.

L26

```
  Instant createdAt,Instant updatedAt,Instant closedAt,long version,boolean
resourceReleasePending,EvaluationReadService.EvaluationView latestEvaluation) {}
```

Campos: createdAt: fecha de creación; updatedAt: fecha de modificación; closedAt: fecha de cierre; null indica abierto; version: versión para control optimista de concurrencia; resourceReleasePending: liberación remota pendiente que impide reutilización.

L27

```
 public record HistoryView(UUID id,UUID requestId,Status previousStatus,Status newStatus,String
actorType,String actorSubject,String reason,String details,String operationKey,Instant occurredAt) {}
```

Declara record inmutable HistoryView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; requestId: UUID de solicitud; previousStatus: estado previo; newStatus: estado resultante; actorType: tipo de actor SYSTEM/DRIVER/OPERATOR; actorSubject: sub del usuario que produjo el evento; reason: motivo legible; details: evidencia adicional; operationKey: clave durable de acción; occurredAt: instante del evento.

L28

```
public record PageView<T>(List<T> content,int page,int size,long totalElements,int totalPages) {}
```

Declara record inmutable PageView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L29

```
public record Created(RequestView request,boolean created) {}
```

Declara record inmutable Created. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L30

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RequestService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RequestService.java`

`SHA-256: 2b8a82ec34e8c82381ae036cbc8a2f51383a1ba799309f6f1afbaa3119658582`

Gestiona propiedad, normalización, exclusividad por vehículo y cancelación previa a primera aceptación. Serializa operaciones con bloqueo del vehículo y conserva huellas de idempotencia.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import java.nio.charset.StandardCharsets;
```

Importa tipos: java.nio.charset.StandardCharsets. Codificación UTF-8 explícita al construir huellas.

L7

```
import java.security.MessageDigest;
```

Importa tipos: java.security.MessageDigest. MessageDigest para SHA-256; no almacena contraseñas.

L8

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L9

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L10

```
import org.springframework.data.domain.*;
```

Importa tipos: org.springframework.data.domain.*. Repositorios Spring Data y estructuras de paginación/orden.

L11

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L12

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L13

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L14

```
import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía.

L15

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L16

```
import static com.auxiruta.assistance.requests.RequestDtos.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.RequestDtos.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L18

```
@Service @Transactional(readOnly=true)
```

Registra lógica de aplicación como bean. Delimita transacción local; readOnly=true declara consulta.

L19

```
public class RequestService {
```

Declara clase RequestService.

L20

```
  private final VehicleRepository vehicles;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L21

```
  private final RequestRepository requests;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L22

```
  private final HistoryRepository histories;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L23

```
  private final RulesConfiguration rules;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L24

```
  private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L25

```
  private final EvaluationReadService evaluations;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L26

```
  @org.springframework.beans.factory.annotation.Autowired private
org.springframework.jdbc.core.JdbcTemplate jdbc;
```

Inyecta dependencia Spring.

L27

```
  public RequestService(VehicleRepository vehicles,RequestRepository requests,HistoryRepository
histories,RulesConfiguration rules,Clock clock,EvaluationReadService evaluations) {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L28

```
   this.vehicles=vehicles;this.requests=requests;this.histories=histories;this.rules=rules;this.clock=clo
ck;this.evaluations=evaluations;
```

Asigna o consulta campos de la instancia.

L29

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L30

```
  private Instant now() { return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS); }
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L31

```
  private static ResponseStatusException missing() { return new
ResponseStatusException(HttpStatus.NOT_FOUND); }
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L32

```
  static void require(boolean ok,HttpStatus status) { if(!ok) throw new ResponseStatusException(status);
}
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L33

```
  private static void key(String value) { require(value!=null && !value.isBlank() &&
value.length()<=100,HttpStatus.BAD_REQUEST); }
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L34

```
  static String plate(String value) { String p=value.replace(" ","").replace("-","").toUpperCase(Locale.R
OOT);require(p.matches("[A-Z0-9]{6,10}"),HttpStatus.BAD_REQUEST);return p; }
```

Quita espacios/guiones, convierte a mayúsculas y exige entre 6 y 10 caracteres alfanuméricos. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L35

```
  private static Pageable page(int p,int size,String sort) { require(p>=0 && size>=1 &&
size<=100,HttpStatus.BAD_REQUEST);return PageRequest.of(p,size,Sort.by(sort).and(Sort.by("id"))); }
```

Construye página ordenada por fecha y UUID, estable incluso cuando coinciden fechas. Valida número/tamaño y aplica orden estable para paginar. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L36

```
  private static <T> PageView<T> paged(Page<T> p) { return new
PageView<>(p.getContent(),p.getNumber(),p.getSize(),p.getTotalElements(),p.getTotalPages()); }
```

Proyecta contenido, página, tamaño, total de elementos y páginas al contrato REST.

L37

```
  private Vehicle owned(UUID id,String subject) { return
vehicles.findByIdAndOwnerSubject(id,subject).orElseThrow(RequestService::missing); }
```

Si no encuentra resultado lanza la excepción indicada. Busca por ID y propietario conjuntamente, ocultando recursos ajenos con 404. Filtra por propietario en la consulta misma. Campos: id: identificador UUID del registro.

L38

```
  private Vehicle locked(UUID id,String subject) { return
vehicles.lockOwned(id,subject).orElseThrow(RequestService::missing); }
```

Si no encuentra resultado lanza la excepción indicada. Campos: id: identificador UUID del registro.

L39

```
  private void free(UUID id) { require(!requests.hasActive(id),HttpStatus.CONFLICT); }
```

Impide editar/reutilizar un vehículo con solicitud activa o liberación pendiente. Campos: id: identificador UUID del registro. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L40

```
  private boolean admin(Jwt jwt) { return TokenSupport.permits(jwt,false,"ADMIN"); }
```

Comprueba ADMIN humano con audiencia/cliente válidos, no solo el texto del rol.

L41

```
  private boolean driver(Jwt jwt) { return TokenSupport.permits(jwt,false,"DRIVER"); }
```

Comprueba DRIVER humano; ADMIN no se trata automáticamente como conductor.

L42

```
  private AssistanceRequest visible(UUID id,Jwt jwt) {
```

Aplica alcance por identidad/rol antes de devolver un recurso. Campos: id: identificador UUID del registro.

L43

```
    AssistanceRequest req=requests.findById(id).orElseThrow(RequestService::missing);
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L44

```
    if(!admin(jwt)) require((driver(jwt) && req.driverSubject.equals(jwt.getSubject())) ||
  (TokenSupport.permits(jwt,false,"OPERATOR") && jdbc.queryForObject("select count(*) from assignments
  where request_id=? and operator_subject=?",Long.class,id,jwt.getSubject())>0),HttpStatus.NOT_FOUND);
```

Identidad obtenida del JWT, no del JSON enviado por el usuario. Consulta assignments contando coincidencias. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L45

```
    return req;
```

Entrega req al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L46

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L47

```
  private VehicleView view(Vehicle v) { return new VehicleView(v.id,v.licensePlate,v.brand,v.model,v.vehi
  cleType,v.declaredWeightKg,v.registrationStatus,v.createdAt,v.updatedAt,v.version); }
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L48

```
  private RequestView view(AssistanceRequest r) { return new RequestView(r.id,r.vehicle.getId(),r.status,
  r.firstAssignedAt,r.closureReasonCode,r.originLatitude,r.originLongitude,r.destinationLatitude,r.destina
  tionLongitude,r.declaredVehicleType,r.declaredWeightKg,r.mobilityCondition,r.description,r.searchAttempt
  Number,r.searchDeadline,r.configurationRevision,r.configurationSnapshot,r.createdAt,r.updatedAt,r.closed
  At,r.version,r.resourceReleasePending,evaluations.latest(r.id)); }
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L49

```
  private HistoryView view(RequestStatusHistory h) { return new HistoryView(h.id,h.request.getId(),h.prev
  iousStatus,h.newStatus,h.actorType,h.actorSubject,h.reason,h.details,h.operationKey,h.occurredAt); }
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L50

```
  public VehicleView vehicle(UUID id,String subject) { return view(owned(id,subject)); }
```

Campos: id: identificador UUID del registro.

L51

```
  public PageView<VehicleView> vehicles(String subject,int p,int size,Registration status) { Pageable
pageable=page(p,size,"createdAt");return paged((status==null?vehicles.findByOwnerSubject(subject,pageabl
e):vehicles.findByOwnerSubjectAndRegistrationStatus(subject,status,pageable)).map(this::view)); }
```

Valida paginación y ordena solicitudes por creación con UUID de desempate. Transforma cada elemento al valor/tipo de salida. Filtra por propietario en la consulta misma. Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L52

```
  @Transactional public VehicleView createVehicle(String subject,CreateVehicle d) {
```

Delimita transacción local; readOnly=true declara consulta.

L53

```
   Vehicle v=new Vehicle();v.ownerSubject=subject;v.licensePlate=plate(d.licensePlate());v.brand=d.brand(
);v.model=d.model();v.vehicleType=d.vehicleType();v.declaredWeightKg=d.declaredWeightKg();v.createdAt=v.
updatedAt=now();vehicles.saveAndFlush(v);return view(v);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción.

L54

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L55

```
  @Transactional public VehicleView editVehicle(UUID id,String subject,EditVehicle d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L56

```
   require(d.licensePlate()!=null || d.brand()!=null || d.model()!=null || d.vehicleType()!=null ||
d.declaredWeightKg()!=null,HttpStatus.BAD_REQUEST);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L57

```
   Vehicle v=locked(id,subject);free(id);
```

Bloquea el vehículo propio y exige ausencia de solicitud activa antes de editar o inactivar.

L58

```
   if(d.licensePlate()!=null)v.licensePlate=plate(d.licensePlate());if(d.brand()!=null)v.brand=d.brand();
if(d.model()!=null)v.model=d.model();if(d.vehicleType()!=null)v.vehicleType=d.vehicleType();if(d.declare
dWeightKg()!=null)v.declaredWeightKg=d.declaredWeightKg();v.updatedAt=now();vehicles.flush();return
view(v);
```

Sincroniza cambios JPA con SQL; no es commit independiente. La rama condicional selecciona la actualización o retorno correspondiente.

L59

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L60

```
  @Transactional public VehicleView registration(UUID id,String subject,RegistrationChange d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro.

L61

```
   Vehicle
v=locked(id,subject);free(id);v.registrationStatus=d.status();v.updatedAt=now();vehicles.flush();return
view(v);
```

Bloquea el vehículo propio y exige ausencia de solicitud activa antes de editar o inactivar. Sincroniza cambios JPA con SQL; no es commit independiente.

L62

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L63

```
  static String digest(String text) { try { return HexFormat.of().formatHex(MessageDigest.getInstance("SH
A-256").digest(text.getBytes(StandardCharsets.UTF_8))); } catch(java.security.NoSuchAlgorithmException
e) { throw new IllegalStateException(e); } }
```

Genera SHA-256 hexadecimal usando UTF-8; es una huella de contenido, no cifrado de contraseñas. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L64

```
  private static String coordinate(BigDecimal n) { return n.stripTrailingZeros().toPlainString(); }
```

Normaliza decimales para que valores equivalentes generen la misma clave o huella.

L65

```
  static String fingerprint(CreateRequest d) {
```

Calcula una huella del contenido para distinguir repetición idéntica de reutilización conflictiva de una clave.

L66

```
    // Length-prefixed fields prevent ambiguities when free text contains delimiters.
```

Comentario de intención o documentación; no se ejecuta.

L67

```
    List<String> fields=List.of(d.vehicleId().toString(),coordinate(d.originLatitude()),coordinate(d.origi
nLongitude()),coordinate(d.destinationLatitude()),coordinate(d.destinationLongitude()),d.mobilityConditi
on().name(),d.description().strip());
```

Define los campos de contenido que identifican semánticamente la solicitud para idempotencia.

L68

```
    return
digest(fields.stream().map(v->v.length()+":"+v).collect(java.util.stream.Collectors.joining()));
```

Prefija longitud de cada campo antes de hashear; evita ambigüedad por delimitadores en texto libre. Transforma cada elemento al valor/tipo de salida.

L69

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L70

```
  @Transactional public Created createRequest(String subject,String operationKey,CreateRequest d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: operationKey: clave durable de acción.

L71

```
    key(operationKey);require(d.originLatitude().compareTo(d.destinationLatitude())!=0 ||
d.originLongitude().compareTo(d.destinationLongitude())!=0,HttpStatus.BAD_REQUEST);
```

Exige clave no nula, no vacía y de hasta cien caracteres antes de operar. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L72

```
    String fingerprint=fingerprint(d);
```

Calcula huella normalizada del cuerpo antes de buscar una operación previa.

L73

```
    // Locking the vehicle also serializes competing creation, edits and cancellation.
```

Comentario de intención o documentación; no se ejecuta.

L74

```
    Vehicle v=locked(d.vehicleId(),subject);
```

Bloquea únicamente el vehículo del conductor autenticado; serializa creación concurrente, edición y cancelación.

L75

```
    var previous=requests.findByDriverSubjectAndIdempotencyKey(subject,operationKey);
```

Busca resultado anterior para repetir sin duplicar.

L76

```
    if(previous.isPresent()) {
  require(previous.get().requestFingerprint.equals(fingerprint),HttpStatus.CONFLICT);return new
  Created(view(previous.get()),false); }
```

Una clave del conductor con mismo contenido devuelve el recurso anterior; contenido distinto es conflicto 409. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L77

```
    require(v.registrationStatus==Registration.ACTIVE,HttpStatus.CONFLICT);free(v.id);
```

Exige vehículo ACTIVE y sin solicitud abierta/liberación pendiente. No impide solicitudes de otros vehículos del mismo conductor. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L78

```
    AssistanceRequest req=new AssistanceRequest();req.vehicle=v;req.driverSubject=subject;req.originLatitu
  de=d.originLatitude();req.originLongitude=d.originLongitude();req.destinationLatitude=d.destinationLatit
  ude();req.destinationLongitude=d.destinationLongitude();req.declaredVehicleType=v.vehicleType;req.declar
  edWeightKg=v.declaredWeightKg;req.mobilityCondition=d.mobilityCondition();req.description=d.description(
  ).strip();req.configurationRevision=rules.revision();req.configurationSnapshot=rules.snapshot();req.idem
  potencyKey=operationKey;req.requestFingerprint=fingerprint;req.createdAt=req.updatedAt=now();req.searchD
  eadline=req.createdAt.plusSeconds(rules.searchSeconds());
```

Crea solicitud con copia de vehículo y reglas, fechas y deadline. Cambiar después el origen de esos datos no reescribe este snapshot. Cambios: req.vehicle recibe v; req.driverSubject recibe subject; req.originLatitude recibe d.originLatitude(); req.originLongitude recibe d.originLongitude(); req.destinationLatitude recibe d.destinationLatitude(); req.destinationLongitude recibe d.destinationLongitude(); req.declaredVehicleType recibe v.vehicleType; req.declaredWeightKg recibe v.declaredWeightKg; req.mobilityCondition recibe d.mobilityCondition(); req.description recibe d.description().strip(); req.configurationRevision recibe rules.revision(); req.configurationSnapshot recibe rules.snapshot(); req.idempotencyKey recibe operationKey; req.requestFingerprint recibe fingerprint; req.createdAt recibe req.updatedAt=now(); req.searchDeadline recibe req.createdAt.plusSeconds(rules.searchSeconds()).

L79

```
    req=requests.saveAndFlush(req);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción.

L80

```
    event(req,null,Status.REQUESTED,subject,"CREATED",null,null);
```

Registra evento inicial CREATED: no hay estado previo y el nuevo es REQUESTED.

L81

```
    return new Created(view(req),true);
```

Devuelve DTO y bandera de creación nueva; el controlador elige 201 en vez de 200 de una repetición.

L82

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L83

```
   private RequestStatusHistory event(AssistanceRequest req,Status previous,Status next,String
  subject,String reason,String operationKey,String fingerprint) {
```

Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible; operationKey: clave durable de acción.

L84

```
   RequestStatusHistory h=new RequestStatusHistory();h.request=req;h.previousStatus=previous;h.newStatus=
 next;h.actorSubject=subject;h.actorType="DRIVER";h.reason=reason;h.operationKey=operationKey;h.requestFi
 ngerprint=fingerprint;h.occurredAt=now();h.details=next==Status.REQUESTED?"searchDeadline="+req.searchDe
 adline:null;histories.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Cambios: h.request recibe req; h.previousStatus recibe previous; h.newStatus recibe next; h.actorSubject recibe subject; h.actorType recibe "DRIVER"; h.reason recibe reason; h.operationKey recibe operationKey; h.requestFingerprint recibe fingerprint; h.occurredAt recibe now(); h.details recibe next==Status.REQUESTED?"searchDeadline="+req.searchDeadline:null.

L85

```
   org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronization(new
 org.springframework.transaction.support.TransactionSynchronization() {
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados.

L86

```
    @Override public void afterCommit() {
 org.slf4j.LoggerFactory.getLogger(RequestService.class).info("Request state changed requestId={} from={}
  to={}",req.id,previous,next); }
```

Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos.

L87

```
   });return h;
```

Termina el callback transaccional y devuelve el evento persistido.

L88

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L89

```
  public RequestView request(UUID id,Jwt jwt) { return view(visible(id,jwt)); }
```

Campos: id: identificador UUID del registro.

L90

```
  public PageView<RequestView> requests(Jwt jwt,int p,int size,Status status) {
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos.

L91

```
   Pageable pageable=page(p,size,"createdAt");
```

Valida paginación y ordena solicitudes por creación con UUID de desempate.

L92

```
   if(!admin(jwt) && !driver(jwt)) return paged(requests.forOperator(jwt.getSubject(),status==null?null:s
 tatus.name(),PageRequest.of(p,size,Sort.by("created_at").and(Sort.by("id")))).map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Identidad obtenida del JWT, no del JSON enviado por el usuario. La rama condicional selecciona la actualización o retorno correspondiente.

L93

```
   return paged(requests.visible(admin(jwt)?null:jwt.getSubject(),status,pageable).map(this::view));
```

Transforma cada elemento al valor/tipo de salida. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L94

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L95

```
 public PageView<HistoryView> history(UUID id,Jwt jwt,int p,int size) { visible(id,jwt);return
paged(histories.findByRequestId(id,page(p,size,"occurredAt")).map(this::view)); }
```

Transforma cada elemento al valor/tipo de salida. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. Campos: id: identificador UUID del registro.

L96

```
 static boolean cancellable(AssistanceRequest r) { return r.firstAssignedAt==null && EnumSet.of(Status.R
EQUESTED,Status.PENDING_EVALUATION,Status.AWAITING_ACCEPTANCE,Status.SEARCHING_ALTERNATIVE).contains(r.s
tatus); }
```

Solo permite cancelar antes de firstAssignedAt y en estados previos a aceptación.

L97

```
@Transactional public HistoryView cancel(UUID id,String subject,String operationKey,Cancellation d) {
```

Delimita transacción local; readOnly=true declara consulta. Campos: id: identificador UUID del registro; operationKey: clave durable de acción.

L98

```
  key(operationKey);
```

Exige clave no nula, no vacía y de hasta cien caracteres antes de operar.

L99

```
  UUID vehicleId=requests.vehicleId(id).orElseThrow(RequestService::missing);locked(vehicleId,subject);
```

Si no encuentra resultado lanza la excepción indicada.

L100

```
  AssistanceRequest req=requests.lockById(id).orElseThrow(RequestService::missing);require(req.driverSub
ject.equals(subject),HttpStatus.NOT_FOUND);
```

Si no encuentra resultado lanza la excepción indicada. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L101

```
  String fingerprint=digest(d.reason().strip());var
previous=histories.findByRequestIdAndOperationKey(id,operationKey);
```

Busca resultado anterior para repetir sin duplicar.

L102

```
  if(previous.isPresent()) { require(previous.get().actorSubject.equals(subject) &&
previous.get().requestFingerprint.equals(fingerprint),HttpStatus.CONFLICT);return view(previous.get());
}
```

Una clave del conductor con mismo contenido devuelve el recurso anterior; contenido distinto es conflicto 409. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L103

```
  require(cancellable(req),HttpStatus.CONFLICT);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L104

```
  if(jdbc.queryForObject("select count(*) from pending_actions p join assignments a on
a.id=p.assignment_id where a.request_id=? and p.kind='CONFIRM' and
p.status<>'COMPLETED'",Long.class,id)>0)throw new
ResponseStatusException(HttpStatus.CONFLICT,"ACCEPTANCE_CONFIRMATION_PENDING");
```

Protege la carrera aceptación/cancelación: una confirmación remota incierta puede haber ocurrido. Impide cancelar hasta reconciliar, con código distinguible en el error. Consulta pending_actions, assignments contando coincidencias. La rama condicional aborta ante la condición inválida.

L105

```
  var assignmentIds=jdbc.queryForList("select id from assignments where request_id=? and status in
('RESERVING','PROPOSED')",UUID.class,id);
```

Consulta assignments con el filtro y proyección escritos.

L106

```
    for(UUID assignment:assignmentIds){
```

Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L107

```
     jdbc.update("update assignments set status='CANCELLED',closed_at=? where
  id=?",java.sql.Timestamp.from(now()),assignment);
```

Actualiza assignments para las filas del filtro.

L108

```
     // Resolve an uncertain RESERVE first; its completion creates the compensating RELEASE.
```

Comentario de intención o documentación; no se ejecuta.

L109

```
     long pending=jdbc.queryForObject("select count(*) from pending_actions where assignment_id=? and
  kind='RESERVE' and status<>'COMPLETED'",Long.class,assignment);
```

Consulta pending_actions contando coincidencias.

L110

```
     if(pending==0)jdbc.update("insert into
  pending_actions(id,assignment_id,kind,status,created_at,updated_at) values (?,?,'RELEASE','PENDING',?,?)
   on conflict(assignment_id,kind) do
  nothing",UUID.randomUUID(),assignment,java.sql.Timestamp.from(now()),java.sql.Timestamp.from(now()));
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. La rama condicional selecciona la actualización o retorno correspondiente.

L111

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L112

```
    req.resourceReleasePending=req.resourceReleasePending || !assignmentIds.isEmpty();
```

Cambios: req.resourceReleasePending recibe req.resourceReleasePending || !assignmentIds.isEmpty().

L113

```
    Status old=req.status;req.status=Status.CANCELLED;req.closureReasonCode="DRIVER_CANCELLED";req.closedA
  t=req.updatedAt=now();requests.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: req.status recibe Status.CANCELLED; req.closureReasonCode recibe "DRIVER_CANCELLED"; req.closedAt recibe req.updatedAt=now().

L114

```
    return view(event(req,old,Status.CANCELLED,subject,d.reason().strip(),operationKey,fingerprint));
```

Persiste y devuelve evento de cancelación con actor, motivo normalizado, clave y huella para repetir sin duplicar.

L115

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L116

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 17: separadores sin instrucciones.

RulesConfiguration (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/RulesConfiguration.java`

`SHA-256: cb8f605437ac46d9c2a37ce3319023df893363885b513def5a7af82c179f5fe2`

Lee rules/mvp.yml y construye el snapshot JSON. Si está activo demo sustituye cinco tiempos de espera; no acelera cuotas ni modifica solicitudes existentes.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.Map;
```

Importa tipos: java.util.Map. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.io.InputStream;
```

Importa tipos: java.io.InputStream. Tipo auxiliar usado por las expresiones del archivo.

L4

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L5

```
import org.springframework.core.io.ClassPathResource;
```

Importa tipos: org.springframework.core.io.ClassPathResource. Tipo auxiliar usado por las expresiones del archivo.

L6

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L7

```
import org.yaml.snakeyaml.Yaml;
```

Importa tipos: org.yaml.snakeyaml.Yaml. Tipo auxiliar usado por las expresiones del archivo.

L8

```
@Component
```

Registra componente inyectable.

L9

```
public class RulesConfiguration {
```

Declara clase RulesConfiguration.

L10

```
  private final String snapshot;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L11

```
  private final Map<String,Object> rules;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
  public RulesConfiguration(ObjectMapper mapper) throws Exception {this(mapper,new
org.springframework.core.env.StandardEnvironment());}
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
  @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L14

```
  public RulesConfiguration(ObjectMapper mapper,org.springframework.core.env.Environment environment)
 throws Exception {
```

Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L15

```
   try(InputStream in=new ClassPathResource("rules/mvp.yml").getInputStream()) {
```

Abre YAML empaquetado en classpath; try-with-resources cierra el stream automáticamente.

L16

```
    Map<String,Object> document=new Yaml().load(in);
```

Parsea YAML a mapa de datos; no ejecuta instrucciones del archivo.

L17

```
    rules=mapper.convertValue(document.get("auxiruta"),new
com.fasterxml.jackson.core.type.TypeReference<Map<String,Object>>() {});
```

Extrae el bloque auxiruta y lo convierte a mapa apto para serialización JSON.

L18

```
    if(java.util.Arrays.asList(environment.getActiveProfiles()).contains("demo")) {
```

La rama condicional selecciona la actualización o retorno correspondiente.

L19

```
     rules.put("rules-version","AR-MVP-5-DEMO");
```

Identifica en el snapshot que los plazos reducidos son de demostración.

L20

```
     override("search","attempt-timeout-seconds",60);override("search","evaluation-interval-seconds",2);o
verride("search","minimum-offer-window-seconds",3);override("fleet","hold-seconds",10);override("executi
on","dispatch-deadline-seconds",20);
```

Demo: búsqueda 60 s, evaluación 2 s, ventana mínima 3 s, retención 10 s y despacho 20 s. Conserva los demás parámetros.

L21

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L22

```
    snapshot=mapper.writeValueAsString(rules);
```

Serializa reglas efectivas una vez para copiarlas a solicitudes nuevas.

L23

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L24

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L25

```
  @SuppressWarnings("unchecked") private void override(String group,String name,int
value){((Map<String,Object>)rules.get(group)).put(name,value);}
```

Sustituye una clave dentro del submapa de reglas; el cast se basa en estructura YAML conocida.

L26

```
  public String snapshot() { return snapshot; }
```

Devuelve JSON de reglas a persistir en la solicitud.

L27

```
  public String revision() { return rules.get("rules-version").toString(); }
```

Devuelve identificador de revisión de reglas.

L28

```
  public int searchSeconds() { return
((Number)((Map<?,?>)rules.get("search")).get("attempt-timeout-seconds")).intValue(); }
```

Devuelve duración de búsqueda del perfil efectivo para calcular deadline.

L29

```
  public Map<String,Object> effective() { return rules; }
```

Devuelve mapa de configuración efectiva usado por consulta administrativa.

L30

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

mvp (assistance-service)

`Ruta: assistance-service/src/main/resources/rules/mvp.yml`

`SHA-256: 0284c4c3bf479f0b6f9a58f45f6f4ae9d308ba19c30ba22901e184bc3a91932d`

Parámetros de referencia del dominio que RulesConfiguration copia a cada solicitud; algunas restricciones también están aplicadas explícitamente en Java y SQL.

L1

```
# AR-MVP-5: referencia funcional; distribuir en cada servicio según su responsabilidad.
```

Comentario o directiva del intérprete; no es regla de negocio.

L2

```
# Valores del proyecto, no límites legales ni garantía de tiempos reales.
```

Comentario o directiva del intérprete; no es regla de negocio.

L3

```
auxiruta:
```

Abre bloque auxiruta; sus hijos configuran este componente.

L4

```
  rules-version: AR-MVP-5
```

Identifica las reglas copiadas al snapshot. Valor: AR-MVP-5.

L5

```
  display-time-zone: America/Argentina/Cordoba
```

Zona de presentación; fechas persistidas en UTC. Valor: America/Argentina/Cordoba.

L6

```
  coverage:
```

Abre bloque coverage; sus hijos configuran este componente.

L7

```
    center-latitude: -31.4167
```

Latitud del centro próximo a Plaza San Martín. Valor: -31.4167.

L8

```
    center-longitude: -64.1833
```

Longitud del centro de cobertura. Valor: -64.1833.

L9

```
    radius-meters: 15000
```

Radio comercial, no límite municipal. Valor: 15000.

L10

```
    haversine-earth-radius-meters: 6371000
```

Radio terrestre usado en Haversine. Valor: 6371000.

L11

```
    boundary-inclusive: true
```

Admite exactamente el borde. Valor: true.

L12

```
  vehicles:
```

Abre bloque vehicles; sus hijos configuran este componente.

L13

```
    allowed-types: [CAR, SUV, PICKUP]
```

Tipos de vehículo admitidos. Valor: [CAR, SUV, PICKUP].

L14

```
    min-weight-kg: 500
```

Peso mínimo admitido del vehículo. Valor: 500.

L15

```
    max-weight-kg: 3000
```

Peso máximo admitido del vehículo. Valor: 3000.

L16

```
    min-truck-capacity-kg: 500
```

Capacidad mínima registrable de grúa. Valor: 500.

L17

```
    max-truck-capacity-kg: 5000
```

Capacidad máxima registrable de grúa. Valor: 5000.

L18

```
    required-qualification: LIGHT_TOWING
```

Categoría exigida. Valor: LIGHT_TOWING.

L19

```
  routing:
```

Abre bloque routing; sus hijos configuran este componente.

L20

```
    profile: driving-car
```

Perfil vial ORS driving-car; distinto del perfil Spring demo. Valor: driving-car.

L21

```
    min-transport-meters: 500
```

Traslado vial mínimo. Valor: 500.

L22

```
    max-transport-meters: 30000
```

Traslado vial máximo. Valor: 30000.

L23

```
    max-approach-meters: 15000
```

Aproximación vial máxima. Valor: 15000.

L24

```
      max-arrival-budget-seconds: 2700
```

Presupuesto de llegada con despacho y margen. Valor: 2700.

L25

```
      max-adjusted-transport-seconds: 5400
```

Duración ajustada máxima del traslado. Valor: 5400.

L26

```
      duration-factor: 1.25
```

Multiplicador conservador de tiempos ORS. Valor: 1.25.

L27

```
      arrival-margin-seconds: 300
```

Margen del presupuesto de llegada. Valor: 300.

L28

```
      planning-contingency-seconds: 600
```

Contingencia del fin estimado. Valor: 600.

L29

```
      max-route-age-seconds: 300
```

Vigencia vial al evaluar/proponer y despachar. Valor: 300.

L30

```
      max-snap-distance-meters: 100
```

Desviación máxima al ajustar puntos a la red vial. Valor: 100.

L31

```
    search:
```

Abre bloque search; sus hijos configuran este componente.

L32

```
      attempt-timeout-seconds: 600
```

Plazo por intento de búsqueda. Valor: 600.

L33

```
      max-offers-per-attempt: 5
```

Cupo de propuestas por intento. Valor: 5.

L34

```
      max-candidates-per-round: 5
```

Máximo de candidatos por preselección. Valor: 5.

L35

```
      evaluation-interval-seconds: 30
```

Intervalo de reevaluación. Valor: 30.

L36

```
      max-reassignments: 2
```

Reemplazos adicionales al intento inicial. Valor: 2.

L37

```
      minimum-offer-window-seconds: 15
```

Tiempo mínimo para publicar una propuesta. Valor: 15.

L38

```
  fleet:
```

Abre bloque fleet; sus hijos configuran este componente.

L39

```
    hold-seconds: 60
```

Retención temporal máxima. Valor: 60.

L40

```
    max-location-age-seconds: 300
```

Vigencia de posición para recibir y aceptar propuestas. Valor: 300.

L41

```
    reconciliation-interval-seconds: 15
```

Intervalo de verificación antes de despacho. Valor: 15.

L42

```
  cancellation:
```

Abre bloque cancellation; sus hijos configuran este componente.

L43

```
    allowed-only-before-first-acceptance: true
```

Prohíbe cancelar tras primera aceptación. Valor: true.

L44

```
  assumptions:
```

Abre bloque assumptions; sus hijos configuran este componente.

L45

```
    declared-vehicle-data-correct: true
```

Establece declared-vehicle-data-correct = true dentro del bloque. Se interpreta al cargar la configuración.

L46

```
    pickup-accessible: true
```

Establece pickup-accessible = true dentro del bloque. Se interpreta al cargar la configuración.

L47

```
    destination-available: true
```

Establece destination-available = true dentro del bloque. Se interpreta al cargar la configuración.

L48

```
    operator-continuity: true
```

Establece operator-continuity = true dentro del bloque. Se interpreta al cargar la configuración.

L49

```
    operational-continuity-after-dispatch: true
```

Establece operational-continuity-after-dispatch = true dentro del bloque. Se interpreta al cargar la configuración.

L50

```
  execution:
```

Abre bloque execution; sus hijos configuran este componente.

L51

```
    dispatch-deadline-seconds: 300
```

Plazo exclusivo para declarar salida. Valor: 300.

L52

```
      rollable-loading-seconds: 900
```

Carga prevista si el vehículo rueda. Valor: 900.

L53

```
      non-rollable-loading-seconds: 1800
```

Carga prevista si no rueda. Valor: 1800.

L54

```
      unloading-seconds: 600
```

Descarga prevista. Valor: 600.

L55

```
    integration:
```

Abre bloque integration; sus hijos configuran este componente.

L56

```
      connect-timeout-seconds: 2
```

Límite de conexión HTTP. Valor: 2.

L57

```
      response-timeout-seconds: 5
```

Límite de respuesta HTTP. Valor: 5.

L58

```
      recovery-delays-seconds: [5, 15, 30]
```

Primeras esperas de recuperación. Valor: [5, 15, 30].

L59

```
      subsequent-recovery-delay-seconds: 60
```

Espera posterior acotada. Valor: 60.

L60

```
      rate-limit-default-wait-seconds: 60
```

Espera si no hay Retry-After utilizable. Valor: 60.

7. Evaluación, rutas e integración externa

Haversine aporta cobertura/preselección; ORS, recorridos viales. Un fallo temporal no justifica rechazo de dominio.

DispatchValidator (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/DispatchValidator.java

SHA-256: 7d0fd5e87bba354fe7c8649ea1c4e9c9375dfb738933d9f8f758166a9dbb96e6

Valida el recurso ya comprometido y refresca evidencia vial antes de despachar. No busca otro candidato ni exige actualizar una ubicación ya aceptada por su antigüedad.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L7

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO.

L9

```
import com.auxiruta.assistance.routing.*;
```

Importa tipos: com.auxiruta.assistance.routing.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L10

```
import static com.auxiruta.assistance.routing.RouteProvider.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RouteProvider.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L12

```
/** Rechecks the currently committed resource, not the list of free candidates. */
```

Comentario de intención o documentación; no se ejecuta.

L13

```
@Service
```

Registra lógica de aplicación como bean.

L14

```
public class DispatchValidator {
```

Declara clase DispatchValidator.

L15

```
 public record Evidence(String json,Instant validUntil,Instant qualificationUntil,long plannedSeconds)
{}
```

Declara record inmutable Evidence. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L16

```
 private final FleetClient fleet;private final RouteProvider routes;private final ObjectMapper
mapper;private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L17

```
 public DispatchValidator(FleetClient fleet,RouteProvider routes,ObjectMapper mapper,Clock
clock){this.fleet=fleet;this.routes=routes;this.mapper=mapper;this.clock=clock;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L18

```
 private static void require(boolean value){if(!value)throw new
ResponseStatusException(HttpStatus.CONFLICT,"DISPATCH_NOT_ELIGIBLE");}
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L19

```
 private static int value(JsonNode rules,String group,String name){return
rules.path(group).path(name).asInt();}
```

Lee un parámetro entero del snapshot JSON.

L20

```
 public Evidence validate(ExecutionStore.Input input){
```

Revalida recurso y recorridos fuera de la transacción SQL y produce evidencia para commit.

L21

```
  try{
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L22

```
   JsonNode rules=mapper.readTree(input.rules());var resource=fleet.eligibility(input.assignmentId());
```

Obtiene snapshot actual del compromiso Fleet, no la lista de grúas disponibles.

L23

```
   if(resource==null || !input.assignmentId().equals(resource.assignmentAttemptId()) ||
!input.requestId().equals(resource.requestId()) || !input.truckId().equals(resource.towTruckId()) ||
!input.operatorId().equals(resource.operatorId()) ||
!input.subject().equals(resource.operatorSubject()))throw new
IllegalStateException("FLEET_INVALID_RESPONSE");
```

Valida identidad del intento, solicitud, grúa, operador y subject de la respuesta Fleet. La rama condicional aborta ante la condición inválida.

L24

```
   require(resource.eligible());
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L25

```
    Instant now=clock.instant();int routeAge=value(rules,"routing","max-route-age-seconds"),snap=value(ru
les,"routing","max-snap-distance-meters");
```

Lee del snapshot vigencia de ruta y snap máximo; captura instante de control. Captura un instante consistente de referencia para comparar condiciones temporales en esta operación.

L26

```
    if(resource.latitude()==null || resource.longitude()==null || resource.locationUpdatedAt()==null ||
resource.qualificationValidUntil()==null || resource.checkedAt()==null)throw new
IllegalStateException("FLEET_INVALID_RESPONSE");
```

La rama condicional aborta ante la condición inválida.

L27

```
    require(!resource.locationUpdatedAt().isAfter(now) && !resource.checkedAt().isAfter(now));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L28

```
    require(EvaluationService.inCoverage(resource.latitude(),resource.longitude(),rules) &&
EvaluationService.inCoverage(input.originLat(),input.originLon(),rules) &&
EvaluationService.inCoverage(input.destinationLat(),input.destinationLon(),rules));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L29

```
    Point origin=new Point(input.originLat(),input.originLon()),destination=new
Point(input.destinationLat(),input.destinationLon()),truck=new
Point(resource.latitude(),resource.longitude());String revision=rules.path("rules-version").asText();
```

Construye puntos origen/destino y, si corresponde, grúa con las coordenadas persistidas.

L30

```
    Route transport=routes.route(origin,destination,revision,routeAge,snap);
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L31

```
    Route approach=truck.latitude().compareTo(origin.latitude())==0 &&
truck.longitude().compareTo(origin.longitude())==0?new Route(BigDecimal.ZERO,BigDecimal.ZERO,transport.o
btainedAt()):routes.route(truck,origin,revision,routeAge,snap);
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L32

```
    BigDecimal factor=rules.path("routing").path("duration-factor").decimalValue();int adjustedTransport=
EvaluationService.adjusted(transport.durationSeconds(),factor),adjustedApproach=EvaluationService.adjust
ed(approach.durationSeconds(),factor),dispatch=value(rules,"execution","dispatch-deadline-seconds");
```

Obtiene factor decimal y calcula tiempos ajustados con redondeo hacia arriba.

L33

```
    require(transport.distanceM().compareTo(BigDecimal.valueOf(value(rules,"routing","min-transport-meter
s")))>=0 &&
transport.distanceM().compareTo(BigDecimal.valueOf(value(rules,"routing","max-transport-meters")))<=0 &&
 adjustedTransport<=value(rules,"routing","max-adjusted-transport-seconds"));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L34

```
    require(approach.distanceM().compareTo(BigDecimal.valueOf(value(rules,"routing","max-approach-meters"
)))<=0 && (long)dispatch+adjustedApproach+value(rules,"routing","arrival-margin-seconds")<=value(rules,"
routing","max-arrival-budget-seconds"));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L35

```
    long planned=(long)dispatch+adjustedApproach+adjustedTransport+value(rules,"execution",input.mobility
().equals("ROLLABLE")?"rollable-loading-seconds":"non-rollable-loading-seconds")+value(rules,"execution"
,"unloading-seconds")+value(rules,"routing","planning-contingency-seconds");
```

Planifica desde ahora despacho, aproximación, traslado, carga, descarga y contingencia para exigir habilitación suficiente.

L36

```
    Instant validUntil=transport.obtainedAt().plusSeconds(routeAge);for(Instant expiry:java.util.List.of(
  approach.obtainedAt().plusSeconds(routeAge),transport.obtainedAt().plusSeconds(routeAge)))if(expiry.isBe
  fore(validUntil))validUntil=expiry;
```

Vence con la más antigua de ambas rutas. No agrega caducidad de ubicación posterior a aceptación. La rama condicional selecciona la actualización o retorno correspondiente. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L37

```
    now=clock.instant();require(!now.isAfter(validUntil) &&
  resource.qualificationValidUntil().isAfter(now.plusSeconds(planned)));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L38

```
    var evidence=mapper.createObjectNode();evidence.put("assignmentId",input.assignmentId().toString());e
  vidence.put("fleetCheckedAt",resource.checkedAt().toString());evidence.put("locationUpdatedAt",resource.
  locationUpdatedAt().toString());evidence.put("latitude",resource.latitude());evidence.put("longitude",re
  source.longitude());evidence.put("approachDistanceM",approach.distanceM());evidence.put("transportDistan
  ceM",transport.distanceM());evidence.put("adjustedApproachSeconds",adjustedApproach);evidence.put("adjus
  tedTransportSeconds",adjustedTransport);evidence.put("approachRouteObtainedAt",approach.obtainedAt().toS
  tring());evidence.put("transportRouteObtainedAt",transport.obtainedAt().toString());evidence.put("qualif
  icationValidUntil",resource.qualificationValidUntil().toString());evidence.put("estimatedServiceEndAt",n
  ow.plusSeconds(planned).toString());
```

Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa.

L39

```
    return new Evidence(evidence.toString(),validUntil,resource.qualificationValidUntil(),planned);
```

Devuelve evidencia JSON, vencimiento vial, fin de habilitación y duración planificada para revalidación transaccional final.

L40

```
   }catch(ResponseStatusException e){throw e;}catch(RoutingFailure e){throw new ResponseStatusException(e
  .domainRejection()?HttpStatus.CONFLICT:HttpStatus.SERVICE_UNAVAILABLE);}catch(Exception e){throw new
  ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE);}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L41

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L42

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 11: separadores sin instrucciones.

EvaluationReadService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationReadService.java`

`SHA-256: e1403870634f6df3d5756fe9a3d7b0635b41d7f4fa60862581253567d93c962f`

Proyecta la evaluación más reciente y sus candidatos hacia DTO para consultar la justificación de la decisión.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L7

```
@Service @Transactional(readOnly=true)
```

Registra lógica de aplicación como bean. Delimita transacción local; readOnly=true declara consulta.

L8

```
public class EvaluationReadService {
```

Declara clase EvaluationReadService.

L9

```
 public record CandidateView(UUID towTruckId,boolean eligible,String reasonCode,BigDecimal
approachDistanceM,Integer adjustedApproachSeconds,Instant estimatedServiceEndAt,Integer rankingPosition)
 {}
```

Declara record inmutable CandidateView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; eligible: aptitud completa del candidato/compromiso; reasonCode: causa codificada; approachDistanceM: distancia vial grúa-recogida; adjustedApproachSeconds: aproximación ajustada; estimatedServiceEndAt: fin planificado para controlar habilitación; rankingPosition: posición entre elegibles, comenzando en 1.

L10

```
 public record EvaluationView(UUID id,String result,String reasonCode,Instant evaluatedAt,BigDecimal
transportDistanceM,Integer adjustedTransportSeconds,List<CandidateView> candidates) {}
```

Declara record inmutable EvaluationView. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; result: resultado de evaluación, distinto del estado de solicitud; reasonCode: causa codificada; evaluatedAt: instante de evaluación; transportDistanceM: distancia vial origen-destino en metros; adjustedTransportSeconds: duración ajustada y redondeada hacia arriba.

L11

```
 private final EvaluationRepository evaluations;private final CandidateEvaluationRepository candidates;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
 public EvaluationReadService(EvaluationRepository evaluations,CandidateEvaluationRepository
candidates){this.evaluations=evaluations;this.candidates=candidates;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
 public EvaluationView latest(UUID id) {return
evaluations.findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(id).map(e->new EvaluationView(e.id,e.result
,e.reasonCode,e.evaluatedAt,e.transportDistanceM,e.adjustedTransportSeconds,candidates.findByEvaluationI
d(e.id).stream().sorted(Comparator.comparing((CandidateEvaluation c)->c.rankingPosition,Comparator.nulls
Last(Comparator.naturalOrder())).thenComparing(c->c.towTruckId.toString())).map(c->new CandidateView(c.t
owTruckId,c.eligible,c.reasonCode,c.approachDistanceM,c.adjustedApproachSeconds,c.estimatedServiceEndAt,
c.rankingPosition)).toList())).orElse(null);}
```

Ordena por el comparador explícito para desempate reproducible. Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. Recupera evaluación más reciente por fecha e ID. Campos: id: identificador UUID del registro.

L14

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

EvaluationService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationService.java`

`SHA-256: 20a5d9c2081214c689b5d5a49084bc6137216fcb0afc6af89e9c1a3682b3aab9`

Calcula cobertura, traslado y aproximación con Fleet/ORS fuera de transacciones largas. Distingue rechazo de dominio de dependencia temporal; rankea por tiempo ajustado, antigüedad de disponibilidad y UUID.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L4

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L5

```
import java.math.*;
```

Importa tipos: java.math.*. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L7

```
import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO.

L8

```
import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía.

L9

```
import com.auxiruta.assistance.routing.*;
```

Importa tipos: com.auxiruta.assistance.routing.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L10

```
import static com.auxiruta.assistance.routing.RouteProvider.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RouteProvider.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L12

```
@Service
```

Registra lógica de aplicación como bean.

L13

```
public class EvaluationService {
```

Declara clase EvaluationService.

L14

```
 private final EvaluationStore store;private final RouteProvider routes;private final FleetClient
fleet;private final Clock clock;private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L15

```
 public EvaluationService(EvaluationStore store,RouteProvider routes,FleetClient fleet,Clock
clock,ObjectMapper
mapper){this.store=store;this.routes=routes;this.fleet=fleet;this.clock=clock;this.mapper=mapper;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L16

```
 static int adjusted(BigDecimal seconds,BigDecimal factor){return
seconds.multiply(factor).setScale(0,RoundingMode.CEILING).intValueExact();}
```

Multiplica segundos por el factor y redondea hacia arriba, evitando subestimar duración.

L17

```
 static boolean inCoverage(BigDecimal lat,BigDecimal lon,JsonNode rules){var
c=rules.path("coverage");return GeoMath.distance(c.path("center-latitude").asDouble(),c.path("center-lon
gitude").asDouble(),lat.doubleValue(),lon.doubleValue(),c.path("haversine-earth-radius-meters").asDouble
())<=c.path("radius-meters").asDouble();}
```

Calcula Haversine y admite el borde inclusive del radio comercial.

L18

```
 static int value(JsonNode rules,String group,String name){return rules.path(group).path(name).asInt();}
```

Lee un parámetro entero del snapshot JSON.

L19

```
 private static boolean same(Point a,Point b){return a.latitude().compareTo(b.latitude())==0 &&
a.longitude().compareTo(b.longitude())==0;}
```

Compara coordenadas por valor numérico, no por escala decimal.

L20

```
 public void evaluate(UUID id) {
```

Campos: id: identificador UUID del registro.

L21

```
  EvaluationStore.Input input=store.load(id);if(input==null)return;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L22

```
  JsonNode rules;try{rules=mapper.readTree(input.rules());}catch(Exception e){throw new
IllegalStateException("INVALID_RULE_SNAPSHOT");}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L23

```
  Evaluation e=new Evaluation();e.searchAttemptNumber=input.attemptNumber();e.originLatitude=input.origi
nLat();e.originLongitude=input.originLon();e.destinationLatitude=input.destinationLat();e.destinationLon
gitude=input.destinationLon();e.vehicleType=input.vehicleType();e.declaredWeightKg=input.weight();e.mobi
lityCondition=input.mobility();e.configurationRevision=rules.path("rules-version").asText();e.maximumArr
ivalSeconds=value(rules,"routing","max-arrival-budget-seconds");e.loadingSeconds=value(rules,"execution"
,input.mobility().equals("ROLLABLE")?"rollable-loading-seconds":"non-rollable-loading-seconds");e.unload
ingSeconds=value(rules,"execution","unloading-seconds");e.operationalMarginSeconds=value(rules,"routing"
,"planning-contingency-seconds");e.coverageAndAdmissionSummary="AR-MVP-5 admission";
```

Cambios: e.searchAttemptNumber recibe input.attemptNumber(); e.originLatitude recibe input.originLat(); e.originLongitude recibe input.originLon(); e.destinationLatitude recibe input.destinationLat(); e.destinationLongitude recibe input.destinationLon(); e.vehicleType recibe input.vehicleType(); e.declaredWeightKg recibe input.weight(); e.mobilityCondition recibe input.mobility(); e.configurationRevision recibe rules.path("rules-version").asText(); e.maximumArrivalSeconds recibe value(rules,"routing","max-arrival-budget-seconds"); e.loadingSeconds recibe value(rules,"execution",input.mobility().equals("ROLLABLE")?"rollable-loading-seconds":"non-rollable-loading-seconds");

e.unloadingSeconds recibe value(rules,"execution","unloading-seconds"); e.operationalMarginSeconds recibe value(rules,"routing","planning-contingency-seconds"); e.coverageAndAdmissionSummary recibe "AR-MVP-5 admission".

L24

```
List<CandidateEvaluation> details=new ArrayList<>();
```

Crea lista que conservará tanto candidatos aptos como descartados y sus motivos.

L25

```
if(!inCoverage(input.originLat(),input.originLon(),rules) || !inCoverage(input.destinationLat(),input.
destinationLon(),rules)){e.result="REJECTED";e.reasonCode="OUTSIDE_COVERAGE";store.save(input,e,details)
;return;}
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: e.result recibe "REJECTED"; e.reasonCode recibe "OUTSIDE_COVERAGE".

L26

```
e.coverageAndAdmissionSummary="Origin and destination inside coverage";
```

Cambios: e.coverageAndAdmissionSummary recibe "Origin and destination inside coverage".

L27

```
int age=value(rules,"routing","max-route-age-seconds"),snap=value(rules,"routing","max-snap-distance-m
eters");BigDecimal factor=rules.path("routing").path("duration-factor").decimalValue();
```

Obtiene factor decimal y calcula tiempos ajustados con redondeo hacia arriba. Lee antigüedad máxima de ruta, tolerancia de snap y factor del snapshot.

L28

```
Point origin=new Point(input.originLat(),input.originLon()),destination=new
Point(input.destinationLat(),input.destinationLon());
```

Construye puntos origen/destino y, si corresponde, grúa con las coordenadas persistidas.

L29

```
try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L30

```
Route transport=routes.route(origin,destination,e.configurationRevision,age,snap);e.transportDistance
M=transport.distanceM();e.transportDurationSeconds=transport.durationSeconds();e.adjustedTransportSecond
s=adjusted(transport.durationSeconds(),factor);e.routeObtainedAt=transport.obtainedAt();
```

Enruta el patrón al microservicio destino; no ejecuta la regla de negocio. Cambios: e.transportDistanceM recibe transport.distanceM(); e.transportDurationSeconds recibe transport.durationSeconds(); e.adjustedTransportSeconds recibe adjusted(transport.durationSeconds(),factor); e.routeObtainedAt recibe transport.obtainedAt().

L31

```
if(transport.distanceM().compareTo(BigDecimal.valueOf(value(rules,"routing","min-transport-meters")))<0
|| transport.distanceM().compareTo(BigDecimal.valueOf(value(rules,"routing","max-transport-meters")))>0)
{e.result="REJECTED";e.reasonCode="TRANSPORT_DISTANCE";store.save(input,e,details);return;}
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: e.result recibe "REJECTED"; e.reasonCode recibe "TRANSPORT_DISTANCE".

L32

```
if(e.adjustedTransportSeconds>value(rules,"routing","max-adjusted-transport-seconds")){e.result="REJE
CTED";e.reasonCode="TRANSPORT_DURATION";store.save(input,e,details);return;}
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: e.result recibe "REJECTED"; e.reasonCode recibe "TRANSPORT_DURATION".

L33

```
List<FleetClient.Candidate> resources=fleet.candidates(input.originLat(),input.originLon(),input.vehi
cleType(),input.weight(),input.mobility().equals("NON_ROLLABLE"));
```

Consulta hasta cinco recursos de Fleet según origen, tipo, peso y movilidad.

L34

```
    Map<UUID,Instant> freeSince=new HashMap<>();boolean pending=false;
```

Prepara antigüedad de disponibilidad para desempate y bandera de dependencia pendiente.

L35

```
    for(var resource:resources) {
```

Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L36

```
     if(resource.towTruckId()==null || resource.operatorId()==null || resource.operatorSubject()==null ||
  resource.capacityKg()==null || resource.supportedTypes()==null ||
 resource.qualificationCategory()==null || resource.qualificationValidFrom()==null ||
 resource.qualificationValidUntil()==null || resource.availableSince()==null || resource.latitude()==null
  || resource.longitude()==null || resource.locationUpdatedAt()==null)throw new
 IllegalStateException("FLEET_INVALID_RESPONSE");
```

La rama condicional aborta ante la condición inválida.

L37

```
     CandidateEvaluation c=new CandidateEvaluation();c.towTruckId=resource.towTruckId();c.operatorId=reso
 urce.operatorId();c.operatorSubject=resource.operatorSubject();c.truckCapacityKg=resource.capacityKg();c
 .qualificationCategory=resource.qualificationCategory();c.qualificationValidUntil=resource.qualification
 ValidUntil();c.truckLatitude=resource.latitude();c.truckLongitude=resource.longitude();c.truckLocationUp
 datedAt=resource.locationUpdatedAt();c.compatible=resource.capacityKg().compareTo(input.weight())>=0 &&
 resource.supportedTypes().contains(input.vehicleType()) && (!input.mobility().equals("NON_ROLLABLE") ||
 resource.supportsNonRollable());c.transportDistanceM=e.transportDistanceM;c.transportDurationSeconds=e.t
 ransportDurationSeconds;c.adjustedTransportSeconds=e.adjustedTransportSeconds;c.transportRouteObtainedAt
 =e.routeObtainedAt;details.add(c);freeSince.put(c.towTruckId,resource.availableSince());
```

Cambios: c.towTruckId recibe resource.towTruckId(); c.operatorId recibe resource.operatorId(); c.operatorSubject recibe resource.operatorSubject(); c.truckCapacityKg recibe resource.capacityKg(); c.qualificationCategory recibe resource.qualificationCategory(); c.qualificationValidUntil recibe resource.qualificationValidUntil(); c.truckLatitude recibe resource.latitude(); c.truckLongitude recibe resource.longitude(); c.truckLocationUpdatedAt recibe resource.locationUpdatedAt(); c.transportDistanceM recibe e.transportDistanceM; c.transportDurationSeconds recibe e.transportDurationSeconds; c.adjustedTransportSeconds recibe e.adjustedTransportSeconds; c.transportRouteObtainedAt recibe e.routeObtainedAt.

L38

```
     Instant validation=clock.instant();
```

Toma instante fresco porque las consultas HTTP previas pueden haber consumido tiempo.

L39

```
     if(!c.compatible){c.reasonCode="INCOMPATIBLE";continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "INCOMPATIBLE".

L40

```
     if(!inCoverage(c.truckLatitude,c.truckLongitude,rules) ||
 c.truckLocationUpdatedAt.isAfter(validation) || c.truckLocationUpdatedAt.isBefore(validation.minusSecond
 s(value(rules,"fleet","max-location-age-seconds")))){c.reasonCode="LOCATION_INVALID";continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "LOCATION_INVALID".

L41

```
     try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L42

```
      Point truck=new Point(c.truckLatitude,c.truckLongitude);Route approach=same(truck,origin)?new Route
 (BigDecimal.ZERO,BigDecimal.ZERO,transport.obtainedAt()):routes.route(truck,origin,e.configurationRevisi
 on,age,snap);
```

Si la grúa ya está exactamente en origen, aproximación cero; en caso contrario pide ruta vial. No inventa una distancia para reemplazar ORS. Enruta el patrón al microservicio destino; no ejecuta la regla de negocio.

L43

```
      c.approachDistanceM=approach.distanceM();c.approachDurationSeconds=approach.durationSeconds();c.adj
  ustedApproachSeconds=adjusted(approach.durationSeconds(),factor);c.approachRouteObtainedAt=approach.obta
  inedAt();
```

Cambios: c.approachDistanceM recibe approach.distanceM(); c.approachDurationSeconds recibe approach.durationSeconds(); c.adjustedApproachSeconds recibe adjusted(approach.durationSeconds(),factor); c.approachRouteObtainedAt recibe approach.obtainedAt().

L44

```
      int dispatch=value(rules,"execution","dispatch-deadline-seconds"),arrivalMargin=value(rules,"routin
  g","arrival-margin-seconds");
```

Lee plazo de despacho y margen de llegada para evaluar el presupuesto total.

L45

```
  if(c.approachDistanceM.compareTo(BigDecimal.valueOf(value(rules,"routing","max-approach-meters")))>0 ||
  (long)dispatch+c.adjustedApproachSeconds+arrivalMargin>e.maximumArrivalSeconds){c.reasonCode="APPROACH_L
  IMIT";continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "APPROACH_LIMIT".

L46

```
      validation=clock.instant();long remaining=Math.min(value(rules,"fleet","hold-seconds"),Duration.bet
  ween(validation,input.deadline()).getSeconds());
```

Recalcula ventana restante: mínimo entre hold máximo y segundos hasta deadline de búsqueda.

L47

```
      if(remaining<value(rules,"search","minimum-offer-window-seconds")){c.reasonCode="SEARCH_WINDOW_TOO_
  SHORT";continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "SEARCH_WINDOW_TOO_SHORT".

L48

```
      c.estimatedServiceEndAt=validation.plusSeconds(remaining+dispatch+c.adjustedApproachSeconds+e.loadi
  ngSeconds+e.adjustedTransportSeconds+e.unloadingSeconds+e.operationalMarginSeconds).truncatedTo(java.tim
  e.temporal.ChronoUnit.MICROS);
```

Suma tiempos de espera y operación para calcular hasta cuándo debe durar la habilitación del operador.

L49

```
      if(!"LIGHT_TOWING".equals(c.qualificationCategory) ||
  resource.qualificationValidFrom().isAfter(validation) || !c.qualificationValidUntil.isAfter(c.estimatedS
  erviceEndAt)){c.reasonCode="QUALIFICATION_EXPIRES";continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "QUALIFICATION_EXPIRES".

L50

```
      if(c.truckLocationUpdatedAt.isBefore(validation.minusSeconds(value(rules,"fleet","max-location-age-
  seconds"))) || c.approachRouteObtainedAt.isBefore(validation.minusSeconds(age)) || c.transportRouteObtai
  nedAt.isBefore(validation.minusSeconds(age))){c.reasonCode="STALE_INPUT";pending=true;continue;}
```

La rama condicional descarta este candidato y sigue con el próximo. Cambios: c.reasonCode recibe "STALE_INPUT".

L51

```
      c.eligible=true;c.reasonCode="ELIGIBLE";
```

El candidato superó filtros; todavía no hay retención. Cambios: c.eligible recibe true; c.reasonCode recibe "ELIGIBLE".

L52

```
     }catch(RoutingFailure
  failure){c.reasonCode=failure.reason().name();pending|=!failure.domainRejection();}
```

Un fallo de dominio invalida ese candidato; uno temporal señala que la evaluación debe mantenerse pendiente si no hay otro viable. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Cambios: c.reasonCode recibe failure.reason().name().

L53

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L54

```
    Instant finished=clock.instant();
```

Captura fin del cálculo para volver a comprobar frescura de todos los candidatos.

L55

```
    for(var c:details)if(c.eligible && (c.truckLocationUpdatedAt.isBefore(finished.minusSeconds(value(rul
es,"fleet","max-location-age-seconds"))) ||
c.approachRouteObtainedAt.isBefore(finished.minusSeconds(age)) || c.transportRouteObtainedAt.isBefore(fi
nished.minusSeconds(age)))){c.eligible=false;c.reasonCode="STALE_INPUT";pending=true;}
```

Invalida evidencia que envejeció durante cálculo de otros candidatos antes de rankear. La rama condicional selecciona la actualización o retorno correspondiente. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo. Cambios: c.eligible recibe false; c.reasonCode recibe "STALE_INPUT".

L56

```
    List<CandidateEvaluation>
ranked=details.stream().filter(c->c.eligible).sorted(Comparator.comparingInt((CandidateEvaluation c)->c.
adjustedApproachSeconds).thenComparing(c->freeSince.get(c.towTruckId)).thenComparing(c->c.towTruckId.toS
tring())).toList();
```

Ordena candidatos elegibles por menor aproximación ajustada, luego por mayor antigüedad de disponibilidad y finalmente por UUID para desempate determinista. Ordena por el comparador explícito para desempate reproducible. Conserva solo elementos que cumplen el predicado. Materializa la secuencia en lista.

L57

```
    for(int i=0;i<ranked.size();i++)ranked.get(i).rankingPosition=i+1;
```

Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L58

```
    e.result=!ranked.isEmpty()?"VIABLE":pending?"PENDING_DEPENDENCY":"SEARCHING";
```

VIABLE si hay candidato; PENDING_DEPENDENCY si falta evidencia temporal; SEARCHING si no hay aptos sin fallo temporal. Cambios: e.result recibe !ranked.isEmpty()?"VIABLE":pending?"PENDING_DEPENDENCY":"SEARCHING".

L59

```
    e.reasonCode=!ranked.isEmpty()?"VIABLE_CANDIDATE":pending?"DEPENDENCY_UNAVAILABLE":resources.isEmpty(
)?"NO_AVAILABLE_RESOURCE":"NO_VIABLE_CANDIDATE";
```

Cambios: e.reasonCode recibe !ranked.isEmpty()?"VIABLE_CANDIDATE":pending?"DEPENDENCY_UNAVAILABLE":resourc
es.isEmpty()?"NO_AVAILABLE_RESOURCE":"NO_VIABLE_CANDIDATE".

L60

```
   }catch(RoutingFailure failure){e.result=failure.domainRejection()?"REJECTED":"PENDING_DEPENDENCY";e.re
asonCode=failure.reason().name();}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Cambios: e.result recibe failure.domainRejection()?"REJECTED":"PENDING_DEPENDENCY"; e.reasonCode recibe failure.reason().name().

L61

```
   catch(Exception failure){e.result="PENDING_DEPENDENCY";e.reasonCode="FLEET_OR_EVALUATION_UNAVAILABLE";
e.reasonDetail=failure.getClass().getSimpleName();}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Cambios: e.result recibe "PENDING_DEPENDENCY"; e.reasonCode recibe "FLEET_OR_EVALUATION_UNAVAILABLE"; e.reasonDetail recibe failure.getClass().getSimpleName().

L62

```
   store.save(input,e,details);
```

Entrega resultado y candidatos al store, que decide bajo bloqueo si aún corresponde persistirlos.

L63

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L64

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 11: separadores sin instrucciones.

EvaluationStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/EvaluationStore.java`

`SHA-256: b4e8d9d8157ca42a8464553bd871438688db7b0c5c39d1bd4b793ec2153ec855`

Carga un snapshot bajo bloqueo, cede una ventana de evaluación y persiste solo si versión/estado/plazo siguen siendo compatibles. Evita sobrescribir una cancelación concurrente.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L7

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L8

```
@Service
```

Registra lógica de aplicación como bean.

L9

```
public class EvaluationStore {
```

Declara clase EvaluationStore.

L10

```
 public record Input(UUID requestId,long version,Status status,int attemptNumber,Instant deadline,String
 rules,BigDecimal originLat,BigDecimal originLon,BigDecimal destinationLat,BigDecimal
destinationLon,String vehicleType,BigDecimal weight,String mobility) {}
```

Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; version: versión para control optimista de concurrencia; status: estado del objeto; su enum o CHECK define valores permitidos; vehicleType: tipo CAR/SUV/PICKUP.

L11

```
 private final RequestRepository requests;private final EvaluationRepository evaluations;private final
CandidateEvaluationRepository candidates;private final HistoryRepository history;private final Clock
clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L12

```
 public EvaluationStore(RequestRepository requests,EvaluationRepository
evaluations,CandidateEvaluationRepository candidates,HistoryRepository history,Clock clock){this.request
s=requests;this.evaluations=evaluations;this.candidates=candidates;this.history=history;this.clock=clock
;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L13

```
@Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L14

```
public Input load(UUID id) {
```

Campos: id: identificador UUID del registro.

L15

```
 var maybe=requests.lockById(id);if(maybe.isEmpty())return null;var r=maybe.get();
```

La rama condicional retorna sin trabajo si no aplica.

L16

```
 if(!Set.of(Status.REQUESTED,Status.PENDING_EVALUATION,Status.SEARCHING_ALTERNATIVE).contains(r.status)
|| r.resourceReleasePending || r.evaluationLeaseUntil.isAfter(clock.instant()) ||
!r.searchDeadline.isAfter(clock.instant()))return null;
```

La rama condicional retorna sin trabajo si no aplica.

L17

```
 Instant lease=clock.instant().plusSeconds(120);r.evaluationLeaseUntil=lease;r.nextEvaluationAt=lease.i
sBefore(r.searchDeadline)?lease:r.searchDeadline;requests.flush();
```

Reserva hasta 120 segundos de trabajo de evaluación y agenda el próximo chequeo; flush hace observable la versión usada por el snapshot. Sincroniza cambios JPA con SQL; no es commit independiente. Cambios: r.evaluationLeaseUntil recibe lease; r.nextEvaluationAt recibe lease.isBefore(r.searchDeadline)?lease:r.searchDeadline.

L18

```
 return new Input(id,r.version,r.status,r.searchAttemptNumber,r.searchDeadline,r.configurationSnapshot,
r.originLatitude,r.originLongitude,r.destinationLatitude,r.destinationLongitude,r.declaredVehicleType.na
me(),r.declaredWeightKg,r.mobilityCondition.name());
```

Devuelve snapshot inmutable con versión/estado y datos necesarios; HTTP se hará fuera del bloqueo local.

L19

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L20

```
@Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L21

```
public boolean save(Input input,Evaluation result,List<CandidateEvaluation> details) {
```

Crea lista que conservará tanto candidatos aptos como descartados y sus motivos. Reingresa a transacción para aplicar solo una evaluación compatible con el snapshot original.

L22

```
   var maybe=requests.lockById(input.requestId());if(maybe.isEmpty())return false;var r=maybe.get();
```

La rama condicional selecciona la actualización o retorno correspondiente.

L23

```
   if(r.version!=input.version() || r.status!=input.status() ||
 !r.searchDeadline.isAfter(clock.instant()))return false;
```

Descarta el resultado si hubo un cambio concurrente de versión/estado o venció la búsqueda. La evaluación externa no pisa una cancelación. La rama condicional selecciona la actualización o retorno correspondiente.

L24

```
   r.evaluationLeaseUntil=Instant.EPOCH;result.request=r;result.evaluatedAt=clock.instant().truncatedTo(j
 ava.time.temporal.ChronoUnit.MICROS);var stored=evaluations.saveAndFlush(result);
```

Usa 1970-01-01 UTC como planificación inicialmente vencida. Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Cambios: r.evaluationLeaseUntil recibe Instant.EPOCH; result.request recibe r; result.evaluatedAt recibe clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS).

L25

```
   for(var c:details){c.evaluation=stored;candidates.save(c);}candidates.flush();
```

Sincroniza cambios JPA con SQL; no es commit independiente. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo. Cambios: c.evaluation recibe stored.

L26

```
   Status previous=r.status;r.status="REJECTED".equals(result.result)?(r.firstAssignedAt==null?Status.REJ
 ECTED:Status.UNFULFILLED):Status.PENDING_EVALUATION;r.updatedAt=result.evaluatedAt;
```

Cambios: r.status recibe "REJECTED".equals(result.result)?(r.firstAssignedAt==null?Status.REJECTED:Status.UNFULFILLED):Status.PENDING_EVALUATION; r.updatedAt recibe result.evaluatedAt.

L27

```
   try{int interval=new com.fasterxml.jackson.databind.ObjectMapper().readTree(r.configurationSnapshot).p
 ath("search").path("evaluation-interval-seconds").asInt();Instant next=result.evaluatedAt.plusSeconds(in
 terval);r.nextEvaluationAt=next.isBefore(r.searchDeadline)?next:r.searchDeadline;}catch(Exception
 e){throw new IllegalStateException("Invalid rules");}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Cambios: r.nextEvaluationAt recibe next.isBefore(r.searchDeadline)?next:r.searchDeadline.

L28

```
   if(r.status==Status.REJECTED ||
 r.status==Status.UNFULFILLED){r.closedAt=result.evaluatedAt;r.closureReasonCode=result.reasonCode;}
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.closedAt recibe result.evaluatedAt; r.closureReasonCode recibe result.reasonCode.

L29

```
   RequestStatusHistory h=new RequestStatusHistory();h.request=r;h.previousStatus=previous;h.newStatus=r.
 status;h.actorType="SYSTEM";h.reason=result.reasonCode;h.details="evaluationId="+stored.id+"
 result="+result.result;h.occurredAt=result.evaluatedAt;history.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción. Cambios: h.request recibe r; h.previousStatus recibe previous; h.newStatus recibe r.status; h.actorType recibe "SYSTEM"; h.reason recibe result.reasonCode; h.details recibe "evaluationId="+stored.id+" result="+result.result; h.occurredAt recibe result.evaluatedAt.

L30

```
   org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronization(new
 org.springframework.transaction.support.TransactionSynchronization(){@Override public void
 afterCommit(){org.slf4j.LoggerFactory.getLogger(EvaluationStore.class).info("Evaluation completed
 requestId={} result={} reason={}",input.requestId(),result.result,result.reasonCode);}});
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos.

L31

```
    return true;
```

Entrega true al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L32

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L33

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

FleetClient (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/FleetClient.java`

`SHA-256: 16a6bcf56553656c7ae00affd1b62c7feb6f3027f4025adf2fa1b505235f6c7d`

Cliente HTTP técnico. Obtiene y reutiliza un token client_credentials; consulta candidatos y opera reservas. No propaga el token del conductor.

L1

```
package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L5

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L7

```
import org.springframework.web.client.RestClient;
```

Importa tipos: org.springframework.web.client.RestClient. Controladores, errores HTTP y clientes REST según la clase importada.

L8

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L9

```
import org.springframework.http.client.JdkClientHttpRequestFactory;
```

Importa tipos: org.springframework.http.client.JdkClientHttpRequestFactory. Estados, headers, tipos de contenido y respuestas HTTP.

L10

```
import org.springframework.core.ParameterizedTypeReference;
```

Importa tipos: org.springframework.core.ParameterizedTypeReference. Tipo auxiliar usado por las expresiones del archivo.

L11

```
import org.springframework.util.LinkedMultiValueMap;
```

Importa tipos: org.springframework.util.LinkedMultiValueMap. Tipo auxiliar usado por las expresiones del archivo.

L12

```
import com.fasterxml.jackson.databind.JsonNode;
```

Importa tipos: com.fasterxml.jackson.databind.JsonNode. Lectura/escritura JSON, nodos y conversión a DTO.

L13

```
@Component
```

Registra componente inyectable.

L14

```
public class FleetClient {
```

Declara clase FleetClient.

L15

```
 public record Candidate(UUID towTruckId,UUID operatorId,String operatorSubject,BigDecimal
capacityKg,Set<String> supportedTypes,boolean supportsNonRollable,String qualificationCategory,Instant
qualificationValidFrom,Instant qualificationValidUntil,Instant availableSince,BigDecimal
latitude,BigDecimal longitude,Instant locationUpdatedAt,double straightDistanceM) {}
```

Declara record inmutable Candidate. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; supportsNonRollable: capacidad para atender vehículos que no ruedan; qualificationCategory: habilitación LIGHT_TOWING; qualificationValidFrom: inicio de habilitación; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado; availableSince: inicio de disponibilidad usado para desempate; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada.

L16

```
 private final RestClient http;private final String base,tokenUrl,clientId,secret;private final Clock
clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L17

```
 private String token;private Instant expires=Instant.EPOCH;
```

Usa 1970-01-01 UTC como planificación inicialmente vencida.

L18

```
 public FleetClient(@Value("${FLEET_URL:http://localhost:8081}") String
base,@Value("${KEYCLOAK_TOKEN_URI:http://localhost:8080/realms/auxiruta/protocol/openid-connect/token}")
 String tokenUrl,@Value("${KEYCLOAK_CLIENT_ID:assistance-service}") String
clientId,@Value("${KEYCLOAK_CLIENT_SECRET:}") String secret,Clock clock) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L19

```
  this.base=base;this.tokenUrl=tokenUrl;this.clientId=clientId;this.secret=secret;this.clock=clock;
```

Asigna o consulta campos de la instancia.

L20

```
   var factory=new JdkClientHttpRequestFactory(java.net.http.HttpClient.newBuilder().connectTimeout(Durat
ion.ofSeconds(2)).followRedirects(java.net.http.HttpClient.Redirect.NEVER).build());factory.setReadTimeo
ut(Duration.ofSeconds(5));http=RestClient.builder().requestFactory(factory).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Limita establecimiento de conexión; no sigue redirecciones a otros destinos. Limita espera de respuesta a cinco segundos.

L21

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L22

```
  private synchronized String token() {
```

Reutiliza el token técnico mientras conserve margen de diez segundos; de lo contrario pide uno nuevo a Keycloak.

L23

```
   if(token!=null && expires.isAfter(clock.instant().plusSeconds(10)))return token;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L24

```
   var form=new LinkedMultiValueMap<String,String>();form.add("grant_type","client_credentials");form.add
("client_id",clientId);form.add("client_secret",secret);
```

Construye formulario OAuth con cliente técnico y secreto, no con usuario/contraseña del conductor.

L25

```
   JsonNode response=http.post().uri(tokenUrl).contentType(MediaType.APPLICATION_FORM_URLENCODED).body(fo
rm).retrieve().body(JsonNode.class);
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L26

```
   if(response==null || response.path("access_token").asText().isBlank())throw new
IllegalStateException("FLEET_AUTH_UNAVAILABLE");
```

La rama condicional aborta ante la condición inválida.

L27

```
   token=response.path("access_token").asText();expires=clock.instant().plusSeconds(response.path("expire
s_in").asLong(0));return token;
```

Guarda token en memoria y calcula vencimiento usando expires_in; no lo persiste en tablas.

L28

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L29

```
  public record Eligibility(UUID assignmentAttemptId,UUID requestId,UUID towTruckId,UUID
operatorId,String operatorSubject,boolean eligible,String reason,BigDecimal latitude,BigDecimal
longitude,Instant locationUpdatedAt,Instant qualificationValidUntil,Instant checkedAt) {}
```

Declara record inmutable Eligibility. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; eligible: aptitud completa del candidato/compromiso; reason: motivo legible; latitude: latitud declarada; longitude: longitud declarada; locationUpdatedAt: instante servidor de última ubicación declarada; qualificationValidUntil: fin de habilitación que debe superar el servicio estimado.

L30

```
  public Eligibility eligibility(UUID id){return http.get().uri(base+"/internal/reservations/"+id+"/eligi
bility").header(HttpHeaders.AUTHORIZATION,"Bearer "+token()).retrieve().body(Eligibility.class);}
```

Consulta si un compromiso confirmado conserva aptitud; no aplica caducidad de ubicación de una propuesta nueva. Campos: id: identificador UUID del registro. Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L31

```
 public java.util.List<JsonNode> statuses(java.util.List<UUID> ids){return
http.post().uri(base+"/internal/reservations/status-queries").header(HttpHeaders.AUTHORIZATION,"Bearer "
+token()).contentType(MediaType.APPLICATION_JSON).body(java.util.Map.of("assignmentAttemptIds",ids)).ret
rieve().body(new org.springframework.core.ParameterizedTypeReference<java.util.List<JsonNode>>(){});}
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L32

```
 public JsonNode reservation(UUID id,String kind,String payload) {
```

Campos: id: identificador UUID del registro; payload: contenido normalizado para reintentos.

L33

```
  if(kind.equals("CONFIRM")){JsonNode
existing=http.get().uri(base+"/internal/reservations/"+id).header(HttpHeaders.AUTHORIZATION,"Bearer
"+token()).retrieve().body(JsonNode.class);if(existing!=null &&
id.toString().equals(existing.path("assignmentAttemptId").asText()) &&
!"HELD".equals(existing.path("status").asText()))return existing;}
```

Devuelve una ruta cacheada solo si su fecha no es futura y su antigüedad no supera el máximo solicitado. Evita consumir cuota en ese caso. Consulta estado remoto previo; si ya no está HELD devuelve ese estado durable. La rama condicional selecciona la actualización o retorno correspondiente. Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L34

```
  var request=kind.equals("RESERVE")?http.put().uri(base+"/internal/reservations/"+id):http.post().uri(b
ase+"/internal/reservations/"+id+(kind.equals("CONFIRM")?"/confirmation":"/release"));
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L35

```
  request.header(HttpHeaders.AUTHORIZATION,"Bearer "+token());
```

Añade Bearer técnico a la operación de reserva.

L36

```
  if(kind.equals("RESERVE"))request.contentType(MediaType.APPLICATION_JSON).body(payload);
```

La rama condicional selecciona la actualización o retorno correspondiente.

L37

```
  JsonNode result=request.retrieve().body(JsonNode.class);
```

Ejecuta HTTP y deserializa respuesta; un estado HTTP de error lanza excepción.

L38

```
  if(result==null || !id.toString().equals(result.path("assignmentAttemptId").asText()))throw new
IllegalStateException("FLEET_INVALID_RESERVATION_RESPONSE");
```

La rama condicional aborta ante la condición inválida.

L39

```
  return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L40

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L41

```
 public List<Candidate> candidates(BigDecimal latitude,BigDecimal longitude,String type,BigDecimal
weight,boolean nonRollable) {
```

Campos: latitude: latitud declarada; longitude: longitud declarada.

L42

```
  try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L43

```
    var uri=org.springframework.web.util.UriComponentsBuilder.fromUriString(base+"/internal/fleet/candida
tes").queryParam("latitude",latitude).queryParam("longitude",longitude).queryParam("vehicleType",type).q
ueryParam("weightKg",weight).queryParam("nonRollable",nonRollable).build().toUri();
```

Construye URL codificando parámetros de búsqueda, sin concatenar datos sin escape.

L44

```
    List<Candidate> result=http.get().uri(uri).header(HttpHeaders.AUTHORIZATION,"Bearer
"+token()).retrieve().body(new ParameterizedTypeReference<List<Candidate>>(){});
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L45

```
    if(result==null || result.size()>5)throw new IllegalStateException("FLEET_INVALID_RESPONSE");return
result;
```

La rama condicional aborta ante la condición inválida.

L46

```
   }catch(Exception e){throw new IllegalStateException("FLEET_UNAVAILABLE");}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L47

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L48

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

OpenRouteServiceClient (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/OpenRouteServiceClient.java`

`SHA-256: d29adb5d66a1a316508a121eebbd9f0f1b642aac6c80c9e49c6e99668c05ca59`

Adaptador HTTP ORS con timeouts, presupuesto persistente, caché por coordenadas/revisión y control del ajuste a red vial. Maneja 429, cuota diaria y respuesta inválida sin reintento inmediato.

L1

```
package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L3

```
import java.time.format.DateTimeFormatter;
```

Importa tipos: java.time.format.DateTimeFormatter. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import java.util.concurrent.ConcurrentHashMap;
```

Importa tipos: java.util.concurrent.ConcurrentHashMap. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L7

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L8

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L9

```
import org.springframework.web.client.*;
```

Importa tipos: org.springframework.web.client.*. Controladores, errores HTTP y clientes REST según la clase importada.

L10

```
import org.springframework.http.*;
```

Importa tipos: org.springframework.http.*. Estados, headers, tipos de contenido y respuestas HTTP.

L11

```
import org.springframework.http.client.JdkClientHttpRequestFactory;
```

Importa tipos: org.springframework.http.client.JdkClientHttpRequestFactory. Estados, headers, tipos de contenido y respuestas HTTP.

L12

```
import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO.

L13

```
import com.auxiruta.commons.GeoMath;
```

Importa tipos: com.auxiruta.commons.GeoMath. Utilidades compartidas de seguridad, errores, reloj o geografía.

L14

```
import static com.auxiruta.assistance.routing.RoutingFailure.Reason.*;
```

Importa miembros estáticos: com.auxiruta.assistance.routing.RoutingFailure.Reason.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L15

```
@Component
```

Registra componente inyectable.

L16

```
public class OpenRouteServiceClient implements RouteProvider {
```

Declara clase OpenRouteServiceClient.

L17

```
  private final RestClient http;private final String key;private final RoutingBudget budget;private final
  Clock clock;private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L18

```
  private final Map<String,Route> cache=new ConcurrentHashMap<>();
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
  public OpenRouteServiceClient(@Value("${ors.base-url:https://api.heigit.org/openrouteservice/v2}")
 String url,@Value("${ors.api-key:}") String key,RoutingBudget budget,Clock clock,ObjectMapper mapper) {
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L20

```
    this.key=key;this.budget=budget;this.clock=clock;this.mapper=mapper;
```

Asigna o consulta campos de la instancia.

L21

```
    var client=java.net.http.HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(2)).followRedirects
 (java.net.http.HttpClient.Redirect.NEVER).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Limita establecimiento de conexión; no sigue redirecciones a otros destinos.

L22

```
    var factory=new JdkClientHttpRequestFactory(client);factory.setReadTimeout(Duration.ofSeconds(5));this
 .http=RestClient.builder().baseUrl(url).requestFactory(factory).build();
```

Construye el objeto final a partir del builder configurado en líneas previas. Asigna o consulta campos de la instancia. Limita espera de respuesta a cinco segundos.

L23

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L24

```
  private static String coordinate(BigDecimal n) { return n.stripTrailingZeros().toPlainString(); }
```

Normaliza decimales para que valores equivalentes generen la misma clave o huella.

L25

```
  @Override public Route route(Point from,Point to,String revision,int maxAgeSeconds,int maxSnapMeters) {
```

Contrato proveedor: dos puntos, revisión, edad máxima y ajuste máximo; devuelve recorrido o falla clasificada. Redefine contrato heredado.

L26

```
    String cacheKey=coordinate(from.latitude())+","+coordinate(from.longitude())+":"+coordinate(to.latitud
 e())+","+coordinate(to.longitude())+":driving-car:"+revision+":"+maxSnapMeters;
```

La clave de caché incluye origen, destino, perfil driving-car, revisión de reglas y tolerancia de ajuste vial. No reutiliza recorridos con coordenadas distintas.

L27

```
    Route existing=cache.get(cacheKey);Instant now=clock.instant();
```

Busca recorrido por clave exacta y captura ahora para controlar su vigencia. Captura un instante consistente de referencia para comparar condiciones temporales en esta operación.

L28

```
    if(existing!=null && !existing.obtainedAt().isAfter(now) &&
 !existing.obtainedAt().isBefore(now.minusSeconds(maxAgeSeconds)))return existing;
```

Devuelve una ruta cacheada solo si su fecha no es futura y su antigüedad no supera el máximo solicitado. Evita consumir cuota en ese caso. La rama condicional selecciona la actualización o retorno correspondiente.

L29

```
    if(key.isBlank())throw new RoutingFailure(CONFIGURATION);
```

Sin clave ORS falla como configuración antes de reservar cuota; la capa de evaluación conserva el resultado pendiente. La rama condicional aborta ante la condición inválida.

L30

```
    if(!budget.reserve())throw new RoutingFailure(RATE_LIMIT);
```

Si no hay presupuesto no sale a la red; reporta límite temporal para reprogramar. La rama condicional aborta ante la condición inválida.

L31

```
    try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L32

```
    ResponseEntity<JsonNode> response=http.post().uri("/directions/driving-car/geojson").header(HttpHeade
rs.AUTHORIZATION,key).contentType(MediaType.APPLICATION_JSON)
```

Prepara HTTP al endpoint indicado; retrieve ejecuta y deserializa o lanza ante error.

L33

```
     .body(Map.of("coordinates",List.of(List.of(from.longitude(),from.latitude()),List.of(to.longitude(),
to.latitude())),"preference","recommended","units","m"))
```

ORS recibe pares [longitud, latitud], aunque Point almacena latitud y longitud. Solicita unidades métricas y ruta recomendada.

L34

```
     .retrieve().toEntity(JsonNode.class);
```

Ejecuta consulta ORS y conserva cuerpo y headers para evaluar cuota.

L35

```
    if("0".equals(response.getHeaders().getFirst("x-ratelimit-remaining")))budget.defer(reset(response.ge
tHeaders(),now,now.plusSeconds(86400)));
```

La rama condicional selecciona la actualización o retorno correspondiente.

L36

```
    Route result=parse(response.getBody(),from,to,maxSnapMeters,clock.instant());if(cache.size()>=500)cac
he.clear();cache.put(cacheKey,result);return result;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L37

```
   } catch(RestClientResponseException e) {
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L38

```
    HttpHeaders headers=e.getResponseHeaders()==null?new HttpHeaders():e.getResponseHeaders();String
body=e.getResponseBodyAsString();int code=0;
```

Extrae headers/cuerpo de error para clasificar cuota, autenticación o ausencia de ruta.

L39

```
    try {code=mapper.readTree(body).path("error").path("code").asInt();}catch(Exception ignored) {}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L40

```
    if((e.getStatusCode().value()==400 || e.getStatusCode().value()==404) && (code==2009 ||
code==2010))throw new RoutingFailure(NO_ROUTE);
```

Solo 400/404 con código reconocido significa ausencia de ruta; no transforma un 5xx en rechazo comercial. La rama condicional aborta ante la condición inválida.

L41

```
    if(e.getStatusCode().value()==429){budget.defer(retryAfter(headers,now));throw new
RoutingFailure(RATE_LIMIT);}
```

La rama condicional aborta ante la condición inválida.

L42

```
    if(e.getStatusCode().value()==403 && (body.toLowerCase(Locale.ROOT).contains("quota") ||
"0".equals(headers.getFirst("x-ratelimit-remaining"))))
{budget.defer(reset(headers,now,now.plusSeconds(86400)));throw new RoutingFailure(DAILY_QUOTA);}
```

La rama condicional aborta ante la condición inválida.

L43

```
    if(e.getStatusCode().value()==401 ||
e.getStatusCode().value()==403){budget.defer(now.plusSeconds(300));throw new
RoutingFailure(CONFIGURATION);}
```

La rama condicional aborta ante la condición inválida.

L44

```
    throw new RoutingFailure(UNAVAILABLE);
```

Clasifica error no recuperado como indisponibilidad, no rechazo comercial.

L45

```
   } catch(ResourceAccessException e) {throw new RoutingFailure(UNAVAILABLE);}
```

Clasifica error no recuperado como indisponibilidad, no rechazo comercial. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L46

```
   catch(RestClientException e) {throw new RoutingFailure(INVALID_RESPONSE);}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L47

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L48

```
  static Instant retryAfter(HttpHeaders headers,Instant now) {
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo.

L49

```
   String value=headers.getFirst("Retry-After");if(value!=null)try{return
now.plusSeconds(Math.max(1,Long.parseLong(value)));}catch(NumberFormatException ignored){try{return Zone
dDateTime.parse(value,DateTimeFormatter.RFC_1123_DATE_TIME).toInstant().isAfter(now)?ZonedDateTime.parse
(value,DateTimeFormatter.RFC_1123_DATE_TIME).toInstant():now.plusSeconds(60);}catch(Exception
ignoredDate){}}
```

La rama condicional selecciona la actualización o retorno correspondiente. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L50

```
   return now.plusSeconds(60);
```

Fallback de espera de un minuto cuando no hay indicación válida del proveedor.

L51

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L52

```
  static Instant reset(HttpHeaders headers,Instant now,Instant fallback) { try {Instant
reset=Instant.ofEpochSecond(Long.parseLong(headers.getFirst("x-ratelimit-reset")));return
reset.isAfter(now)?reset:fallback;}catch(Exception e){return fallback;} }
```

Interpreta el instante de reinicio de cuota y utiliza el fallback si es inválido o anterior. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L53

```
  static Route parse(JsonNode root,Point from,Point to,int maxSnap,Instant now) {
```

Interpreta GeoJSON y valida métricas, geometría y distancia de ajuste a red.

L54

```
   try {
```

Inicia bloque de operaciones que pueden fallar; los catch siguientes clasifican o transforman el error.

L55

```
    JsonNode feature=root.path("features").path(0),summary=feature.path("properties").path("summary"),geo
metry=feature.path("geometry").path("coordinates"),way=feature.path("properties").path("way_points");
```

Selecciona primera feature, resumen, geometría y waypoints de la respuesta ORS.

L56

```
    if(!summary.path("distance").isNumber() || !summary.path("duration").isNumber() || geometry.size()<2
|| way.size()!=2)throw new RoutingFailure(INVALID_RESPONSE);
```

Valida resumen, geometría y dos waypoints antes de aceptar el GeoJSON. La rama condicional aborta ante la condición inválida.

L57

```
    BigDecimal
distance=summary.get("distance").decimalValue(),duration=summary.get("duration").decimalValue();
```

Extrae distancia y duración como decimales para evitar pérdida innecesaria de precisión.

L58

```
    if(distance.signum()<0 || duration.signum()<0)throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida.

L59

```
    int first=way.get(0).asInt(-1),last=way.get(1).asInt(-1);if(first<0 || last<first ||
last>=geometry.size())throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida.

L60

```
    JsonNode a=geometry.get(first),z=geometry.get(last);
```

Obtiene puntos de geometría correspondientes a los dos waypoints validados.

L61

```
    if(a.size()<2 || z.size()<2 || !a.get(0).isNumber() || !a.get(1).isNumber() || !z.get(0).isNumber()
|| !z.get(1).isNumber())throw new RoutingFailure(INVALID_RESPONSE);
```

La rama condicional aborta ante la condición inválida.

L62

```
    for(JsonNode point:List.of(a,z))if(!Double.isFinite(point.get(0).asDouble()) ||
!Double.isFinite(point.get(1).asDouble()) || Math.abs(point.get(0).asDouble())>180 ||
Math.abs(point.get(1).asDouble())>90)throw new RoutingFailure(INVALID_RESPONSE);
```

Verifica extremos finitos y dentro de rangos geográficos. La rama condicional aborta ante la condición inválida. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L63

```
    if(GeoMath.distance(from.latitude().doubleValue(),from.longitude().doubleValue(),a.get(1).asDouble(),
a.get(0).asDouble(),6371000)>maxSnap || GeoMath.distance(to.latitude().doubleValue(),to.longitude().doub
leValue(),z.get(1).asDouble(),z.get(0).asDouble(),6371000)>maxSnap)throw new
RoutingFailure(SNAP_TOO_FAR);
```

Compara puntos pedidos con extremos ajustados a la red vial; si la desviación supera el máximo devuelve SNAP_TOO_FAR. La rama condicional aborta ante la condición inválida.

L64

```
    return new Route(distance,duration,now.truncatedTo(java.time.temporal.ChronoUnit.MICROS));
```

Devuelve recorrido métrico con instante truncado a microsegundos.

L65

```
   }catch(RoutingFailure e){throw e;}catch(Exception e){throw new RoutingFailure(INVALID_RESPONSE);}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L66

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L67

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RouteProvider (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RouteProvider.java`

`SHA-256: 4385788a5526ea79cd300d7d131c234b5092a70defcbaca15ea47dc37de80213`

Contrato local del proveedor de rutas; Point y Route expresan coordenadas, distancia, duración y fecha de obtención.

L1

```
package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L3

```
import java.time.Instant;
```

Importa tipos: java.time.Instant. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
public interface RouteProvider {
```

Declara interfaz RouteProvider.

L5

```
  record Point(BigDecimal latitude,BigDecimal longitude) {}
```

Declara record inmutable Point. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: latitude: latitud declarada; longitude: longitud declarada.

L6

```
  record Route(BigDecimal distanceM,BigDecimal durationSeconds,Instant obtainedAt) {}
```

Declara record inmutable Route. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L7

```
  Route route(Point from,Point to,String revision,int maxAgeSeconds,int maxSnapMeters);
```

Contrato proveedor: dos puntos, revisión, edad máxima y ajuste máximo; devuelve recorrido o falla clasificada.

L8

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RoutingBudget (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RoutingBudget.java`

`SHA-256: ef307446ae0a26eb8a16f0d8c8f7daf2a1fabeb40628ae01650b08700cfa35e7`

Reserva cuota antes de llamar a ORS mediante una transacción con bloqueo. Conserva ventanas móviles de 60 s y 24 h y un bloqueo hasta fecha indicada por el proveedor.

L1

```
package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L3

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L4

```
import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables.

L5

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L6

```
import org.springframework.beans.factory.annotation.Value;
```

Importa tipos: org.springframework.beans.factory.annotation.Value. Inyección de dependencias y propiedades de entorno.

L7

```
@Service
```

Registra lógica de aplicación como bean.

L8

```
public class RoutingBudget {
```

Declara clase RoutingBudget.

L9

```
  private final JdbcTemplate jdbc;private final Clock clock;private final int perMinute,perDay;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L10

```
  public RoutingBudget(JdbcTemplate jdbc,Clock clock,@Value("${ors.per-minute:40}") int
minute,@Value("${ors.per-day:2000}") int day) {
this.jdbc=jdbc;this.clock=clock;this.perMinute=minute;this.perDay=day; }
```

Inyecta la propiedad nombrada; el valor después de dos puntos es fallback cuando existe. No lee credenciales desde el cuerpo de una petición. Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L11

```
  @Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L12

```
  public boolean reserve() {
```

Reserva una unidad de presupuesto de ORS de manera transaccional. Devuelve false si hay bloqueo o cuota agotada; true solo después de registrar la llamada.

L13

```
   Instant now=clock.instant();java.sql.Timestamp blocked=jdbc.queryForObject("select blocked_until from
routing_budget where id=1 for update",java.sql.Timestamp.class);
```

Captura un instante consistente de referencia para comparar condiciones temporales en esta operación. Consulta routing_budget con el filtro y proyección escritos. Bloquea las filas durante la transacción.

L14

```
   if(blocked!=null && blocked.toInstant().isAfter(now))return false;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L15

```
   jdbc.update("delete from routing_calls where called_at <
 ?",java.sql.Timestamp.from(now.minusSeconds(86400)));
```

Poda llamadas fuera de la ventana móvil de 24 h; esta tabla es control de cuota, no historial permanente de todas las consultas ORS. Elimina de routing_calls solo las filas del filtro.

L16

```
   long daily=jdbc.queryForObject("select count(*) from routing_calls",Long.class);
```

Consulta routing_calls contando coincidencias.

L17

```
   long minute=jdbc.queryForObject("select count(*) from routing_calls where called_at >
 ?",Long.class,java.sql.Timestamp.from(now.minusSeconds(60)));
```

Consulta routing_calls contando coincidencias.

L18

```
   if(daily>=perDay || minute>=perMinute)return false;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L19

```
   jdbc.update("insert into routing_calls(id,called_at) values
 (?,?)",java.util.UUID.randomUUID(),java.sql.Timestamp.from(now));return true;
```

Genera ID nuevo en la aplicación. Inserta en routing_calls.

L20

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L21

```
  @Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L22

```
  public void defer(Instant until) { jdbc.update("update routing_budget set
 blocked_until=greatest(blocked_until,?) where id=1",java.sql.Timestamp.from(until)); }
```

Extiende blocked_until sin acortarlo por una respuesta posterior con espera menor. Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención. Actualiza routing_budget para las filas del filtro.

L23

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

RoutingFailure (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/routing/RoutingFailure.java`

`SHA-256: 5237ce34ccfae8f08e66a714f92fa0ef2bf189ca62b2b652d789f7b85a376cc6`

Clasifica fallos del proveedor. Algunas causas justifican rechazo del dominio y otras dejan evaluación pendiente.

L1

```
 package com.auxiruta.assistance.routing;
```

Ubica el archivo en com.auxiruta.assistance.routing. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
 public class RoutingFailure extends RuntimeException {
```

Declara clase RoutingFailure.

L3

```
  public enum Reason { NO_ROUTE, SNAP_TOO_FAR, RATE_LIMIT, DAILY_QUOTA, CONFIGURATION, UNAVAILABLE,
 INVALID_RESPONSE }
```

Declara enumeración cerrada Reason.

L4

```
  private final Reason reason;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L5

```
  public RoutingFailure(Reason reason) { super(reason.name());this.reason=reason; }
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L6

```
  public Reason reason() { return reason; }
```

Permite a la capa de negocio conocer categoría del fallo sin depender de la excepción HTTP.

L7

```
  public boolean domainRejection() { return reason==Reason.NO_ROUTE || reason==Reason.SNAP_TOO_FAR; }
```

Solo NO_ROUTE y SNAP_TOO_FAR representan rechazo de dominio; el resto exige recuperación temporal/configuración.

L8

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

8. Propuestas, aceptación y recuperación durable

AssignmentCoordinator es una clase técnica, no un rol humano. Las acciones remotas inciertas se conservan y recuperan.

AssignmentController (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentController.java

SHA-256: dcfa69fae37f3ee9158a0cb48afe8bae4ae56de89f222a4e8c35aee6bdd148ac

Expone consultas y respuestas de propuestas. Solo el operador destinatario puede aceptar/rechazar; ADMIN consulta pero no actúa por él.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas.

L4

```
import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L5

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad.

L7

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L8

```
import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación.

L9

```
import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación.

L10

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación.

L11

```
@RestController @RequestMapping("/api/v1") @SecurityRequirement(name="bearerAuth")
```

Expone HTTP y serializa JSON. Documenta autenticación; no concede permisos. Ruta /api/v1 combinada con el prefijo de clase cuando corresponda.

L12

```
@ApiResponses({@ApiResponse(responseCode="400",description="Entrada
invalida"),@ApiResponse(responseCode="401",description="JWT ausente o
invalido"),@ApiResponse(responseCode="403",description="Rol
insuficiente"),@ApiResponse(responseCode="404",description="Recurso inexistente o
ajeno"),@ApiResponse(responseCode="409",description="Conflicto de estado, plazo o
idempotencia"),@ApiResponse(responseCode="503",description="Dependencia temporalmente no disponible;
consultar estado antes de reintentar")})
```

Documenta código HTTP.

L13

```
public class AssignmentController {
```

Declara clase AssignmentController.

L14

```
  public record Rejection(@NotBlank @Size(min=10,max=500) String reason) {}
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Rejection. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: reason: motivo legible.

L15

```
  private final AssignmentStore store;private final AssignmentCoordinator coordinator;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L16

```
  public AssignmentController(AssignmentStore store,AssignmentCoordinator
coordinator){this.store=store;this.coordinator=coordinator;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L17

```
  @GetMapping("/operator/assignments") @Operation(summary="OPERATOR: consultar propuestas y servicios
propios")
```

Ruta /operator/assignments combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR: consultar propuestas y servicios propios.

L18

```
  public RequestDtos.PageView<AssignmentStore.View> own(@AuthenticationPrincipal Jwt
jwt,@RequestParam(defaultValue="0") int page,@RequestParam(defaultValue="20") int size){return
store.own(jwt.getSubject(),page,size);}
```

Obtiene JWT validado, no identidad suministrada por el cliente. Lee parámetro/filtro URL y su default. Identidad obtenida del JWT, no del JSON enviado por el usuario.

L19

```
  @GetMapping("/assignments/{id}") @Operation(summary="Consultar asignación propia; ADMIN puede consultar
  globalmente")
```

Ruta /assignments/{id} combinada con el prefijo de clase cuando corresponda. Operación: Consultar asignación propia; ADMIN puede consultar globalmente.

L20

```
  public AssignmentStore.View get(@PathVariable UUID id,@AuthenticationPrincipal Jwt jwt){return
store.visible(id,jwt);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Campos: id: identificador UUID del registro.

L21

```
  @PostMapping("/assignments/{id}/acceptance") @Operation(summary="OPERATOR destinatario: confirmar una
propuesta vigente con Idempotency-Key")
```

Ruta /assignments/{id}/acceptance combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR destinatario: confirmar una propuesta vigente con Idempotency-Key.

L22

```
 public AssignmentStore.View accept(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String
key){coordinator.execute(store.respond(id,jwt.getSubject(),key,true,null));return
store.visible(id,jwt);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L23

```
 @PostMapping("/assignments/{id}/rejection") @Operation(summary="OPERATOR destinatario: rechazar con
motivo y liberar antes de otra propuesta")
```

Ruta /assignments/{id}/rejection combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR destinatario: rechazar con motivo y liberar antes de otra propuesta.

L24

```
 public AssignmentStore.View reject(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key,@Valid @RequestBody Rejection
body){coordinator.execute(store.respond(id,jwt.getSubject(),key,false,body.reason()));var
result=store.visible(id,jwt);coordinator.propose(result.requestId());return result;}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L25

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

AssignmentCoordinator (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentCoordinator.java`

`SHA-256: a93536c376dc9f9c463f4ac74db86c243af44ed869dd15c91f8155edda212a7e`

Orquestador técnico, no rol humano. Ejecuta HTTP fuera de las transacciones de AssignmentStore y reprograma acciones inciertas con el mismo identificador.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L4

```
import com.auxiruta.assistance.routing.FleetClient;
```

Importa tipos: com.auxiruta.assistance.routing.FleetClient. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L5

```
import org.springframework.web.client.RestClientResponseException;
```

Importa tipos: org.springframework.web.client.RestClientResponseException. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
@Service
```

Registra lógica de aplicación como bean.

L7

```
public class AssignmentCoordinator {
```

Declara clase AssignmentCoordinator.

L8

```
 private final AssignmentStore store;private final FleetClient fleet;private final java.time.Clock
clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L9

```
 public AssignmentCoordinator(AssignmentStore store,FleetClient
fleet){this(store,fleet,java.time.Clock.systemUTC());}
```

Crea reloj de sistema con zona UTC; las pruebas pueden sustituirlo por Clock fijo. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L10

```
 @org.springframework.beans.factory.annotation.Autowired
```

Inyecta dependencia Spring.

L11

```
 public AssignmentCoordinator(AssignmentStore store,FleetClient fleet,java.time.Clock
clock){this.store=store;this.fleet=fleet;this.clock=clock;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L12

```
 private int retryAfter(RestClientResponseException e){return retryAfter(e.getResponseHeaders()==null?nu
ll:e.getResponseHeaders().getFirst("Retry-After"),clock.instant());}
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo.

L13

```
 static int retryAfter(String header,java.time.Instant now){
```

Interpreta Retry-After en segundos o fecha HTTP y aplica una espera segura si no puede leerlo.

L14

```
  if(header==null)return 60;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L15

```
  try{return
(int)Math.min(Integer.MAX_VALUE,Math.max(1,Long.parseLong(header.strip())));}catch(Exception ignored){}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L16

```
  try{var until=java.time.ZonedDateTime.parse(header,java.time.format.DateTimeFormatter.RFC_1123_DATE_TI
ME).toInstant();return (int)Math.min(Integer.MAX_VALUE,Math.max(1,java.time.Duration.between(now,until).
getSeconds()+1));}catch(Exception ignored){return 60;}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L17

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L18

```
 public void recoverDue(){for(var action:store.dueActions())execute(action);}
```

Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L19

```
  public void propose(UUID request){for(int i=0;i<5;i++){var
 action=store.propose(request);if(action==null)return;execute(action);}}
```

La rama condicional selecciona la actualización o retorno correspondiente. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L20

```
  public void recover(UUID request){for(var action:store.pendingForRequest(request))execute(action);}
```

Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L21

```
  public void execute(AssignmentStore.Action action){
```

Ejecuta una intención durable con claim y manejo de errores, sin transacción extendida alrededor de HTTP.

L22

```
   // Each network call is outside database transactions. Continuations are compensation, not retries.
```

Comentario de intención o documentación; no se ejecuta.

L23

```
   for(int step=0;action!=null && step<3;step++){
```

Máximo tres continuaciones compensatorias, no tres reintentos inmediatos del fallo. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L24

```
    if(!store.claim(action))return;
```

Solo quien obtiene ventana de trabajo hace HTTP; el otro abandona. La rama condicional selecciona la actualización o retorno correspondiente.

L25

```
    try{var response=fleet.reservation(action.assignmentId(),action.kind(),action.payload());action=store
 .completed(action,response);}
```

Realiza operación remota, luego registra respuesta en transacción local; puede devolver una compensación siguiente.

L26

```
    catch(RestClientResponseException e){if(action.kind().equals("RESERVE") &&
 (e.getStatusCode().value()==409 || e.getStatusCode().value()==404)){store.reserveRejected(action);return
 ;}store.defer(action,e.getStatusCode().value()==401 ||
 e.getStatusCode().value()==403?300:e.getStatusCode().value()==429?retryAfter(e):0);return;}
```

La rama condicional selecciona la actualización o retorno correspondiente. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L27

```
    catch(Exception e){store.failed(action);return;}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L28

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L29

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L30

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

AssignmentStore (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AssignmentStore.java

SHA-256: 0ff54422ace9150b40b4faace6bc000878c24d52f7af3e101c8be4672144356d

Persistencia transaccional de propuestas y acciones remotas pendientes. Excluye al operador que rechazó durante toda la solicitud y conserva aceptación/cancelación idempotentes.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.sql.*;
```

Importa tipos: java.sql.*. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables.

L7

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L8

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L9

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L10

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L11

```
import com.auxiruta.commons.security.TokenSupport;
```

Importa tipos: com.auxiruta.commons.security.TokenSupport. Utilidades compartidas de seguridad, errores, reloj o geografía.

L12

```
import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO.

L13

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L14

```
@Service @Transactional
```

Registra lógica de aplicación como bean. Delimita transacción local; readOnly=true declara consulta.

L15

```
public class AssignmentStore {
```

Declara clase AssignmentStore.

L16

```
  public record View(UUID id,UUID requestId,UUID towTruckId,UUID operatorId,String operatorSubject,String
  status,Instant expiresAt,Instant acceptedAt,String reason) {}
```

Declara record inmutable View. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; requestId: UUID de solicitud; towTruckId: UUID de grúa; operatorId: UUID del perfil operador; operatorSubject: sub del operador destinatario; status: estado del objeto; su enum o CHECK define valores permitidos; expiresAt: fin exclusivo de retención/propuesta; acceptedAt: instante de aceptación efectiva; reason: motivo legible.

L17

```
  public record Action(UUID id,UUID assignmentId,String kind,String payload) {}
```

Declara record inmutable Action. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: id: identificador UUID del registro; payload: contenido normalizado para reintentos.

L18

```
  private final JdbcTemplate jdbc;private final RequestRepository requests;private final
CandidateEvaluationRepository candidates;private final EvaluationRepository evaluations;private final
HistoryRepository history;private final Clock clock;private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L19

```
  public AssignmentStore(JdbcTemplate jdbc,RequestRepository requests,CandidateEvaluationRepository
candidates,EvaluationRepository evaluations,HistoryRepository history,Clock clock,ObjectMapper mapper){t
his.jdbc=jdbc;this.requests=requests;this.candidates=candidates;this.evaluations=evaluations;this.histor
y=history;this.clock=clock;this.mapper=mapper;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L20

```
  private int rule(AssistanceRequest r,String group,String key){try{return
mapper.readTree(r.configurationSnapshot).path(group).path(key).asInt();}catch(Exception e){throw new
IllegalStateException("Invalid rules");}}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L21

```
  private Instant now(){return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);}
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L22

```
  private static Timestamp ts(Instant i){return i==null?null:Timestamp.from(i);}
```

Convierte Instant a Timestamp JDBC; la variante nullable preserva null.

L23

```
  private static Instant instant(ResultSet r,String name)throws SQLException {Timestamp
t=r.getTimestamp(name);return t==null?null:t.toInstant();}
```

Lee columna temporal JDBC y la convierte a Instant sin inventar fecha cuando es nula.

L24

```
  private static void require(boolean ok){if(!ok)throw new ResponseStatusException(HttpStatus.CONFLICT);}
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L25

```
  private static ResponseStatusException missing(){return new
ResponseStatusException(HttpStatus.NOT_FOUND);}
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L26

```
  private View map(ResultSet r,int row)throws SQLException{return new View(r.getObject("id",UUID.class),r
.getObject("request_id",UUID.class),r.getObject("tow_truck_id",UUID.class),r.getObject("operator_id",UUI
D.class),r.getString("operator_subject"),r.getString("status"),instant(r,"expires_at"),instant(r,"accept
ed_at"),r.getString("reason"));}
```

Convierte una fila ResultSet a un record; respeta tipos UUID y fechas opcionales.

L27

```
  private View get(UUID id){return jdbc.query("select * from assignments where
id=?",this::map,id).stream().findFirst().orElseThrow(AssignmentStore::missing);}
```

Si no encuentra resultado lanza la excepción indicada. Consulta assignments con el filtro y proyección escritos. Campos: id: identificador UUID del registro.

L28

```
  private AssistanceRequest lock(UUID requestId){UUID
vehicle=requests.vehicleId(requestId).orElseThrow(AssignmentStore::missing);jdbc.queryForObject("select
id from vehicles where id=? for update",UUID.class,vehicle);return
requests.lockById(requestId).orElseThrow(AssignmentStore::missing);}
```

Si no encuentra resultado lanza la excepción indicada. Adquiere bloqueos en orden para serializar operaciones competidoras. Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Campos: requestId: UUID de solicitud.

L29

```
  public View visible(UUID id,Jwt jwt){View a=get(id);if(!TokenSupport.permits(jwt,false,"ADMIN") &&
!a.operatorSubject().equals(jwt.getSubject()))throw missing();return a;}
```

Comprueba ADMIN humano con audiencia/cliente válidos, no solo el texto del rol. Identidad obtenida del JWT, no del JSON enviado por el usuario. Aplica alcance por identidad/rol antes de devolver un recurso. Campos: id: identificador UUID del registro. La rama condicional aborta ante la condición inválida.

L30

```
  public RequestDtos.PageView<View> own(String subject,int page,int size){if(page<0 || size<1 ||
size>100)throw new ResponseStatusException(HttpStatus.BAD_REQUEST);long
count=jdbc.queryForObject("select count(*) from assignments where
operator_subject=?",Long.class,subject);return new RequestDtos.PageView<>(jdbc.query("select * from
assignments where operator_subject=? order by created_at,id limit ? offset
?",this::map,subject,size,(long)page*size),page,size,count,(int)((count+size-1)/size));}
```

Consulta assignments contando coincidencias. Consulta assignments con el filtro y proyección escritos. La rama condicional aborta ante la condición inválida.

L31

```
  private void event(AssistanceRequest r,Status next,String reason,String actor){Status previous=r.status
;r.status=next;r.updatedAt=now();r.nextEvaluationAt=now();if(next==Status.ASSIGNED)r.nextReconciliationA
t=now();RequestStatusHistory h=new RequestStatusHistory();h.request=r;h.previousStatus=previous;h.newSta
tus=next;h.actorType=actor==null?"SYSTEM":"OPERATOR";h.actorSubject=actor;h.reason=reason;h.occurredAt=n
ow();history.save(h);
```

Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.status recibe next; r.updatedAt recibe now(); r.nextEvaluationAt recibe now(); r.nextReconciliationAt recibe now(); h.request recibe r; h.previousStatus recibe previous; h.newStatus recibe next; h.actorType recibe actor==null?"SYSTEM":"OPERATOR"; h.actorSubject recibe actor; h.reason recibe reason; h.occurredAt recibe now().

L32

```
   org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronization(new
org.springframework.transaction.support.TransactionSynchronization(){@Override public void
afterCommit(){org.slf4j.LoggerFactory.getLogger(AssignmentStore.class).info("Request state changed
requestId={} from={} to={} reason={}",r.id,previous,next,reason);}});
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos.

L33

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L34

```
  private Action action(UUID assignment,String kind,String payload){UUID
id=UUID.randomUUID();jdbc.update("insert into
pending_actions(id,assignment_id,kind,status,created_at,updated_at) values (?,?,?,'PENDING',?,?) on
conflict(assignment_id,kind) do nothing",id,assignment,kind,ts(now()),ts(now()));return
jdbc.query("select id from pending_actions where assignment_id=? and kind=? and
status<>'COMPLETED'",(rs,n)->new Action(rs.getObject(1,UUID.class),assignment,kind,payload),assignment,k
ind).stream().findFirst().orElse(null);}
```

Crea una acción pendiente única por asignación y tipo. Si ya existe y no se completó, devuelve la misma; si está completada no crea otra. Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila. Consulta pending_actions con el filtro y proyección escritos. Campos: payload: contenido normalizado para reintentos; id: identificador UUID del registro.

L35

```
  public Action propose(UUID requestId){
```

Inicia la decisión local de seleccionar candidato y persistir una intención RESERVE. Devuelve null cuando todavía no puede proponer. Campos: requestId: UUID de solicitud.

L36

```
   AssistanceRequest r=lock(requestId);if(r.closedAt!=null || r.resourceReleasePending ||
r.evaluationLeaseUntil.isAfter(now()) || r.status!=Status.PENDING_EVALUATION ||
!r.searchDeadline.isAfter(now()))return null;
```

No propone si la solicitud está cerrada, queda liberación pendiente, otra evaluación conserva su lease, el estado no admite propuestas o venció la búsqueda. La rama condicional retorna sin trabajo si no aplica.

L37

```
   if(jdbc.queryForObject("select count(*) from assignments where request_id=? and (status in
('RESERVING','PROPOSED','ACCEPTED'))",Long.class,requestId)>0)return null;
```

Si ya hay una asignación en reserva, propuesta o aceptación, impide crear otra activa para la misma solicitud. Consulta assignments contando coincidencias. La rama condicional retorna sin trabajo si no aplica.

L38

```
   if(jdbc.queryForObject("select count(*) from assignments where request_id=? and
search_attempt_number=? and expires_at is not
null",Long.class,requestId,r.searchAttemptNumber)>=rule(r,"search","max-offers-per-attempt"))return
null;
```

Cuenta ofertas del intento que llegaron a tener vencimiento; si alcanzó max-offers-per-attempt no publica otra. Consulta assignments contando coincidencias. IS NOT NULL filtra filas que ya tienen valor en la columna indicada. La rama condicional retorna sin trabajo si no aplica.

L39

```
   var maybe=evaluations.findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(requestId);if(maybe.isEmpty()
|| !"VIABLE".equals(maybe.get().result))return null;
```

Solo intenta proponer usando la evaluación más reciente y de resultado VIABLE; ausencia o resultado distinto no produce reserva. Recupera evaluación más reciente por fecha e ID. La rama condicional retorna sin trabajo si no aplica.

L40

```
   var e=maybe.get();if(e.evaluatedAt.isBefore(now().minusSeconds(300)))return null;
```

Descarta una evaluación de más de 300 segundos antes de emitir una propuesta; no significa reevaluar cada aceptación. La rama condicional retorna sin trabajo si no aplica.

L41

```
   Set<UUID> tried=new HashSet<>(jdbc.queryForList("select tow_truck_id from assignments where
request_id=? and search_attempt_number=? and (expires_at is not null or created_at>?)",UUID.class,reques
tId,r.searchAttemptNumber,ts(now().minusSeconds(rule(r,"search","evaluation-interval-seconds")))));
```

Recupera grúas ya ofrecidas en este intento y conflictos recientes; evita repetir inmediatamente el mismo recurso mientras la búsqueda conserva su plazo. Consulta assignments con el filtro y proyección escritos. IS NOT NULL filtra filas que ya tienen valor en la columna indicada.

L42

```
   Set<UUID> rejectedOperators=new HashSet<>(jdbc.queryForList("select distinct operator_id from
assignments where request_id=? and response_kind='REJECT'",UUID.class,requestId));
```

Recupera todos los operadores que rechazaron explícitamente esta solicitud, sin limitarse al intento actual. Un vencimiento de despacho no figura como REJECT y no genera esta exclusión. Consulta assignments con el filtro y proyección escritos.

L43

```
   var choice=candidates.findByEvaluationId(e.id).stream().filter(c->c.eligible &&
c.rankingPosition!=null && !tried.contains(c.towTruckId) && !rejectedOperators.contains(c.operatorId)).s
orted(Comparator.comparingInt(c->c.rankingPosition)).findFirst();if(choice.isEmpty())return null;
```

Elige el candidato elegible de mejor ranking que no tenga grúa ya intentada en esta ronda ni operador que rechazó en cualquier intento. Si no existe, no emite propuesta. Ordena por el comparador explícito para desempate reproducible. Conserva solo elementos que cumplen el predicado. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos. La rama condicional retorna sin trabajo si no aplica.

L44

```
   var c=choice.get();if(c.truckLocationUpdatedAt.isBefore(now().minusSeconds(300)) ||
c.approachRouteObtainedAt.isBefore(now().minusSeconds(300)) ||
c.transportRouteObtainedAt.isBefore(now().minusSeconds(300)) || Duration.between(now(),r.searchDeadline)
.getSeconds()<rule(r,"search","minimum-offer-window-seconds"))return null;
```

Comprueba frescura de ubicación y de ambas rutas al emitir, y que reste la ventana mínima de respuesta. Si alguna condición falla, espera otra evaluación. La rama condicional retorna sin trabajo si no aplica.

L45

```
   // Preserve the planned duration while moving its reference instant to reservation time.
```

Comentario de intención o documentación; no se ejecuta.

L46

```
   Instant end=now().plusSeconds(Math.min(rule(r,"fleet","hold-seconds"),Duration.between(now(),r.searchD
eadline).getSeconds())+rule(r,"execution","dispatch-deadline-seconds")+c.adjustedApproachSeconds+e.loadi
ngSeconds+e.adjustedTransportSeconds+e.unloadingSeconds+e.operationalMarginSeconds);
```

Calcula fin estimado desde la reserva: suma ventana de respuesta, despacho, aproximación, carga, traslado, descarga y margen. Esa fecha se usa para exigir habilitación suficientemente larga.

L47

```
   var body=mapper.createObjectNode();body.put("holdSeconds",rule(r,"fleet","hold-seconds"));body.put("mi
nimumOfferSeconds",rule(r,"search","minimum-offer-window-seconds"));body.put("requestId",requestId.toStr
ing());body.put("towTruckId",c.towTruckId.toString());body.put("operatorId",c.operatorId.toString());bod
y.put("vehicleType",r.declaredVehicleType.name());body.put("weightKg",r.declaredWeightKg);body.put("nonR
ollable",r.mobilityCondition==Mobility.NON_ROLLABLE);body.put("searchDeadline",r.searchDeadline.toString
());body.put("estimatedServiceEndAt",end.toString());
```

Prepara contrato técnico de reserva: plazos del snapshot, IDs, tipo/peso/movilidad y fin estimado para que Fleet revalide antes de comprometer. Construye JSON explícito para petición remota o evidencia; evita exponer la entidad completa.

L48

```
   UUID id=UUID.randomUUID();jdbc.update("insert into assignments(id,request_id,candidate_evaluation_id,t
ow_truck_id,operator_id,operator_subject,status,created_at,search_attempt_number,reserve_payload) values
 (?,?,?,?,?,?,'RESERVING',?,?,?)",id,requestId,c.id,c.towTruckId,c.operatorId,c.operatorSubject,ts(now()
),r.searchAttemptNumber,body.toString());
```

Inserta la asignación RESERVING con candidato, recursos, operador, número de intento y payload de reserva. Todavía no publica una propuesta ni afirma que Fleet haya retenido. Genera ID nuevo en la aplicación. Inserta en assignments. Campos:

id: identificador UUID del registro.

L49

```
    return action(id,"RESERVE",body.toString());
```

Persiste/recupera intención RESERVE con UUID y payload del intento para que el coordinador haga HTTP.

L50

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L51

```
  public Action respond(UUID id,String subject,String key,boolean accept,String reason){
```

Campos: id: identificador UUID del registro; reason: motivo legible.

L52

```
    if(key==null || key.isBlank() || key.length()>100)throw new
ResponseStatusException(HttpStatus.BAD_REQUEST);
```

La rama condicional aborta ante la condición inválida.

L53

```
    View initial=get(id);if(!initial.operatorSubject().equals(subject))throw missing();AssistanceRequest
r=lock(initial.requestId());View a=get(id);
```

La rama condicional aborta ante la condición inválida.

L54

```
    String kind=accept?"ACCEPT":"REJECT",fingerprint=RequestService.digest(kind+":"+(reason==null?"":reaso
n.strip()));
```

Distingue ACCEPT/REJECT y hashea tipo más motivo normalizado para idempotencia de respuesta.

L55

```
    var previous=jdbc.queryForMap("select response_key,response_fingerprint,response_kind from assignments
 where id=?",id);
```

Consulta assignments con el filtro y proyección escritos.

L56

```
    if(previous.get("response_key")!=null){require(key.equals(previous.get("response_key")) &&
fingerprint.equals(previous.get("response_fingerprint")));return pending(id);}
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L57

```
    require(r.closedAt==null && a.status().equals("PROPOSED") && a.expiresAt().isAfter(now()) &&
r.searchDeadline.isAfter(now()));
```

Exige propuesta PROPOSED, solicitud abierta y vencimientos de propuesta/búsqueda todavía futuros antes de registrar una respuesta. Alcanzar el instante límite ya no permite aceptar. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L58

```
    jdbc.update("update assignments set response_key=?,response_fingerprint=?,response_kind=?,reason=?
where id=?",key,fingerprint,kind,reason,id);
```

Actualiza assignments para las filas del filtro.

L59

```
    // An active proposal retains its admitted route evidence until expiresAt; Fleet locks location while
HELD.
```

Comentario de intención o documentación; no se ejecuta.

L60

```
    // Dispatch independently revalidates routes. Acceptance still requires Fleet confirmation before its
deadline.
```

Comentario de intención o documentación; no se ejecuta.

L61

```
    if(accept)return action(id,"CONFIRM",null);
```

Persiste la intención CONFIRM; no acepta definitivamente ni recalcula ORS aquí. Solo la respuesta confirmada de Fleet fija ASSIGNED. La rama condicional selecciona la actualización o retorno correspondiente.

L62

```
    jdbc.update("update assignments set status='REJECTED',closed_at=? where id=?",ts(now()),id);r.resource
  ReleasePending=true;event(r,Status.PENDING_EVALUATION,"OPERATOR_REJECTED",subject);return
  action(id,"RELEASE",null);
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Actualiza assignments para las filas del filtro. Cambios: r.resourceReleasePending recibe true.

L63

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L64

```
   public Action pending(UUID id){return jdbc.query("select p.id,p.kind,a.reserve_payload from
  pending_actions p join assignments a on a.id=p.assignment_id where a.id=? and p.status<>'COMPLETED'
  order by p.created_at limit 1",(rs,n)->new Action(rs.getObject(1,UUID.class),id,rs.getString(2),rs.getSt
  ring(3)),id).stream().findFirst().orElse(null);}
```

Busca trabajo durable sin completar; no presupone que Fleet no lo haya aplicado. Consulta pending_actions, assignments con el filtro y proyección escritos. Campos: id: identificador UUID del registro.

L65

```
   public List<Action> pendingForRequest(UUID requestId){return jdbc.query("select
  p.id,p.assignment_id,p.kind,a.reserve_payload from pending_actions p join assignments a on
  a.id=p.assignment_id where a.request_id=? and p.status<>'COMPLETED' order by p.created_at",(rs,n)->new A
  ction(rs.getObject(1,UUID.class),rs.getObject(2,UUID.class),rs.getString(3),rs.getString(4)),requestId);
  }
```

Recupera acciones de todas las asignaciones de una solicitud. Consulta pending_actions, assignments con el filtro y proyección escritos. Campos: requestId: UUID de solicitud.

L66

```
   public boolean claim(Action action){return jdbc.update("update pending_actions set next_attempt_at=?
  where id=? and status<>'COMPLETED' and
  next_attempt_at<=?",ts(now().plusSeconds(30)),action.id(),ts(now()))==1;}
```

Adquiere por SQL una ventana de 30 s para que otro worker no ejecute simultáneamente la misma acción. Actualiza pending_actions para las filas del filtro.

L67

```
   public static int retryDelay(int failures){return failures<=0?5:failures==1?15:failures==2?30:60;}
```

Escalona la recuperación a 5, 15, 30 y luego 60 segundos.

L68

```
   public void failed(Action action){defer(action,0);}
```

Reprograma con backoff estándar sin mínimo adicional del proveedor.

L69

```
   public void defer(Action action,int minimum){Integer failures=jdbc.queryForObject("select failure_count
   from pending_actions where id=?",Integer.class,action.id());int
  delay=Math.max(minimum,retryDelay(failures));jdbc.update("update pending_actions set
  status='RETRYING',failure_count=failure_count+1,updated_at=?,next_attempt_at=? where id=? and
  status<>'COMPLETED'",ts(now()),ts(now().plusSeconds(delay)),action.id());}
```

Usa el mayor de backoff local y espera externa; conserva intención e incrementa fallos. Pospone la recuperación; respeta la espera mínima del proveedor y no descarta la intención. Consulta pending_actions con el filtro y proyección escritos. Actualiza pending_actions para las filas del filtro.

L70

```
 public List<Action> dueActions(){return jdbc.query("select
p.id,p.assignment_id,p.kind,a.reserve_payload from pending_actions p join assignments a on
a.id=p.assignment_id where p.status<>'COMPLETED' and p.next_attempt_at<=? order by
p.next_attempt_at,p.id limit 100",(rs,n)->new Action(rs.getObject(1,UUID.class),rs.getObject(2,UUID.clas
s),rs.getString(3),rs.getString(4)),ts(now()));}
```

Selecciona hasta cien acciones cuyo próximo intento ya venció. Consulta pending_actions, assignments con el filtro y proyección escritos.

L71

```
 public Action completed(Action action,JsonNode remote){
```

Procesa respuesta remota durable de RESERVE/CONFIRM/RELEASE y produce siguiente compensación si hace falta.

L72

```
  View initial=get(action.assignmentId());AssistanceRequest r=lock(initial.requestId());View
a=get(initial.id());
```

Localiza solicitud, bloquea vehículo/solicitud en orden y vuelve a leer asignación bajo ese bloqueo.

L73

```
  String actionState=jdbc.queryForObject("select status from pending_actions where
id=?",String.class,action.id());if("COMPLETED".equals(actionState))return pending(a.id());
```

Consulta pending_actions con el filtro y proyección escritos. La rama condicional selecciona la actualización o retorno correspondiente.

L74

```
  String status=remote.path("status").asText();require(Set.of("HELD","CONFIRMED","RELEASED","EXPIRED").c
ontains(status));
```

Campos: status: estado del objeto; su enum o CHECK define valores permitidos. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L75

```
  jdbc.update("update pending_actions set status='COMPLETED',updated_at=? where
id=?",ts(now()),action.id());
```

Actualiza pending_actions para las filas del filtro.

L76

```
  if(action.kind().equals("RESERVE")){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L77

```
   if(!remote.path("expiresAt").isMissingNode() && !remote.path("expiresAt").isNull() &&
(status.equals("HELD") || remote.path("createdHold").asBoolean()))jdbc.update("update assignments set
expires_at=? where id=?",ts(Instant.parse(remote.path("expiresAt").asText())),a.id());
```

Actualiza assignments para las filas del filtro. La rama condicional selecciona la actualización o retorno correspondiente.

L78

```
   if(r.closedAt!=null){r.resourceReleasePending=true;return action(a.id(),"RELEASE",null);}
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.resourceReleasePending recibe true.

L79

```
   if(status.equals("HELD")){Instant expiry=Instant.parse(remote.path("expiresAt").asText());
```

La rama condicional selecciona la actualización o retorno correspondiente.

L80

```
    if(Duration.between(now(),expiry).getSeconds()<rule(r,"search","minimum-offer-window-seconds")){jdbc
.update("update assignments set status='EXPIRED',closed_at=? where id=?",ts(now()),a.id());r.resourceRel
easePending=true;event(r,Status.PENDING_EVALUATION,"PROPOSAL_WINDOW_TOO_SHORT",null);return
action(a.id(),"RELEASE",null);}
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Actualiza assignments para las filas del filtro. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.resourceReleasePending recibe true.

L81

```
     jdbc.update("update assignments set status='PROPOSED',expires_at=? where
 id=?",ts(expiry),a.id());event(r,Status.AWAITING_ACCEPTANCE,"PROPOSAL_CREATED",null);}
```

Actualiza assignments para las filas del filtro.

L82

```
    else{jdbc.update("update assignments set status='EXPIRED',closed_at=? where
 id=?",ts(now()),a.id());event(r,Status.PENDING_EVALUATION,"RESERVATION_EXPIRED",null);}
```

Actualiza assignments para las filas del filtro.

L83

```
   }else if(action.kind().equals("CONFIRM")){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L84

```
    if(status.equals("CONFIRMED")){require(r.closedAt==null);Instant
 confirmed=Instant.parse(remote.path("confirmedAt").asText());jdbc.update("update assignments set
 status='ACCEPTED',accepted_at=? where id=?",ts(confirmed),a.id());if(r.firstAssignedAt==null)r.firstAssi
 gnedAt=confirmed;event(r,Status.ASSIGNED,"OPERATOR_ACCEPTED",a.operatorSubject());}
```

Aplica la confirmación durable: marca ACCEPTED con el instante recibido de Fleet, fija primera aceptación solo si no existía y pasa solicitud a ASSIGNED. Nunca restituye cancelación al reintentar. Actualiza assignments para las filas del filtro. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.firstAssignedAt recibe confirmed.

L85

```
    else{jdbc.update("update assignments set status='EXPIRED',closed_at=? where id=?",ts(now()),a.id());r
 .resourceReleasePending=true;event(r,Status.PENDING_EVALUATION,"CONFIRMATION_NOT_VALID",null);return
 action(a.id(),"RELEASE",null);}
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Actualiza assignments para las filas del filtro. Cambios: r.resourceReleasePending recibe true.

L86

```
   }else{require(status.equals("RELEASED") || status.equals("EXPIRED"));r.resourceReleasePending=false;}
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. Cambios: r.resourceReleasePending recibe false.

L87

```
   return null;
```

Entrega null al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L88

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L89

```
  public void reserveRejected(Action action){View a=get(action.assignmentId());AssistanceRequest
 r=lock(a.requestId());jdbc.update("update assignments set
 status='EXPIRED',closed_at=?,reason='RESOURCE_UNAVAILABLE' where
 id=?",ts(now()),a.id());jdbc.update("update pending_actions set status='COMPLETED',updated_at=? where id
 =?",ts(now()),action.id());if(r.closedAt==null)event(r,Status.PENDING_EVALUATION,"RESOURCE_UNAVAILABLE",
 null);else r.resourceReleasePending=false;}
```

Cierra el intento de reserva rechazado explícitamente y vuelve a evaluar si la solicitud sigue abierta. Actualiza assignments para las filas del filtro. Actualiza pending_actions para las filas del filtro. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.resourceReleasePending recibe false.

L90

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

9. Ejecución y automatización

El despacho revalida rutas; luego el operador declara etapas en orden. Los reemplazos automáticos son anteriores al despacho.

AutomationJobs (assistance-service)

Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AutomationJobs.java

SHA-256: ec885eea4cf7f288dc1d6b83a21525e58116792ac6b7917904c4325ace75ecdb

Dispara recuperación y avance periódico; reúne consultas de estado a Fleet y aísla fallos por solicitud. No actúa sobre etapas posteriores al despacho.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.stereotype.Component;
```

Importa tipos: org.springframework.stereotype.Component. Estereotipos Service/Component para registro e inyección.

L4

```
import org.springframework.scheduling.annotation.*;
```

Importa tipos: org.springframework.scheduling.annotation.*. Tareas periódicas y scheduler.

L5

```
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
```

Importa tipos: org.springframework.boot.autoconfigure.condition.ConditionalOnProperty. Arranque/autoconfiguración y condiciones de activación.

L6

```
import com.auxiruta.assistance.routing.FleetClient;
```

Importa tipos: com.auxiruta.assistance.routing.FleetClient. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L7

```
@Component @EnableScheduling
@ConditionalOnProperty(name="auxiruta.automation.enabled",havingValue="true",matchIfMissing=true)
```

Registra componente inyectable. Habilita tareas programadas. Activa bean solo si cumple propiedad configurada.

L8

```
public class AutomationJobs {
```

Declara clase AutomationJobs.

L9

```
 private final AutomationStore store;private final EvaluationService evaluations;private final
AssignmentCoordinator assignments;private final FleetClient fleet;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L10

```
 public AutomationJobs(AutomationStore store,EvaluationService evaluations,AssignmentCoordinator
assignments,FleetClient
fleet){this.store=store;this.evaluations=evaluations;this.assignments=assignments;this.fleet=fleet;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L11

```
@org.springframework.context.annotation.Bean
```

Registra el objeto retornado en Spring.

L12

```
public org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler taskScheduler(){var
scheduler=new org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler();scheduler.setPoolSize(
2);scheduler.setThreadNamePrefix("automation-");return scheduler;}
```

Dos hilos separan recuperación y avance.

L13

```
@org.springframework.context.event.EventListener(org.springframework.boot.context.event.ApplicationRead
yEvent.class)
```

Responde al evento de arranque.

L14

```
public void resume(){store.resumeAfterRestart();}
```

Al arrancar, vuelve a programar trabajo abierto sin reiniciar el recorrido despachado.

L15

```
@Scheduled(fixedDelay=1000,initialDelay=1000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa.

L16

```
public void recover(){try{assignments.recoverDue();}catch(Exception e){log("recovery",e);}}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L17

```
@Scheduled(fixedDelay=1000,initialDelay=2000)
```

Tarea periódica: fixedDelay se mide desde fin de ejecución previa.

L18

```
public void process(){
```

Tarea que avanza búsquedas, propone recursos y reúne reconciliaciones de Fleet.

L19

```
List<AutomationStore.Work> checks=new ArrayList<>();
```

Acumula revisiones de compromisos para consulta remota por lote.

L20

```
for(UUID id:store.due())try{var work=store.advance(id);if(work==null)continue;if(work.evaluate()){eval
uations.evaluate(id);assignments.propose(id);}else checks.add(work);}catch(Exception
e){log("request="+id,e);}
```

Campos: id: identificador UUID del registro. La rama condicional descarta este candidato y sigue con el próximo. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L21

```
if(!checks.isEmpty())try{var states=fleet.statuses(checks.stream().map(AutomationStore.Work::assignmen
tId).toList());if(states!=null)for(var work:checks)states.stream().filter(s->work.assignmentId().toStrin
g().equals(s.path("assignmentAttemptId").asText())).findFirst().ifPresent(s->store.reconcile(work,s));}c
atch(Exception e){log("fleet-reconciliation",e);}
```

Cruza respuesta por UUID; ausencia de respuesta no prueba liberación. Conserva solo elementos que cumplen el predicado. Transforma cada elemento al valor/tipo de salida. Materializa la secuencia en lista. La rama condicional selecciona la actualización o retorno correspondiente. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L22

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L23

```
private void log(String task,Exception e){org.slf4j.LoggerFactory.getLogger(getClass()).warn("Automatic
task deferred task={} type={}",task,e.getClass().getSimpleName());}
```

Registra los campos indicados, sin token ni secreto.

L24

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

AutomationStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/AutomationStore.java`

`SHA-256: 5c8580d6bccb4eb6328937ab727bb8b4e9680c1cc39ec879bd40335aff638025`

Máquina de decisiones persistentes: plazos, propuestas vencidas, reemplazos y reconciliación. No realiza HTTP dentro de sus transacciones.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java.

L5

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L6

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L7

```
import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables.

L8

```
import com.fasterxml.jackson.databind.*;
```

Importa tipos: com.fasterxml.jackson.databind.*. Lectura/escritura JSON, nodos y conversión a DTO.

L9

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L10

```
/** Durable planning and local decisions. No network calls in these transactions. */
```

Comentario de intención o documentación; no se ejecuta.

L11

```
@Service @Transactional
```

Registra lógica de aplicación como bean. Delimita transacción local; readOnly=true declara consulta.

L12

```
public class AutomationStore {
```

Declara clase AutomationStore.

L13

```
  public record Work(UUID requestId,UUID assignmentId,boolean evaluate) {}
```

Declara record inmutable Work. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud.

L14

```
  private final JdbcTemplate jdbc;private final RequestRepository requests;private final
HistoryRepository history;private final EvaluationRepository evaluations;private final ObjectMapper
mapper;private final Clock clock;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L15

```
  public AutomationStore(JdbcTemplate jdbc,RequestRepository requests,HistoryRepository
history,EvaluationRepository evaluations,ObjectMapper mapper,Clock clock){this.jdbc=jdbc;this.requests=r
equests;this.history=history;this.evaluations=evaluations;this.mapper=mapper;this.clock=clock;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L16

```
  private Instant now(){return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);}
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L17

```
  private static Timestamp ts(Instant t){return Timestamp.from(t);}
```

Convierte Instant a Timestamp JDBC; la variante nullable preserva null.

L18

```
  private int rule(AssistanceRequest r,String group,String key){try{return
mapper.readTree(r.configurationSnapshot).path(group).path(key).asInt();}catch(Exception e){throw new
IllegalStateException("Invalid rules");}}
```

Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L19

```
  private AssistanceRequest lock(UUID id){UUID
vehicle=requests.vehicleId(id).orElseThrow();jdbc.queryForObject("select id from vehicles where id=? for
 update",UUID.class,vehicle);return requests.lockById(id).orElseThrow();}
```

Si no encuentra resultado lanza la excepción indicada. Adquiere bloqueos en orden para serializar operaciones competidoras. Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Campos: id: identificador UUID del registro.

L20

```
  public void resumeAfterRestart(){jdbc.update("update assistance_requests set
evaluation_lease_until='1970-01-01T00:00:00Z',next_evaluation_at=? where closed_at is null and status in
 ('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','SEARCHING_ALTERNATIVE','ASSIGNED')",ts(now()))
;}
```

Restablece leases de evaluación y agenda trabajo abierto tras reinicio, sin reiniciar el proceso ya despachado. Actualiza assistance_requests para las filas del filtro.

L21

```
  public List<UUID> due(){return jdbc.queryForList("select id from assistance_requests where closed_at is
  null and status in
('REQUESTED','PENDING_EVALUATION','AWAITING_ACCEPTANCE','SEARCHING_ALTERNATIVE','ASSIGNED') and
next_evaluation_at<=? order by created_at,id limit 100",UUID.class,ts(now()));}
```

Selecciona trabajo abierto y vencido para procesar por lotes. Consulta assistance_requests con el filtro y proyección escritos.

L22

```
  private void event(AssistanceRequest r,Status next,String reason){Status
old=r.status;r.status=next;r.updatedAt=now();RequestStatusHistory h=new RequestStatusHistory();h.request
=r;h.previousStatus=old;h.newStatus=next;h.actorType="SYSTEM";h.reason=reason;h.occurredAt=now();history
.save(h);org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronizati
on(new org.springframework.transaction.support.TransactionSynchronization(){@Override public void
afterCommit(){org.slf4j.LoggerFactory.getLogger(AutomationStore.class).info("Automatic transition
requestId={} from={} to={} reason={}",r.id,old,next,reason);}});}
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos. Persiste el cambio de estado con actor/motivo y difiere el log hasta confirmar la transacción. Campos: reason: motivo legible. Cambios: r.status recibe next; r.updatedAt recibe now(); h.request recibe r; h.previousStatus recibe old; h.newStatus recibe next; h.actorType recibe "SYSTEM"; h.reason recibe reason; h.occurredAt recibe now().

L23

```
  private long pending(UUID id,String kind){return jdbc.queryForObject("select count(*) from
pending_actions p join assignments a on a.id=p.assignment_id where a.request_id=? and
p.status<>'COMPLETED'"+(kind==null?"":" and p.kind=?"),Long.class,kind==null?new Object[]{id}:new
Object[]{id,kind});}
```

Busca trabajo durable sin completar; no presupone que Fleet no lo haya aplicado. Consulta pending_actions, assignments contando coincidencias. Campos: id: identificador UUID del registro.

L24

```
  private void release(UUID assignment){jdbc.update("insert into
pending_actions(id,assignment_id,kind,status,created_at,updated_at) values (?,?,'RELEASE','PENDING',?,?)
 on conflict(assignment_id,kind) do nothing",UUID.randomUUID(),assignment,ts(now()),ts(now()));}
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila.

L25

```
  private void close(AssistanceRequest r,String reason){
```

Campos: reason: motivo legible.

L26

```
   for(var row:jdbc.queryForList("select id,status,expires_at from assignments where request_id=? and
status in ('RESERVING','PROPOSED')",r.id)){
```

Consulta assignments con el filtro y proyección escritos. Recorre el conjunto/intervalo indicado; los límites escritos acotan el trabajo.

L27

```
    UUID id=(UUID)row.get("id");Timestamp expires=(Timestamp)row.get("expires_at");String
state=expires!=null && !expires.toInstant().isAfter(now())?"EXPIRED":"CANCELLED";
```

Campos: id: identificador UUID del registro.

L28

```
    jdbc.update("update assignments set status=?,closed_at=? where
id=?",state,ts(now()),id);r.resourceReleasePending=true;
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Actualiza assignments para las filas del filtro. Cambios: r.resourceReleasePending recibe true.

L29

```
    if(jdbc.queryForObject("select count(*) from pending_actions where assignment_id=? and kind='RESERVE'
 and status<>'COMPLETED'",Long.class,id)==0)release(id);
```

Consulta pending_actions contando coincidencias. La rama condicional selecciona la actualización o retorno correspondiente.

L30

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L31

```
    r.replacementPending=false;r.closedAt=now();r.closureReasonCode=reason;event(r,Status.UNFULFILLED,reas
  on);
```

Cambios: r.replacementPending recibe false; r.closedAt recibe now(); r.closureReasonCode recibe reason.

L32

```
    }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L33

```
   private String deadlineReason(AssistanceRequest r){if(pending(r.id,null)>0)return
  "DEPENDENCY_TIMEOUT";return evaluations.findFirstByRequestIdOrderByEvaluatedAtDescIdDesc(r.id).map(e->"P
  ENDING_DEPENDENCY".equals(e.result)?"DEPENDENCY_TIMEOUT":"NO_AVAILABLE_RESOURCE".equals(e.reasonCode)?"N
  O_AVAILABLE_RESOURCE":"NO_VIABLE_CANDIDATE").orElse("DEPENDENCY_TIMEOUT");}
```

Transforma cada elemento al valor/tipo de salida. Distingue falta de candidatos de una dependencia que no pudo resolverse. Recupera evaluación más reciente por fecha e ID. La rama condicional selecciona la actualización o retorno correspondiente.

L34

```
  public Work advance(UUID id){
```

Campos: id: identificador UUID del registro.

L35

```
    AssistanceRequest r=lock(id);Instant now=now();if(r.closedAt!=null || !Set.of(Status.REQUESTED,Status.
  PENDING_EVALUATION,Status.AWAITING_ACCEPTANCE,Status.SEARCHING_ALTERNATIVE,Status.ASSIGNED).contains(r.s
  tatus) || r.nextEvaluationAt.isAfter(now))return null;
```

Serializa cambio mediante bloqueo de vehículo/solicitud antes de validar precondiciones. La rama condicional retorna sin trabajo si no aplica.

L36

```
    r.nextEvaluationAt=now.plusSeconds(5);
```

Cambios: r.nextEvaluationAt recibe now.plusSeconds(5).

L37

```
    // A remote confirmation may already have committed: never infer nonacceptance from elapsed time.
```

Comentario de intención o documentación; no se ejecuta.

L38

```
    if(pending(id,"CONFIRM")>0)return null;
```

No cierra ni reemplaza mientras existe confirmación sin resolver; el paso del tiempo no prueba que el operador no haya aceptado. La rama condicional retorna sin trabajo si no aplica.

L39

```
    var accepted=jdbc.queryForList("select id,accepted_at from assignments where request_id=? and
  status='ACCEPTED'",id);
```

Consulta assignments con el filtro y proyección escritos.

L40

```
    if(r.status==Status.ASSIGNED){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L41

```
     if(accepted.isEmpty())return null;var a=accepted.get(0);UUID assignment=(UUID)a.get("id");Instant dea
  dline=((Timestamp)a.get("accepted_at")).toInstant().plusSeconds(rule(r,"execution","dispatch-deadline-se
  conds"));
```

La rama condicional retorna sin trabajo si no aplica.

L42

```
    if(!deadline.isAfter(now)){replace(r,assignment,"DISPATCH_TIMEOUT");return null;}
```

La rama condicional retorna sin trabajo si no aplica.

L43

```
    if(!r.nextReconciliationAt.isAfter(now)){r.nextReconciliationAt=now.plusSeconds(rule(r,"fleet","recon
ciliation-interval-seconds"));r.nextEvaluationAt=deadline.isBefore(r.nextReconciliationAt)?deadline:r.ne
xtReconciliationAt;return new Work(id,assignment,false);}
```

La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.nextReconciliationAt recibe now.plusSeconds(rule(r,"fleet","reconciliation-interval-seconds")); r.nextEvaluationAt recibe deadline.isBefore(r.nextReconciliationAt)?deadline:r.nextReconciliationAt.

L44

```
    r.nextEvaluationAt=deadline.isBefore(r.nextReconciliationAt)?deadline:r.nextReconciliationAt;return
null;
```

Cambios: r.nextEvaluationAt recibe deadline.isBefore(r.nextReconciliationAt)?deadline:r.nextReconciliationAt.

L45

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L46

```
   if(r.status==Status.SEARCHING_ALTERNATIVE && r.replacementPending && !r.resourceReleasePending &&
pending(id,null)==0){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L47

```
    if(r.searchAttemptNumber>=1+rule(r,"search","max-reassignments")){close(r,"REASSIGNMENT_LIMIT_REACHED
");return null;}
```

Cierra al agotar un intento inicial más los dos reemplazos admitidos. Se evalúa después de liberar los recursos previos. La rama condicional retorna sin trabajo si no aplica.

L48

```
    r.replacementPending=false;r.searchAttemptNumber++;r.searchDeadline=now.plusSeconds(rule(r,"search","
attempt-timeout-seconds"));event(r,Status.PENDING_EVALUATION,"REPLACEMENT_SEARCH_STARTED");
```

Inicia el siguiente intento después de la liberación: incrementa contador y concede un nuevo plazo de búsqueda; conserva firstAssignedAt. Cambios: r.replacementPending recibe false; r.searchDeadline recibe now.plusSeconds(rule(r,"search","attempt-timeout-seconds")).

L49

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L50

```
   if(r.status==Status.SEARCHING_ALTERNATIVE && r.replacementPending && r.resourceReleasePending)return
null;
```

Suspende el reemplazo hasta confirmar la liberación remota para no comprometer dos recursos. La rama condicional retorna sin trabajo si no aplica.

L51

```
   if(!r.searchDeadline.isAfter(now)){close(r,deadlineReason(r));return null;}
```

La rama condicional retorna sin trabajo si no aplica.

L52

```
   if(r.resourceReleasePending || pending(id,null)>0)return null;
```

La rama condicional retorna sin trabajo si no aplica.

L53

```
   var proposals=jdbc.queryForList("select id,expires_at from assignments where request_id=? and
 status='PROPOSED'",id);
```

Consulta assignments con el filtro y proyección escritos.

L54

```
   if(!proposals.isEmpty()){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L55

```
    var a=proposals.get(0);Instant expires=((Timestamp)a.get("expires_at")).toInstant();
```

Toma la propuesta activa y su vencimiento para decidir espera o liberación.

L56

```
    if(!expires.isAfter(now)){UUID assignment=(UUID)a.get("id");jdbc.update("update assignments set
 status='EXPIRED',closed_at=? where id=?",ts(now),assignment);r.resourceReleasePending=true;release(assig
 nment);event(r,Status.SEARCHING_ALTERNATIVE,"PROPOSAL_EXPIRED");}
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Actualiza assignments para las filas del filtro. La rama condicional selecciona la actualización o retorno correspondiente. Cambios: r.resourceReleasePending recibe true.

L57

```
    else r.nextEvaluationAt=expires.isBefore(r.searchDeadline)?expires:r.searchDeadline;return null;
```

Cambios: r.nextEvaluationAt recibe expires.isBefore(r.searchDeadline)?expires:r.searchDeadline.

L58

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L59

```
   if(jdbc.queryForObject("select count(*) from assignments where request_id=? and
 search_attempt_number=? and expires_at is not null",Long.class,id,r.searchAttemptNumber)>=rule(r,"search
 ","max-offers-per-attempt")){close(r,"OFFER_LIMIT_REACHED");return null;}
```

Consulta assignments contando coincidencias. IS NOT NULL filtra filas que ya tienen valor en la columna indicada. La rama condicional retorna sin trabajo si no aplica.

L60

```
   if(r.status==Status.SEARCHING_ALTERNATIVE)event(r,Status.PENDING_EVALUATION,"SEARCH_RESUMED");
```

La rama condicional selecciona la actualización o retorno correspondiente.

L61

```
   r.nextEvaluationAt=now.plusSeconds(rule(r,"search","evaluation-interval-seconds"));return new
 Work(id,null,true);
```

Cambios: r.nextEvaluationAt recibe now.plusSeconds(rule(r,"search","evaluation-interval-seconds")).

L62

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L63

```
  private void replace(AssistanceRequest r,UUID assignment,String reason){jdbc.update("update assignments
  set status='CLOSED',closed_at=?,reason=? where id=?",ts(now()),reason,assignment);r.resourceReleasePend
 ing=true;r.replacementPending=true;r.nextEvaluationAt=now();release(assignment);event(r,Status.SEARCHING
 _ALTERNATIVE,reason);}
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Cierra la asignación previa y crea intención de liberación antes de buscar reemplazo. Actualiza assignments para las filas del filtro. Campos: reason: motivo legible. Cambios: r.resourceReleasePending recibe true; r.replacementPending recibe true; r.nextEvaluationAt recibe now().

L64

```
  public void reconcile(Work work,JsonNode remote){
```

Compara compromiso local con Fleet; solo reemplaza ante evidencia válida, no por ausencia o fallo de infraestructura.

L65

```
   AssistanceRequest r=lock(work.requestId());if(r.status!=Status.ASSIGNED || r.closedAt!=null ||
pending(r.id,"CONFIRM")>0)return;
```

Reconciliación limitada a solicitudes asignadas abiertas; no sustituye un servicio que ya comenzó su ejecución. La rama condicional selecciona la actualización o retorno correspondiente.

L66

```
   if(jdbc.queryForObject("select count(*) from assignments where id=? and request_id=? and
status='ACCEPTED'",Long.class,work.assignmentId(),r.id)==0)return;
```

Consulta assignments contando coincidencias. La rama condicional selecciona la actualización o retorno correspondiente.

L67

```
   if(remote==null ||
!work.assignmentId().toString().equals(remote.path("assignmentAttemptId").asText()))return;
```

La rama condicional selecciona la actualización o retorno correspondiente.

L68

```
   String status=remote.path("status").asText();if(status.equals("MISSING"))return; // Infrastructure
inconsistency is not authority to free a commitment.
```

La ausencia de registro remoto es inconsistencia de infraestructura; no autoriza a liberar unilateralmente el compromiso. Campos: status: estado del objeto; su enum o CHECK define valores permitidos. La rama condicional selecciona la actualización o retorno correspondiente.

L69

```
   if((status.equals("CONFIRMED") && remote.path("resource").has("eligible") &&
!remote.path("resource").path("eligible").asBoolean()) ||
Set.of("RELEASED","EXPIRED").contains(status))replace(r,work.assignmentId(),"RESOURCE_NOT_ELIGIBLE");
```

La rama condicional selecciona la actualización o retorno correspondiente.

L70

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L71

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ExecutionController (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionController.java`

`SHA-256: 2d10574e2a26c7ba41b0cd5a07803fd4ff0b295fef8c39bd5ad7b95dbc5ef539`

Acciones declaradas por el operador: despacho, llegada, carga, inicio y entrega. Cada paso requiere clave de idempotencia.

L1

```
 package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
 import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
 import jakarta.validation.Valid;
```

Importa tipos: jakarta.validation.Valid. Restricciones de DTO y activación de validación de entradas.

L4

```
import jakarta.validation.constraints.*;
```

Importa tipos: jakarta.validation.constraints.*. Restricciones de DTO y activación de validación de entradas.

L5

```
import org.springframework.web.bind.annotation.*;
```

Importa tipos: org.springframework.web.bind.annotation.*. Controladores, errores HTTP y clientes REST según la clase importada.

L6

```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
```

Importa tipos: org.springframework.security.core.annotation.AuthenticationPrincipal. Validación JWT, Authentication, autorización y cadenas de seguridad.

L7

```
import org.springframework.security.oauth2.jwt.Jwt;
```

Importa tipos: org.springframework.security.oauth2.jwt.Jwt. Validación JWT, Authentication, autorización y cadenas de seguridad.

L8

```
import io.swagger.v3.oas.annotations.Operation;
```

Importa tipos: io.swagger.v3.oas.annotations.Operation. Metadatos OpenAPI de rutas, respuestas y autenticación.

L9

```
import io.swagger.v3.oas.annotations.responses.*;
```

Importa tipos: io.swagger.v3.oas.annotations.responses.*. Metadatos OpenAPI de rutas, respuestas y autenticación.

L10

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Importa tipos: io.swagger.v3.oas.annotations.security.SecurityRequirement. Metadatos OpenAPI de rutas, respuestas y autenticación.

L11

```
import static com.auxiruta.assistance.requests.ExecutionStore.Step;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.ExecutionStore.Step. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L12

```
@RestController @RequestMapping("/api/v1/assistance-requests") @SecurityRequirement(name="bearerAuth")
```

Expone HTTP y serializa JSON. Documenta autenticación; no concede permisos. Ruta /api/v1/assistance-requests combinada con el prefijo de clase cuando corresponda.

L13

```
@ApiResponses({@ApiResponse(responseCode="400",description="Entrada
invalida"),@ApiResponse(responseCode="401",description="JWT ausente o
invalido"),@ApiResponse(responseCode="403",description="Rol
insuficiente"),@ApiResponse(responseCode="404",description="Recurso inexistente o
ajeno"),@ApiResponse(responseCode="409",description="Conflicto de estado, plazo o
idempotencia"),@ApiResponse(responseCode="503",description="Dependencia temporalmente no disponible;
consultar estado antes de reintentar")})
```

Documenta código HTTP.

L14

```
public class ExecutionController {
```

Declara clase ExecutionController.

L15

```
 public record Completion(@NotBlank @Size(min=10,max=500) String note) { public Completion {
if(note!=null)note=note.strip(); } }
```

Exige texto no vacío ni solo espacios. Limita caracteres/elementos según min/max. Declara record inmutable Completion. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. La rama condicional selecciona la actualización o retorno correspondiente.

L16

```
  private final ExecutionService service;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L17

```
  public ExecutionController(ExecutionService service){this.service=service;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L18

```
  @PostMapping("/{id}/dispatch") @Operation(summary="OPERATOR asignado: revalidar compromiso y rutas y
declarar salida")
```

Ruta /{id}/dispatch combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR asignado: revalidar compromiso y rutas y declarar salida.

L19

```
  public RequestDtos.HistoryView dispatch(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key){return
service.perform(id,jwt.getSubject(),key,Step.DISPATCH,null);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L20

```
  @PostMapping("/{id}/arrival") @Operation(summary="OPERATOR asignado: registrar llegada al origen
después del despacho")
```

Ruta /{id}/arrival combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR asignado: registrar llegada al origen después del despacho.

L21

```
  public RequestDtos.HistoryView arrival(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key){return
service.perform(id,jwt.getSubject(),key,Step.ARRIVAL,null);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L22

```
  @PostMapping("/{id}/loading") @Operation(summary="OPERATOR asignado: declarar vehículo cargado después
de la llegada")
```

Ruta /{id}/loading combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR asignado: declarar vehículo cargado después de la llegada.

L23

```
  public RequestDtos.HistoryView loading(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key){return
service.perform(id,jwt.getSubject(),key,Step.LOADING,null);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L24

```
  @PostMapping("/{id}/transport-start") @Operation(summary="OPERATOR asignado: iniciar traslado después
de cargar el vehículo")
```

Ruta /{id}/transport-start combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR asignado: iniciar traslado después de cargar el vehículo.

L25

```
 public RequestDtos.HistoryView start(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key){return
service.perform(id,jwt.getSubject(),key,Step.TRANSPORT_START,null);}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L26

```
 @PostMapping("/{id}/completion") @Operation(summary="OPERATOR asignado: registrar entrega con nota y
liberar recursos")
```

Ruta /{id}/completion combinada con el prefijo de clase cuando corresponda. Operación: OPERATOR asignado: registrar entrega con nota y liberar recursos.

L27

```
 public RequestDtos.HistoryView complete(@PathVariable UUID id,@AuthenticationPrincipal Jwt
jwt,@RequestHeader("Idempotency-Key") String key,@Valid @RequestBody Completion body){return
service.perform(id,jwt.getSubject(),key,Step.COMPLETION,body.note());}
```

Extrae ID de la ruta. Obtiene JWT validado, no identidad suministrada por el cliente. Lee header indicado, normalmente Idempotency-Key. Valida DTO antes del cuerpo del endpoint. Lee cuerpo JSON. Identidad obtenida del JWT, no del JSON enviado por el usuario. Campos: id: identificador UUID del registro.

L28

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ExecutionService (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionService.java`

`SHA-256: 724434f17972ffe848794db12efb38e8b9cdf194d2a4c88afc429a3704a11d25`

Coordina preparación, revalidación de despacho, commit y recuperación de liberación sin mantener bloqueos durante HTTP.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L2

```
import java.util.UUID;
```

Importa tipos: java.util.UUID. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L3

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L4

```
import static com.auxiruta.assistance.requests.ExecutionStore.Step;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.ExecutionStore.Step. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L5

```
@Service
```

Registra lógica de aplicación como bean.

L6

```
public class ExecutionService {
```

Declara clase ExecutionService.

L7

```
  private final ExecutionStore store;private final DispatchValidator dispatch;private final
AssignmentCoordinator assignments;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L8

```
  public ExecutionService(ExecutionStore store,DispatchValidator dispatch,AssignmentCoordinator
assignments){this.store=store;this.dispatch=dispatch;this.assignments=assignments;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L9

```
  public RequestDtos.HistoryView perform(UUID id,String subject,String key,Step step,String note){
```

Campos: id: identificador UUID del registro.

L10

```
   var prepared=store.prepare(id,subject,key,step,note);RequestDtos.HistoryView
result=prepared.previous();
```

Prepara paso bajo transacción; previous no nulo indica repetición ya registrada.

L11

```
   if(result==null){var evidence=step==Step.DISPATCH?dispatch.validate(prepared.input()):null;result=stor
e.commit(prepared.input(),subject,key,step,note,evidence);}
```

Solo un paso DISPATCH nuevo consulta evidencia externa; el store revalida antes de confirmar transición. La rama condicional selecciona la actualización o retorno correspondiente.

L12

```
   if(step==Step.COMPLETION)assignments.recover(id);
```

Reintenta liberar incluso al repetir entrega; no duplica COMPLETED. La rama condicional selecciona la actualización o retorno correspondiente.

L13

```
   return result;
```

Entrega result al llamador. El objeto o valor fue preparado y validado por las líneas precedentes de este método.

L14

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L15

```
}
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

ExecutionStore (assistance-service)

`Ruta: assistance-service/src/main/java/com/auxiruta/assistance/requests/ExecutionStore.java`

`SHA-256: 3e939df38caaf222bfea670421c83a67015cbc903392ca2c1a0e06ccd1ea963e`

Transacciones locales de ejecución. Prepara y revalida versión/asignación antes de cambiar el estado. Al completar, guarda intención durable de liberar recursos.

L1

```
package com.auxiruta.assistance.requests;
```

Ubica el archivo en com.auxiruta.assistance.requests. Organiza nombres y alcance; no crea un servicio por sí solo.

L3

```
import java.time.*;
```

Importa tipos: java.time.*. Instant, Duration y Clock para instantes UTC, plazos y reloj reemplazable en pruebas.

L4

```
import java.util.*;
```

Importa tipos: java.util.*. UUID, colecciones, Optional y comparadores utilizados en identidad, resultados y ranking.

L5

```
import java.math.BigDecimal;
```

Importa tipos: java.math.BigDecimal. BigDecimal y redondeo para pesos, coordenadas y duraciones sin aritmética binaria inexacta.

L6

```
import java.sql.Timestamp;
```

Importa tipos: java.sql.Timestamp. Timestamp y ResultSet para puentear fechas/filas de JDBC con objetos Java.

L7

```
import org.springframework.stereotype.Service;
```

Importa tipos: org.springframework.stereotype.Service. Estereotipos Service/Component para registro e inyección.

L8

```
import org.springframework.transaction.annotation.Transactional;
```

Importa tipos: org.springframework.transaction.annotation.Transactional. Límites y callbacks de transacciones SQL locales.

L9

```
import org.springframework.jdbc.core.JdbcTemplate;
```

Importa tipos: org.springframework.jdbc.core.JdbcTemplate. JdbcTemplate para SQL parametrizado, resultados y acciones durables.

L10

```
import org.springframework.http.HttpStatus;
```

Importa tipos: org.springframework.http.HttpStatus. Estados, headers, tipos de contenido y respuestas HTTP.

L11

```
import org.springframework.web.server.ResponseStatusException;
```

Importa tipos: org.springframework.web.server.ResponseStatusException. Controladores, errores HTTP y clientes REST según la clase importada.

L12

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

Importa tipos: com.fasterxml.jackson.databind.ObjectMapper. Lectura/escritura JSON, nodos y conversión a DTO.

L13

```
import static com.auxiruta.assistance.requests.DomainTypes.*;
```

Importa miembros estáticos: com.auxiruta.assistance.requests.DomainTypes.*. Contratos y componentes locales de Assistance; no una entidad de Fleet compartida.

L15

```
/** Local transactions only; no remote calls are performed while holding these locks. */
```

Comentario de intención o documentación; no se ejecuta.

L16

```
@Service
```

Registra lógica de aplicación como bean.

L17

```
@Transactional
```

Delimita transacción local; readOnly=true declara consulta.

L18

```
public class ExecutionStore {
```

Declara clase ExecutionStore.

L19

```
  public enum Step {
```

Declara enumeración cerrada Step.

L20

```
    DISPATCH(Status.ASSIGNED,Status.DISPATCHED,"dispatched_at"),
```

Mapea ASSIGNED a DISPATCHED y fecha dispatched_at.

L21

```
    ARRIVAL(Status.DISPATCHED,Status.ARRIVED,"arrived_at"),
```

Mapea DISPATCHED a ARRIVED y fecha arrived_at.

L22

```
    LOADING(Status.ARRIVED,Status.VEHICLE_LOADED,"loaded_at"),
```

Mapea ARRIVED a VEHICLE_LOADED y fecha loaded_at.

L23

```
    TRANSPORT_START(Status.VEHICLE_LOADED,Status.IN_TRANSIT,"transport_started_at"),
```

Mapea VEHICLE_LOADED a IN_TRANSIT y fecha transport_started_at.

L24

```
    COMPLETION(Status.IN_TRANSIT,Status.COMPLETED,"completed_at");
```

Mapea IN_TRANSIT a COMPLETED y fecha completed_at.

L25

```
    final Status previous,next;final String column;
```

Cada paso conserva predecesor, sucesor y columna temporal segura.

L26

```
    Step(Status previous,Status next,String
  column){this.previous=previous;this.next=next;this.column=column;}
```

Asigna o consulta campos de la instancia.

L27

```
    public boolean admits(Status state){return previous==state;}
```

Solo admite exactamente el estado predecesor; no permite saltar etapas.

L28

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L29

```
   public record Input(UUID requestId,long version,UUID assignmentId,UUID truckId,UUID operatorId,String
  subject,Instant acceptedAt,String rules,BigDecimal originLat,BigDecimal originLon,BigDecimal
  destinationLat,BigDecimal destinationLon,String mobility) {}
```

Declara record inmutable Input. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla. Campos: requestId: UUID de solicitud; version: versión para control optimista de concurrencia; operatorId: UUID del perfil operador; acceptedAt: instante de aceptación efectiva.

L30

```
  public record Preparation(Input input,RequestDtos.HistoryView previous) {}
```

Declara record inmutable Preparation. Java genera constructor, accesores, equals/hashCode; DTO no implica tabla.

L31

```
  private final JdbcTemplate jdbc;private final RequestRepository requests;private final
HistoryRepository history;private final Clock clock;private final ObjectMapper mapper;
```

Conserva dependencias/configuración; final impide reasignar referencias.

L32

```
  public ExecutionStore(JdbcTemplate jdbc,RequestRepository requests,HistoryRepository history,Clock
clock,ObjectMapper
mapper){this.jdbc=jdbc;this.requests=requests;this.history=history;this.clock=clock;this.mapper=mapper;}
```

Asigna o consulta campos de la instancia. Constructor del componente; recibe dependencias, o impide instanciación si es utilidad privada.

L33

```
  private Instant now(){return clock.instant().truncatedTo(java.time.temporal.ChronoUnit.MICROS);}
```

Obtiene el instante del reloj inyectado; truncar a microsegundos mantiene consistencia con PostgreSQL.

L34

```
  private static ResponseStatusException missing(){return new
ResponseStatusException(HttpStatus.NOT_FOUND);}
```

Construye 404; puede representar recurso inexistente o ajeno para no revelar su existencia.

L35

```
  private static void require(boolean value){if(!value)throw new
ResponseStatusException(HttpStatus.CONFLICT);}
```

Expresa una precondición; si resulta falsa se aborta con el estado HTTP indicado. La rama condicional aborta ante la condición inválida.

L36

```
  static String fingerprint(Step step,String note){return
RequestService.digest(step.name()+":"+(note==null?"":note.strip()));}
```

Calcula una huella del contenido para distinguir repetición idéntica de reutilización conflictiva de una clave.

L37

```
  private RequestDtos.HistoryView view(RequestStatusHistory h){return new RequestDtos.HistoryView(h.id,h.
request.getId(),h.previousStatus,h.newStatus,h.actorType,h.actorSubject,h.reason,h.details,h.operationKe
y,h.occurredAt);}
```

Transforma entidad/datos internos en DTO; no cambia la base por realizar esta proyección.

L38

```
  private AssistanceRequest lock(UUID id){UUID
vehicle=requests.vehicleId(id).orElseThrow(ExecutionStore::missing);jdbc.queryForObject("select id from
vehicles where id=? for update",UUID.class,vehicle);return
requests.lockById(id).orElseThrow(ExecutionStore::missing);}
```

Si no encuentra resultado lanza la excepción indicada. Adquiere bloqueos en orden para serializar operaciones competidoras. Consulta vehicles con el filtro y proyección escritos. Bloquea las filas durante la transacción. Campos: id: identificador UUID del registro.

L39

```
  public Preparation prepare(UUID id,String subject,String key,Step step,String note){
```

Campos: id: identificador UUID del registro.

L40

```
   if(key==null || key.isBlank() || key.length()>100)throw new
ResponseStatusException(HttpStatus.BAD_REQUEST);
```

La rama condicional aborta ante la condición inválida.

L41

```
   AssistanceRequest r=lock(id);
```

Serializa cambio mediante bloqueo de vehículo/solicitud antes de validar precondiciones.

L42

```
   if(jdbc.queryForObject("select count(*) from assignments where request_id=? and
 operator_subject=?",Long.class,id,subject)==0)throw missing();
```

Consulta assignments contando coincidencias. La rama condicional aborta ante la condición inválida.

L43

```
   var old=history.findByRequestIdAndOperationKey(id,key);
```

Busca resultado anterior para repetir sin duplicar.

L44

```
   if(old.isPresent()){var h=old.get();require(subject.equals(h.actorSubject) &&
 fingerprint(step,note).equals(h.requestFingerprint));return new Preparation(null,view(h));}
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L45

```
   var assignments=jdbc.query("select id,tow_truck_id,operator_id,operator_subject,accepted_at from
 assignments where request_id=? and status='ACCEPTED'",(rs,n)->new Input(id,r.version,rs.getObject("id",U
 UID.class),rs.getObject("tow_truck_id",UUID.class),rs.getObject("operator_id",UUID.class),rs.getString("
 operator_subject"),rs.getTimestamp("accepted_at").toInstant(),r.configurationSnapshot,r.originLatitude,r
 .originLongitude,r.destinationLatitude,r.destinationLongitude,r.mobilityCondition.name()),id);
```

Consulta assignments con el filtro y proyección escritos.

L46

```
   if(assignments.isEmpty())throw new ResponseStatusException(HttpStatus.CONFLICT);
```

La rama condicional aborta ante la condición inválida.

L47

```
   var input=assignments.get(0);if(!input.subject().equals(subject))throw
 missing();require(step.admits(r.status) && r.closedAt==null && !r.resourceReleasePending);
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional aborta ante la condición inválida.

L48

```
   if(step==Step.DISPATCH)require(dispatchOpen(input.acceptedAt(),now(),dispatchSeconds(input.rules())));
```

Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L49

```
   return new Preparation(input,null);
```

Devuelve datos preparados para operación nueva, sin evento previo de idempotencia.

L50

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L51

```
  static boolean dispatchOpen(Instant acceptedAt,Instant now,int seconds){return
 !now.isBefore(acceptedAt) && now.isBefore(acceptedAt.plusSeconds(seconds));}
```

Exige ahora mayor o igual a aceptación y estrictamente anterior al vencimiento de despacho. Campos: acceptedAt: instante de aceptación efectiva.

L52

```
  private int dispatchSeconds(String rules){try{return
 mapper.readTree(rules).path("execution").path("dispatch-deadline-seconds").asInt();}catch(Exception
 e){throw new IllegalStateException("Invalid rule snapshot");}}
```

Lee el plazo del snapshot de la solicitud, no de una configuración posterior. Captura el fallo para clasificarlo o reprogramarlo; no significa éxito remoto.

L53

```
 public RequestDtos.HistoryView commit(Input expected,String subject,String key,Step step,String
note,DispatchValidator.Evidence evidence){
```

Vuelve a validar el estado y aplica transición e historial atómicamente; para despacho comprueba evidencia aún vigente.

L54

```
  Preparation
preparation=prepare(expected.requestId(),subject,key,step,note);if(preparation.previous()!=null)return
preparation.previous();
```

La rama condicional selecciona la actualización o retorno correspondiente.

L55

```
  Input current=preparation.input();require(current.version()==expected.version() &&
current.assignmentId().equals(expected.assignmentId()));
```

Tras el HTTP vuelve a comprobar versión y asignación. Si cambiaron, aborta en vez de aplicar evidencia a otro estado. Cada require exige su condición; el helper aborta con el HTTP definido si falla.

L56

```
  AssistanceRequest r=requests.findById(expected.requestId()).orElseThrow();Instant at=now();
```

Si no encuentra resultado lanza la excepción indicada. Spring deriva el filtro del nombre findBy; Optional expresa ausencia y Page incluye metadatos.

L57

```
  if(step==Step.DISPATCH)require(evidence!=null && !at.isAfter(evidence.validUntil()) &&
evidence.qualificationUntil().isAfter(at.plusSeconds(evidence.plannedSeconds())));
```

Antes de guardar despacho exige evidencia aún vigente y habilitación estrictamente posterior al fin planificado. Cada require exige su condición; el helper aborta con el HTTP definido si falla. La rama condicional selecciona la actualización o retorno correspondiente.

L58

```
  RequestStatusHistory h=new RequestStatusHistory();h.request=r;h.previousStatus=r.status;h.newStatus=st
ep.next;h.actorType="OPERATOR";h.actorSubject=subject;h.reason=step==Step.COMPLETION?note.strip():step.n
ame();h.details=evidence==null?null:evidence.json();h.operationKey=key;h.requestFingerprint=fingerprint(
step,note);h.occurredAt=at;
```

Cambios: h.request recibe r; h.previousStatus recibe r.status; h.newStatus recibe step.next; h.actorType recibe "OPERATOR"; h.actorSubject recibe subject; h.reason recibe step==Step.COMPLETION?note.strip():step.name(); h.details recibe evidence==null?null:evidence.json(); h.operationKey recibe key; h.requestFingerprint recibe fingerprint(step,note); h.occurredAt recibe at.

L59

```
  r.status=step.next;r.updatedAt=at;
```

Aplica el estado sucesor del enum y actualiza la fecha local; el historial y la etapa de asignación se guardan en esta misma transacción. Cambios: r.status recibe step.next; r.updatedAt recibe at.

L60

```
  // Column comes exclusively from the enum, never from client input.
```

Comentario de intención o documentación; no se ejecuta.

L61

```
  jdbc.update("update assignments set "+step.column+"=? where
id=?",Timestamp.from(at),current.assignmentId());
```

Actualiza la fecha de la etapa usando una columna tomada del enum cerrado Step; el cliente no puede inyectar nombres SQL. Actualiza assignments para las filas del filtro.

L62

```
  if(step==Step.COMPLETION){
```

La rama condicional selecciona la actualización o retorno correspondiente.

L63

```
   r.closedAt=at;r.closureReasonCode="SERVICE_COMPLETED";r.resourceReleasePending=true;
```

Marca que todavía se debe confirmar liberación en Fleet; el estado local por sí solo no demuestra que los recursos remotos estén libres. Cambios: r.closedAt recibe at; r.closureReasonCode recibe "SERVICE_COMPLETED"; r.resourceReleasePending recibe true.

L64

```
    jdbc.update("update assignments set status='CLOSED',closed_at=?,completion_note=? where
id=?",Timestamp.from(at),note.strip(),current.assignmentId());
```

Actualiza assignments para las filas del filtro.

L65

```
    jdbc.update("insert into pending_actions(id,assignment_id,kind,status,created_at,updated_at) values
(?,?,'RELEASE','PENDING',?,?) on conflict(assignment_id,kind) do
nothing",UUID.randomUUID(),current.assignmentId(),Timestamp.from(at),Timestamp.from(at));
```

Genera ID nuevo en la aplicación. Inserta en pending_actions. DO NOTHING evita duplicar la intención/fila.

L66

```
   }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L67

```
   history.saveAndFlush(h);
```

Persiste y fuerza sincronización SQL para detectar restricciones antes de retornar; commit ocurre al terminar transacción.

L68

```
   org.springframework.transaction.support.TransactionSynchronizationManager.registerSynchronization(new
org.springframework.transaction.support.TransactionSynchronization(){@Override public void
afterCommit(){org.slf4j.LoggerFactory.getLogger(ExecutionStore.class).info("Request execution
requestId={} from={} to={}",r.id,h.previousStatus,h.newStatus);}});
```

Registra un callback del ciclo transaccional; no ejecuta el log antes del commit. Implementa callback afterCommit para anunciar solo cambios confirmados. Redefine contrato heredado. Emite log solo después del commit, no anuncia cambios revertidos.

L69

```
   return view(h);
```

Devuelve DTO del evento confirmado, no la entidad persistente completa.

L70

```
  }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

L71

```
 }
```

Cierra/delimita la estructura anterior; no introduce una operación de negocio.

Líneas vacías 2, 14: separadores sin instrucciones.

10. Pruebas y utilidades de demostración

Las pruebas no se empaquetan en app.jar. Integración requiere contenedores y puede reiniciarlos; los seeds demo dejan datos persistentes.

BackendClient (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/BackendClient.java`

`SHA-256: 37e1e52f1d9128a0b27c8552d87951bef1d15de8cd01a464b551bb82a35af2e0`

Infraestructura de pruebas: HTTP, PKCE, tokens en memoria, comandos Docker e identidades temporales. No se empaqueta en el JAR productivo.

Métodos/casos localizables: encode, form, json, claims, initialize, login, call, direct, admin, secondDriver, cleanup, id, key.

No se ejecuta sola: otras pruebas utilizan sus métodos de apoyo.

BackendIntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/BackendIntegrationTest.java`

`SHA-256: e8eb109c7a6d0787f357233483c7f620e6b9269bd868a89cdf0ce5041fa3655c`

Keycloak/Gateway/BD reales: seguridad, flota, vehículos, solicitudes, propiedad, concurrencia y persistencia tras reinicio; rutas reales bajo indicador optativo.

Métodos/casos localizables: setup, finish, api, a, o, d, dk, identityAndSecurityRegression, fleetAdministrationRegression, vehiclesRequestsAndCancellation, requestSnapshotAndHistorySurviveRestart, realRouteAndFleetEvaluation, routingBudgetState.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.integration=true"
"-Dtest=BackendIntegrationTest"
```

Requiere Docker y entorno local.

Añadir "-Dauxiruta.realRouting=true" para exigir rutas reales.

DeliveryArtifacts (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryArtifacts.java`

`SHA-256: 1689846e1e39a549a9af942054b345440dd61d895635414dfb2933edcc584763`

Define el plan de 47 pasos, scripts/capturas Postman y entorno sin secretos. Es soporte de entrega, no controlador de negocio.

Métodos/casos localizables: requestBody, write, event, initScript, testScript.

No se ejecuta sola: otras pruebas utilizan sus métodos de apoyo.

DeliveryArtifactsTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryArtifactsTest.java`

`SHA-256: 5ca569dba91b8d233ae2526eab1b412ec6f2570c0712f28b498aa56ac3541178`

Compara colección y entorno publicados con la definición Java; exporta solo bajo indicador explícito.

Métodos/casos localizables: committedCollectionMatchesExecutableJavaPlan.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dtest=DeliveryArtifactsTest"
```

No requiere Docker: dependencias controladas.

DeliveryIntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DeliveryIntegrationTest.java`

`SHA-256: 0e011c7fbb4b218c2daba64571ab6755759abdb7c2752b3b6affed5b36fa1cd5`

Comprueba contratos OpenAPI y ejecuta por Java las llamadas de la colección. No ejecuta el sandbox JavaScript de Postman.

Métodos/casos localizables: login, openApiMatchesControllersAndExport, executePublishedCollectionWithJava, resolve, cleanupOwnFixtures.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.delivery=true"
"-Dtest=DeliveryIntegrationTest"
```

Requiere Docker y entorno local.

DemoDataSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/DemoDataSetupTest.java`

`SHA-256: 210c2da461acab48ffaa33775481440141d576e19d7e0756ee4030ac4157f1c3`

Prepara operador principal y habilitación; incorpora preparación persistente de privacidad. Requiere recurso libre.

Métodos/casos localizables: prepareOperator, prepare.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.seed=true" "-Dtest=DemoDataSetupTest"
```

Requiere Docker y entorno local.

Conserva datos de demostración al finalizar.

KeycloakScopeSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/KeycloakScopeSetupTest.java`

`SHA-256: ff7aa4917e79e847b87054181aa2c2fe79e64e8f3c6fb712b30b44053eb2746d`

Reparación optativa e idempotente del scope basic en instalaciones antiguas de Keycloak.

Métodos/casos localizables: ensureBasicScopeWithoutDeletingRealm.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.repairIdentity=true"
"-Dtest=KeycloakScopeSetupTest"
```

Requiere Docker y entorno local.

Phase7IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase7IntegrationTest.java`

`SHA-256: 3c3b73ab92c72241f77e6fd0a0d1629e9e05bd3ceb830a97dd04c047f3803e10`

Reservas exclusivas, carrera aceptación/cancelación, rechazo, recuperación de confirmación y evidencia conservada durante oferta.

Métodos/casos localizables: initializeClient, resources, request, proposal, exclusiveReservationAndIdempotence, acceptanceAndCancellationRace, acceptsDurablyAndBlocksAdministration, rejectionReleasesAndIsIdempotent, lostConfirmationRecoveredAfterRestart, sql, internal, cleanupFixtures, fixtureSql, activeProposalKeepsItsRouteEvidenceUntilAcceptance.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase7=true"
"-Dtest=Phase7IntegrationTest"
```

Requiere Docker y entorno local.

Phase8IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase8IntegrationTest.java`

`SHA-256: 731ff9f5a04005ab9067b9e30111bb53f0344ff57c36a750f464ae08657e2886`

Recorrido hasta COMPLETED, pasos inválidos, despacho vencido y entrega durante caída de Fleet.

Métodos/casos localizables: initializeClient, removeTemporaryIdentity, setup, path, action, toTransit, completesServiceAndPreservesIdempotentHistory, expiredDispatchIsRejected, completionKeepsPendingReleaseAcrossFleetOutage, sql, container, startFleet, cleanup.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase8=true"
"-Dtest=Phase8IntegrationTest"
```

Requiere Docker y entorno local.

Phase9IntegrationTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/Phase9IntegrationTest.java`

`SHA-256: 39392ce34bdbf43fbd80abf4dcea978985dc4f03158858fff0b40e033b7d16b3`

Perfil demo: búsqueda, vencimientos, cupos, reemplazos, exclusión por rechazo y recuperación tras reinicios.

Métodos/casos localizables: initializeClient, resources, request, current, ready, await, active, proposal, accept, sql, uuid, availabilityTriggersSearchAndOfferExpiryPreservesAttempt, emptySearchClosesAutomatically, twoReassignmentsThenCloseWithoutRestoringCancellation, confirmationIsRecoveredAfterRestartAndDeadline, cancelledUncertainReserveIsCompensatedAutomatically, releaseSurvivesRestartWithoutRepeatingCompletion, fiveExpiredOffersExhaustTheSearchWithoutChangingDeadline, startFleet, cleanup, rejectingOperatorIsExcludedInNextAttempt.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.phase9=true"
"-Dtest=Phase9IntegrationTest"
```

Requiere Docker y entorno local.

Arrancar con perfil docker,demo y restaurar docker al terminar.

PrivacyDemoSetupTest (api-gateway)

`Ruta: api-gateway/src/test/java/com/auxiruta/gateway/integration/PrivacyDemoSetupTest.java`

`SHA-256: 7dd5b8f3a5f933c0fa30a42f416945c514545181c22ac1318fe3af0a0010ae24`

Crea/reutiliza cuentas secundarias y vehículos demo; comprueba 200 propios, 404 cruzados y listado aislado. Deja datos persistentes.

Métodos/casos localizables: prepareAndVerifyPrivacy, prepare, ensureIdentity, ensureOperator, ensureVehicle, findVehicle.

Ejecución desde raíz

```
.\api-gateway\mvnw.cmd -f api-gateway/pom.xml test "-Dauxiruta.privacySeed=true"
"-Dtest=PrivacyDemoSetupTest"
```

Requiere Docker y entorno local.

Conserva datos de demostración al finalizar.

SecurityConfigurationTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/config/SecurityConfigurationTest.java`

`SHA-256: 01120fe023d8a825d9bf495291a22c2342c06519811c1d4ddbe5bda0fc996233`

Acceso sin token, rol permitido, rol incorrecto y separación de rutas internas.

Métodos/casos localizables: publicRoute, internalRoute, noTokenRejected, authorizedRoleAccepted, wrongRoleRejected, humanCannotCallInternal.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=SecurityConfigurationTest"
```

No requiere Docker: dependencias controladas.

AssignmentCoordinatorTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AssignmentCoordinatorTest.java`

`SHA-256: a5bc0455d50bb8fba33105447af0fd59106f968b0084853137393c6b70b72408`

Respuesta perdida, compensación, recuperación con payload original, claim y Retry-After.

Métodos/casos localizables: allowClaims, lostConfirmationResponseRemainsDurableWithoutImmediateRetry, cancellationCompensatesTheSameReservationAttempt, recoveryReusesPersistedAttemptAndPayload, anotherWorkerCannotExecuteAnAlreadyClaimedAction, retryAfterHonorsSecondsAndHttpDate.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AssignmentCoordinatorTest"
```

No requiere Docker: dependencias controladas.

AutomationConfigurationTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AutomationConfigurationTest.java`

`SHA-256: a92903c084bd25c39b5a117ba13f37d6e61f2525b0020da121cc3abc7df26b7b`

Tiempos demo y backoff, preservando cuotas y planificación.

Métodos/casos localizables: demoChangesOnlyWaitsNotTravelPlanningOrQuotas, recoveryBackoffRemainsBoundedAndNeverDiscards.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AutomationConfigurationTest"
```

No requiere Docker: dependencias controladas.

AutomationStoreTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/AutomationStoreTest.java`

`SHA-256: cdeeb7a2ca208b7a8805716962f44d069cec13bf71d37ca27d8e45f1c0f278ae`

Cierres, incertidumbre de confirmación, liberación previa al reemplazo y cupos.

Métodos/casos localizables: setup, finish, pendingConfirmationPreventsDeadlineClosure, deadlineClosesWithoutExtendingSearch, dispatchExpiryWaitsForReleaseBeforeStartingNewDeadline, expiredOfferDuringReplacementDoesNotConsumeAnotherReassignment, thirdAcceptedAttemptCannotStartFourthSearch, fiveCreatedOffersCloseTheAttempt, executionStatesAreNeverAutomaticallyReassigned.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=AutomationStoreTest"
```

No requiere Docker: dependencias controladas.

DispatchValidatorTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/DispatchValidatorTest.java`

`SHA-256: 4dfc1af05b94a56ff4771262e6d4c2614054d23a74dcfc1301d417f021b0d812`

Compromiso, cobertura, rutas, habilitación y tolerancia a la edad de ubicación después de aceptar.

Métodos/casos localizables: setup, resource, validResourceProducesAuditableEvidence, acceptedLocationDoesNotExpireForDispatch, qualificationMustOutlivePlannedEndStrictly, unconfirmedResourceCannotDispatch, quotaIsTemporaryFailure, confirmedMissingRouteIsDomainConflict, malformedFleetResponseIsTechnicalFailure.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=DispatchValidatorTest"
```

No requiere Docker: dependencias controladas.

EvaluationServiceTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/EvaluationServiceTest.java`

`SHA-256: 66fc898356eae58ec079b6d33408edcca99a29f1728e72255665ddae5d440807`

Cobertura, redondeo de duración, límites de recorridos, ranking y vigencia de habilitación.

Métodos/casos localizables: setup, distanceLimitsAreInclusiveAndNotRounded, adjustedSecondsRoundUp, transportDurationBoundary, coverageFailureAvoidsExternalCalls, dependencyFailureDoesNotReject, missingRouteIsARecordedDomainRejection, rankingUsesDurationThenFreeTimeThenLexicalUuid, qualificationMustExtendStrictlyBeyondPlannedEnd, expiredOrClosedRequestIsNotEvaluated, cancellationRaceDiscardsResultAtCommit.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=EvaluationServiceTest"
```

No requiere Docker: dependencias controladas.

EvaluationStoreTest (assistance-service)

Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/EvaluationStoreTest.java

SHA-256: 10dd13e2bc6d843d83a20f975fa6f37f43d391ef6005a42ac916155cc44974c5

Descartar una evaluación incompatible con una cancelación o modificación concurrente.

Métodos/casos localizables: concurrentCancellationCannotBeOverwritten.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=EvaluationStoreTest"
```

No requiere Docker: dependencias controladas.

ExecutionRulesTest (assistance-service)

Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/ExecutionRulesTest.java

SHA-256: acd734498001083ef595d7234c9e85ed35b8258e4bd8c867dcdb5db61d3553ae

Predecesor de cada paso, plazo exclusivo de despacho y huella de operación.

Métodos/casos localizables: everyActionAcceptsOnlyItsPredecessor, dispatchDeadlineIsExclusive, sameKeyCannotRepresentAnotherStepOrNote.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=ExecutionRulesTest"
```

No requiere Docker: dependencias controladas.

ExecutionServiceTest (assistance-service)

Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/ExecutionServiceTest.java

SHA-256: 2a7e0d3449a713536069b5afc3ddc10ba2799750a312245e88125e1ebf5d3369

Fallo de despacho sin transición y repetición de entrega sin duplicar el evento.

Métodos/casos localizables: replayedCompletionRetriesReleaseWithoutAnotherTransition, failedDispatchDoesNotChangeState.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=ExecutionServiceTest"
```

No requiere Docker: dependencias controladas.

RequestRulesTest (assistance-service)

Ruta: assistance-service/src/test/java/com/auxiruta/assistance/requests/RequestRulesTest.java

SHA-256: 726d75690196cc5736d7ae889eae9953ab5498c5a6a7917d6a26c31bc4cce7ef

Patentes, fingerprints, snapshot, exclusividad de cancelación y copia de datos del vehículo.

Métodos/casos localizables: fingerprintNormalizesDecimalScale, changedBodyHasDifferentFingerprint, previousAcceptanceBlocksCancellationEvenDuringAlternativeSearch, terminalAndExecutionStatesCannotBeCancelled, onlyPreAcceptanceStatesCanBeCancelled, plateNormalizedAndInvalidRejected, snapshotIncludesAllParametersAndCorrectDeadline, creationUsesInjectedClockAndCopiesVehicleData.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=RequestRulesTest"
```

No requiere Docker: dependencias controladas.

OpenRouteServiceClientTest (assistance-service)

Ruta: assistance-service/src/test/java/com/auxiruta/assistance/routing/OpenRouteServiceClientTest.java

SHA-256: 3d10ddcc97a2f9043e30919a923ce9bc94c3ba141af999c4cf82bcfc0ae3e4e7

Servidor HTTP Java local: caché, parsing, errores ORS, cuota, snap y clave ausente; no consume ORS real.

Métodos/casos localizables: setup, stop, parsesMetricRouteAndCachesSameRevision, noRouteIsDomainFailure, unknownServerFailureIsNotDomainRejection, quotaRespectsRetryAfterWithoutImmediateRetry, dailyQuotaDiffersFromInvalidCredentials, localBudgetBlocksNetwork, malformedSuccessRemainsPending, excessiveSnapIsRejected, missingKeyDoesNotConsumeQuota.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=OpenRouteServiceClientTest"
```

No requiere Docker: dependencias controladas.

RoutingBudgetTest (assistance-service)

`Ruta: assistance-service/src/test/java/com/auxiruta/assistance/routing/RoutingBudgetTest.java`

`SHA-256: 0ce78f1278046df3a21cce3e29db2b274ac98a1db7174cf6d5d41d5bd8ff4e00`

Presupuesto por minuto/día y bloqueo impuesto por el proveedor con JDBC simulado.

Métodos/casos localizables: setup, consumesBudgetBeforeNetworkCall, minuteLimitPreventsReservation, dailyLimitPreventsReservation, providerBackoffPreventsReservation.

Ejecución desde raíz

```
.\assistance-service\mvnw.cmd -f assistance-service/pom.xml test "-Dtest=RoutingBudgetTest"
```

No requiere Docker: dependencias controladas.

TokenSupportTest (auxiruta-commons)

`Ruta: auxiruta-commons/src/test/java/com/auxiruta/commons/security/TokenSupportTest.java`

`SHA-256: 248337d5dfa9a88f968a8aec3834e2b3c849195908ee3a111839bf6df6188ca7`

JWT válido/inválido, emisor, audiencias, roles humanos/técnicos y propiedad.

Métodos/casos localizables: correctAudienceAndIssuerAccepted, wrongIssuerRejected, wrongAudienceRejected, expiredRejected, adminCannotImpersonateDriver, driverAllowed, technicalClientSeparated, wrongAuthorizedPartyRejected, unknownRolesIgnored, ownerAllowedAndForeignResourceHidden, missingAudienceRejectedWithoutException, missingExpiryOrSubjectRejected.

Ejecución desde raíz

```
.\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml test "-Dtest=TokenSupportTest"
```

No requiere Docker: dependencias controladas.

CandidateServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/CandidateServiceTest.java`

`SHA-256: cebcd6daab572672dec489245009258d66e14ea79daa67afbbd23f8a10c8ec06`

Edad inclusiva de ubicación, capacidad, movilidad, habilitación, cobertura y máximo de candidatos.

Métodos/casos localizables: locationAgeInclusive, incompatibleCapacityAndMobilityExcluded, expiredQualificationAndOutsideCoverageExcluded, atMostFiveCandidates.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=CandidateServiceTest"
```

No requiere Docker: dependencias controladas.

FleetServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/FleetServiceTest.java`

`SHA-256: 9155a873cbddcc15342e90ba0e19c7b08835374bbfe8252df2c95468de665343`

Disponibilidad idempotente, habilitación, reloj, perfil propio y bloqueo de ubicación de recursos ocupados.

Métodos/casos localizables: reservations, repeatedAvailabilityDoesNotResetFreeTime, invalidQualificationDoesNotWrite, locationTimestampComesFromClockWithDatabasePrecision, missingOwnProfileIsHidden, replayReturnsOriginalEventWithoutReapplyingOldState, committedTruckCannotChangeLocationButReleasedTruckCan.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=FleetServiceTest"
```

No requiere Docker: dependencias controladas.

ReservationServiceTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/catalog/ReservationServiceTest.java`

`SHA-256: 319bffe3914af506b27f9db2917393664937d7cc0f71de8629f213d8f9f17c87`

Reserva/confirmación/liberación, límites de tiempo, idempotencia, compatibilidad demo y ubicación antigua tras aceptar.

Métodos/casos localizables: setup, holdExpiresAtSixtySeconds, shortSearchWindowCreatesDurableTerminalAttempt, activeCommitmentExcludesNewAttempt, confirmationAtExactExpiryIsRejected, confirmedReservationDoesNotExpire, invalidQualificationReleasesHold, releaseIsIdempotent, confirmationCommitsBeforeDeadline, demoWindowComesFromTheRequestSnapshot, legacyPayloadWithoutTimingFieldsRemainsIdempotent, confirmedLocationDoesNotExpire, oldLocationCannotBeConfirmed.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=ReservationServiceTest"
```

No requiere Docker: dependencias controladas.

SecurityConfigurationTest (fleet-service)

`Ruta: fleet-service/src/test/java/com/auxiruta/fleet/config/SecurityConfigurationTest.java`

`SHA-256: f2d8c568a8bbd02df25c0554a6ab6b98bbc3a9dc3f63ff5396cb609f354d0e36`

Acceso sin token, rol permitido, rol incorrecto y separación de rutas internas.

Métodos/casos localizables: publicRoute, internalRoute, noTokenRejected, authorizedRoleAccepted, wrongRoleRejected, humanCannotCallInternal, technicalClientAcceptedInternally.

Ejecución desde raíz

```
.\fleet-service\mvnw.cmd -f fleet-service/pom.xml test "-Dtest=SecurityConfigurationTest"
```

No requiere Docker: dependencias controladas.

11. Maven Wrapper: herramientas distribuidas

Maven Wrapper es código de terceros distribuido con cada módulo. Se explica uso/configuración, sin reproducir implementaciones repetidas de descarga/plataforma como lógica propia.

api-gateway/.mvn/wrapper/maven-wrapper.properties

`Ruta: api-gateway/.mvn/wrapper/maven-wrapper.properties`

`SHA-256: 821602a8cceb1f8f5945f80b82b27617ba88b74a8afae230a5df80affcb6a8ab`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
.9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

api-gateway/mvnw

`Ruta: api-gateway/mvnw`

`SHA-256: cae96cef89ebea3531221f4ae17c23cf8edf67d00eae8306d4186ae1bbed4d02`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\api-gateway\mvnw.cmd -f api-gateway/pom.xml verify. install de Commons publica el JAR en .m2.

api-gateway/mvnw.cmd

`Ruta: api-gateway/mvnw.cmd`

`SHA-256: 46eedb8419bd14fe70d5bb2916d7b6f51806e51b39d5b76a42610384ca929c1c`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\api-gateway\mvnw.cmd -f api-gateway/pom.xml verify. install de Commons publica el JAR en .m2.

assistance-service/.mvn/wrapper/maven-wrapper.properties

`Ruta: assistance-service/.mvn/wrapper/maven-wrapper.properties`

`SHA-256: 821602a8cceb1f8f5945f80b82b27617ba88b74a8afae230a5df80affcb6a8ab`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
.9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

assistance-service/mvnw

`Ruta: assistance-service/mvnw`

`SHA-256: cae96cef89ebea3531221f4ae17c23cf8edf67d00eae8306d4186ae1bbed4d02`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\assistance-service\mvnw.cmd -f assistance-service/pom.xml verify. install de Commons publica el JAR en .m2.

assistance-service/mvnw.cmd

`Ruta: assistance-service/mvnw.cmd`

`SHA-256: 46eedb8419bd14fe70d5bb2916d7b6f51806e51b39d5b76a42610384ca929c1c`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\assistance-service\mvnw.cmd -f assistance-service/pom.xml verify. install de Commons publica el JAR en .m2.

auxiruta-commons/.mvn/wrapper/maven-wrapper.properties

`Ruta: auxiruta-commons/.mvn/wrapper/maven-wrapper.properties`

`SHA-256: 821602a8cceb1f8f5945f80b82b27617ba88b74a8afae230a5df80affcb6a8ab`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
.9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

auxiruta-commons/mvnw

`Ruta: auxiruta-commons/mvnw`

`SHA-256: cae96cef89ebea3531221f4ae17c23cf8edf67d00eae8306d4186ae1bbed4d02`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml verify. install de Commons publica el JAR en .m2.

auxiruta-commons/mvnw.cmd

`Ruta: auxiruta-commons/mvnw.cmd`

`SHA-256: 46eedb8419bd14fe70d5bb2916d7b6f51806e51b39d5b76a42610384ca929c1c`

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\auxiruta-commons\mvnw.cmd -f auxiruta-commons/pom.xml verify. install de Commons publica el JAR en .m2.

fleet-service/.mvn/wrapper/maven-wrapper.properties

Ruta: fleet-service/.mvn/wrapper/maven-wrapper.properties

SHA-256: 821602a8cceb1f8f5945f80b82b27617ba88b74a8afae230a5df80affcb6a8ab

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 3 líneas de herramienta distribuida.

L1

```
wrapperVersion=3.3.4
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L2

```
distributionType=only-script
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

L3

```
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.11/apache-maven-3
.9.11-bin.zip
```

Versión, modalidad o URL de distribución Maven Wrapper; no endpoint del backend.

fleet-service/mvnw

Ruta: fleet-service/mvnw

SHA-256: cae96cef89ebea3531221f4ae17c23cf8edf67d00eae8306d4186ae1bbed4d02

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 295 líneas de herramienta distribuida.

Uso: .\fleet-service\mvnw.cmd -f fleet-service/pom.xml verify. install de Commons publica el JAR en .m2.

fleet-service/mvnw.cmd

Ruta: fleet-service/mvnw.cmd

SHA-256: 46eedb8419bd14fe70d5bb2916d7b6f51806e51b39d5b76a42610384ca929c1c

mvnw inicia Maven en Unix; mvnw.cmd en Windows. Detectan Java, descargan distribución si falta y propagan argumentos/código de salida. No son lógica de negocio. 189 líneas de herramienta distribuida.

Uso: .\fleet-service\mvnw.cmd -f fleet-service/pom.xml verify. install de Commons publica el JAR en .m2.

12. JAR: binarios y contenido empaquetado

Los binarios presentes se inspeccionan sin recompilar. No se afirma que correspondan a cambios de fuente posteriores. La explicación línea por línea de Java está en las secciones de fuente.

api-gateway/target/app.jar

Ruta: api-gateway/target/app.jar

SHA-256: de3c991d58fcd70316045c01b1dcb69430f447b09f6c699ca50dbe8070fe5e38

Tamaño: 39.41 MiB; 216 entradas; 5 clases del proyecto; 74 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
api-gateway
Implementation-Version: 1.0.0
Main-Class:
org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.auxiruta.gateway.GatewayApplication

Spring-Boot-Version: 3.5.16
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/

Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

BOOT-INF/classes/com/auxiruta/gateway/config/GatewayRoutes.class

BOOT-INF/classes/com/auxiruta/gateway/config/RequestLoggingFilter.class

BOOT-INF/classes/com/auxiruta/gateway/config/SecurityConfiguration.class

BOOT-INF/classes/com/auxiruta/gateway/config/SecurityErrors.class

BOOT-INF/classes/com/auxiruta/gateway/GatewayApplication.class

Dependencias empaquetadas

spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar

log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar

spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar

spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar

spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | slf4j-api-2.0.18.jar | android-json-0.0.20131108.vaadin1.jar

spring-core-6.2.19.jar | spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar | spring-cloud-starter-gateway-server-webflux-4.3.5.jar | spring-cloud-starter-4.3.3.jar

spring-cloud-context-4.3.3.jar | spring-cloud-commons-4.3.3.jar | bcprov-jdk18on-1.80.2.jar | spring-cloud-gateway-server-webflux-4.3.5.jar | spring-cloud-gateway-server-4.3.5.jar

tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar | jboss-logging-3.6.3.Final.jar | classmate-1.7.3.jar

reactor-extra-3.5.5.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar | jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar

jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | reactor-netty-http-1.2.18.jar | netty-codec-http-4.1.135.Final.jar | netty-common-4.1.135.Final.jar

netty-buffer-4.1.135.Final.jar | netty-transport-4.1.135.Final.jar | netty-codec-4.1.135.Final.jar | netty-handler-4.1.135.Final.jar | netty-codec-http2-4.1.135.Final.jar

netty-resolver-dns-4.1.135.Final.jar | netty-resolver-4.1.135.Final.jar | netty-codec-dns-4.1.135.Final.jar | netty-resolver-dns-native-macos-4.1.135.Final-osx-x86_64.jar | netty-resolver-dns-classes-macos-4.1.135.Final.jar

netty-transport-native-epoll-4.1.135.Final-linux-x86_64.jar | netty-transport-native-unix-common-4.1.135.Final.jar | netty-transport-classes-epoll-4.1.135.Final.jar | reactor-netty-core-1.2.18.jar | netty-handler-proxy-4.1.135.Final.jar

netty-codec-socks-4.1.135.Final.jar | spring-webflux-6.2.19.jar | reactor-core-3.7.19.jar | reactive-streams-1.0.4.jar | spring-boot-properties-migrator-3.5.16.jar

spring-boot-configuration-metadata-3.5.16.jar | auxiruta-commons-1.0.0.jar | spring-web-6.2.19.jar | spring-boot-jarmode-tools-3.5.16.jar

api-gateway/target/app.jar.original

Ruta: api-gateway/target/app.jar.original

SHA-256: f4e450bbee84b0b09f089be35cd4eda2de672cd5d5b6932b6e4e1e798f041368

Tamaño: 0.01 MiB; 18 entradas; 5 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
api-gateway
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/gateway/config/GatewayRoutes.class

com/auxiruta/gateway/config/RequestLoggingFilter.class

com/auxiruta/gateway/config/SecurityConfiguration.class

com/auxiruta/gateway/config/SecurityErrors.class

com/auxiruta/gateway/GatewayApplication.class

assistance-service/target/app.jar

Ruta: assistance-service/target/app.jar

SHA-256: 59fe59f95202e8f421665b7be6986f7bede3da837455fd489aa03a03ac97f9b0

Tamaño: 61.03 MiB; 311 entradas; 78 clases del proyecto; 86 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
assistance-service
Implementation-Version: 1.0.0
Main-Class:
org.springframework.boot.loader.launch.JarLauncher
Start-Class:
com.auxiruta.assistance.AssistanceApplication
Spring-Boot-Version: 3.5.16
Spring-Boot-Classes:
BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/
Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx

Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

BOOT-INF/classes/com/auxiruta/assistance/AssistanceApplication.class

BOOT-INF/classes/com/auxiruta/assistance/config/ClockConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/OpenApiConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/RequestLoggingFilter.class

BOOT-INF/classes/com/auxiruta/assistance/config/SecurityConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/config/SecurityErrors.class

BOOT-INF/classes/com/auxiruta/assistance/exceptions/ApiExceptionHandler.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentController$Rejection.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentCoordinator.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$Action.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore$View.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssignmentStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AssistanceRequest.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationJobs.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore$Work.class

BOOT-INF/classes/com/auxiruta/assistance/requests/AutomationStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/CandidateEvaluation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/CandidateEvaluationRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DispatchValidator$Evidence.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DispatchValidator.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Mobility.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Registration.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$Status.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes$VehicleType.class

BOOT-INF/classes/com/auxiruta/assistance/requests/DomainTypes.class

BOOT-INF/classes/com/auxiruta/assistance/requests/Evaluation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService$CandidateView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService$EvaluationView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationReadService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore$Input.class

BOOT-INF/classes/com/auxiruta/assistance/requests/EvaluationStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionController$Completion.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Input.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Preparation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore$Step.class

BOOT-INF/classes/com/auxiruta/assistance/requests/ExecutionStore.class

BOOT-INF/classes/com/auxiruta/assistance/requests/HistoryRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestController.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$Cancellation.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$Created.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$CreateRequest.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$CreateVehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$EditVehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$HistoryView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$PageView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$RegistrationChange.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$RequestView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos$VehicleView.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestDtos.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestRepository.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestService$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestService.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RequestStatusHistory.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RulesConfiguration$1.class

BOOT-INF/classes/com/auxiruta/assistance/requests/RulesConfiguration.class

BOOT-INF/classes/com/auxiruta/assistance/requests/Vehicle.class

BOOT-INF/classes/com/auxiruta/assistance/requests/VehicleRepository.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$1.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$2.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$Candidate.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient$Eligibility.class

BOOT-INF/classes/com/auxiruta/assistance/routing/FleetClient.class

BOOT-INF/classes/com/auxiruta/assistance/routing/OpenRouteServiceClient.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider$Point.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider$Route.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RouteProvider.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingBudget.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingFailure$Reason.class

BOOT-INF/classes/com/auxiruta/assistance/routing/RoutingFailure.class

Dependencias empaquetadas

spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar

log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar

spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar

spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar

spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar

jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar | jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | tomcat-embed-core-10.1.55.jar

tomcat-embed-websocket-10.1.55.jar | spring-web-6.2.19.jar | spring-webmvc-6.2.19.jar | HikariCP-6.3.3.jar | spring-jdbc-6.2.19.jar

hibernate-core-6.6.53.Final.jar | jakarta.persistence-api-3.1.0.jar | jakarta.transaction-api-2.0.1.jar | jboss-logging-3.6.3.Final.jar | hibernate-commons-annotations-7.0.3.Final.jar

jandex-3.2.0.jar | classmate-1.7.3.jar | byte-buddy-1.17.8.jar | jaxb-runtime-4.0.9.jar | jaxb-core-4.0.9.jar

angus-activation-2.0.3.jar | txw2-4.0.9.jar | istack-commons-runtime-4.1.2.jar | jakarta.inject-api-2.0.1.jar | antlr4-runtime-4.13.2.jar

spring-data-jpa-3.5.13.jar | spring-data-commons-3.5.13.jar | spring-orm-6.2.19.jar | spring-tx-6.2.19.jar | slf4j-api-2.0.18.jar

spring-aspects-6.2.19.jar | aspectjweaver-1.9.25.1.jar | tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar

spring-security-oauth2-client-6.5.11.jar | oauth2-oidc-sdk-9.43.6.jar | content-type-2.2.jar | lang-tag-1.7.jar | jakarta.xml.bind-api-4.0.5.jar

jakarta.activation-api-2.1.4.jar | json-smart-2.5.2.jar | accessors-smart-2.5.2.jar | asm-9.7.1.jar | spring-core-6.2.19.jar

spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar | postgresql-42.7.11.jar | springdoc-openapi-starter-webmvc-ui-2.10.0.jar | springdoc-openapi-starter-webmvc-api-2.10.0.jar

springdoc-openapi-starter-common-2.10.0.jar | commons-lang3-3.17.0.jar | swagger-core-jakarta-2.2.55.jar | swagger-annotations-jakarta-2.2.55.jar | swagger-models-jakarta-2.2.55.jar

jackson-dataformat-yaml-2.21.4.jar | swagger-ui-5.34.0.jar | webjars-locator-lite-1.1.3.jar | jspecify-1.0.0.jar | auxiruta-commons-1.0.0.jar

spring-boot-jarmode-tools-3.5.16.jar

assistance-service/target/app.jar.original

Ruta: assistance-service/target/app.jar.original

SHA-256: 4eebd9ffd57000220034a735f76eb76d147a4b9d23a3ff81a504347c5198d4c0

Tamaño: 0.15 MiB; 102 entradas; 78 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
assistance-service
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/assistance/AssistanceApplication.class

com/auxiruta/assistance/config/ClockConfiguration.class

com/auxiruta/assistance/config/OpenApiConfiguration.class

com/auxiruta/assistance/config/RequestLoggingFilter.class

com/auxiruta/assistance/config/SecurityConfiguration.class

com/auxiruta/assistance/config/SecurityErrors.class

com/auxiruta/assistance/exceptions/ApiExceptionHandler.class

com/auxiruta/assistance/requests/AssignmentController$Rejection.class

com/auxiruta/assistance/requests/AssignmentController.class

com/auxiruta/assistance/requests/AssignmentCoordinator.class

com/auxiruta/assistance/requests/AssignmentStore$1.class

com/auxiruta/assistance/requests/AssignmentStore$Action.class

com/auxiruta/assistance/requests/AssignmentStore$View.class

com/auxiruta/assistance/requests/AssignmentStore.class

com/auxiruta/assistance/requests/AssistanceRequest.class

com/auxiruta/assistance/requests/AutomationJobs.class

com/auxiruta/assistance/requests/AutomationStore$1.class

com/auxiruta/assistance/requests/AutomationStore$Work.class

com/auxiruta/assistance/requests/AutomationStore.class

com/auxiruta/assistance/requests/CandidateEvaluation.class

com/auxiruta/assistance/requests/CandidateEvaluationRepository.class

com/auxiruta/assistance/requests/DispatchValidator$Evidence.class

com/auxiruta/assistance/requests/DispatchValidator.class

com/auxiruta/assistance/requests/DomainTypes$Mobility.class

com/auxiruta/assistance/requests/DomainTypes$Registration.class

com/auxiruta/assistance/requests/DomainTypes$Status.class

com/auxiruta/assistance/requests/DomainTypes$VehicleType.class

com/auxiruta/assistance/requests/DomainTypes.class

com/auxiruta/assistance/requests/Evaluation.class

com/auxiruta/assistance/requests/EvaluationReadService$CandidateView.class

com/auxiruta/assistance/requests/EvaluationReadService$EvaluationView.class

com/auxiruta/assistance/requests/EvaluationReadService.class

com/auxiruta/assistance/requests/EvaluationRepository.class

com/auxiruta/assistance/requests/EvaluationService.class

com/auxiruta/assistance/requests/EvaluationStore$1.class

com/auxiruta/assistance/requests/EvaluationStore$Input.class

com/auxiruta/assistance/requests/EvaluationStore.class

com/auxiruta/assistance/requests/ExecutionController$Completion.class

com/auxiruta/assistance/requests/ExecutionController.class

com/auxiruta/assistance/requests/ExecutionService.class

com/auxiruta/assistance/requests/ExecutionStore$1.class

com/auxiruta/assistance/requests/ExecutionStore$Input.class

com/auxiruta/assistance/requests/ExecutionStore$Preparation.class

com/auxiruta/assistance/requests/ExecutionStore$Step.class

com/auxiruta/assistance/requests/ExecutionStore.class

com/auxiruta/assistance/requests/HistoryRepository.class

com/auxiruta/assistance/requests/RequestController.class

com/auxiruta/assistance/requests/RequestDtos$Cancellation.class

com/auxiruta/assistance/requests/RequestDtos$Created.class

com/auxiruta/assistance/requests/RequestDtos$CreateRequest.class

com/auxiruta/assistance/requests/RequestDtos$CreateVehicle.class

com/auxiruta/assistance/requests/RequestDtos$EditVehicle.class

com/auxiruta/assistance/requests/RequestDtos$HistoryView.class

com/auxiruta/assistance/requests/RequestDtos$PageView.class

com/auxiruta/assistance/requests/RequestDtos$RegistrationChange.class

com/auxiruta/assistance/requests/RequestDtos$RequestView.class

com/auxiruta/assistance/requests/RequestDtos$VehicleView.class

com/auxiruta/assistance/requests/RequestDtos.class

com/auxiruta/assistance/requests/RequestRepository.class

com/auxiruta/assistance/requests/RequestService$1.class

com/auxiruta/assistance/requests/RequestService.class

com/auxiruta/assistance/requests/RequestStatusHistory.class

com/auxiruta/assistance/requests/RulesConfiguration$1.class

com/auxiruta/assistance/requests/RulesConfiguration.class

com/auxiruta/assistance/requests/Vehicle.class

com/auxiruta/assistance/requests/VehicleRepository.class

com/auxiruta/assistance/routing/FleetClient$1.class

com/auxiruta/assistance/routing/FleetClient$2.class

com/auxiruta/assistance/routing/FleetClient$Candidate.class

com/auxiruta/assistance/routing/FleetClient$Eligibility.class

com/auxiruta/assistance/routing/FleetClient.class

com/auxiruta/assistance/routing/OpenRouteServiceClient.class

com/auxiruta/assistance/routing/RouteProvider$Point.class

com/auxiruta/assistance/routing/RouteProvider$Route.class

com/auxiruta/assistance/routing/RouteProvider.class

com/auxiruta/assistance/routing/RoutingBudget.class

com/auxiruta/assistance/routing/RoutingFailure$Reason.class

com/auxiruta/assistance/routing/RoutingFailure.class

auxiruta-commons/target/auxiruta-commons-1.0.0.jar

Ruta: auxiruta-commons/target/auxiruta-commons-1.0.0.jar

SHA-256: 77e2583fdd3d48ac97b6a6ed53a0d35ed57df4813fec9b6722bcd12a4233795b

Tamaño: 0.01 MiB; 17 entradas; 4 clases del proyecto; 0 dependencias anidadas.

Biblioteca regular instalada en .m2; no se inicia como servidor.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
auxiruta-commons
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/commons/errors/Problems.class

com/auxiruta/commons/GeoMath.class

com/auxiruta/commons/security/TokenSupport.class

com/auxiruta/commons/time/UtcClock.class

fleet-service/target/app.jar

Ruta: fleet-service/target/app.jar

SHA-256: 82e346bff2fce417befd97b7602feff4cc3184925f498a12f5b8c40fa6530bb9

Tamaño: 59.43 MiB; 272 entradas; 51 clases del proyecto; 79 dependencias anidadas.

Ejecutable Spring Boot: BOOT-INF/classes contiene clases/recursos; BOOT-INF/lib las dependencias. El launcher inicia la Start-Class. Usar java -jar con entorno y dependencias configurados.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
fleet-service
Implementation-Version: 1.0.0
Main-Class:
org.springframework.boot.loader.launch.JarLauncher
Start-Class: com.auxiruta.fleet.FleetApplication

Spring-Boot-Version: 3.5.16
Spring-Boot-Classes: BOOT-INF/classes/
Spring-Boot-Lib: BOOT-INF/lib/

Spring-Boot-Classpath-Index: BOOT-INF/classpath.idx
Spring-Boot-Layers-Index: BOOT-INF/layers.idx
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateService$Candidate.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/CandidateService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetController$AdministrationRules.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$AvailabilityChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$CreateOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$CreateTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$EditOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$EditTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$LinkOperator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$Location.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$OperatorView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$PageView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$QualificationChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$RegistrationChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$StatusChange.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$StatusView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos$TruckView.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetDtos.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetService$1.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Availability.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Operational.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Qualification.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$Registration.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes$VehicleType.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/FleetTypes.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/Operator.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/OperatorRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationController$StatusQueries.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationController.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationJobs.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$Eligibility.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$Input.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$StatusQuery.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService$View.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ReservationService.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/ResourceReservation.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TowTruck.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TowTruckRepository.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TruckStatusHistory.class

BOOT-INF/classes/com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.class

BOOT-INF/classes/com/auxiruta/fleet/config/ClockConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/OpenApiConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/RequestLoggingFilter.class

BOOT-INF/classes/com/auxiruta/fleet/config/SecurityConfiguration.class

BOOT-INF/classes/com/auxiruta/fleet/config/SecurityErrors.class

BOOT-INF/classes/com/auxiruta/fleet/exceptions/ApiExceptionHandler.class

BOOT-INF/classes/com/auxiruta/fleet/FleetApplication.class

Dependencias empaquetadas

spring-boot-3.5.16.jar | spring-boot-autoconfigure-3.5.16.jar | logback-classic-1.5.34.jar | logback-core-1.5.34.jar | log4j-to-slf4j-2.24.3.jar

log4j-api-2.24.3.jar | jul-to-slf4j-2.0.18.jar | jakarta.annotation-api-2.1.1.jar | snakeyaml-2.4.jar | spring-security-config-6.5.11.jar

spring-aop-6.2.19.jar | spring-beans-6.2.19.jar | spring-context-6.2.19.jar | spring-security-core-6.5.11.jar | spring-security-crypto-6.5.11.jar

spring-expression-6.2.19.jar | micrometer-observation-1.15.12.jar | micrometer-commons-1.15.12.jar | spring-security-oauth2-resource-server-6.5.11.jar | spring-security-oauth2-core-6.5.11.jar

spring-security-oauth2-jose-6.5.11.jar | nimbus-jose-jwt-9.37.4.jar | jcip-annotations-1.0-1.jar | jackson-databind-2.21.4.jar | jackson-annotations-2.21.jar

jackson-core-2.21.4.jar | jackson-datatype-jdk8-2.21.4.jar | jackson-datatype-jsr310-2.21.4.jar | jackson-module-parameter-names-2.21.4.jar | tomcat-embed-core-10.1.55.jar

tomcat-embed-websocket-10.1.55.jar | spring-web-6.2.19.jar | spring-webmvc-6.2.19.jar | HikariCP-6.3.3.jar | spring-jdbc-6.2.19.jar

hibernate-core-6.6.53.Final.jar | jakarta.persistence-api-3.1.0.jar | jakarta.transaction-api-2.0.1.jar | jboss-logging-3.6.3.Final.jar | hibernate-commons-annotations-7.0.3.Final.jar

jandex-3.2.0.jar | classmate-1.7.3.jar | byte-buddy-1.17.8.jar | jaxb-runtime-4.0.9.jar | jaxb-core-4.0.9.jar

angus-activation-2.0.3.jar | txw2-4.0.9.jar | istack-commons-runtime-4.1.2.jar | jakarta.inject-api-2.0.1.jar | antlr4-runtime-4.13.2.jar

spring-data-jpa-3.5.13.jar | spring-data-commons-3.5.13.jar | spring-orm-6.2.19.jar | spring-tx-6.2.19.jar | slf4j-api-2.0.18.jar

spring-aspects-6.2.19.jar | aspectjweaver-1.9.25.1.jar | tomcat-embed-el-10.1.55.jar | hibernate-validator-8.0.3.Final.jar | jakarta.validation-api-3.0.2.jar

jakarta.xml.bind-api-4.0.5.jar | jakarta.activation-api-2.1.4.jar | spring-core-6.2.19.jar | spring-jcl-6.2.19.jar | spring-security-web-6.5.11.jar

postgresql-42.7.11.jar | springdoc-openapi-starter-webmvc-ui-2.10.0.jar | springdoc-openapi-starter-webmvc-api-2.10.0.jar | springdoc-openapi-starter-common-2.10.0.jar | commons-lang3-3.17.0.jar

swagger-core-jakarta-2.2.55.jar | swagger-annotations-jakarta-2.2.55.jar | swagger-models-jakarta-2.2.55.jar | jackson-dataformat-yaml-2.21.4.jar | swagger-ui-5.34.0.jar

webjars-locator-lite-1.1.3.jar | jspecify-1.0.0.jar | auxiruta-commons-1.0.0.jar | spring-boot-jarmode-tools-3.5.16.jar

fleet-service/target/app.jar.original

Ruta: fleet-service/target/app.jar.original

SHA-256: 3606e55dd11b7ba08a028d823f14d977221c3d2d6ddb6ee4659a030430bfb61f

Tamaño: 0.08 MiB; 70 entradas; 51 clases del proyecto; 0 dependencias anidadas.

JAR previo al reempaquetado Boot: clases/recursos sin ensamblado completo de dependencias; Docker usa app.jar.

Manifest real

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.2
Build-Jdk-Spec: 21
Implementation-Title:
fleet-service
Implementation-Version: 1.0.0
```

Main-Class identifica el launcher; Start-Class la aplicación delegada; Build-Jdk-Spec la versión de Java de construcción, cuando esos campos están presentes.

Clases del proyecto incluidas

com/auxiruta/fleet/catalog/CandidateController.class

com/auxiruta/fleet/catalog/CandidateService$Candidate.class

com/auxiruta/fleet/catalog/CandidateService.class

com/auxiruta/fleet/catalog/FleetController$AdministrationRules.class

com/auxiruta/fleet/catalog/FleetController.class

com/auxiruta/fleet/catalog/FleetDtos$AvailabilityChange.class

com/auxiruta/fleet/catalog/FleetDtos$CreateOperator.class

com/auxiruta/fleet/catalog/FleetDtos$CreateTruck.class

com/auxiruta/fleet/catalog/FleetDtos$EditOperator.class

com/auxiruta/fleet/catalog/FleetDtos$EditTruck.class

com/auxiruta/fleet/catalog/FleetDtos$LinkOperator.class

com/auxiruta/fleet/catalog/FleetDtos$Location.class

com/auxiruta/fleet/catalog/FleetDtos$OperatorView.class

com/auxiruta/fleet/catalog/FleetDtos$PageView.class

com/auxiruta/fleet/catalog/FleetDtos$QualificationChange.class

com/auxiruta/fleet/catalog/FleetDtos$RegistrationChange.class

com/auxiruta/fleet/catalog/FleetDtos$StatusChange.class

com/auxiruta/fleet/catalog/FleetDtos$StatusView.class

com/auxiruta/fleet/catalog/FleetDtos$TruckView.class

com/auxiruta/fleet/catalog/FleetDtos.class

com/auxiruta/fleet/catalog/FleetService$1.class

com/auxiruta/fleet/catalog/FleetService.class

com/auxiruta/fleet/catalog/FleetTypes$Availability.class

com/auxiruta/fleet/catalog/FleetTypes$Operational.class

com/auxiruta/fleet/catalog/FleetTypes$Qualification.class

com/auxiruta/fleet/catalog/FleetTypes$Registration.class

com/auxiruta/fleet/catalog/FleetTypes$VehicleType.class

com/auxiruta/fleet/catalog/FleetTypes.class

com/auxiruta/fleet/catalog/Operator.class

com/auxiruta/fleet/catalog/OperatorRepository.class

com/auxiruta/fleet/catalog/ReservationController$StatusQueries.class

com/auxiruta/fleet/catalog/ReservationController.class

com/auxiruta/fleet/catalog/ReservationJobs.class

com/auxiruta/fleet/catalog/ReservationRepository.class

com/auxiruta/fleet/catalog/ReservationService$Eligibility.class

com/auxiruta/fleet/catalog/ReservationService$Input.class

com/auxiruta/fleet/catalog/ReservationService$StatusQuery.class

com/auxiruta/fleet/catalog/ReservationService$View.class

com/auxiruta/fleet/catalog/ReservationService.class

com/auxiruta/fleet/catalog/ResourceReservation.class

com/auxiruta/fleet/catalog/TowTruck.class

com/auxiruta/fleet/catalog/TowTruckRepository.class

com/auxiruta/fleet/catalog/TruckStatusHistory.class

com/auxiruta/fleet/catalog/TruckStatusHistoryRepository.class

com/auxiruta/fleet/config/ClockConfiguration.class

com/auxiruta/fleet/config/OpenApiConfiguration.class

com/auxiruta/fleet/config/RequestLoggingFilter.class

com/auxiruta/fleet/config/SecurityConfiguration.class

com/auxiruta/fleet/config/SecurityErrors.class

com/auxiruta/fleet/exceptions/ApiExceptionHandler.class

com/auxiruta/fleet/FleetApplication.class

13. Glosario de lectura

Entidad / DTO

Entidad: objeto JPA persistente. DTO: contrato de entrada/salida que no representa una tabla.

Controller / Service / Repository / Store

Controller traduce HTTP; Service aplica reglas; Repository consulta; Store delimita operaciones transaccionales en este diseño.

JWT / sub / aud / azp

JWT firmado: sub identifica usuario, aud destinatario, azp cliente que lo obtuvo. No son contraseñas.

Rol / propiedad

Rol habilita tipo de acción; propietario/asignatario limita el recurso sobre el que puede actuar.

401 / 403 / 404 / 409 / 503

Autenticación / permiso / inexistente o ajeno / conflicto / dependencia temporal.

Snapshot

Copia de datos/reglas usados al decidir, conservada aunque el origen cambie.

Idempotencia / fingerprint

La clave identifica intención; la huella impide reutilizarla con otro contenido.

Bloqueo pesimista / optimista

Uno serializa acceso mediante lock; otro detecta cambios con @Version.

Transacción / compensación

SQL local no engloba ambos servicios. Las intenciones durables y compensaciones recuperan operaciones HTTP inciertas.

HELD / CONFIRMED

Retención temporal de propuesta / compromiso tras aceptación. Distintos del estado de solicitud y del registro de grúa.

Caché / cuota

Caché reutiliza evidencia vigente; cuota limita llamadas reales. Rutas en memoria, presupuesto persistido.

Perfil demo / usuario demo

El perfil reduce tiempos; la cuenta es identidad. No se eliminan permisos ni dependencia ORS.

JAR / original

Bytecode y recursos; el ejecutable Boot añade dependencias/launcher. original es el artefacto anterior al reempaquetado.

Cobertura: 100 archivos propios, 3152 líneas no vacías comentadas, 28 archivos de pruebas/soporte y 7 binarios. Los restantes son Maven Wrapper.